



Towards Long-Horizon Agents: A Survey

Foundation, Evolution, Harness, Optimization, Application, and Frontier



面向长-时域智能体：综述——基础、演化、控制、优化、应用与前沿领域

Guanting Dong^{1,*}, Xiaoshuai Song^{1,*}, Yuyang Hu^{1,*}, Jiajie Jin^{1,*}, Chenghao Zhang^{1,*}, Yifei Chen¹, Xiaoxi Li¹, Huaying Yuan¹, Xinyu Yang¹, Tongyu Wen¹, Jiejun Tan¹, Hongjin Qian², Shijue Huang⁵, Juntaing Lu², Zhenyu Li³, Wanjun Zhong⁴, Yutao Zhu¹, Tat-Seng Chua⁶, Zhicheng Dou^{1,†}, Ji-Rong Wen¹

董广婷¹、宋晓帅¹、胡玉阳¹、金佳杰¹、张承浩¹、陈一飞¹、李晓曦¹、袁慧阳¹、杨欣宇¹、文彤宇¹、谭继军¹、钱鸿锦²、黄世杰⁵、卢俊廷²、李振宇³、钟万军⁴、朱宇涛¹、Chua Tat-Seng⁶、Dou Zhicheng^{1,†}、Wen Ji-Rong¹

* Core Contributors † Corresponding author

* 核心贡献者† 通讯作者

¹Renmin University of China, ²Peking University, ³Tsinghua University, ⁴Sun Yat-Sen University,

¹中国人民大学, ²北京大学, ³清华大学, ⁴中山大学,

⁵Hong Kong University of Science and Technology, ⁶National University of Singapore

⁵香港科技大学、⁶新加坡国立大学

With the rapid advancement of LLM capabilities, expectations for AI agents are shifting from solving simple, single-turn tasks toward carrying out long-horizon tasks in the real world. We refer to such systems as **long-horizon agents**: agents that plan over extended horizons, interact with real-world environments, recover from their own mistakes, and adapt their strategies during execution. This capability is rapidly becoming the central bottleneck for practical agent intelligence. Despite this progress, the field still lacks a shared definition and taxonomy for long-horizon agents. Related concepts such as “self-evolving” or “autonomous” agents are often used interchangeably, none of which clearly captures what it means to build a *long-horizon agent*. This gap leaves the community without a principled way to attribute advances in long-horizon competence.

随着大语言模型（LLM）能力的飞速提升，人们对人工智能智能体的期望正从解决简单的单轮任务转向执行现实世界中的长周期任务。我们将此类系统称为**长时程智能体**：即能够进行远期规划、与现实环境交互、从自身错误中恢复，并在执行过程中动态调整策略的智能体。这一能力正迅速成为实际智能体研究的核心瓶颈。尽管已取得诸多进展，该领域仍缺乏针对长时程智能体的统一定义与分类体系。诸如“自进化”或“自主”智能体等相关概念常被混用，但这些概念均未能清晰地阐明构建**长时程智能体**的含义。这一空白使得学界缺乏一种基于原则的途径，用以归因于长周期能力提升。

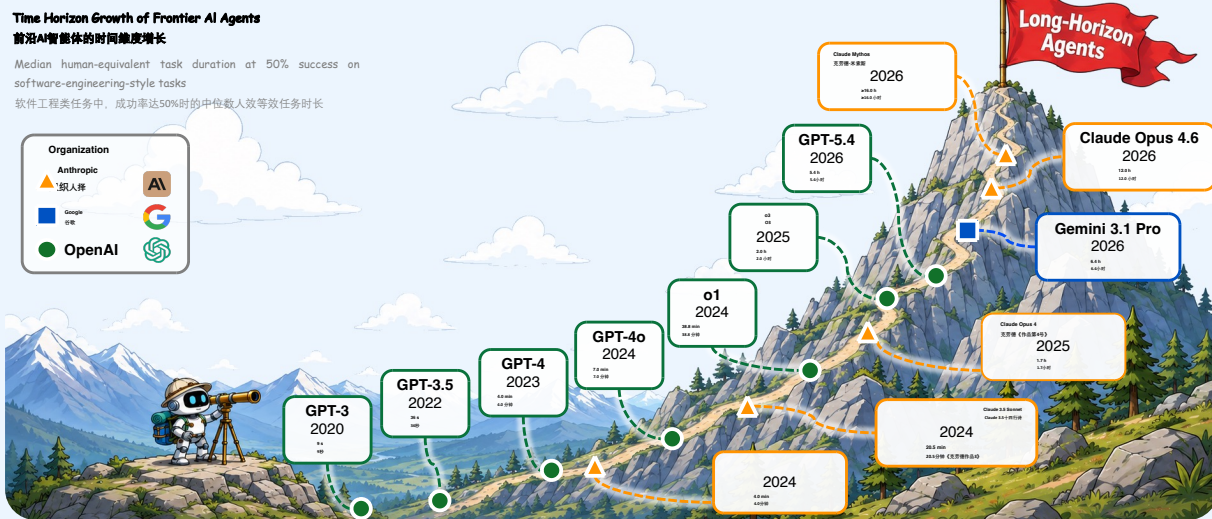
This paper offers a unified landscape by framing long-horizon agency as the co-evolution of an **externalized harness** and an **internalized optimization**, and organizes the paper around six connected perspectives: **Foundation, Evolution, Harness, Optimization, Application, and Frontier**. We first formalize long-horizon agency as a harness-coupled decision process and distinguish it from neighboring concepts such as long-running execution, autonomy, and self-evolution. We then trace the field’s evolution from prompt-level control to runtime agent systems, classifying existing work through the complementary lenses of externalized harnesses and internalized optimization. Building on this view, we organize five application forms of long-horizon agents by their interfaces, and review the corresponding benchmarks and resources. Finally, we discuss key challenges and frontier directions. Looking ahead, we hope this paper serves not only as a reference for existing work, but also as a foundation for building the next generation of capable, reliable long-horizon agents.

本文通过将长周期主体性问题界定为**外部化调控机制**与**内部化优化过程**的协同演化，构建了一个统一的理论框架，并围绕六个相互关联的视角展开论述：**基础部分进化、优化、优化、应用及前沿**。我们首先将长时程代理问题形式化为一种“约束-耦合”决策过程，并将其与“长期执行”“自主性”及“自我进化”等邻近概念区分开来。随后，我们追溯了该领域的演进历程——从即时控制阶段发展至运行时智能体系统，并通过“外部化调控机制”与“内部化优化”这两个互补视角对现有研究进行分类。基于这一观点，我们按接口对长时程智能体的五种应用场景进行了分类，并综述了相应的基准测试集与资源；最后，我们探讨了关键挑战与前沿研究方向。展望未来，我们希望本文不仅能为现有研究提供参考，还能为构建新一代高性能、高可靠性长时程智能体奠定基础。

✉ dongguanting@ruc.edu.cn, dou@ruc.edu.cn
✉ dongguanting@ruc.edu.cn, dou@ruc.edu.cn

🌐 <https://github.com/RUC-NLPIR/Awesome-Long-Horizon-Agents>

🌐 <https://Long-Horizon-Agents.github.io>



*The empirical time-horizon trend of frontier agents: the task-completion horizon has climbed from seconds to over twelve hours, doubling roughly every seven months (<https://metr.org/time-horizons>).

*前沿参与者的经验性时间范围趋势：任务完成时间范围已从几秒延长至超过 12 小时，大约每 7 个月翻一番（<https://metr.org/time-horizons>）。

Contents
目录

1 Introduction 4
1 引言 4
2 Foundation: Formalizing Long-Horizon Agents 7
2 基础：长时程智能体的形式化建模 7
2.1 Definition 7
2.1 定义 7
2.2 Three Levels of Long-Horizon Tasks and Capabilities 10
2.2 长周期任务与能力的三级划分 10
2.3 Scaling the Time Horizon of Frontier-Agent Execution 11
2.3 扩展前沿智能体执行的时间范围 11
2.4 Long-Horizon Agent vs. Neighboring Concepts 13
2.4 长时程智能体与邻近概念对比 13
2.4.1 Long-Horizon Agent vs. Long-Running Agent 13
2.4.1 长时程智能体与长运行智能体 13
2.4.2 Long-Horizon Agent vs. Autonomous Agent 13
2.4.2 长时程智能体与自主智能体 13
2.4.3 Long-Horizon Agent vs. Self-Evolving Agent 14
2.4.3 长时程智能体与自进化智能体 14
3 Evolution: From Prompting to Runtime 15
3 演进历程：从提示阶段到运行时 15
3.1 Stage I: Prompt Engineering (2020–2023) 16
3.1 第一阶段：提示工程（2020–2023年） 16
3.2 Stage II: Context Engineering (2023–2025) 17
3.2 第二阶段：语境工程（2023–2025年） 17
3.3 Stage III: Runtime Harnesses (2025–Present) 18
3.3 第三阶段：运行时套件（2025年至今） 18
4 Harness: Externalizing Long-Horizon Capability 20
4 吊带：外化长周期能力 20
4.1 Loops and Workflows 20
4.1 循环与 workflow 20
4.1.1 Linear Workflows 21
4.1.1 线性 workflow 21
4.1.2 Plan-Execute Workflows 21
4.1.2 计划-执行 workflow 21
4.1.3 Branching Workflows 22
4.1.3 分支 workflow 22
4.2 Context and Memory 23
4.2 语境与记忆 23
4.2.1 Working Context 23
4.2.1 工作环境 23
4.2.2 Persistent Memory 24
4.2.2 持久内存 24
4.3 Tools, MCP, and Skills 26
4.3 工具、MCP 与技能 26
4.3.1 Tool Interfaces and Protocols 27
4.3.1 工具接口与协议 27
4.3.2 Active Tool Discovery 27
4.3.2 主动工具发现 27
4.3.3 Skill Libraries 28
4.3.3 技能库 28
4.4 Orchestration 29
4.4 编排 29
4.4.1 Decomposition and Roles 29
4.4.1 分解与作用 29
4.4.2 Coordination Topologies 30
4.4.2 协调拓扑结构 30
4.4.3 Orchestration Optimization 30
4.4.3 编排优化 30
4.4.4 Agent Protocols 30
4.4.4 代理协议 30
4.5 Hooks and Middleware 31
4.5 钩子与中间件 31
4.5.1 Pre-defined Rule-based Hooks 31
4.5.1 预定义的基于规则的挂钩 31
4.5.2 Custom User-defined Hooks 32
4.5.2 自定义用户定义钩子 32
4.5.3 Runtime-adaptive Hooks 33
4.5.3 运行时自适应钩子 33
4.6 Verification 34
4.6 验证 34
4.6.1 Assessment Targets 34
4.6.1 评估目标 34
4.6.2 Verification Levels 36
4.6.2 验证级别 36
4.6.3 Verifier Strategies 36
4.6.3 验证器策略 36
5 Optimization: Internalizing Long-Horizon Capability 37
5 优化：内化长周期能力 37
5.1 Architectural Substrate 37
5.1 建筑基材 37
5.1.1 Explicit-Context Architectures 38
5.1.1 显式上下文架构 38
5.1.2 Compressed-State Architectures 39
5.1.2 压缩态架构 39
5.1.3 Hybrid Architectures 40
5.1.3 混合架构 40
5.1.4 High-Throughput Mechanisms 40
5.1.4 高通量机制 40
5.2 Data and Environment Synthesis 40
5.2 数据与环境综合分析 40
5.2.1 Task Synthesis 41
5.2.1 任务综合 41
5.2.2 Environment Synthesis 41
5.2.2 环境综合设计 41
5.2.3 Trajectory Synthesis 42
5.2.3 轨迹合成 42
5.3 Pre-training and Mid-training 43
5.3 预训练与中期训练 43
5.3.1 Reasoning Priors 44
5.3.1 推理先验 44
5.3.2 Long-Context State 44
5.3.2 长上下文状态 44
5.3.3 Multimodal Perception 44
5.3.3 多模态感知 44
5.3.4 Data Mixture and Training Strategies 44
5.3.4 数据混合与训练策略 44

5.4	Fine-tuning	45
5.4	微调.	45
5.4.1	Instruction Selection and Mixing	45
5.4.1	指令选择与混合.	45
5.4.2	Curriculum Learning	46
5.4.2	课程学习.	46
5.4.3	Distillation	46
5.4.3	蒸馏.	46
5.5	Agentic Reinforcement Learning	47
5.5	代理强化学习.	47
5.5.1	Credit Assignment	48
5.5.1	信用分配.	48
5.5.2	Policy Optimization	50
5.5.2	策略优化.	50
5.5.3	Sampling Strategy	50
5.5.3	抽样策略.	50
5.5.4	Interaction Patterns	51
5.5.4	交互模式.	51
5.6	On-Policy Distillation	52
5.6	在线策略蒸馏.	52
5.6.1	Teacher-guided OPD	52
5.6.1	教师引导的门诊诊疗服务.	52
5.6.2	Self-improving OPD	53
5.6.2	自改进OPD.	53
5.7	Self-evolution	53
5.7	自进化.	53
5.7.1	Offline Self-evolution	54
5.7.1	离线自进化.	54
5.7.2	Online Self-evolution	54
5.7.2	在线自我进化.	54
5.7.3	Agent–Environment Co-evolution	55
5.7.3	智能体–环境共进化.	55
6	Application: Long-Horizon Agents in Practice	55
6	应用场景：长时程智能体在实际中的应用	55
6.1	Software Engineering	56
6.1	软件工程.	56
6.1.1	Repository Grounding	56
6.1.1	存储库接地.	56
6.1.2	Workflow-level Planning	57
6.1.2	工作流级规划.	57
6.1.3	Feedback-driven Repair	58
6.1.3	反馈驱动式修复.	58
6.2	Information Seeking	58
6.2	信息寻求.	58
6.2.1	Deep Search	58
6.2.1	深度搜索.	58
6.2.2	Wide Search	59
6.2.2	广域搜索.	59
6.2.3	Multimodal Grounding	59
6.2.3	多模态接地.	59
6.2.4	Research Synthesis	59
6.2.4	研究综述.	59
6.3	Computer Use	60
6.3	计算机使用.	60
6.3.1	Browser Agents	60
6.3.1	浏览器代理.	60
6.3.2	Desktop GUI Agents	60
6.3.2	桌面图形用户界面代理.	60
6.3.3	Mobile Agents	60
6.3.3	移动代理.	60
6.4	Multimodal Agents	61
6.4	多模态智能体.	61
6.4.1	Multimodal Understanding	61
6.4.1	多模态理解.	61
6.4.2	Multimodal Generation	61
6.4.2	多模态生成.	61
6.4.3	Omnimodal Agency	62
6.4.3	全模态机构.	62
6.5	General-purpose Agents	62
6.5	通用智能体.	62
6.5.1	Personal Assistants	62
6.5.1	个人助理.	62
6.5.2	Embodied Agents and World Models	63
6.5.2	具身智能体与世界模型.	63
6.5.3	Productive Agents	63
6.5.3	生产性因子.	63
6.6	Benchmarks and Resources	64
6.6	基准测试与资源.	64
7	Frontier: Open Challenges and Outlooks	67
7	前沿：开放性挑战与展望	67
7.1	Evolution: Generalization and Learning Over Time	67
7.1	进化：泛化与随时间推移的学习.	67
7.1.1	Self-evolving Harness and Agents	67
7.1.1	自进化约束与代理.	67
7.1.2	Harness Generalization and Transferability	69
7.1.2	Harness Generalization and Transferability	69
7.1.3	Continual and Lifelong Learning	69
7.1.3	持续终身学习.	69
7.2	Effectiveness: Acting in Realistic Environments	70
7.2	有效性：在真实环境中进行操作.	70
7.2.1	Real-world Environment Interaction	70
7.2.1	真实环境交互.	70
7.2.2	From Digital to Embodied Agents	71
7.2.2	从数字智能体到具身智能体.	71
7.3	Efficiency: Spending Compute, Context, and Modality Budgets	71
7.3	效率：计算、上下文及模态预算分配.	71
7.3.1	Cost- and Budget-aware Agency	72
7.3.1	成本与预算导向型机构.	72
7.3.2	Multimodal and Omni Harnesses	72
7.3.2	多模态与全场景安全带系统.	72
7.4	Trustworthiness: Robust and Governed Autonomy	73
7.4	可信性：稳健且受监管的自主性.	73
7.4.1	Reflection and Error Robustness	73
7.4.1	反射与误差鲁棒性.	73
7.4.2	Safety and Governance	74
7.4.2	安全与治理.	74
8	Conclusion	74
8	结论	74

Landscape of Long-Horizon Agent Research
长时程智能体研究全景图

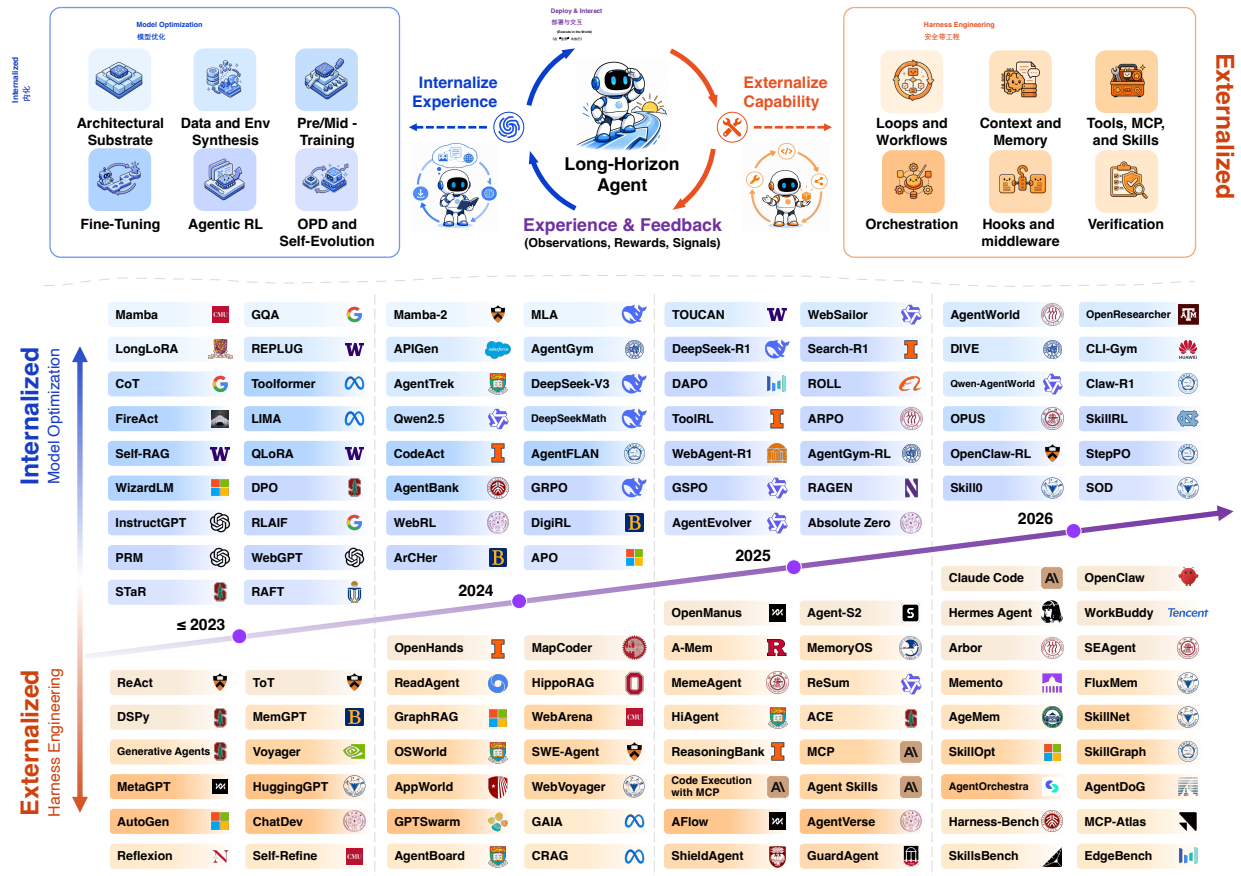


Figure 1 The landscape of long-horizon agent research. This paper adopts a co-evolutionary view of long-horizon agents, organized around *externalized harness engineering* (bottom) and *internalized model optimization* (top).

图1：长时程智能体研究的全景概览。 本文采用长时程智能体的共进化视角，其架构围绕外部化控制工程（下图）与内部化模型优化（上图）展开。

1 Introduction

1 引言

Over the past few years, large language models (LLMs) have moved from single-turn chatbots to the decision-making core of autonomous agents in software engineering [260, 747], general-purpose assistance [431], scientific discovery [678], computer use [716, 885], and multimodal interaction [141]. These domains differ on the surface, but they share one decisive requirement that we call *long horizon*: they require persistent iteration across reasoning, tool use, observation, and revision over many interdependent steps—from those completable within a single context window to those that span windows, sessions, or open-ended task streams. To meet this requirement, *long-horizon agents* have emerged, expanding the boundaries of agent applications through continual, sustained interaction with real-world environments.

过去几年间，大语言模型（LLMs）已从单轮对话机器人发展为软件工程领域自主智能体的决策核心[260, 747]，以及通用辅助工具[431]。科学发现[678]、计算机应用[716, 885]以及多模态交互[141]。这些领域在表面上各不相同，但它们都满足一项至关重要的共同要求——我们称之为*长时程*：它们都需要在推理、工具使用及观察等环节中进行持续的迭代。以及对多个相互关联的步骤进行修订——这些步骤既可在一个单一上下文窗口内完成，也可跨越不同窗口、会话或开放式的任务流来完成。为满足这一需求，*长时程智能体*应运而生，通过与现实环境进行持续、长期的交互，拓展了智能体应用的边界。

Long-horizon competence is now advancing rapidly and increasingly marks the practical line for delegating productive work to agents. For instance, frontier coding agents are already able to run for hours on a single project, occasionally close to a full day [14, 456]. Moreover, METR [286] reports that the time horizon of advanced agentic task execution has doubled roughly every seven months over the past several years, accelerating to about every four months for the most recent models. For example, GLM-5.1 is reported to sustain independent execution on a single task for up to eight hours, covering planning, implementation, testing, iterative refinement, and final delivery [179]. Notably, this capability cannot be obtained *simply* by optimizing and scaling the model’s internal parameters or enlarging its context window alone; its capability boundary also varies sharply with the *external harness* surrounding the model [123, 378, 468, 511]. Therefore, the central thesis of this paper is that *long-horizon agency is a system-level capability jointly shaped by two co-evolving components: externalized harness engineering and internalized model optimization*.

长周期能力正在快速演进，正日益成为将高成效工作委托给代理人的实践准则。例如，前沿编码代理已能够在一个项目上连续运行数小时，有时甚至接近全天运行[14, 456]。此外，METR[286]指出，在过去数年间，高级代理任务执行的时间跨度大约每7个月翻一番；而在最新模型中，这一时间跨度已加速至约每4个月一次。例如，据报告，GLM-5.1能够在单个任务上独立运行长达8小时，涵盖规划、实施、测试、迭代优化及最终交付等全流程[179]。值得注意的是，这种能力无法仅通过优化和缩放模型的内部参数或单纯扩大其上下文窗口来获得；其能力边界还会随外部因素而发生剧烈变化。模型周围的束带[123, 378, 468, 511]。因此，本文的核心论点在于：长期代理能力是一种系统级能力，是由两个协同演进的组成部分共同塑造的——即外

部化的系统工程与内部化的模型优化。

Why long-horizon task execution is hard for agents. The difficulty of long-horizon task execution does not reduce to per-step accuracy: it arises from the *composition* of many tightly coupled steps into a single trajectory, rather than from any individual decision. Across the systems we organize, three representative failure modes recur throughout long-horizon execution and structure much of the technical discussion.

为何长时程任务执行对智能体而言较为困难。长时程任务执行的困难并非仅源于单步精度问题：其根源在于将众多紧密耦合的步骤组合为一条单一轨迹，而非源于任何单个决策。在我们所组织的各类系统中，三种典型的故障模式在长周期执行过程中反复出现，并构成了大部分技术讨论的核心内容。

- **Goal drift and compounding error.** Over thousands of steps an agent can slowly diverge from its original objective, and because no step is perfectly reliable, local errors accumulate along the trajectory and eventually steer it onto a wrong reasoning path [20, 549].
- **目标漂移与复合误差。**在数千步迭代过程中，智能体可能会逐渐偏离其原始目标；由于没有任何一步是绝对可靠的，局部误差会在轨迹上不断累积，最终导致其转向错误的推理路径[20, 549]。
- **Context rot and context-limit pressure.** As the working context fills, an agent tends to degrade abruptly once it passes a certain context-utilization threshold [378, 511]; conversely, an agent that perceives its context limit may terminate prematurely and declare partial progress a success [14].
- **上下文衰减与上下文限制压力。**随着工作环境的扩展，智能体一旦超过特定的上下文利用阈值[378, 511]，往往会突然性能下降；反之，若智能体感知到其上下文限制，则可能会过早终止并声明为部分终止。进展取得成功[14]。
- **Sparse, delayed rewards and irreversibility.** Many long-horizon tasks return a reward only at the end, leaving the agent with an extremely sparse learning signal over its intermediate decisions [808]; moreover, the longer an agent acts, the more likely it is to take a risky, irreversible action, so monitoring its execution safety and recoverability becomes essential.
- **稀疏、延迟的奖励与不可逆性。**许多长周期任务仅在最终阶段返回奖励，导致智能体在中间决策过程中获得的学习信号极为稀疏[808]；此外，智能体行动时间越长，其采取冒险行动的可能性就越大。不可逆操作，因此对其执行安全性与可恢复性进行监控至关重要。

Why long-horizon agents need a new taxonomy. Recent works have begun to clarify parts of this landscape, but its taxonomies still exhibit two structural limitations that motivate a new one.

为何长时域智能体需要新的分类体系？近期的研究已开始阐明这一领域中的部分问题，但现有分类体系仍存在两个结构性局限性，这正是催生新分类体系的动因。

- **Limited and unsystematic coverage.** Existing papers typically examine a *single* technical component of long-horizon agents in isolation, such as memory [694], context engineering [425, 841], agent harnesses [468, 876], self-evolution [165, 709], or agentic reinforcement learning [808]. Each is a genuine piece of the long-horizon agent, but none offers a comprehensive account of what such an agent *is*, nor of how to build an efficient and trustworthy long-horizon agent that stays reliable across a full trajectory.
- **覆盖范围有限且缺乏系统性。**现有文献通常会孤立地研究长时程智能体的单一技术组件，例如记忆[694]、上下文工程[425, 841]以及智能体接口[468, 876]。自进化[165, 709]或代理强化学习[808]。这些研究各自都是对长时程智能体的独立探讨，但均未对这类智能体究竟是什么，亦未阐明如何构建一个高效且可信、并在整个运行轨迹中始终保持可靠性的长时程智能体，提供全面的阐述。
- **Unclear definitions and overlapping terminology.** As interest in capable agents grows, the concept itself has become broad and blurred. Related notions such as long-context, long-running, autonomous, and self-evolving agents are routinely used interchangeably, even though each emphasizes a different facet, and reported gains entangle changes in the model, the harness, the data pipeline, and the evaluation protocol. Most works cannot say precisely what counts as long-horizon agency or how these techniques relate, which leaves improvements hard to attribute and hard to transfer.
- **定义不明确且术语存在重叠。**随着对智能代理研究兴趣的日益增长，该概念本身已变得宽泛且模糊。诸如“长上下文”、“长运行”、“自主”及“自进化”等相关概念常被混用，尽管它们各自强调不同的方面；此外，所报告的性能提升往往涉及模型、控制机制及数据流水线的多重变化。以及评估方案。大多数研究无法明确界定何为长时程代理，亦无法阐明这些技术之间的关联关系，这使得改进成果难以归因，亦难以迁移。

These findings highlight the need for a unified taxonomy for long-horizon agent research.

这些发现凸显了为长时程智能体研究建立统一分类体系的必要性。

We therefore organize the long-horizon agent field in this paper through a *co-evolutionary* view (Figure 1). Our organizing principle is that long-horizon agency is realized through the co-evolution of **externalized harness engineering** and **internalized model optimization**, two complementary halves that together constitute a long-horizon agent. Concretely, **externalized harness engineering** supplies runtime structure such as loops and workflows, context and memory, tools, orchestration, hooks and middleware, and verification; **internalized model optimization** improves the agent policy through architecture, data, environments, fine-tuning, reinforcement learning, and self-evolution. Neither half is sufficient on its own; long-horizon agency emerges only through their co-evolution, as harness mechanisms are gradually internalized into the policy and a stronger policy in turn unlocks more advanced harnesses.

因此，本文从**共进化**视角（图1）出发，对长时程智能体领域进行了组织。我们的组织原则是：长周期代理机制通过**外部化控制工程**与**内部化模型优化**的协同演化来实现。两个互补的半部分，共同构成一个长时程智能体。具体而言，**外部化约束工程**可提供运行时结构（如循环与工作流）、上下文与内存、工具、编排机制、钩子与中间件以及验证功能；而**内部化模型优化**则指相关过程。通过架构设计、数据、环境设置、微调、强化学习及自我进化等手段，优化智能体策略。二者单独均不足以发挥作用；长周期代理机制唯有通过两者的协同演化方能形成——即，调控机制会逐步内化于政策之中，而更强大的政策又会进而解锁更先进的调控机制。

Building on this co-evolutionary view, this paper aims to reconcile the fragmented definitions, connect the emerging trends, and articulate the foundational principles of long-horizon agents. Concretely, we organize the discussion around the following key questions:

基于这一共进化视角，本文旨在弥合现有定义间的碎片化状态，衔接新兴趋势，并阐明长时程智能体的基础原则。具体而言，我们围绕以下关键问题组织讨论：

Key Questions

关键问题

- Q1. Foundation.** How should long-horizon agency be formalized, and how does it differ from long-running execution, autonomy, and self-evolution?

Q1. 基础概念。长周期机构应如何进行规范化构建？其与长期执行、自主性及自我进化有何区别？

Q2. Evolution. How did the control surface widen from prompts, to context, to runtime agent systems?

问题 2：演进。控制面是如何从指令驱动模式，发展到上下文驱动模式，最终演变为运行时智能体系统？

Q3. Harness. How does externalized harness engineering keep long-horizon agent execution specifiable, steerable, verifiable, and recoverable at runtime?

Q3. 捕获机制。外部化的捕获工程如何确保长时程智能体执行过程在运行时能够保持可定义性、可控性、可验证性和可恢复性？

Q4. Optimization. How can internalized model optimization build long-horizon competence across the agent’s architectural substrate, data and environment synthesis, and full training lifecycle?

Q4. 优化。内置模型优化如何在智能体的架构基底、数据与环境合成以及完整的训练生命周期中构建长周期能力？

Q5. Application. What forms do long-horizon agents take across domains, and how should their long-horizon capability be systematically evaluated?

Q5. 应用场景。长视野智能体在不同领域中呈现何种形式？其长视野能力应如何进行系统性评估？

Q6. Frontier. What open problems define the next stage as horizons grow?

Q6. 前沿领域。随着研究视野的拓展，定义下一阶段的关键开放性问题有哪些？

To address these questions, the paper follows this co-evolutionary decomposition through six connected perspectives. **Foundation** formalizes the agent and separates it from adjacent concepts, while **Evolution** traces how the locus of control widened from prompts to context to runtime systems. The two halves of the argument are then treated in turn: **Harness** studies how long-horizon capability is *externalized* into a runtime layer, and **Optimization** studies how it is *internalized* into the policy. Finally, **Application** shows how the same long-horizon pressures recur across domains and how such systems are evaluated, and **Frontier** identifies what remains open. Crucially, these two halves are not independent: capability continually migrates between them, so throughout we treat externalized harness engineering and internalized model optimization as coupled rather than as separate tracks.

为解答上述问题，本文通过六个相互关联的视角，对这一共进化分解过程进行了探讨。**基础层**将智能体进行规范化，并将其与相邻概念区分开来；而**演化层**则追溯了控制权如何从提示符扩展到上下文，最终延伸至运行时系统。随后，本文对这两个论证部分分别进行探讨：“**Harness**”研究了长时程能力如何被**外部化**为一个运行时层；而“**Optimization**”则研究了该能力是如何被**内部化**的。纳入政策框架。最后，“**应用**”部分展示了相同的长周期压力如何在不同领域中反复出现，以及此类系统是如何进行评估的；而“**前沿**”部分则指出了尚待解决的问题。至关重要的是，这两个部分并非相互独立：能力在二者之间持续迁移，因此，在整个过程中，我们将外部化的系杆工程与内部化的模型优化视为耦合关系，而非视为独立的两条路径。

Contributions and Roadmap. This paper makes the following contributions, each answering one of the key questions and mapping to one part of the paper. Figure 2 presents an overview of our taxonomy.

贡献与路线图。本文做出了以下贡献，每项贡献均针对其中一个问题并对应于论文的相应部分。图2展示了我们分类体系的概览。

1. **Foundation (Section 2).** We formalize long-horizon agency as a coupled model–harness decision process and organize its difficulty into three nested task levels (H1–H3) with matching capabilities (C1–C3): intra-context reasoning, cross-context memory, and cross-task experience. Building on METR’s data, we note that the frontier time horizon is scaling with an accelerating doubling time. We further separate long-horizon agency from long-running execution, autonomy, and self-evolution, giving the field a shared vocabulary for attributing where long-horizon competence comes from.

1. **基础（第2节）。**我们将长时程代理问题形式化为一个耦合的模型-决策过程，并将该问题的难度划分为三个嵌套的任务层级（H1–H3），以及相应的匹配能力（C1–C3）：即上下文内推理、跨上下文记忆与跨任务经验。基于METR的数据，我们注意到前沿时间范围正随加速的倍增时间而扩展。我们进一步将长时程代理机制与长期执行、自主性及自我进化区分开来，为该领域提供了一个共同的术语体系，以便于界定长时程能力的来源。

2. **Evolution (Section 3).** We trace how the control surface migrated from prompts, to context, to runtime systems, framing today’s long-horizon agents as the current endpoint of a capability-driven trajectory and clarifying why control first moved outward before it could be internalized.

2. **演化（第3节）。**我们追溯了控制权如何从提示符、到上下文，再到运行时系统进行迁移——从而将当今的长视野智能体界定为能力驱动演进路径的当前终点，并阐明了为何控制权在能够实现之前，首先会向外部转移。内化。

3. **Harness (Section 4).** We organize externalized harness engineering by component: loops and workflows, context and memory, tools, MCP, and skills, orchestration and multi-agent control, hooks and middleware, and verification. Together these components show how a runtime layer keeps long execution specifiable, steerable, verifiable, and recoverable.

3. **车辆安全带（第4节）。**我们按组件对外部化约束工程进行划分：循环与 workflows、上下文与内存、工具、MCP与技能、编排与多智能体控制、钩子与中间件，以及验证。这些组件共同展示了运行时层如何确保长时间执行过程能够保持可指定、可操控、可验证及可恢复的特性。

4. **Optimization (Section 5).** We organize internalized model optimization along the pipeline, from the architectural substrate and data and environment synthesis to agentic pre/mid-training, fine-tuning, reinforcement learning with long-horizon credit assignment, on-policy distillation, and self-evolution, tracing how capabilities first scaffolded through externalized harness engineering are progressively internalized into the policy.

4. **优化（第5节）。**我们沿整个流程组织内部模型优化过程，涵盖从架构基础、数据与环境合成到代理式预训练/中期训练、微调，乃至采用长时程信用分配机制的强化学习等环节。在线策略蒸馏与自我进化机制，可追溯出：通过外部化控制工程初步构建的能力，如何逐步内化为策略的过程。

5. **Application (Section 6).** Organized by the agent–environment interface, we map long-horizon pressures across software engineering, information seeking, computer use, multimodal agents, and general-purpose agents, revealing a common long-horizon core beneath surface-level domain differences, and consolidate horizon-aware evaluation resources.

5. **申请（第6节）。**通过代理与环境接口的组织方式，我们对软件工程、信息寻求、计算机使用、多模态代理及通用代理领域中的长周期性压力因素进行了映射，揭示了存在于表层领域之下的共同长周期性核心。差异分析，并整合时域感知评估资源。

6. **Frontier (Section 7).** We organize open problems into four axes, namely evolution, effectiveness, efficiency, and trustworthiness, spanning nine concrete directions that range from self-evolving and transferable harnesses, to real-world environments and error robustness, to cost- and modality-aware efficiency, and to safety, governance, and embodiment, marking what must be solved as horizons keep growing.

6. 《边境法》（第7节）。我们将开放性问题划分为四个维度——即演化、有效性、效率与可信度，涵盖九个具体方向，范围从自进化及可迁移的调控装置，到现实应用场景与误差鲁棒性。兼顾成本与技术模式方面的效率，以及安全性、治理与实体化问题——随着发展边界不断拓展，这些议题亟待解决。

Taken together, these contributions serve three purposes: a *clear definition* of long-horizon agency; a *high-quality taxonomy* (Figure 2) built around the paper’s six perspectives; and a *roadmap* sketching the directions toward capable long-horizon agents. Beyond organizing existing work, this co-evolutionary view also offers a vantage point for reasoning about how the balance between externalized harness and internalized optimization may shift as horizons grow, and thus for anticipating the frontier forms that long-horizon agents may take.

综上所述，这些贡献可实现三个目标：一是对长周期代理关系给出*清晰的定义*；二是基于本文的六个视角构建一个*高质量分类体系*（图2）；三是提供一份*路线图*。勾勒出面向高能力长视野智能体的发展方向。除组织现有工作外，这种共进化视角还为探讨随着视野的拓展，外部化控制与内部化优化之间的平衡可能如何发生转变提供了一个独特的视角。因此，可用于预测长时程智能体可能采取的前沿形态。

Scope and Boundaries. We delimit our scope along four boundaries. **(1)** This is not a general investigation of LLM agents, memory, or reinforcement learning: we treat memory, reinforcement learning, and self-evolution through the co-evolution of externalized harness engineering and internalized model optimization, and only as they bear on long-horizon competence. **(2)** We exclude general ML platform infrastructure (e.g., container orchestration, GPU-cluster scheduling), and consider inference and decoding only where they materially affect long-trajectory execution. **(3)** We address model-architecture internals only for their effect on horizon, i.e., making long trajectories representable, trainable, and affordable to decode, while still treating reasoning and long-context modeling as foundational capabilities. **(4)** We draw on official documentation, open-source implementations, and emerging specifications alongside indexed academic publications, prioritizing documented engineering practice.

范围与边界。 我们根据四个边界来界定我们的研究范围。**(1)** 本研究并非对大型语言模型（LLM）智能体、记忆或强化学习进行广义探讨：我们通过外部化系统工程与内部化模型之间的协同演化，来探讨记忆、强化学习及自我进化问题。优化，且仅适用于与长周期能力相关的场景。**(2)** 我们排除通用机器学习平台基础设施（例如）。（如容器编排、GPU集群调度等），并仅在推理与解码环节对长轨迹执行产生实质性影响时才予以考虑。**(3)** 我们仅针对模型架构内部因素对其“视距”产生的影响进行分析，即：该方法可使长轨迹得以表示、可进行训练且易于解码，同时仍将推理与长上下文建模视为其基础能力。**(4)** 我们参考了官方文档、开源实现及新兴规范，同时亦参考了索引化的学术出版物，并优先采用有文献记载的工程实践。

2 Foundation: Formalizing Long-Horizon Agents

2 基础：长时程智能体的形式化建模

Overview. Section 1 positioned long-horizon agency as a system-level capability, jointly shaped by the internalized model optimization and the externalized harness engineering that surrounds it. This section makes that claim precise and lays the conceptual groundwork for the rest of the paper, organized around four questions that follow the section’s structure:

概述。 第1节将长视距机构定位为一种系统级能力，该能力由内部的模型优化与围绕其展开的外部的总成工程共同塑造。本节对这一主张进行了精确阐述，并为论文后续部分奠定了概念基础；后文将围绕本节的结构所对应的四个问题展开论述：

- **What is a long-horizon agent? (Section 2.1)** We define it by the coupled logical dependencies of a task rather than by elapsed time, and cast the agent as a base policy coupled to a surrounding harness, so long-horizon agency becomes a property of the model-harness system rather than of the foundation model alone.
- **什么是长时程智能体？（第2.1节）** 我们通过任务的耦合逻辑依赖关系而非经过的时间来定义它，并将智能体表述为与周边控制框架耦合的基础策略；因此，长时域智能体特性便成为模型-控制框架系统本身的属性，而非仅属于该智能体本身。仅使用基础模型。
- **How does its difficulty decompose? (Section 2.2)** We use how far execution stretches beyond one working context as an observable proxy for the underlying dependency burden, giving three nested levels (H1–H3) and their capabilities (C1–C3).
- **其难度如何分解？（第2.2节）** 我们采用执行过程超出单一工作上下文的范围作为衡量底层依赖负担的可观测代理指标，由此构建出三个嵌套层级（H1–H3）及其对应能力（C1–C3）。
- **How fast is the frontier moving? (Section 2.3)** We quantify the rising time horizon of frontier agents as an empirical scaling trend and compare its full-period and recent-window doubling times.
- **前沿的移动速度有多快？（第2.3节）** 我们通过实证尺度趋势来量化前沿主体的上升时间范围，并对比其全周期与近期窗口的倍增时间。
- **How does it differ from neighboring notions? (Section 2.4)** We separate long-horizon agency from long-running execution, autonomy, and self-evolution, showing that it integrates these rather than reducing to any one.
- **它与邻近概念有何区别？（第2.4节）** 我们将长时程代理与长期执行、自主性及自我演化区分开来，论证了长时程代理能够整合这些要素，而非将其简化为其中任何一种。

2.1 Definition

2.1 定义

We begin with the two objects of study: the task and the agent that seeks to complete it. As introduced in Section 1, we characterize long-horizon agency not by elapsed time or token count, but by the *coupled logical dependencies* that distribute a task across many interdependent steps. It is this chain of decisions, together with the interaction it entails, rather than duration alone, that determines the length of the horizon.

我们首先从两个研究对象入手：任务与旨在完成该任务的智能体。如第1节所述，我们并非通过已流逝的时间或令牌计数来刻画长时程代理机制，而是通过*耦合的逻辑依赖关系*——这些依赖关系将任务分布至多个相互关联的步骤之中。决定视野范围长度的并非仅仅是时间跨度，而是这一决策链及其所引发的相互作用。

Long-horizon Tasks. A **long-horizon task** is a goal that may be under-specified at the outset and may require progressive clarification during solving, and whose solution path admits many possibilities and typically couples several complex subtasks together. As a result, it cannot be answered by a single-turn response in isolation, but only by *composing many interdependent decisions* into a single coherent trajectory [286, 811].

Concretely, a long-horizon task requires the agent to reason over many steps while interacting with the environment and, where relevant, the user: it progressively narrows down the solution path and continually tests and revises its reasoning direction until the goal is reached. Such a task may run from minutes to days

长周期任务。 长周期任务是指其目标在初始阶段可能不够明确、在求解过程中可能需要逐步澄清，且其求解路径存在多种可能性，并通常将多个复杂的子任务紧密耦合在一起的任务。因此，该问题无法通过单一回合的响应单独回答，而必须将*诸多相互依赖的决策*整合为一条连贯的轨迹[286, 811]。具体而言，长时程任务要求智能体在与环境交互的过程中，对多个时间步进行推理，并在相关情况下予以处理。用户：系统会逐步缩小解决方案的搜索范围，并持续对推理方向进行测试与调整，直至达成目标。此类任务的运行时间可能从几分钟到数天不等。

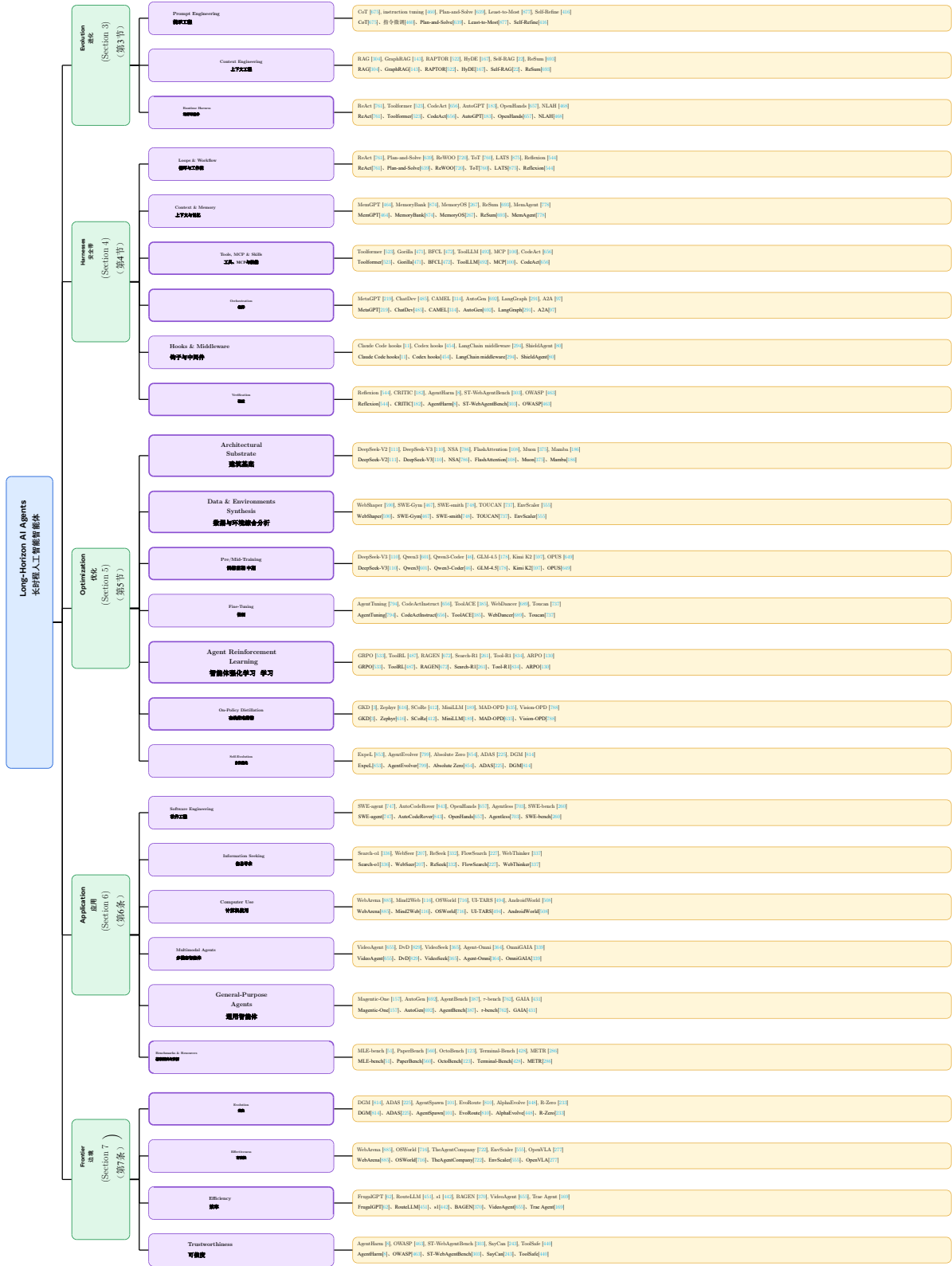


Figure 2 A taxonomy of long-horizon agent research. The paper is structured according to this taxonomy, progressing from **Evolution** (Section 3) through **Harnesses** (Section 4), **Optimization** (Section 5), and **Applications** (Section 6), to open **Frontiers** (Section 7). Each leaf lists representative works or systems discussed in the corresponding part of the paper. and may fit within one context window or stretch far beyond it; its defining feature is not duration itself but this sustained, feedback-driven dependence among steps and the continual interaction required to manage it.

图2：长时程智能体研究的分类体系。 本文的结构遵循该分类体系，从**演化**（第3节）逐步推进至**约束条件**（第4节），以及**优化部分**。（第5节）、以及**应用部分**（第6节），关于**前沿领域**（第7节）。每一页列出了在该论文相应章节中讨论的代表性作品或系统。它可能仅适用于一个特定的语境窗口，也可能远超出该窗口范围；其定义特征并非时长本身，而是步骤之间这种持续的、由反馈驱动的依存关系，以及为管理这种关系而所需的持续互动。

Long-horizon Agents. A **long-horizon agent** is a system designed to solve such tasks in an environment with state, feedback, and side effects [640]. Although the stated intent may be simple, successful execution requires resolving a large and tightly coupled chain of logical dependencies. Unlike a single-turn responder, it does not produce one answer and stop; instead, it must sustain reasoning through repeated cycles of planning, action, observation, and revision, progressively clarifying an under-specified goal as the trajectory unfolds. As argued in Section 1, this competence is not a property of the foundation model alone. It arises from the joint design of an **externalized harness engineering**, which supplies runtime structure such as tools, memory, orchestration, and checks, and an **internalized model optimization** process, which improves the policy through data, training, and experience.

长时程智能体。 **长时程智能体**是指一种旨在具有状态、反馈及副作用的环境中解决此类任务的系统[640]。尽管声明的意图可能很简单，但要成功实现，则需解决一个庞大且紧密耦合的逻辑依赖链。与单轮响应器不同，它不会生成单一答案后即停止；相反，它必须通过反复的规划、行动、观察及修正循环来持续进行推理。随着过程的推进，对未明确的目标会逐步予以澄清。如第1节所述，这种能力并非仅由基础模型所具备的属性。该机制源于**外部化硬件工程**与**内部化模型优化**的联合设计——前者提供运行时所需的结构支持（如工具、内存、调度及校验功能），后者则负责模型的内部优化。该流程通过数据、训练和经验来优化策略。

Formalization: Long-Horizon Agent

形式化：长时程智能体

We model a long-horizon agent as a foundation-model *base policy* π_θ coupled to a surrounding structure $\mathcal{H}$ that closes the loop around it:

我们将长时程智能体建模为一个基础模型*基础策略* π_θ ，该策略与周围结构 $\mathcal{H}$ 相耦合，从而闭合其周围的控制环路：

$$\text{Agent} = \pi_\theta \oplus \mathcal{H} \quad (1)$$

$$\text{代理} = \pi_\theta \oplus \mathcal{H} \quad (1)$$

Here π_θ proposes actions, while $\mathcal{H}$ comprises the loops and workflows, system prompts, tools and skills, context and memory, orchestration logic, hooks, and verification mechanisms that govern *when* the model is invoked, *how* its actions are validated and executed, and *what* state persists across steps. The operator $\oplus$ denotes runtime coupling rather than simple addition: $\mathcal{H}$ closes the action–observation–memory loop around π_θ , so long-horizon agency is a property of the coupled system and its training, not of π_θ alone. Equation (1) describes the *deployed* coupling that produces long-horizon behavior at run time; how it *evolves* over time, as π_θ internalizes competence through training and the system accumulates experience, is the co-evolving recipe we develop in Section 5 and Section 3. We refer to these two halves as **Pillar I**, the externalized harness $\mathcal{H}$ (Section 4), and **Pillar II**, the internalized optimization of π_θ (Section 5).

此处 π_θ 负责提出操作，而 $\mathcal{H}$ 则包含循环与 workflow、系统提示、工具与技能、上下文与记忆、编排逻辑、钩子以及控制*何时*执行的验证机制。调用该模型时，*其*操作如何得到验证并执行，以及*哪些*状态会在不同步骤间持续存在。运算符 $\oplus$ 表示运行时耦合而非简单的加法： $\mathcal{H}$ 将作用-观测-记忆环路围绕 π_θ 完成闭环。因此，长时程代理特性是耦合系统及其训练过程的属性，而非仅由 π_θ 决定。公式(1)描述了*部署的*耦合机制，该耦合机制在运行时会产生长时程行为；以及该耦合机制如何随时间*演变*，即随着 π_θ 的变化而演变。通过培训实现能力内化，且系统能够积累经验——这正是我们在第5节和第3节中所阐述的协同进化机制。我们将这两部分分别称为**Pillar I**——外部化约束 $\mathcal{H}$ （第4节），以及**Pillar II**—— π_θ 的内部化优化（第5节）。）。

The Long-horizon Agent–Environment Loop. Since a long-horizon agent makes progress precisely by interacting with its environment step after step, its execution is most naturally described as a continual loop in which it acts, observes the consequences, and updates its state before acting again. Following the standard LLM-based agent formulation [228, 640, 761], we place the coupled system in an environment. Let $\mathcal{S}$ be an environment state space, $\mathcal{O}$ an observation space, and $\mathcal{A}$ an action space. The environment evolves by a transition kernel $s_{t+1} \sim \Psi(\cdot \mid s_t, a_t)$ and emits observations through an observation kernel $o_t \sim Z(\cdot \mid s_t)$, which is partial and in general stochastic. We distinguish the base policy π_θ from the *agent policy* $\pi = \pi_\theta \oplus \mathcal{H}$ that the harness closes around it: at each step the harness assembles a contextual signal c_t and the base policy acts on it,

长时程智能体–环境循环。 由于长时程智能体正是通过逐步与环境进行交互来取得进展的，其执行过程最自然地可描述为一个持续循环：该智能体先采取行动，再观察其产生的结果。在再次执行操作前，系统会更新其状态。遵循标准的基于大语言模型的智能体建模方法[228, 640, 761]，我们将该耦合系统置于一个环境中。设 $\mathcal{S}$ 为环境状态空间， $\mathcal{O}$ 为观测空间， $\mathcal{A}$ 为动作空间。环境通过转移核 $s_{t+1} \sim \Psi(\cdot \mid s_t, a_t)$ 进行演化，并通过观测核 $o_t \sim Z(\cdot \mid s_t)$ 发生观测值。 $(\cdot \mid s_t)$ ，该函数是部分的且通常具有随机性。我们区分了基础策略 π_θ 与*智能体策略* $\pi = \pi_\theta \oplus \mathcal{H}$ ——即套索围绕该策略进行闭合：在每一步中，套索会生成一个上下文信号 c_t 。而基础政策会对其产生影响。

$$c_t = \mathcal{H}(o_{0:t}, a_{0:t-1}, \mathcal{Q}), \quad a_t \sim \pi_\theta(\cdot \mid c_t),$$

$$c_t = \mathcal{H}(o_{0:t}, a_{0:t-1}, \mathcal{Q}), \quad a_t \sim \pi_\theta(\cdot \mid c_t),$$

where $\mathcal{Q}$ is the task specification and c_t is the contextual signal that the harness assembles at step t from the interaction history: the prompt window with retrieved memories, prior tool outputs, reasoning traces, and other state. Execution induces a trajectory $\tau = (s_0, o_0, a_0, s_1, o_1, a_1, \dots, s_T)$ that runs until termination, i.e. until the first step $T = \min\{t : s_t \in \mathcal{S}_{\text{term}}\}$ at which the state enters a terminal set $\mathcal{S}_{\text{term}}$ (the goal is reached, a budget is exhausted, or the agent halts). Because o_t is only a partial view of s_t and c_t is a lossy summary of history, the coupled system is naturally modeled, following standard POMDP formulations for sequential decision making under partial observability [264], as a partially observable Markov decision process (POMDP). Much of what $\mathcal{H}$ does is the machinery of acting under partial observability. The task assigns each trajectory a utility $R(\tau) \in \mathbb{R}$, typically sparse and delayed and often revealed only at T , so that credit for the intermediate decisions that actually drive success or failure must be inferred from a single terminal signal. This is the long-horizon credit-assignment problem that is central to reinforcement learning methods [579].

其中， $\mathcal{Q}$ 表示任务规范，而 c_t 表示总成线在第 t 步时采集的上下文信号。基于交互历史：提示窗口中包含检索到的记忆、先前的工具输出、推理轨迹及其他状态。执行过程会生成一条轨迹 $\tau = (s_0, o_0, a_0, s_1, o_1, a_1, \dots, s_T)$ 该进程将持续运行直至终止，即：直至第一步 $T = \min\{t : s_t \in \mathcal{S}_{\text{term}}\}$ ，此时状态进入终端集 $\mathcal{S}_{\text{term}}$ （即目标已达成或预算已耗尽）；或代理终止）。由于 o_t 仅是 s_t 的局部视图，而 c_t 则是历史数据的有损摘要，因此耦合系统可自然地采用标准POMDP框架进行建模，以处理部分信息下的序列决策问题。可观测性[264]可视为一种部分可观测马尔可夫决策过程（POMDP）。 $\mathcal{H}$ 的大部分功能均涉及在部分可观测条件下执行动作的机制。该任务为每条轨迹分配一个效用值 $R(\tau) \in \mathbb{R}$ ，该值通常为稀疏且存在延迟，且往往仅在 T 时刻才被揭示。因此，对实际决定成败的中间决策所赋予的贡献度，必须从单一的终端信号中推断出来。这是长期远期信用分配问题，也是强化学习方法的核心问题[579]。

Three Levels of Long Horizon Tasks & Capabilities

H1 ⊂ H2 ⊂ H3 | C1 ⊂ C2 ⊂ C3

三个层级的长程任务与能力: H1 H2 H3 C1 ⊂ C2 ⊂ C3

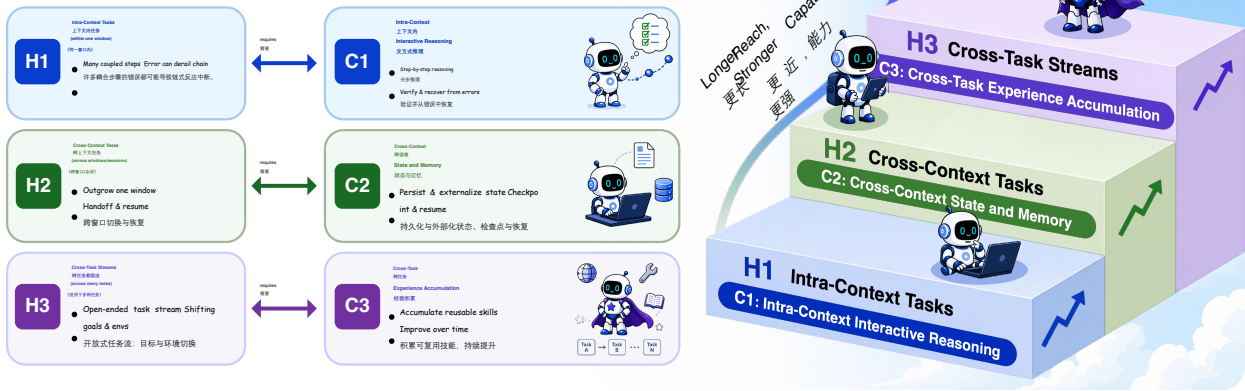


Figure 3 Three levels of long-horizon tasks and the capabilities they demand. As the required horizon widens from intra-context tasks (H1), to cross-context tasks (H2), to cross-task streams (H3), the agent must add intra-context interactive reasoning (C1), cross-context state and memory (C2), and cross-task experience accumulation (C3) capabilities.

图3：三级长周期任务及其所需能力。随着所需处理范围从内部上下文任务（H1）扩展到跨上下文任务（H2），再到跨任务流（H3），智能体必须引入内部上下文交互推理（C1）以及跨上下文状态与记忆机制（C2）。以及跨任务经验积累（C3）能力。

2.2 Three Levels of Long-Horizon Tasks and Capabilities

2.2 长周期任务与能力的三级划分

Although the notion of a *long-horizon* agent is by now widely used, it remains conceptually vague, and prior work still lacks a principled way to partition long-horizon competence into distinct levels. Building on the definition above, we organize long-horizon tasks by an operational axis: *how far execution must stretch beyond a single working context*, ranging from intra-context tasks completable within one window (H1), to cross-context tasks that span windows or sessions (H2), to cross-task streams in which an overarching goal unfolds over many episodes (H3). This axis is a proxy for, rather than the definition of, logical dependency: a context boundary matters when coupled decisions and task state must survive it, whereas a long but decomposable batch process need not be long-horizon. The proxy is useful because no step is perfectly reliable and errors compound along the dependency graph, so trajectory-level reliability rather than per-step accuracy governs success [286]. It yields three nested operational levels of increasing reach [14, 228], and we pair each level with the capability an agent needs to succeed at it (Figure 3). Because every level adds a new failure mode on top of the one below, the corresponding demands accumulate. The time ranges below are therefore descriptive of current systems, not definitional thresholds.

尽管长时程智能体的概念目前已得到广泛应用，但其概念仍较为模糊，且现有研究尚未建立一种基于原理的、能够将长时程能力划分为不同层级的方法。基于上述定义，我们根据一个操作维度对长周期任务进行分类：*执行范围需延伸至多远才能跨越单一工作上下文* 这些任务涵盖范围广泛：从可在单个时间窗口内完成的同语境任务（H1），到跨越多个时间窗口或会话的跨语境任务（H2），再到在多个场景中逐步展开总体目标的跨任务流（H3）。该轴线是逻辑依赖关系的代理指标，而非其定义：当耦合决策与任务状态必须跨越该上下文边界时，该上下文边界便具有重要意义；而一个长度较长但可分解的批处理流程则无需具备长时域特性。代理方法之所以有用，是因为没有任何一步是完全可靠的，且误差会在依赖图中累积；因此，决定成功的关键在于轨迹级可靠性而非单步精度[286]。该框架可生成三个层级递进的操作层级，其覆盖范围依次递增[14, 228]；我们将每个层级与智能体在该层级上取得成功所需的相应能力进行配对（图3）。由于每个层级都在下一层级的失效模式之上新增一种新的失效模式，相应的要求便会不断累积；因此，下方的时间范围仅用于描述当前系统状态，而非定义性阈值。

Three Levels of Long-Horizon Tasks (H1 < H2 < H3)

三级长程任务 (H1 H2 H3)

H1 — Intra-context, within one window (~minutes).

H1 — 上下文内，同一窗口内（ 分钟）。

A single task can be completed within one working context window, yet it is far from a one-shot query: it requires composing many interdependent decisions into one coherent trajectory, with repeated interaction with the environment and, where relevant, the user to test and revise the reasoning direction until the goal is reached [286, 544].

单个任务可在一个工作上下文窗口内完成，但这远非“一次性查询”：它需要将诸多相互依赖的决策整合为一条连贯的行动轨迹，并需与环境进行多次交互——在必要时亦需如此。用户可对推理方向进行测试与修订，直至达成目标[286, 544]。

H2 — Cross-context, across windows/sessions (~hours to days).

H2 — 跨上下文、跨窗口/会话（ 小时至数天）。

A single task far exceeds one context window or session and must be carried across multiple context windows or by several agents working in concert. The agent(s) must therefore actively compress and externalize historical context, track interaction state in real time, and resume progress without assuming that the full history remains visible at every step [14, 378, 511].

单一任务往往远超单个上下文窗口或会话范围，必须跨越多个上下文窗口，或由多个协同工作的智能体共同完成。因此，代理必须主动对历史上下文进行压缩与外部化处理，实时追踪交互状态，并在恢复进度时无需假设完整的历史记录在每一步骤下均可见[14, 378, 511]。

H3 — Cross-task, open-ended task stream (~lifelong).

H3 — 跨任务、开放式任务流（ 终身）。

Tasks arrive as an open-ended stream rather than a closed episode. A single overarching goal may therefore span many tasks, each of which may itself require multiple context windows to complete. In deployment, such settings often demand persistent, and sometimes lifelong, operation across heterogeneous environments, tools, and sessions [165, 228, 580, 808].

任务以开放式的连续流形式出现，而非作为封闭的独立事件。因此，一个总体目标可能涵盖多项任务；而每项任务本身又可能需要多个上下文窗口才能完成。在部署过程中，此类设置通常要求系统能够在异构环境、不同工具及会话之间实现持久运行，有时甚至需支持终身运行[165, 228, 580, 808]。

So far we have organized long-horizon *tasks* into three difficulty levels; we now turn from tasks to *agents* and identify, for each level, the capability that its difficulty demands. Each level exposes a distinct insufficiency. H1 shows that per-step accuracy alone is insufficient: the agent must sustain reasoning through continual interaction rather than produce a single response. H2 shows that a larger context window is insufficient: once a task spans multiple windows or agents, state must be compressed, externalized, and tracked even when the full history is not continuously in view. H3 shows that episodic success is insufficient: an agent that completes an individual task may still fail to accumulate reusable competence across an open-ended stream of subsequent tasks. Each level therefore demands a capability of matching reach, denoted C1–C3 respectively, and these capabilities compose cumulatively: C2 presupposes C1, and C3 presupposes both.

到目前为止，我们将长时程任务划分为三个难度等级；现在，我们将视角从任务转向智能体，并为每个等级确定其所需具备的能力。每个层级都揭示了不同的不足之处。H1表明，仅依赖单步准确率是不够的：智能体必须通过持续交互来维持推理过程，而非仅产生单一响应。图H2表明，采用较大的上下文窗口是不够的：当任务涉及多个窗口或多个智能体时，即使完整的历史记录并非始终处于视野范围内，状态仍需进行压缩、外化及追踪。H3表明，单次成功尚不充分：即使智能体完成了某个特定任务，它仍可能无法在后续一系列开放式的任务流中积累可复用的能力。因此，每个层级均需具备相匹配的可达性能力（分别记为C1–C3），这些能力可进行累积组合：C2预设了C1，而C3则同时预设了C1与C2。

Capabilities Demanded by Each Level (C1 < C2 < C3)

各层级所需能力（C1 C2 C3）

C1 — Intra-context interactive reasoning (for H1).
C1 — 上下文内交互推理（针对H1）。
Within a single context, sustain continuous reasoning through interaction with the environment and the user: take actions, absorb feedback, and revise the trajectory so that many interdependent decisions compose into one coherent path toward the goal [286, 544].
在单一语境下，通过与环境及用户的交互实现持续推理：执行操作、吸收反馈。并修正路径，使得诸多相互依赖的决策能够整合为一条指向目标的连贯路径[286, 544]。

C2 — Cross-context state and memory (for H2).
C2 —— 跨上下文状态与记忆（适用于H2）。
Building on C1, maintain task state when execution spans windows or agents: compress and externalize historical context, read and write memory, checkpoint progress, and faithfully resume after hand-offs [14, 378, 511, 694].
基于C1，当任务执行跨越多个窗口或代理时，需保持任务状态：压缩并外部化历史上下文、读写内存、记录进度，并在任务转移后准确恢复[14, 378, 511, 694]。

C3 — Cross-task experience accumulation (for H3).
C3——跨任务经验积累（适用于H3）。
Building on C2, turn experience into lasting competence across an open-ended task stream: switch among complex tasks, each of which may itself span multiple context windows, accumulate reusable skills [630], generalize from past experience [462], and continually improve as goals, environments, and data distributions shift [165, 228, 580].
基于C2框架，将经验转化为贯穿开放式任务流的持久能力：在各类复杂任务之间进行切换——每项任务本身可能跨越多个上下文窗口，并在此过程中积累可复用技能[630]。基于过往经验进行泛化[462]，并在目标、环境及数据分布发生变化时持续改进[165, 228, 580]。

Because the levels nest, so do the capabilities, in that an agent operating at H3 must still satisfy C1 and C2 within every task it touches. This is where the paper’s central question enters. Each capability can be supplied either from *outside* the model, through the harness, or from *inside* it, through training. H1 pressures are often met by C1 within a single context, or absorbed into the model weights; H2 pressures push C2 out of the window into durable external state, namely compressed context, memory, and checkpoints; and H3 pressures demand C3, which turns deployment experience into lasting competence. In many H3 settings, however, neither route is likely to suffice on its own, and the highest level of long-horizon competence is therefore inherently a *system-level* property, jointly constructed by internalized model weights and the externalized harness rather than achievable by either in isolation (Figure 1). We attach each method to the level it primarily serves.

由于层级结构是嵌套的，其能力亦然——即，在H3层级下运行的智能体，在执行任何任务时，仍须满足C1与C2。这正是本文核心问题所在。每项能力均可通过以下方式提供：来自模型外部（经由总线），或来自模型内部（经由训练）。H1压力通常由C1在单一上下文中处理，或被吸收至模型权重中；而H2压力则会将C2从窗口中推离，转而进入持久的外部状态——即压缩上下文、记忆及检查点；H3压力要求具备C3能力，这能将部署经验转化为持久的胜任力。然而，在许多 H3 情境下，单独采用任一途径都可能难以充分应对；因此，最高水平的长周期能力本质上属于系统级能力。该特性是由内化模型权重与外化约束共同构建的，而非任一因素单独作用所能实现（图1）。我们将每种方法对应到其主要适用的层面。

2.3 Scaling the Time Horizon of Frontier-Agent Execution

2.3 扩展前沿智能体执行的时间范围

In the previous subsection, we organized long-horizon difficulty and its corresponding capabilities into a stair-stepped ladder *qualitatively*; here, we quantify how an observable proxy for the frontier scales over time. We build on the measurements of METR [286], which define the *50%-task-completion time horizon* as the human-expert completion time for the class of agentic engineering tasks that a given agent is predicted to complete with 50% reliability. These tasks span software engineering, machine learning engineering, and computer use, providing a more objective proxy for task difficulty than task labels alone.

在前一小节中，我们将长周期难度及其对应能力整理为一个阶梯状梯子定性地呈现；而在本节中，我们将量化用于衡量前沿的可观测代理指标如何随时间变化。我们基于 METR[286]的测量数据构建模型，该指标定义了50% 任务完成时间范围。指对于特定智能体预计以50%可靠性完成的代理型工程任务类别，人类专家所需的完成时间。这些任务涵盖软件工程、机器学习工程及计算机应用领域，相较于仅依赖任务标签，能为衡量任务难度提供更为客观的替代指标。

Figure 4 plots METR’s live estimates through May 8, 2026 [286, 429]¹. Because several early models were not re-evaluated under the newer TH1.1 methodology, the full historical series combines TH1.0 estimates for those models with TH1.1 estimates for later models. Under this stitched series, the time horizon of frontier agents rises from seconds for early models (e.g. GPT-2) to approximately twelve hours for Claude Opus 4.6 and at least sixteen hours for an early Claude Mythos Preview by May 2026, spanning nearly four orders of magnitude within only a few years.

图4展示了METR截至2026年5月8日的实时估算值[286, 429]¹。由于部分早期模型未在新的TH1.1方法论下进行重新评估，因此完整的历史序列整合了TH1数据。对于包含TH1.1的模型，其估计值为0；而对于后续模型，则采用TH1.1的估计值。在这一拼接序列下，前沿智能体的时间范围从早期模型（如GPT-2）的数秒，逐步扩展至Claude Opus 4的约12小时。6，且至少需16小时即可参与2026年5月前的“Claude Mythos”抢先预览活动——该活动在短短数年内便实现了近四个数量级的跨越。

¹METR live dashboard, <https://metr.org/time-horizons/>, accessed May 8, 2026.

¹METR实时仪表板, <https://metr.org/time-horizons/>, 访问日期: 2026年5月8日。

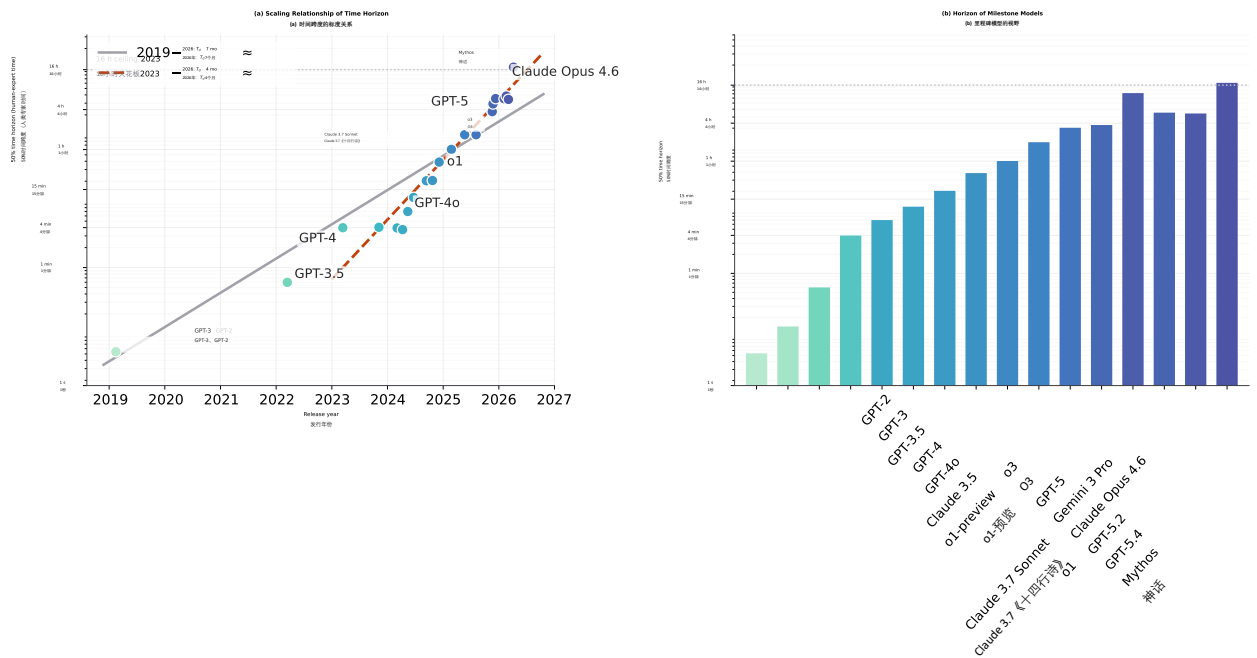


Figure 4 Empirical growth of frontier-agent time horizons. Frontier agents’ 50%-task-completion time horizons over model release dates, shown for the full fitted series in **(a)** and selected milestone agents in **(b)**.

图4：前沿主体时间跨度的实证增长趋势。前沿代理在模型发布日期后的50%任务完成时间范围，分别展示于（a）中的完整拟合序列及（b）中选定的关键节点代理数据中。

The fitted trend in Figure 4 summarizes this rapid expansion. Across the stitched full-period series, the estimated doubling time is approximately 196.5 days, or 6.5 months. Restricting the fit to models released from 2023 onward under TH1.1 yields a shorter doubling time of approximately 130.8 days, or 4.3 months [286, 429, 683]. The dotted line marks the approximately sixteen-hour region above which METR considers estimates unreliable under the current task suite. Consequently, both individual estimates and fitted trends near or beyond this boundary carry substantial uncertainty.

图4中的拟合趋势曲线概括了这一快速扩张过程。在拼接后的完整周期序列中，估计的倍增时间约为196.5天，即6.5个月。将模型拟合范围限定于2023年及之后发布、基于TH1.1版本的模型时，所得出的倍增时间较短，约为130.8天或4.3个月[286, 429, 683]。虚线标出了约16小时的时间区间；在该区间之上，METR认为基于当前任务集所得到的估算结果不可靠。因此，无论是个体估计值还是位于该边界附近或超出该边界的拟合趋势，均存在显著的不确定性。

To summarize the trend formally, we borrow the functional spirit of the compute-optimal scaling law of Hoffmann et al. [217] and model the frontier horizon in terms of two quantities: the release date of a frontier agent and the human-expert duration of tasks that the evaluated system can complete. We write this relationship as a compact descriptive summary:

为正式总结这一趋势，我们借鉴了Hoffmann等人提出的计算最优标度律的函数化思路。[217] 并可基于两个量来建模前沿边界：前沿智能体的发布日期，以及被评估系统能够完成的任务所需的人工专家耗时。我们将这一关系表述为一个简洁的描述性公式：

$$\log_2 H(t); \approx; \log_2 H_0 + \frac{t - t_0}{T_d}, \quad \text{equivalently} \quad H(t); \approx; H_0 \cdot 2^{(t-t_0)/T_d}, \quad (2)$$

$$\log_2 H(t); \approx; \log_2 H_0 + \frac{t - t_0}{T_d}, \quad \text{等价于} H(t); \approx; H_0 \cdot 2^{(t-t_0)/T_d}, \quad (2)$$

where $H(t)$ is the estimated time horizon of a frontier agent whose base model was released at date t , t_0 is a reference date, $H_0 = H(t_0)$ is the fitted horizon at that date, and T_d is the *doubling time*. Under this functional form, the full-period and 2023-onward fits correspond to $T_d \approx 196.5$ days and $T_d \approx 130.8$ days, respectively. Although these estimates are derived from self-contained, automatically scorable software-style tasks and become less precise near the upper boundary of the benchmark, these limitations primarily constrain the exact fitted values rather than the overall direction of the trend.

其中， $H(t)$ 表示一个前沿智能体的估计时间范围，该智能体的基础模型于日期 t 发布； t_0 为参考日期； $H_0 = H(t_0)$ 为该日期的拟合预测期，而 T_d 为倍增时间。在此函数形式下，全周期及2023年及以后的拟合结果分别对应于 $T_d \approx 196.5$ 天与 $T_d \approx 130.8$ 天。尽管这些估计值源自独立的、可自动评分的软件式任务，且在基准测试的上边界附近其精确度有所下降，但这些局限性主要制约的是精确拟合值，而非其他方面。趋势的总体方向。

The decisive signal is that the task duration frontier agents can reliably handle has increased approximately exponentially. The shorter doubling time obtained for the 2023-onward subset further suggests that this expansion may have accelerated, although differences in evaluation coverage and methodology prevent treating the two fitted values as definitive evidence of acceleration. The trend nevertheless aligns directly with our capability hierarchy: as completable task horizons expand from minutes to hours and then to tens of hours, frontier agents move beyond intra-context H1 tasks, increasingly address H2 tasks that require cross-context state and memory, and begin to approach the boundary of cross-task, open-ended H3. In this sense, the sustained expansion of the measured time horizon is an observable projection of long-horizon competence climbing the H1→H2→H3 ladder. At the current pace, many multi-hour tasks that remain at the frontier today may become routine in the foreseeable future. That said, the trend characterizes *progress over time*; it does not explain its *source*. In particular, it does not distinguish whether the gains arise from a stronger policy π_θ , a better harness $\mathcal{H}$, or the co-evolution of the two. Attributing observed progress among these factors is the central question of this paper. We therefore turn from measuring the frontier to asking what drives it.

决定性信号在于：前沿智能体能够可靠应对的任务时长边界已近似呈指数级增长。在2023年及之后的子集中观察到的较短倍增时间进一步表明，此次扩张可能已加速；然而，由于评估覆盖范围和研究方法学上的差异，尚无法将这两个拟合值视为确凿证据。加速度。不过，这一趋势与我们的能力层级结构完全吻合：随着可完成任务的时间范围从几分钟扩展至数小时，再扩展至数十小时，前沿智能体便不再局限于上下文内的H1任务，而开始越来越多地处理需要更高层次能力的H2任务。跨上下文状态与记忆，开始向跨任务、开放式的H3边界靠拢。从这个意义上说，测得时间范围的持续扩展，正是长时程能力沿H1 → H2 → H3阶梯攀升过程的可观测投影。按照当前的节奏，许多目前仍处于前沿领域的多小时任务，在可预见的未来可能会变得常规化。不过，这一趋势体现了随时间推移而取得的进步；但它并未解释其根源。具体而言，该方法并不区分收益是源于更强的策略 π_θ 、更优的资源利用方案 H ，还是两者的协同进化所致。将这些因素中观察到的进展归因于何种因素，正是本文的核心问题。因此，我们不再聚焦于测算前沿，而是转而探讨驱动该前沿的因素。

2.4 Long-Horizon Agent vs. Neighboring Concepts

2.4 长时程智能体与邻近概念对比

Long-horizon agency is often conflated with three adjacent notions: long-running execution, autonomy, and self-evolution. They are related because all appear in modern agent systems, but they answer different questions. Long-running asks how long execution persists, autonomy asks how much human control remains, and self-evolution asks whether the agent improves itself over time. Long-horizon agency asks a different question: can the coupled system sustain interactive reasoning within a context (C1), maintain state and memory across contexts when needed (C2), and accumulate experience across tasks when deployment demands it (C3)? Figure 5 summarizes the overlap and separation.

长周期代理概念常与三个相邻的概念混为一谈：长期执行、自主性以及自我进化。这三者之间存在关联，因为它们均可见于现代智能体系统中，但各自所回答的问题亦有所不同。“长期运行”问题探讨执行过程会持续多久；“自主性”问题探究人类控制力还保留到何种程度；而“自我进化”问题则关注智能体是否会在时间推移中实现自我提升。长时程代理提出一个不同的问题：耦合系统能否在特定语境（C1）内支持交互式推理，并在需要时跨语境维持状态与记忆（C2）？当部署需求要求时，能否在不同任务间积累经验（C3）？图5总结了这些重叠与分离情况。

2.4.1 Long-Horizon Agent vs. Long-Running Agent

2.4.1 长时程智能体与长运行智能体

A long-running agent is defined by duration: it keeps making progress over hours to weeks across many windows, sessions, or sandboxes, with checkpointing and resumption [14]. It often appears in long-horizon settings, but duration and difficulty are not the same.

长运行代理由其持续时间定义：它能在多个窗口、会话或沙箱中，历时数小时至数周持续推进，并支持检查点记录与恢复功能[14]。这类问题常出现于长周期场景中，但其持续时间与难度并不相同。

Overlap. Many genuinely long-horizon tasks are also long-running, because their trajectories outgrow one context window or one execution session. In these cases the same engineering mechanisms that instantiate C2 matter: compressing and externalizing historical context, durable memory, progress logs, checkpoints, and faithful resumption. Long-running engineering is therefore a common implementation requirement in H2 settings, where task state must survive resets, context boundaries, and hand-offs without assuming that full history remains visible.

重叠。许多真正的长周期任务也属于长运行任务，因为其执行轨迹会超出单个上下文窗口或单次执行会话的范围。在这些情况下，实现C2的相同工程机制同样至关重要：包括压缩与外部化历史上下文、持久化存储、进度日志、检查点以及精确的恢复功能。因此，在H2环境下，长运行工程是常见的实现需求——即任务状态必须能够在系统重置、上下文边界切换及任务交接过程中保持有效，而无需假定完整的历史记录仍可被访问。

Distinctions. The key distinction is that duration and *logical coupling* are independent properties. A task may run for days while consisting mostly of logically decoupled, independently checkpointable steps; a script that loops over a large dataset and applies the same operation runs for a long time, yet its steps carry few interdependencies, so it demands neither dense, sustained interactive reasoning (C1) nor live management of cross-context state (C2). Conversely, a task may complete within a single window while still requiring many interdependent decisions whose direction must be continually tested and revised through interaction with the environment. Long-running engineering thus optimizes persistence, throughput, and cost over time, whereas long-horizon agency concerns whether an agent can sustain reasoning through continual interaction and, when the task demands it, maintain the state and memory needed to carry that reasoning across contexts [286]. Accordingly, execution duration captures temporal persistence, whereas long-horizon agency denotes the capacity to sustain coupled reasoning and state management across the task’s dependency structure.

区别。其核心区别在于：持续时间与*逻辑耦合*是两个独立的属性。一项任务可能持续数天运行，其过程主要由逻辑上解耦且可独立进行检查点保存的步骤组成；而一个对大型数据集进行循环遍历并执行相同操作的脚本虽运行时间较长，但其各步骤间的依赖关系却十分有限。因此，它既无需进行密集且持续的交互式推理（C1），也无需实时管理跨上下文状态（C2）。相反，一项任务可能在单个时间窗口内完成，但同时仍需做出诸多相互依赖的决策——这些决策的方向必须通过与环境的交互不断进行测试和修正。长期运行的工程方案能够随时间推移优化持久性、吞吐量与成本；而长周期规划则关注智能体是否能够通过持续交互维持推理能力，以及在任务要求时能否实现该功能。维持执行该推理跨越不同语境所需的**状态与记忆**[286]。因此，执行时长可捕捉时间上的持续性，而长时程代理则体现了在任务的依赖结构中维持耦合推理与状态管理的能力。

2.4.2 Long-Horizon Agent vs. Autonomous Agent

2.4.2 长时程智能体与自主智能体

An autonomous agent is defined by how much it can act without user involvement, often formalized as roles ranging from operator to observer [155]. Autonomy and long-horizon agency are strongly related, but they describe different properties of a system.

自主智能体的定义基于其能够在无需用户干预的情况下执行多少操作，这一概念通常可表述为从“操作员”到“观察者”的各类角色[155]。自主性与长周期能动性密切相关，但二者描述的是系统的不同属性。

Overlap. The two overlap because long-horizon tasks make constant human supervision expensive, so most long-horizon agents are largely automated: they interact with external environments and tools on their own, and autonomously decide when to revise their strategy. If an agent must take many steps, interact with tools and the environment, absorb feedback, and revise its trajectory, then requiring approval at every step can destroy the usefulness of the system, even when the task itself fits within H1. At the same time, autonomy raises the safety stakes of long-horizon execution, since more unattended steps and more coupled side effects mean a larger blast radius, so permission gates and oversight remain central.

重叠。这两者存在重叠之处，因为长周期任务使得持续的人工监督成本高昂，因此大多数长周期智能体都基本实现了自动化：它们能够独立与外部环境及工具进行交互，并自主决定何时对自身策略进行调整。策略：若智能体需执行大量操作步骤、与工具及环境进行交互、吸收反馈并修正其行动轨迹，则在每一步骤均要求获得人工批准可能会削弱该系统的实用性——即便该任务本身符合假设 H1 的要求。与此同时，自主性会加剧长周期执行过程中的安全风险——更多的无人值守步骤与更多的耦合副作用意味着更大的风险扩散范围，因此，权限闸门与监督机制仍至关重要。

Distinctions. The distinction is that autonomy is only a partial, governance-oriented dimension of a system, not a measure of its capability. A strong long-horizon agent can be run with low autonomy behind approval gates, whereas a highly autonomous agent may execute only a short, single-window task, such as a single web action [452]. Moreover, long-horizon agents need not be fully autonomous: their harnesses can introduce permission gates, approval checkpoints, and escalation policies that selectively involve humans at high-impact or uncertain steps. Autonomy therefore asks who remains in the loop, whereas long-horizon agency asks

区分。其区别在于：自主性仅是系统的一个部分——即一个以治理为导向的维度，而非衡量该系统能力的指标。强长时程智能体可在审批闸口下以较低自主性运行，而高度自主的智能体则可能仅执行短时、单窗口的任务，例如单次网络操作[452]。此外，长周期智能体未必需要完全自主：其控制机制可引入权限闸门、审批节点及**escalation**策略，从而在高影响或不确定的环节中选择性地引入人类干预。因此，“自主性”问题关注的是谁仍参与决策过程，而“长周期代理性”问题则关注.....

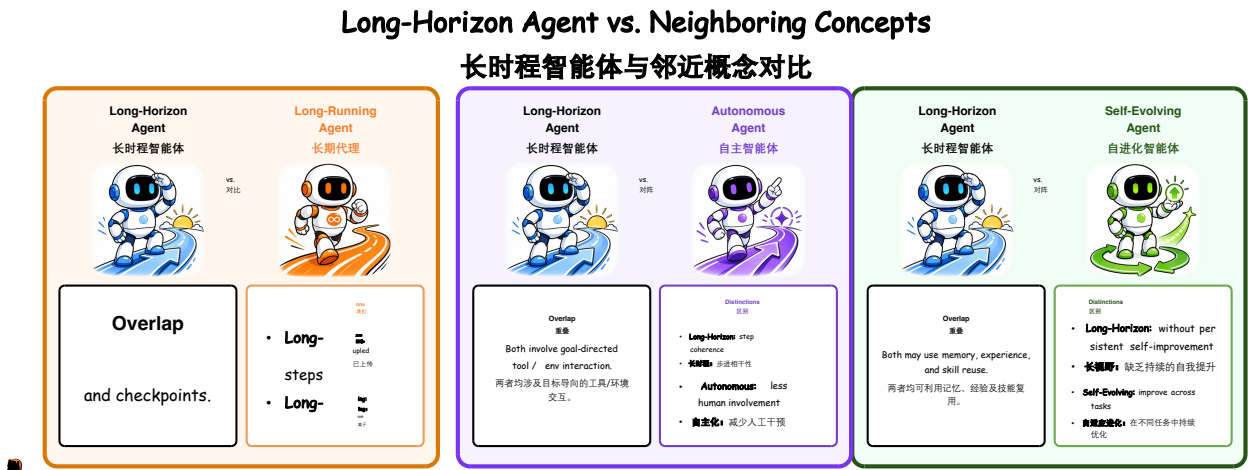


Figure 5 Long-horizon agent and three neighboring concepts. Long-running execution, autonomy, and self-evolution each emphasize a different facet; long-horizon agency asks whether the coupled model–harness system can sustain interactive reasoning (C1), maintain state and memory across contexts when needed (C2), and accumulate experience across tasks when deployment demands it (C3).

图5：长视野智能体及其三个邻近概念。长期执行、自主性与自我演化分别强调不同的维度：长周期代理机制旨在探讨耦合的模型-驾驭系统是否能够维持交互式推理（C1），以及在需要时能否在不同上下文中保持状态与记忆（C2）。并在部署需求时积累跨任务经验（C3）。

whether the coupled system can sustain the reasoning, state management, and, at H3, experience accumulation that the task horizon demands. In short, autonomy controls the degree of human involvement, whereas long-horizon agency measures whether execution can be carried through the required horizon.

该耦合系统是否能够维持任务视野所要求的推理、状态管理以及在H3层级下的经验积累。简而言之，自主性决定了人类参与的程度，而长时程代理性则用于衡量执行任务是否能在规定的时域内完成。

2.4.3 Long-Horizon Agent vs. Self-Evolving Agent

2.4.3 长时程智能体与自进化智能体

A self-evolving agent is defined by experience-driven, *persistent* change to its own parameters, memory, tools, or topology, with the goal of improving future performance. It is the closest neighbor to long-horizon agency because both concern competence beyond a single model call, but they still differ in scope.

自进化智能体是指通过经验驱动的、*持续的*对其自身参数、记忆、工具或拓扑结构的修改，以提升未来性能为目标的智能体。它与“长视域代理”最为接近，因为两者都涉及超越单次模型调用范围内的能力，但二者的适用范围仍有所不同。

Overlap. At the cross-task (H3) scale the two overlap strongly. A long-horizon agent that switches among complex tasks, accumulates reusable skills, consolidates memory, and reuses experience across episodes begins to approach self-evolution. Both also fit the “era of experience” view that agent competence should grow from interaction rather than only from a frozen pretraining corpus [580]. In this paper, self-evolution is one internalized mechanism for realizing C3, namely persistent cross-task experience accumulation.

重叠。在跨任务（H3）量表上，两者存在显著重叠。一种具备长时程规划能力的智能体——能够在复杂任务间切换、积累可复用技能、整合记忆并能在不同任务序列中复用经验——正逐步迈向自我进化阶段。这两者也均符合“经验时代”的观点，即智能体的能力应通过交互来提升，而非仅依赖于一个固定的预训练语料库[580]。本文中，自我演化是实现C3的一种内化机制，即持续的跨任务经验积累。

Distinctions. The distinction is whether experience persistently changes the agent for the future. Self-evolution requires such persistent change and is therefore only one facet of the C3 capability, cross-task experience accumulation, rather than the whole of long-horizon agency, which also rests on core capabilities such as interactive reasoning, tool use, and long-context reasoning. Consequently, many long-horizon agents operating at H1 or H2 complete difficult tasks with a fixed policy and temporary state, finishing a trajectory without updating their weights, tools, or lasting memory. Such agents lack any self-evolution capability, yet they remain long-horizon agents.

区分问题。关键区别在于：经验是否会对智能体的未来产生持续的影响。自我进化需要如此持续的变革，因此仅是C3能力——跨任务经验积累——的一个方面，而非长时程主体能力的全部；后者同样依赖于交互推理等核心能力。工具使用与长上下文推理能力。因此，许多在H1或H2层级运行的长时程智能体，能够通过采用固定策略和临时状态来完成复杂的任务——即在完成整个轨迹的过程中，无需更新其权重、工具或持久记忆。此类智能体不具备任何自进化能力，但仍属于长时程智能体。

Self-evolution therefore concerns persistent improvement *between* tasks, while long-horizon agency additionally concerns reliably carrying each task *through* the extended interaction, state management, tool use, and iterative reasoning that its dependency horizon demands.

因此，自我进化关注的是*任务之间的*持续改进，而长时程代理还涉及可靠地完成每项任务*的过程*。其依赖范围所要求的扩展交互、状态管理、工具使用及迭代推理。

Overall, this section has laid the conceptual foundation on which the rest of the paper builds. We defined a long-horizon agent as a coupled model–harness system, $\pi_\theta \oplus \mathcal{H}$, rather than as the foundation model alone (Section 2.1); organized its difficulty and matching capabilities into three nested levels, H1–H3 and C1–C3 (Section 2.2); quantified the empirical growth of a frontier-agent time-horizon proxy and compared alternative fitting windows (Section 2.3); and separated long-horizon agency from long-running execution, autonomy, and self-evolution (Section 2.4). The methodological takeaway is that long-horizon agency is a high-level, system-level capability that integrates these adjacent properties rather than reducing to any one of them. An agent may exhibit sustained running, autonomy, and self-evolution and yet fail an H1 task when it cannot

总体而言，本节为本文其余部分奠定了概念基础。我们将长时程智能体定义为一个耦合的模型-驾驭系统 $\pi_\theta \oplus \mathcal{H}$ ，而非仅将其视为基础模型（第2.1节）。); 将其难度与匹配能力划分为三个嵌套层级：H1–H3 和 C1–C3（第 2.2 节）；量化了前沿代理模型的时间跨度代理变量的经验增长，并对比了不同的拟合窗口（第2.3节）。)；并将长周期机构与长期执行、自主权及自我演进相分离（第2.4节）。该方法论启示在于：长周期代理是一种高层级、系统级的能力，它能够整合这些相邻特性，而非简化为其中任何单一特性。智能体可能表现出持续奔跑、自主性及自我进化能力，但在执行H1任务时却无法成功。

Table 1 Three stages in the co-evolution of long-horizon agents. Each stage widens the *control unit*, opens a new *control surface*, and advances a set of *key technical routes*. The graded *long-horizon profile* on the right scores each stage against the three accumulating capabilities defined in Section 2.2, namely C1 intra-context interactive reasoning, C2 cross-context state and memory, and C3 cross-task experience accumulation (● full, ◐ partial, ○ absent).

表1：长周期智能体共进化过程的三个阶段。每个阶段均会扩展*控制单元*、开辟新的*控制面*，并推进一系列*关键技术路径*。右侧的*梯度长周期性能曲线*将每个阶段与第2.2节中定义的各项累积能力进行对比评分。即：C1 纵向上下文内交互推理、C2 跨上下文状态与记忆、以及 C3 跨任务经验累积（● 全部、◐ 部分、○ 缺失）。

Stage (era) 时代（阶段）	Control unit & surface 控制单元与表面	Key technical routes 关键技术路线	Long-horizon profile 长周期曲线		
			C1	C2	C3
I. Prompt engineering I. 提示工程 (2020–2023)	Single query 单次查询 <i>language of the prompt</i> 提示词的语言	● Chain-of-thought reasoning ● 链式思维推理 ● Decomposition, planning, and search ● 分解、规划与搜索 ● Prompt optimization and internalization ● 快速优化与内化	◐	○	○
		● Retrieval-augmented generation ● 检索增强生成 ● Tool use and function calling ● 工具使用与函数调用 ● Long context, memory, and context engineering ● 长上下文、记忆与上下文工程	◐	◐	○
III. Runtime harnesses III. 运行时套件 (2025–Present) (2025年一至今)	Whole trajectory 完整轨迹 <i>runtime around the model</i> 模型周围的运行时	● Agent loops (reason–act–observe) ● 代理循环（原因–行动–观察） ● Harness engineering ● 安全带工程设计 ● Internalizing the loop ● 内化环	●	●	◐

sustain interactive reasoning through environment feedback, or fail an H2–H3 task when it lacks the state management or experience accumulation that those horizons require. Competence therefore shifts from the model alone to the coupled system $\pi_\theta \oplus \mathcal{H}$ and its training, and evaluation must track task completion, robustness over interaction, cross-context recovery, cross-task transfer, cost, and safety rather than static per-call accuracy. Having framed long-horizon agency as a coupled model–harness property organized along H1–H3 and C1–C3, we now turn to a historical question, namely how the field progressively moved capability from prompts, to context, to persistent runtime systems. Section 3 traces that progression.

通过环境反馈维持交互式推理；若H2–H3任务缺乏这些推理层级所要求的状态管理或经验积累，则该任务可能失败。因此，能力转移从单纯模型转向耦合系统 $\pi_\theta \oplus \mathcal{H}$ 。其训练与评估过程必须关注任务完成度、交互过程中的鲁棒性、跨上下文恢复能力、跨任务迁移能力、成本及安全性，而非仅关注静态的每次调用准确率。在将长时程代理建模为一个沿H1–H3与C1–C3维度组织的耦合模型–资源利用机制之后，我们现转向一个历史问题：即该领域如何逐步将能力从提示符层面迁移至上下文层面，最终发展至持久运行时系统层面。第3节追溯了这一进程。

3 Evolution: From Prompting to Runtime

3 演进历程：从提示阶段到运行时

As stated in the previous sections, we define a long-horizon agent as the joint product of *internalized model optimization* and *externalized harness engineering*, and reading the field through the lens of evolution shows why this coupling is not a recent design choice but a recurring necessity. Across the history of LLMs, the base model and the engineering structure around it have advanced together, each repeatedly compensating for the other’s limits. A base model is a powerful but fragile next-step predictor that, on its own, does not reliably decompose a goal, ground its steps in the world, sustain reasoning through environment interaction, or maintain the cross-context state that longer horizons require (Sections 1 and 2.2), and supplying this missing structure has always been the task of the surrounding engineering. Following this co-evolutionary thread, this section traces the development of long-horizon agents along its two driving forces, model optimization and harness engineering, and organizes the representative work that defines each stage.

如前文所述，我们定义“长周期智能体”为*内部化模型优化*与*外部化系统集成工程*的结合产物。通过进化视角审视该领域，可揭示这种耦合并非近期的刻意设计选择，而是一种反复出现的必然需求。在大型语言模型的发展历程中，基础模型及其配套的工程架构始终同步演进，二者相互弥补彼此的局限性。基础模型是一种功能强大但较为脆弱的后续预测器——其本身无法可靠地分解目标、将步骤锚定于现实世界、或通过与环境的交互来维持推理过程。或维持较长时间跨度所需的跨上下文状态（第1节与第2.2节），而填补这一缺失结构始终是周边工程领域的任务。基于这一协同进化脉络，本节将追溯长时程智能体的发展历程——其发展过程受两大驱动力——模型优化与能效优化的驱动，并梳理了界定各发展阶段的代表性研究成果。

As shown in Figure 6, we trace this co-evolution through three nested, chronologically and thematically coherent stages, each opened by an expansion in the base model’s own capability, organized around a new control surface exposed by the surrounding harness, and closed by the long-horizon limit that forces the next stage. **(1) Prompt Engineering** (2020–2023) steers behavior through the *language* of a single query, first eliciting multi-step reasoning from a fixed model; **(2) Context Engineering** (2023–2025) extends control to the *information* supplied to the model, grounding the agent in retrieved evidence, tools, and memory; and **(3) Runtime Harnesses** (2025–present) make the unit of control the whole *trajectory*, so that the long-horizon agent proper emerges as a model coupled to a runtime that sustains one goal over many dependent steps. The three are not successive replacements but nested layers, in which each new control surface absorbs the previous one rather than discarding it. Throughout this chapter, the years attached to each stage denote the period in which its control surface became the *dominant* paradigm, not the publication date of any individual work; a stage may therefore rest on foundational methods that predate it and preview ideas that mature later. Table 1 summarizes the three stages along a common set of axes.

如图6所示我们通过三个相互嵌套、在时间线与主题上均具连贯性的阶段来追溯这一协同演化过程；每个阶段的开启均源于基础模型自身能力的扩展，而该扩展则围绕着由周边总成所呈现的新控制面展开。最终通过长时程极限完成闭环，从而推动进入下一阶段。**(1) 提示工程**(2020–2023) 通过单个查询的*语言*来引导模型行为，首先从固定模型中引出多步推理过程；**(2) 上下文工程**(2023–2025年)将控制权扩展至模型所接收的*信息*，使智能体能够基于检索到的证据、工具及记忆进行运作；而**(3) 运行时框架**（2025年至今）则将控制单元设定为整个*轨迹*。由此，长时程智能体本身便演变为一个与运行时耦合的模型——该运行时能够在多个相互依赖的步骤中持续维持某一目标。这三者并非连续替换关系，而是嵌套层级结构——其中每个新的控制面均会吸收前一个控制面，而非将其丢弃。在本章中，每个阶段所标注的年份表示该阶段的控制面成为主导范式所处的时间段，而非任何单部著作的出版日期；因此，某一阶段可能建立在其之前存在的基础性方法之上，并预示着日后成熟的思想。表1总结了沿同一组轴线划分的这三个阶段。

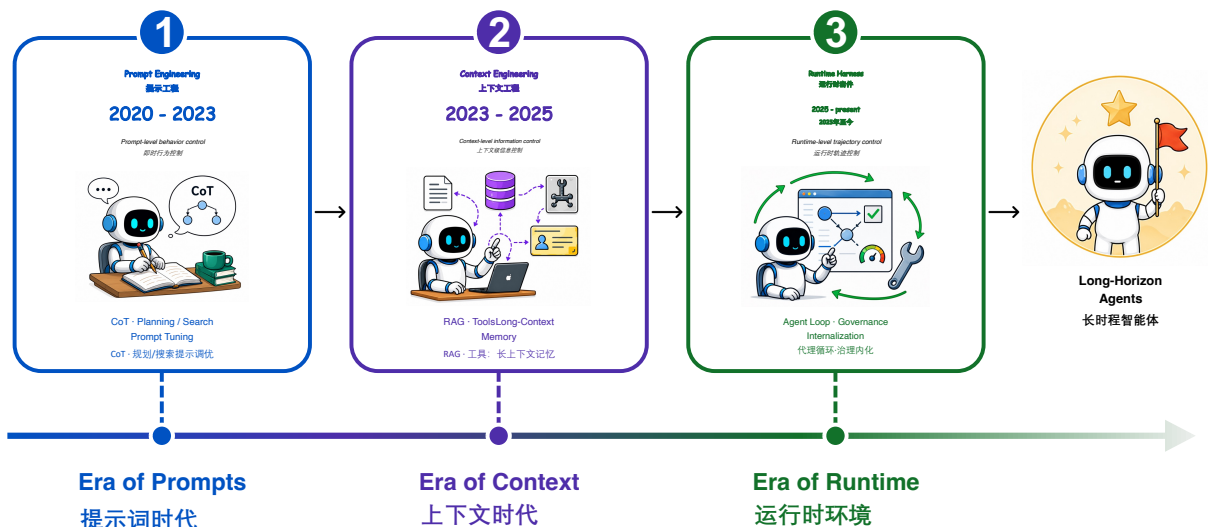


Figure 6 The co-evolution of long-horizon agents across three stages. Control widens from the *language* of a single prompt (2020–2023), to the *information* supplied to the model before it acts, to the whole *trajectory* sustained by a runtime harness (2025–present); each stage absorbs the previous rather than replacing it.

图6：长时程智能体在三个阶段中的协同进化过程。控制范围已从单个提示的 *语言*（2020–2023年），扩展至模型执行前提供的 *信息*，乃至运行时框架所支撑的整个 *轨迹*（2025年至今）；每个阶段均吸收前一阶段的内容，而非取而代之。

3.1 Stage I: Prompt Engineering (2020–2023)

3.1 第一阶段：提示工程（2020–2023年）

The prompting stage is the prototype of long-horizon agency, in which the model carries most task capability in its weights and the only surface a builder can touch is the language of the query. The stage opens with *prompt learning*, made widely visible by GPT-3 [40] and consolidated by dedicated papers on prompting and in-context learning [132, 379], which let a pre-trained model perform a new task from a handful of in-prompt examples with no weight update and thereby moved the focus of system building from training the model to shaping its input at inference time. Its central discovery is that the right prompt can *elicit* latent abilities the base model does not show by default, including the embryonic forms of planning and execution that long-horizon tasks demand. Concretely, the prompting stage surfaces this latent competence along three representative threads, which we take up in turn: chain-of-thought reasoning, decomposition and planning, and the optimization and internalization of prompts themselves.

提示阶段是长时程代理机制的原型：在此阶段，模型的大部分任务能力都蕴含在其权重参数中，而构建者能够触及的唯一层面仅限于查询语言。本章开篇介绍 *提示学习*，该领域因GPT-3[40]而变得广为人知，并通过关于提示学习与上下文学习的专门论文[132, 379]得到了进一步巩固。该方法允许预训练模型仅基于少量提示内示例即可执行新任务，且无需进行权重更新，从而将系统构建的重点从模型的训练转向在推理阶段对模型输入进行塑造。其核心发现在于：恰当的提示能够激发基础模型默认下并未展现的潜在能力，其中包括长时程任务所要求的规划与执行的雏形。具体而言，提示阶段会通过三个代表性路径来凸显这种潜在能力——我们依次探讨这三方面：思维链推理、分解与规划，以及对提示本身的优化与内化。

Chain-of-Thought Reasoning. The first thread showed that a fixed model reasons far better when prompted to externalize intermediate steps rather than answer directly. Initially, the scratchpad [450] emitted intermediate computation for multi-step problems; chain-of-thought (CoT) prompting [675] then turned this into a few-shot recipe that elicits a reasoning trace before the answer; and zero-shot CoT [281] further showed that the same behavior could be triggered by a single instruction, confirming that the capacity was latent in the weights and merely needed surfacing. Building on CoT, a family of variants quickly followed that automate demonstration construction, scale reasoning effort with problem complexity, and modularize the prompt into reusable sub-steps [160, 275, 851]. The shared lesson, that the same model exhibits qualitatively different competence depending on how its input is phrased, recurs throughout the paper.

链式推理。第一段讨论表明，当被提示将中间步骤外部化而非直接作答时，固定模型的推理效果要好得多。最初，Scratchpad[450]会为多步问题输出中间计算结果；随后，思维链（CoT）提示机制[675]将其转化为一个“少样本示例方案”，在得出答案之前引导出推理轨迹；而零样本CoT[281]进一步表明，相同的行为可由单条指令触发，这证实了该能力原本存在于权重中，仅需将其“激活”即可显现。基于CoT，一系列变体随即涌现，这些变体能够自动完成演示构建、根据问题复杂度扩展推理工作量，并将提示词模块化为可复用的子步骤[160, 275, 851]。本文贯穿始终的一个共同启示是：同一模型会根据其输入的表述方式，展现出质上不同的能力。

Decomposition, Planning, and Search. A second thread widened the single chain into the structures that prefigure an agent, using prompting not merely to reason but to make a fixed model *decompose* a goal, *plan* a course of action, and *execute* it step by step, which is the earliest recognizable form of an agent. Self-consistency [659] samples many chains and aggregates them by majority vote, trading one fragile path for a more reliable distribution; least-to-most prompting [877] made decomposition explicit by solving ordered subproblems; and Plan-and-Solve prompting [639] split an explicit planning step from execution, producing the first clean “plan then act” template that later agent loops inherit. A parallel line grounded execution in code, with PAL [168] and Program of Thoughts [70] offloading computation to an interpreter rather than the model’s own arithmetic, while Tree-of-Thoughts [760] recast reasoning as deliberate search over a tree of partial solutions with look-ahead and backtracking. A closely related line then pushed the same plan-then-act template into *interactive* environments, where SayCan and Inner Monologue grounded language plans in affordances and environment feedback [236, 243], and AdaPlanner, ADaPT, and DEPS closed the loop with adaptive re-planning from execution feedback in embodied and text worlds such as ALFWorld [482, 545, 569, 673]. Read together, this line forms the embryonic agent skeleton, which decomposes a goal, proposes and executes steps, evaluates progress, and revises. Prompt learning alone cannot yet yield a long-horizon agent, since all of this unfolds without a persistent runtime harness, durable state, or robust recovery. What the stage does provide is a soft external constraint on model behavior and an early exploration of the harness-mediated control that later stages systematize, together with the planning-and-execution

skeleton here and the elicitation-then-internalization dynamic below.

分解、规划与搜索。第二条线将单链扩展为预示智能体的结构，其采用提示机制不仅用于推理，更用于使固定模型分解目标、规划行动路径以及执行。它是一个逐步构建的过程，这是代理最早可识别的形式。自治法[659]会生成多个链路，并通过多数投票将这些链路聚合，从而用一条较为脆弱的路径来换取更可靠的分布；最小到最大提示法[877]通过求解有序子问题将分解过程显式化；而“规划-求解”策略[639]将显式的规划步骤与执行过程分离，从而产生了首个清晰的“先规划后行动”模板，该模板随后被智能体循环所继承。代码中的并行接地执行机制，其中PAL[168]与“思维树”[70]将计算任务卸载至解释器，而非交由模型自身的算术运算处理；而“思维树”[760]则采用另一种机制。将推理过程重新定义为对部分解树的有预见性与回溯相结合的有意识搜索。随后，一个密切相关的研究团队将“规划-执行”这一模式引入了交互式环境；在这些环境中，SayCan和Inner Monologue将语言规划与可用性及环境反馈相结合[236, 243]。AdaPlanner、ADaPT和DEPS通过在具身环境及文本环境中利用执行反馈进行自适应重规划，从而实现了闭环控制，例如在ALFWorld等环境中[482, 545, 569, 673]。这条指令共同构成了胚胎型代理骨架，它负责分解目标、提出并执行操作步骤、评估进展以及进行调整。仅靠即时学习尚无法构建出长时程智能体，因为上述所有过程均在缺乏持久运行时环境、持久状态或鲁棒恢复机制的情况下展开。阶段机制所提供的，是对模型行为的柔性外部约束，以及对“调控介导控制”机制的早期探索——而这一机制将在后续阶段被系统化。与本文中的规划-执行框架以及下文所述的“启发-内化”动态过程共同构成。

Prompt Optimization and Internalization. A third thread responded to a property this stage exposed and the rest of the paper inherits, namely that elicited capability is *fragile*, swinging with wording, example order, example choice, or output format [403, 433, 862], which made prompt-based systems hard to reproduce, test, and compose. The community answered by making prompt construction systematic rather than handcrafted. One line recast prompt design as an explicit search or optimization problem [483, 889], while another treated prompts as composable, program-like artifacts [514]. In parallel, what began as a prompting trick started to be *internalized*, as instruction tuning together with reinforcement learning from human feedback let models follow intent without elaborate templates, most visibly in InstructGPT [460], and the same dynamic would later fold full chain-of-thought traces into the weights, which we revisit in Section 5. This is the first concrete instance of the harness→policy migration the paper tracks across all three stages.

提示优化与内化。第三条讨论线对本阶段揭示且论文其余部分均沿用的某一特性作出了回应，即：所引出的能力是脆弱的——会随措辞、示例排序、示例选择或输出格式的改变而波动[403, 433, 862]。这一特性使得基于提示的系统难以复现、测试和组合。为此，社区采取了将提示构建过程系统化而非依赖人工设计的方法来应对这一问题。有一种观点将提示词重写问题重新表述为一个显式的搜索或优化问题[483, 889]，而另一种观点则将提示词视为可组合的、类程序化的人工制品[514]。与此同时，最初只是作为一种引导性技巧的行为，逐渐被内化。通过指令微调与基于人类反馈的强化学习相结合，模型无需依赖复杂的模板即可遵循意图，这一点在InstructGPT[460]中尤为明显。该动态机制随后可将完整的链式推理轨迹折叠至权重中，我们将在第5节中重新探讨这一问题。这是该论文所追踪的、贯穿所有三个阶段的→带式策略迁移方案的第一个具体实例。

Although prompt engineering by itself does not constitute a long-horizon agent, its core idea, shaping a model’s reasoning behavior through external constraints rather than weight updates, remains a guiding principle for long-horizon agents to this day. Yet prompting controls only *language* over a fixed knowledge base. A single call can use only the text in the prompt and the knowledge already stored in the parameters [266, 479, 516], so no phrasing can supply information the model never saw. As applications moved toward current events, private data, large repositories, and changing APIs, the binding question shifted from how the instruction is worded to what the model is allowed to see, pushing engineering one layer outward, from the language of the prompt to the information assembled into it.

尽管“即时工程”本身并不构成一种长时程智能体，但其核心思想——即通过外部约束而非权重更新来塑造模型的推理行为——至今仍是指导长时程智能体设计的重要原则。然而，提示控制机制仅能在固定的知识库上进行语言操作。单次调用仅能使用提示中的文本及参数中已存储的知识[266, 479, 516]，因此任何表述都无法提供模型从未见过的信息。随着应用程序逐渐转向关注时事、私有数据、大型数据仓库以及动态变化的API，绑定问题也发生了转变——从“指令应如何表述”转变为“模型被允许查看哪些内容”，从而使工程实践向更宏观的层面延伸。从提示词的语言表述到其中所整合的信息。

3.2 Stage II: Context Engineering (2023–2025)

3.2 第二阶段：情境工程（2023–2025年）

Where Stage I engineered the model’s *behavior* through the language of the query, Stage II turns to its *information*, that is, the evidence, tools, and history that a single call is allowed to condition on. Broadening this input, rather than rewording the instruction, becomes the primary means of extending the model’s capability boundary. The representative directions of this stage are retrieval-augmented generation, tool use and function calling, and context management and memory, which we detail in turn below.

第一阶段通过查询语言对模型的行为进行设计；第二阶段则转而利用模型的信息——即单次调用所允许基于的证据、工具及历史记录。拓宽输入范围——而非对指令进行改写——已成为扩展模型能力边界的主要途径。该阶段的代表性任务包括检索增强生成、工具使用与功能调用，以及上下文管理与记忆——我们将在下文依次对这些任务进行详细阐述。

Retrieval-Augmented Generation. Retrieval-augmented generation (RAG) loosened the boundary between the model and the world by attaching external evidence to the prompt at inference time [304], building on a line of retrieval- and memorization-augmented models that coupled parametric knowledge with an external corpus [38, 200, 246, 247, 270, 273] and organized systematically in dedicated RAG reviews. Its core result is that grounding generation in retrieved text reduces hallucination and improves factuality on knowledge-intensive tasks. Initially a single retrieve-then-read pass, the technique soon evolved into a richer family. HyDE-style hypothetical-document embeddings first improved recall [167], and hierarchical indexing summarized corpora into multi-resolution trees [522]. Retrieval was then interleaved with reasoning, through multi-hop methods that chain retrieval across steps [612] and active retrieval that lets the model decide *when* to look something up [258]. More recently, self-reflective retrieval further lets the model critique and re-retrieve its own evidence [22].

检索增强生成。检索增强生成（RAG）通过在推理阶段将外部证据附加至提示词，从而消除了模型与外部世界之间的边界[304]；该方法是在一系列基于检索与记忆增强的模型基础上发展而来的。将参数化知识与外部语料库[38, 200, 246, 247, 270, 273]相结合，并通过专门的RAG综述进行系统化整理。其核心结论是：在检索所得文本中进行接地生成能够减少幻觉现象，并提升知识密集型任务中的事实准确性。该技术最初仅采用一次“检索-阅读”流程，但随后迅速演进为一个更为丰富的技术体系。HyDE风格的假设文档嵌入方法首先提升了召回率[167]，而层次化索引则将语料库汇总为多分辨率树[522]。检索过程随后与推理过程交替进行：通过多跳方法将检索操作在不同步骤间串联[612]，以及通过主动检索机制让模型决定何时查询特定内容[258]。近年来，自我反思检索还使模型能够对自身的证据进行批判并重新检索[22]。

Tool Use and Function Calling. Where retrieval lets a model *read* the world, tool use lets it *query and act on* systems beyond text. Early work established the mechanics. Toolformer [523] showed that a model could teach itself, in a self-supervised way, when and how to call APIs; ToolAlpaca [585] and ToolLLM [492], with the StableToolBench testbed [198], then scaled tool use to thousands of real-world APIs; Gorilla [471] grounded API selection in documentation to curb hallucinated calls; and WebGPT [445] grounded long-form answering in live browsing. As the practice consolidated, function calling and structured outputs standardized the interface so that model decisions could be parsed and dispatched reliably [385, 472], a maturation charted by dedicated tool-learning papers [493, 497]. The decisive bridge is ReAct [761], which interleaves reasoning with tool calls so that each observation feeds the next thought, while HuggingGPT [537] further showed how an LLM could route tasks across specialized models in a tool ecosystem. Both sit at the boundary of two

stages, turning one-shot tool use into the beginnings of the loop that Stage III makes central.

工具使用与函数调用。检索机制使模型能够 *读取* 外部世界；而工具使用则使其能够 *查询并操作* 超越文本范围的系统。早期的研究奠定了其运作机制的基础。Toolformer[523]证明了模型能够以自监督的方式自主学习何时以及如何调用 API；而 ToolAlpaca[585]与 ToolLLM[492]则依托于 StableToolBench 测试平台[198]进行验证。随后，Gorilla[471]将工具调用范围扩展至数千个真实世界API；通过基于文档进行API选择来抑制幻觉调用；而WebGPT[445]则将长文式回答构建于实时浏览之上。随着该实践的不断成熟，函数调用与结构化输出使接口得以标准化，从而能够可靠地解析模型决策并进行分发[385, 472]；这一进程通过专门的工具学习论文逐步得以完善[493, 497]。。决定性桥梁是ReAct[761]，它将推理与工具调用交替进行，使得每个观察结果都能为下一轮思考提供输入；而HuggingGPT[537]此外，该研究还展示了大型语言模型（LLM）如何在工具生态系统中将任务路由至不同的专用模型。这两者均处于两个阶段的交界处，将单次工具使用场景转化为循环运作的起点——而这一循环正是第三阶段的核心所在。

Long Context, Memory, and Context Engineering. The third development enlarged the container itself. Efficient-attention kernels such as FlashAttention [108] improved the practicality of long-sequence serving, while sparse-attention architectures such as Longformer [32] and BigBird [792] explored algorithmic routes to longer inputs; later frontier systems exposed hundred-thousand- to million-token windows [593], and state-space architectures offered near-linear alternatives to attention [186]. This made it physically possible to condition on whole documents, codebases, and histories rather than summarizing them away, which shifted the question from whether the model can reach information to whether it can use the information well once present. LongBench [28] was among the first benchmarks to quantify how poorly models handle genuinely long-context tasks, a gap that dedicated long-context suites continue to make precise [7, 220, 833]. Because no window is unbounded, a complementary line externalized state into persistent memory, with MemGPT treating context as a paging hierarchy over external storage [464] and generative agents maintaining a long-running memory stream with retrieval and reflection over remembered experience [470]. This abundance, however, created its own problem, since more context is not better context. Long inputs are read unevenly, with material in the middle systematically ignored [378]; irrelevant evidence actively distracts reasoning [541]; and utilization past a threshold degrades sharply, a failure measured as *context rot* [511]. The methodological response, named by practitioners *context engineering*, is the systematic selection, organization, compression, and refresh of the model’s input under hard token budgets [13, 425], with a toolkit spanning prompt compression [254, 351, 773], working-memory mechanisms [224, 268, 778], and rolling summarization for long runs [693, 890]. More recently, this curation is itself becoming agentic, as *agentic context engineering* lets the agent treat its own context as an evolving object that it revises and improves over successive steps [824]. Together, retrieval, tools, longer windows, memory, and context engineering shifted the relevant capacity from how much knowledge sits inside the model to how effectively the surrounding structure can selectively surface, organize, and use relevant information at each step; here too capability was partly internalized, as retrieval, tool use, and long-context handling increasingly became native, trainable model skills rather than bolted-on calls.

长上下文、记忆与上下文工程。第三项发展扩大了容器本身的范畴。高效的注意力核（如FlashAttention [108]）提升了长序列处理的实用性，而稀疏注意力架构（如Longformer[32]和BigBird[792]）则实现了类似效果。研究了适用于更长输入的算法路径；随后，前沿系统实现了数十万至百万级词元窗口[593]，而状态空间架构则为注意力机制提供了近乎线性的替代方案[186]。这使得在物理层面上能够对完整文档、代码库及历史记录进行条件化处理，而非对其进行摘要化处理。这使得问题的焦点从“模型能否获取信息”转变为“模型在获得信息后能否有效利用该信息”。LongBench[28]是首批能够量化模型在处理真正长上下文任务时表现如何不佳的基准测试之一；这一差距至今仍由专门针对长上下文任务设计的基准测试套件得以精确揭示[7, 220, 833]。由于不存在无界窗口，互补线性外部化状态被映射至持久内存，其中MemGPT将上下文视为外部存储上的分页层次结构[464]。生成式智能体能够维持一个长期记忆流，可对所记忆的经验进行检索与反思[470]。然而，这种丰富的记忆资源也带来了自身的难题：更多的上下文并不等同于更优的上下文。长输入的读取过程并不均匀，其中位于中间位置的材料会被系统性地忽略[378]；不相关的证据会主动干扰推理过程[541]；当利用率超过阈值时，性能会急剧下降，该故障可通过*context rot*[511]来衡量。该方法论响应——由从业者称为*上下文工程*——是指在严格的令牌预算约束下，对模型输入进行系统性选择、组织、压缩与更新的过程[13, 425]。其工具包涵盖了提示压缩[254, 351, 773]、工作记忆机制[224, 268, 778]以及针对长文本序列的滚动摘要技术[693, 890]。近期，这种内容策展本身正逐渐具备“智能体”特性——因为*智能体上下文工程*使得智能体能够将自身的上下文视作一个动态演变的对象，并在连续的处理步骤中对其进行修正与优化[824]。检索、工具、更长的窗口期、记忆机制以及上下文工程共同作用，将相关容量从“模型内部存储了多少知识”这一维度，转向了“周边结构能够如何有效地选择性地呈现并组织这些知识”的维度。并在每个步骤中运用相关信息；在此过程中，能力也部分实现了内化——即检索、工具使用及长上下文处理等技能，逐渐转变为本源性的、可训练的模型能力，而非外置的调用机制。

For all its reach, context engineering keeps the structural assumption it inherited from prompting, in that it optimizes the input to a *single* model call. The unit is wider, but it remains a one-shot, open-loop conditioning step that shapes what the model sees *before* it acts, not what happens after. Once deployments began chaining many such calls into consequential actions on tools, files, browsers, and operating systems, the dominant failures became properties of the *trajectory*, as errors compounded across steps, belief state drifted from the world, and side effects became irreversible. Closing that loop requires control over the runtime itself, which is the surface Stage III opens.

尽管上下文工程的适用范围广泛，但它仍保留了其从提示（prompting）继承的结构假设，即它会对单个模型调用的输入进行优化。该单元宽度较大，但其仍属于单次执行的开环预处理步骤，用于塑造模型在采取行动前所感知的信息，而非处理行动之后的情况。一旦部署过程开始将众多此类调用链式衔接为对工具、文件、浏览器及操作系统的后续操作时，主要故障便表现为*流程轨迹*的特性。随着错误在各个步骤中不断累积、信念状态偏离了现实世界，且副作用变得不可逆转，要闭合这一循环就需要对运行时本身进行控制——而这正是“表面第三阶段”所开启的领域。

3.3 Stage III: Runtime Harnesses (2025–Present)

3.3 第三阶段：运行时套件（2025年至今）

In Stage III, capability expansion manifests as a shift from answering questions to acting in environments, and the long-horizon agent takes its recognizable form. This also marks the transition of agents from chatbots toward genuinely productive settings, in which they carry out consequential work rather than merely converse. The transition was made concrete by executable environments and benchmarks such as WebShop [759], WebArena [885], SWE-bench [260], OSWorld [716], and Terminal-Bench [428], which score completed actions in web, code, desktop, and terminal settings rather than static answers, and thereby supply the trajectory-level feedback long-horizon agents are optimized against. The unit of engineering is no longer a prompt or a single conditioned call but a *trajectory*, comprising repeated cycles of reasoning, action, observation, verification, and recovery. We summarize the stage in three movements, namely the loop that makes the agent act, the harness that governs the loop, and the internalization that absorbs harness capabilities back into the model. 在第三阶段，能力扩展表现为从回答问题转向在环境中采取行动，此时长时程智能体呈现出其典型的形态。这也标志着智能体正从聊天机器人向真正具有生产力的场景转变——在这些场景中，它们并非仅进行对话，而是能够执行具有重要影响的工作。这一转变通过可执行环境及基准测试工具得以具体实现，例如WebShop [759]、WebArena[885]、SWE-bench[260]、OSWorld[716]以及Terminal-Bench[428]。该评分机制针对的是在Web、代码、桌面及终端设置中完成的实际操作（而非静态答案），从而为长时程智能体提供其优化所依据的轨迹级反馈。工程单元已不再是即时响应或单次条件调用，而是一个*轨迹*——该轨迹由推理、行动、观察、验证及恢复等重复循环组成。我们可将该阶段划分为三个部分：即驱动智能体执行动作的循环、调控该循环的约束机制，以及将约束机制的能力重新整合回模型的内化过程。

Agent Loops. The core engineering unit is a *trajectory* that unfolds repeatedly in an environment, in which the agent reasons, acts, reads feedback, and decides what to do next, following the widely adopted thought–action–observation cycle. Early work such as ReAct, Reflexion, and Self-Refine established the basic pattern of interleaving reasoning with tool use and verbal reflection after failure [416, 544, 761], while

languageagent tree search wrapped the loop in value-guided search with backtracking [875]; around the same time, open-source autonomous agents such as AutoGPT and BabyAGI popularized the self-directed loop for a broad audience [183, 444]. Since 2025, the loop has tightened along three axes rather than inventing a new paradigm. First, *action representation* became more executable and modular, as CodeAct unified step-level actions as code [656] and ReWOO separated planning from tool execution [720]. Second, *reliability* moved inward to the loop itself, as ReAct grounds decisions in goal-state reflection [276] and test-and-repair loops such as CRITIC introduce tool-interactive verify-then-correct steps that catch and revise errors mid-trajectory [182], complementing the reflect-and-retry pattern of Reflexion and the value-guided search of LATS above. Third, *task form* widened from QA-style interaction to open-ended, multi-tool trajectories in benchmarks such as GAIA and AssistantBench [431, 774], to vision-language control in SeeAct [864], and to multi-stage research pipelines that interleave planning, retrieval, and writing [258, 613, 871]. Alongside these free-form loops, a complementary line re-emphasizes structured *agent workflows*, in which recurring procedures are captured as reusable graphs or memories and replayed across tasks, as in Agent Workflow Memory [674] and automated workflow generation [353]. The loop has thus become a runtime primitive that sustain one goal across many coupled steps while continuously changing external state, and the remaining question is how to govern it.

代理循环。核心工程单元是一个轨迹，该轨迹在特定环境中反复展开；在此过程中，智能体会进行推理、采取行动、读取反馈，并根据广泛采用的“思考-行动-观察”循环来决定下一步行动。早期的研究（如 ReAct、Reflexion 和 Self-Refine）确立了在失败后，将推理与工具使用及言语反思相结合的基本模式 [416, 544, 761]。而 `languageagent` 的树搜索机制则将循环封装在基于价值引导的搜索及回溯过程中 [875]；大约在同一时期，诸如 AutoGPT 和 BabyAGI 等开源自主智能体使“自驱动循环”概念得以向广大受众普及 [183, 444]。自 2025 年起，该循环在三个维度上不断收紧，而非开创全新的范式。首先，*动作表征* 变得更加可执行且模块化——CodeAct 将步骤级动作统一为代码 [656]，而 ReWOO 则将规划与工具执行分离 [720]。第二，*可靠性* 向内延伸至循环本身，因为 ReAct 将决策建立在目标状态反思之上 [276]。诸如 CRITIC 之类的测试与修复循环引入了工具交互式“验证后再修正”步骤，可于轨迹执行过程中捕获并修正错误 [182]。该机制与 Reflexion 的“反射-重试”模式以及上文所述 LATS 的值导向搜索相结合。第三，*任务形式* 已从问答式交互扩展至 GAIA 和 AssistantBench 等基准测试中的开放式、多工具交互轨迹 [431, 774]，乃至 SeeAct 中的视觉-语言控制 [864]。以及支持将规划、检索与写作相互交织的多阶段研究流程 [258, 613, 871]。除这些自由形式的循环外，另一条互补线则进一步强调了结构化的 *智能体工作流*——在此模式下，重复出现的流程会被捕获为可复用的图或记忆，并可在不同任务间进行重放，如“智能体工作流记忆” [674] 所述。以及自动化工作流生成 [353]。因此，循环已成为一种运行时原语：它能够在多个相互耦合的步骤中持续实现某一目标，同时不断改变外部状态；而剩下的问题在于如何对其进行管控。

Harness Engineering. Once a loop stretches over a horizon, failures become properties of the *execution process* rather than of any single input, as errors compound, goals drift, success is declared too early, state distorts as context fills, and side effects escape authorization. The engineering response must therefore enter the runtime itself. *Harness engineering* names the organization of prompts, context, tools, state, permissions, verification, and recovery into a system-level control surface. The field’s first response was to make multi-step execution explicit. Communicative multi-agent efforts led by ChatDev and MetaGPT cast long-horizon software work as role-specialized message passing [219, 485], AutoGen turned that pattern into a programmable conversation runtime [692], and Magentic-One added dual ledgers that separate outer replanning from inner dispatch [157]. As trajectories grew longer and more inspectable, control migrated from ad hoc dialogue to graph-structured runtimes exemplified by LangGraph and packaged into deep-research systems such as Deep Agents and Flash-Searcher, where state can be resumed, branched, and audited [291, 293, 491]. In parallel, the interface to the outside world was standardized, as MCP unified heterogeneous tools behind a discoverable, authorizable service layer [99], while AGENTS.md and Agent Skills externalized project rules as portable harness configuration [16, 98]. Long-running deployment then forced a third concern, what an agent is allowed to do, into the foreground through hooks, middleware, pre-action gates, sandboxes, and policy layers [17, 18, 41, 463, 617]. These layers are what product systems ultimately integrate, including repository-oriented harnesses such as OpenHands and SWE-agent, whose Agent-Computer Interface showed that interface design alone can determine long-horizon success [657, 747], local personal-agent runtimes such as Claude Code and OpenClaw [10, 457, 513], and GUI-facing systems such as Operator [12, 452]. A later turn makes the harness itself an object of search and improvement, as in workflow discovery by AFlow and ADAS [225, 383, 662, 816, 897] and self-editing harness code in the Darwin Gödel Machine [814]. Reliable long-horizon agency is therefore a system property to be engineered and learned. Practitioners increasingly describe this outward shift as *loop engineering*, a move from writing one-shot prompts to designing loops that dispatch, monitor, verify, and recover agents over a trajectory.

Harness Engineering. 一旦循环跨越地平线，故障便成为 *执行过程* 的特性，而非任何单一输入的特性——因为错误会相互叠加、目标会发生偏移、成功判定过早、状态在上下文填充过程中发生畸变。且副作用无法获得授权，因此，工程响应必须直接在运行小时内执行。*赋能工程* 将提示、上下文、工具、状态、权限、验证及恢复机制整合为一个系统级控制界面。该领域的首个应对措施是将多步骤执行过程显式化。由 ChatDev 和 MetaGPT 主导的通信式多智能体协作机制，将长时程软件任务转化为角色专用的消息传递模式 [219, 485]；而 AutoGen 则将这一模式转化为可编程的对话运行时 [692]。Magentic-One 还新增了双账簿，可将外部重新规划与内部调度分开 [157]。随着轨迹的延长与可检测性增强，控制机制从临时性对话逐步转向基于图结构的运行时——以 LangGraph 为代表，并被集成至诸如 Deep Agents 和 Flash-Searcher 等深度研究系统中，其中状态可被恢复。具有分支结构，并经过审计 [291, 293, 491]。与此同时，与外部世界的接口也得到了标准化——MCP 通过一个可发现且可授权的服务层统一了异构工具 [99]，而 AGENTS.md 则……Agent Skills 将项目规则外化为可移植的套件配置 [16, 98]。长期的部署实践进而迫使第三个关注点——即代理被允许执行哪些操作——通过钩子、中间件、预操作闸门、沙箱及策略层 [17, 18, 41, 463, 617] 而凸显出来。这些层级正是产品系统最终所整合的内容，其中包括基于存储库的集成框架（例如 OpenHands 和 SWE-agent）；其中对 SWE-agent 的“代理-计算机接口”分析表明，仅凭接口设计即可决定长期成功 [657, 747]。本地个人代理运行时环境（例如 Claude Code 和 OpenClaw [10, 457, 513]），以及面向图形用户界面的系统（例如 Operator [12, 452]）。后续的转向使得调用链本身成为需要搜索与改进的对象，例如 AFlow 和 ADAS 对工作流的发现 [225, 383, 662, 816, 897]，以及 Darwin Gödel Machine 中对调用链代码的自编辑功能 [814]。因此，可靠的长时程代理是需要通过工程设计与学习来实现的系统特性。从业者们日益将这种向外转变描述为 *循环工程*——即从编写一次性提示词转向设计能够沿整个轨迹调度、监控、验证及恢复智能体的循环结构。

Internalizing the Loop. Stage III is also where the paper’s two halves become visible together, as behaviors first organized by the harness are increasingly absorbed into model weights through trajectory-level SFT and RL. Post-2024 work no longer pursues a single generic agent-tuning recipe; instead, internalization *tracks harness type*, because each runtime exposes a different action grammar, feedback channel, and failure mode. Search-centric harnesses are distilled by agentic-search RL such as Search-R1 [261]; repository harnesses by software-engineering RL such as SWE-RL, increasingly trained inside neural environments like SWE-World [572, 680]; terminal harnesses by systems such as RLAnything that unify shell, tool-use, and coding rollouts [663]; GUI harnesses by UI-TARS-2 [632]; and personal runtimes by Claw-R1, which routes rich OpenClaw trajectories into RL backends through step-level middleware [457, 627]. Agent-World pushes this further by coupling environment-task discovery with continual self-evolving training, so that internalization is no longer confined to fixed benchmarks [131]. Evaluation moves in the same direction, being less concerned with a single end-to-end score under one hand-built harness and more with whether a model natively carries the capability a harness would otherwise supply. Recent benchmarks accordingly probe cross-industry computer-use workflows [575], long-horizon tool and MCP use [322, 700], local control and multi-day collaboration [426, 764], and productive office reasoning [458]. Internalization does not make the harness disappear; it shifts the field from hand-assembling a runtime for every model toward distilling runtime experience into weights and validating transfer with harness-native benchmarks, a theme developed systematically in Section 5.

内化循环。第三阶段也是论文两部分内容共同呈现的阶段——在此阶段，最初由约束条件组织的行为，通过轨迹级的 SFT 和 RL 技术逐渐被整合到模型权重中。2024 年之后的开发工作不再追求单一的通用代理调参方

案；相反，内部化机制会追踪使用类型的差异，因为每个运行时环境都采用了不同的动作语法、反馈通道及故障模式。搜索导向的库套件通过代理式搜索强化学习（如Search-R1[261]）进行提炼；而代码库套件则通过软件工程强化学习（如SWE-RL）实现，且这些模型正越来越多地在诸如SWE-World[572, 680]等神经网络环境中进行训练；由诸如RLAnything等系统提供的终端套件，可统一整合外壳、工具使用及编码部署流程[663]；由UI-TARS-2提供的GUI套件[632]；以及由Claw-R1提供的个人运行时，该运行时通过步级中间件[457, 627]将丰富的OpenClaw轨迹路由至强化学习后端。Agent-World通过将环境–任务发现与持续自我演进的训练相结合，进一步推进了这一方向，使得内化过程不再局限于固定的基准测试[131]。评估方法也顺应这一趋势：不再过分关注基于单一手工构建的硬件套件所生成的单一端到端评分，而更侧重于模型是否已原生具备该套件原本能提供的功能。近期的基准测试相应地考察了跨行业的计算机使用工作流程[575]、长周期工具与MCP的使用[322, 700]、本地控制及多日协作[426, 764]。以及高效的办公推理能力[458]。内部化并不会使“Harness”消失：它将开发模式从为每个模型手动构建运行时，转向将运行时经验提炼为权重，并利用“Harness”原生基准测试来验证迁移效果。该主题在第5节中进行了系统阐述。

This is the co-evolution at the heart of the paper. Long-horizon capability is supplied *jointly* by an externalized harness and an internalized optimization process, with experience traces carrying capability between them. The next two chapters take up these halves in turn, treating the externalized harness in Section 4 and the internalized optimization in Section 5.

本文的核心在于协同进化机制：长时程能力由外部化的约束机制与内部化的优化过程共同提供，而经验轨迹则在二者之间传递该能力。接下来的两章将依次探讨这两部分内容：第4节讨论外部化约束，而第5节则探讨内部化优化。

4 Harness: Externalizing Long-Horizon Capability

4 驾驭机制：外化长周期能力

To analyze long-horizon capability, we start from the externalization perspective. Long-horizon capabilities need not originate in weights: before a model learns to plan, remember, or recover through training, these capacities can be *externalized*, that is, engineered into the runtime system through the harness that wraps the model at inference time and mediates every interaction it has with the world. It contains the machinery of loops, memory, tools, coordination, and oversight that extends a model into a long-horizon agent without changing what the model knows. Where Section 5 asks what training can absorb, this chapter asks what harness can supply. As shown in Table 2, we organize the harness into six components that recur across essentially every contemporary agent, ordered from the control loop outward to the systems that govern it:

为分析长周期可行性，我们从外部化视角出发。长时程能力未必源自权重：在模型通过训练学习规划、记忆或恢复之前，这些能力可被外部化。即，通过在推理时封装模型并介导其与外界所有交互的封装机制，在运行时系统中进行了工程化植入。该系统包含循环机制、内存、工具、协调与监督功能，能够将模型扩展为具备长时程能力的智能体，且无需改变模型原有的认知能力。第5节探讨的是何种培训能够吸收，而本章则探讨的是何种安全防护装置能够提供。如表2所示，我们将控制回路组织为六个基本组件——这些组件在几乎所有现代智能体中均可见，并按从控制回路向外延伸至其调控系统的顺序排列：

The Six Components of an Agent Harness
代理系统集成框架的六大组成部分

1. **Loops and Workflows** (Section 4.1): specifies how execution moves across states, including when the model is called, how actions are taken, and how control returns after observations.

1. 循环与工作流（第4.1节）：规定了执行过程如何在不同状态间转移，包括模型被调用时的时机、采取哪些操作，以及在观测结果后控制权如何返回。

2. **Context and Memory** (Section 4.2): manages the state available to the model, from the active context of the current run to persistent memory across windows, tasks, and sessions.

2. 上下文与记忆（第4.2节）：负责管理模型可访问的状态，涵盖当前运行的活动上下文以及跨窗口、任务和会话的持久化记忆。

3. **Tools, MCP, and Skills** (Section 4.3): defines how the agent accesses external capabilities, represents actions, and reuses acquired procedures.

3. 工具、MCP 与技能（第 4.3 节）：定义了智能体如何访问外部能力、表示动作以及复用已获取的流程。

4. **Orchestration** (Section 4.4): organizes work beyond a single agent loop, including task decomposition, role assignment, routing, and inter-agent coordination.

4. 编排（第4.4节）：负责组织超出单个智能体循环范围的工作，包括任务分解、角色分配、路由选择以及智能体间协调。

5. **Hooks and Middleware** (Section 4.5): provides predefined interception points in the runtime where execution can be inspected, modified, paused, or handed off.

5. 钩子与中间件（第4.5节）：在运行时提供预定义的拦截点，可在这些点上对执行过程进行检查、修改、暂停或转发。

6. **Verification** (Section 4.6): checks whether states, actions, outputs, or side effects satisfy task, safety, and quality constraints before execution continues or results are accepted.

6. 验证（第 4.6 节）：在执行继续或接受结果之前，检查状态、动作、输出或副作用是否满足任务、安全性和质量约束条件。

4.1 Loops and Workflows

4.1 循环与工作流

The innermost layer of a long-horizon agent harness is the workflow that advances execution from one state to the next. At this layer, the harness specifies the control pattern by which model calls, tool calls, state updates, checks, and feedback are sequenced. Recent workflow papers describe agent workflows as execution structures in which agents perceive state, reason, decide, act, receive feedback, and adapt under a workflow engine [777]. An example of a long-horizon run can be found in Figure 7.

长时程代理驱动程序的最内层是负责将执行流程从一个状态推进到下一个状态的工作流。在该层中，总线用于指定模型调用、工具调用、状态更新、检查及反馈等操作的执行顺序控制模式。近期的工作流论文将代理工作流描述为一种执行结构，其中代理在工作流引擎[777]的驱动下，会感知状态、进行推理、做出决策、执行操作、接收反馈并进行自适应调整。一个长周期运行示例可参见图 7。

For long-horizon agents, the central design question is how much of the trajectory is governed by this external control pattern rather than by the model’s next-token policy alone. A workflow may simply repeat an observe–act loop, maintain an explicit plan across steps, or branch over multiple candidate trajectories before committing. We therefore classify harness workflows into three structures: *linear workflows*, which advance through a single trajectory; *plan-execute workflows*, which externalize task decomposition and progress tracking; and *branching workflows*, which search over alternatives using comparison, pruning, or backtracking. 对于长时程智能体而言，核心设计问题在于：轨迹中有多少部分是由该外部控制模式主导的，而非仅由模型的“下一个标记”策略单独决定。工作流可能仅重复执行“观察–行动”循环，或在多个步骤间维持一个明确的计划，亦或在最终提交前对多种候选轨迹进行分支选择。因此，我们将约束工作流划分为三种结构：*线性工作流*——沿单一轨迹推进；*计划-执行工作流*——将任务分解与进度跟踪外部化；以及*分支式工作流*。该算法通过比较、剪枝或回溯等方法对备选方案进行搜索。

Table 2 The agent harness by component. *Harness responsibility* states what the component must supply at runtime; *representative mechanisms* lists the component’s sub-mechanisms, and *representative systems* pairs each sub-mechanism with two concrete systems on the same line.

表2：按组件划分的代理总线。 “*责任*” 部分规定了组件在运行时必须提供的内容； “*代表机制*” 部分列出了组件的子机制，而 “*代表系统*” 部分则.....将每个子机制与同一条线上的两个具体系统进行配对。

Component (\$) 组件 (\$)	Harness responsibility 勇于承担责任	Representative mechanisms 代表性作用机制	Representative systems 代表性系统
Loop & Workflow 循环与 workflow (§4.1)	Schedules model calls, action 调度模型调用、操作 execution, and observation feedback across steps 执行过程及各步骤间的观察反馈	• Linear workflows	ReAct [761], Self-Refine [416]
		• 线性工作流	ReAct[761]、Self-Refine[416]
		• Plan–execute	ReWOO [720], Arbor [262]
		• 计划–执行	ReWOO[720]、Arbor[262]
Context & Memory 上下文与记忆 (§4.2)	Bounds in-run state and persists what must survive across windows and sessions 边界运行状态，用于保存必须跨窗口和 会话持久存在的数据。	• Branching	Tree-of-Thoughts [760], LATS [875]
		• 分支	思维树法[760]、LATS[875]
		• Working context	
		• 工作环境	ReSum [693], MemAgent [778] Mem0 [88], MemoryBank [874] ExpeL [853], ReasoningBank [462]
Tools, MCP & 工具、MCP及 Skills (§4.3) 技能 (§4.3)	Interfaces model decisions with 接口用于对决策进行建模 external capabilities and reusable procedures 外部功能与可复用流程	• Factual memory	
		• 事实记忆	
		• Tool interfaces & MCP	Toolformer [523], MCP [100]
		• 工具接口与MCP	Toolformer[523]、MCP[100]
Orchestration 编排 (§4.4)	Decomposes goals, assigns roles, and coordinates work beyond a single loop 分解目标、分配角色，以及 coordinates work beyond a single loop 可协调超出单一范围的工作 loop 循环	• Active discovery	AnyTool [140], ToolGen [647] Voyager [630], SkillNet [360]
		• 主动发现	AnyTool[140]、ToolGen[647]、Voyager [630]、SkillNet[360]
		• Skill libraries	
		• 技能库	AnyTool[140]、ToolGen[647]、Voyager [630]、SkillNet[360]
Hooks & Middleware (§4.5) 钩子与中间件 (§ 4.5)	Enforces allow, block, or hand-off at predefined points on the action path 强制执行允许、阻断或切换操作于 predefined points on the action path 动作路径上的预定义点	• Decomposition	MetaGPT [219], CAMEL [314]
		• 分解	MetaGPT[219]、CAMEL[314]
		• Topologies	Magentic-One [157], LangGraph [291]
		• 拓扑结构	Magentic-One[157]、LangGraph[291]
Verification (§4.6) 验证 (§4.6)	Scores states and outputs for correctness, safety, and task fit 准确性、安全性及任务适配性 before continuing or accepting results 在继续或接受结果之前	• Routing & protocols	GPTSwarm [897], A2A [97]
		• 路由与协议	GPTSwarm[897]、A2A[97]
		• Rule-based hooks	AEGIS [783], IsolateGPT [698]
		• 基于规则的挂钩	AEGIS[783]、IsolateGPT[698]
		• User-defined policies	AgentSpec [633], ShieldAgent [80]
		• 用户自定义策略	AdaptiveGuard [754], AGrail [408]
		• Adaptive guards	AgentSpec[633]、ShieldAgent[80]、Adap- tiveGuard[754]、AGrail[408]
		• 自适应防护装置	
		• Assessment targets	SelfCheckGPT [418], VeriGuard [432]
		• 评估目标	SelfCheckGPT[418]、VeriGuard[432]
		• Verification levels	PRMs [363], LLM-as-a-Judge [867]
		• 验证等级	PRM[363]、LLM作为裁判[867]
		• Verifier strategies	Math-Shepherd [643], RLV [521]
		• 验证器策略	Math-Shepherd[643]、RLV[521]

4.1.1 Linear Workflows

4.1.1 线性工作流

Linear workflows are the simplest way to keep an agent moving: the harness repeatedly renders the current state to the model, executes the selected action, records the resulting observation, and feeds the updated state into the next call. The workflow follows one evolving trace rather than maintaining a separate plan or search tree. This makes it suitable for interactive tasks where the agent must react to fresh evidence, such as web navigation, debugging, tool use, and exploratory search. ReAct [761] is the canonical instance of this pattern, where reasoning and acting are interleaved and each observation conditions the next model invocation.

线性工作流是保持智能体持续运行的最简单方式：该流程会不断将当前状态渲染至模型，执行所选动作，记录生成的观测结果，并将更新后的状态输入至下一次调用。该工作流遵循一条动态演变的轨迹，而非维护独立的计划或搜索树。这使得该方法适用于需要智能体对新证据做出响应的交互任务，例如网页导航、调试、工具使用及探索性搜索。ReAct[761]是该模式的典型范例，其中推理与行动相互交织，且每次观测结果都会对下一次模型调用产生条件约束。

The strength of a linear workflow is responsiveness, but its weakness is that all progress depends on a single trace. Early mistakes can contaminate later steps, repeated reasoning can exhaust the context window, and the model may loop, drift, or stop prematurely. Many variants therefore wrap feedback, critique, retrieval, memory, or self-correction around the basic chain [22, 416, 544, 567]. In this sense, a bare linear workflow is only the inner loop, while the emerging practice of *loop engineering* designs the recurring outer system that automatically prompts, monitors, verifies, persists, and resumes the agent. This outer system adds external stop conditions, persistent state, verification gates, cost limits, and recovery policies, turning the workflow into an unattended long-horizon run in which a human need not decide every next prompt.

线性工作流的优势在于响应速度，但其劣势在于所有进度均依赖于单一追踪记录。早期的错误可能污染后续步骤，重复推理可能会耗尽上下文窗口，导致模型出现循环、漂移或过早终止的情况。因此，许多变体将反馈、批判、检索、记忆或自我校正机制封装在基础链式结构之中[22, 416, 544, 567]。从这个意义上说，裸线性工作流仅是内层循环，而新兴的*循环工程*实践则负责设计可重复的外层系统，该系统能够自动触发、监控、验证、持久化及恢复代理。该外部系统可添加外部终止条件、持久化状态、验证门控机制、成本限制及恢复策略，从而将工作流转化为无需人工干预的长周期运行模式——在此模式下，人类无需决定每一次后续提示。

4.1.2 Plan-Execute Workflows

4.1.2 计划–执行工作流

Plan-execute workflows address the short-sightedness of purely reactive loops by making task structure explicit. The harness first creates or maintains a plan, then uses it to guide later actions through subgoals, dependencies, progress markers, budgets, and failure conditions. Early forms ask the model to plan before solving, while more structured variants plan tool calls, decompose tasks recursively, or revise the plan during execution [482, 639, 720]. Across these systems, the plan becomes an execution object that the harness can inspect, update, resume, invalidate, or assign to different modules.

计划-执行工作流通过使任务结构显式化，有效解决了纯反应式循环的短视问题。约束机制首先会创建或维护一份计划，随后利用该计划通过子目标、依赖关系、进度标记、预算及失败条件来指导后续操作。早期形式要求模型在求解前进行规划；而更结构化的变体则会规划工具调用、递归分解任务，或在执行过程中对规划进行修订[482, 639, 720]。在这些系统中，该计划可成为一个执行对象，Harness 可对该对象进行检查、更新、恢复、失效或分配给不同的模块。

This structure is especially useful for long-horizon loops because an unattended agent cannot safely run on a vague instruction such as “improve the project”. It needs a goal that can be decomposed, monitored, and verified. A maintained plan provides this control surface: it translates an underspecified objective into concrete checkpoints and failure conditions, while allowing the model to make local decisions under persistent task commitments. Recent long-horizon systems therefore couple planning with execution monitoring, verification, replanning, or learning-based policy improvement [763, 832]. Arbor [262] extends this paradigm to autonomous research by maintaining a persistent hypothesis tree across coordinator–executor cycles, where each branch records hypotheses, experimental artifacts, evidence, and distilled insights that propagate across iterations, turning isolated attempts into a cumulative strategy-driven search.

该结构对于长周期循环尤为实用，因为无人值守的智能体无法在诸如“改进项目”这类模糊指令下安全运行：它需要一个能够被分解、监控和验证的目标。持续计划可提供此类控制机制：它能将未明确的目标转化为具体的检查点与失败条件，同时允许模型在持续的任务承诺下做出局部决策。因此，近期的长时程系统将规划与执行监控、验证、再规划或基于学习的策略改进相结合[763, 832]。Arbor[262]将这一范式扩展至自主

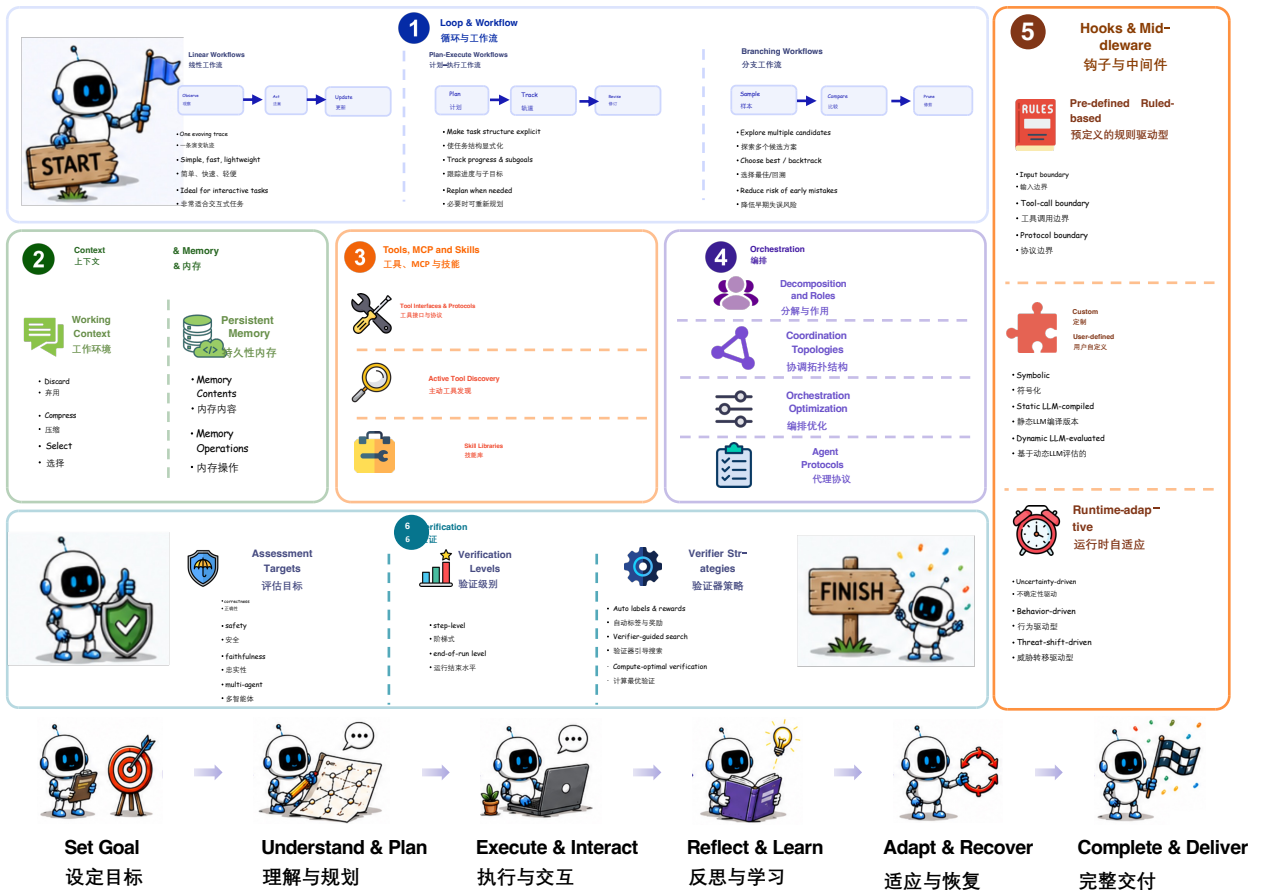


Figure 7 Overview of the agent harness. Six core harness components support the end-to-end agent lifecycle, from goal formulation and planning to execution, adaptation, verification, and delivery, enabling coherent long-horizon execution across interdependent steps.

图 7 代理套件概览。六项核心约束组件可支持端到端智能体生命周期——涵盖从目标制定与规划，到执行、自适应、验证及交付的全过程，从而在相互依赖的各步骤之间连贯的长周期执行。

研究领域，通过在协调器与执行器的循环运行过程中维护一棵持久性的假设树来实现；其中每个分支均用于记录假设、实验数据及证据。这些提炼出的洞见可在迭代过程中传播，将零散的尝试转化为一种累积的、基于策略的搜索过程。

4.1.3 Branching Workflows

4.1.3 分支工作流

Branching workflows are useful when one wrong commitment can derail the whole task. Instead of immediately following the first plausible action, plan, or answer, the harness generates several alternatives and evaluates them before deciding which one to pursue. This pattern appears in reasoning trees, tree-search-based acting, graph-based thought, code-generation search, and memory-augmented search variants [33, 91, 760, 875]. The defining feature is evaluation before commitment: the harness controls the branching factor, scoring function, pruning rule, backtracking policy, and final selection procedure [319].

分支工作流在存在单次错误承诺就可能使整个任务受阻的情况下非常实用。该系统并非直接执行第一个合理的行动、计划或答案，而是会生成若干备选方案，并对这些方案进行评估，再决定选择哪一项来执行。该模式出现于推理树、基于树搜索的行动规划、基于图结构的思维过程、代码生成搜索以及记忆增强型搜索变体中[33, 91, 760, 875]。其核心特征在于“在承诺前进行评估”：该框架负责控制分支因子、评分函数、剪枝规则、回溯策略以及最终选择流程[319]。

For long-horizon agents, branching reduces the risk of spending a large trajectory budget on a weak early decision [280]. It is also closely related to loop engineering, where unattended loops need reliable verification rather than repeated self-approval. A branching workflow can separate generation and checking: one branch proposes a patch or plan, another critiques it, a verifier tests constraints, and the harness decides whether to continue, revise, or stop. This maker-checker separation helps reduce self-evaluation bias and Goodhart-style failures, where the agent satisfies the visible completion condition while violating the user’s underlying intent. 对于长时程智能体而言，分支机制可降低在薄弱的早期决策上耗费大量轨迹预算的风险[280]。这还与环路工程密切相关——在无人值守的环路场景下，需要的是可靠的验证机制，而非重复的自我审批流程。分支化工作流可将生成与验证环节分离：一个分支提出补丁或计划，另一个分支对其进行评审，验证器负责测试约束条件，而集成测试框架则决定是否继续、修订或终止流程。这种“制造者-检验者分离”机制有助于减少自我评估偏差及“Goodhart式失败”——即智能体虽满足可见完成条件，却违背了用户的深层意图。

Branching is most useful when partial states can be evaluated cheaply and meaningfully. It is less attractive when feedback is weak, delayed, or expensive, because search can multiply token and tool costs, create incompatible partial worlds, and make provenance harder to audit [515]. Thus, branching workflows are best viewed as selective search mechanisms invoked when the cost of a bad commitment is higher than the cost of evaluating alternatives [343]. Across linear, plan-execute, and branching workflows, the common requirement is that the harness must maintain the state on which these workflows operate. We therefore turn next to the

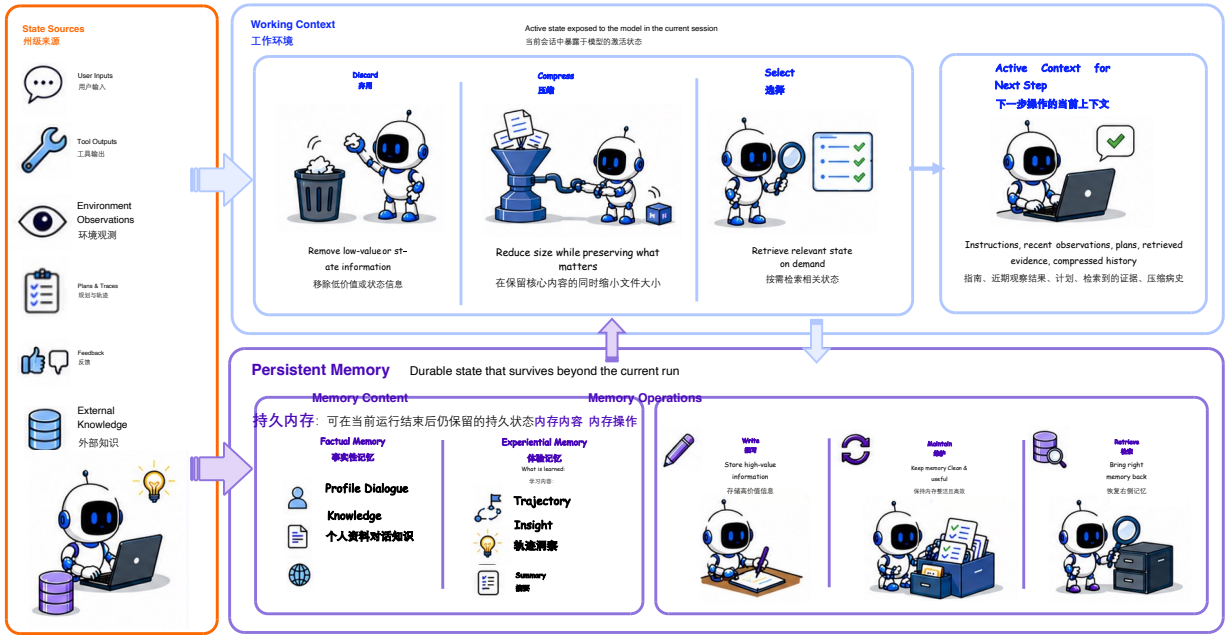


Figure 8 Context and memory in an agent harness. The harness governs the state interface between the workflow and the model: it selects, compresses, or discards active context under a token budget, and reads from and writes to persistent memory that outlives a single window, task, or session.

图8：智能体框架中的上下文与记忆。约束机制负责管理工作流与模型之间的状态接口：它可在令牌预算范围内选择、压缩或丢弃活跃上下文，并可从以及向一个独立于单个窗口、任务或会话的持久存储中进行读写操作。

state machinery that supports long-horizon execution.

分支结构在部分状态能够以较低成本且具有意义地进行评估时最为实用。当反馈较弱、延迟或成本较高时，该方案的吸引力会降低，因为搜索可能加剧令牌与工具的成本，引发不兼容的部分世界，并使溯源审计变得更加困难[515]。因此，分支式工作流最好被视为一种选择性搜索机制——当做出错误承诺所涉及的成本高于评估其他备选方案所涉及的成本时，便会触发该机制[343]。在线性、计划执行及分支型工作流中，一个共同的要求是：Harness必须能够维护这些工作流运行时的状态。因此，我们接下来将探讨支持长周期执行的状态机制。

4.2 Context and Memory

4.2 语境与记忆

If workflows define how an agent proceeds, context and memory define what state it proceeds with [228]. For long-horizon agents, the harness cannot simply pass along an ever-growing transcript. It must decide what the next model call can see, what should be compressed or discarded when the context window fills, and what information should persist beyond the current window, task, or session [876]. In this sense, context and memory are the state interface between the workflow and the model: the harness reads from prior observations, tool outputs, plans, traces and stored knowledge, then writes the selected state back into the next step.

若工作流定义了智能体的执行流程，则上下文与记忆则决定了智能体基于何种状态继续执行[228]。对于长时程智能体而言，运行环境无法简单地传递一个不断增长的对话记录。该模块必须决定：下一个模型调用可访问哪些信息、当上下文窗口填满时应压缩或丢弃哪些信息，以及哪些信息应在当前窗口、任务或会话之外持续保留[876]。从这个意义上说，上下文与记忆是工作流与模型之间的状态接口：该框架会读取先前的观测结果、工具输出、计划、追踪记录及存储知识，随后将所选状态写回下一步骤。

We distinguish context and memory by their time horizon and role at runtime. *Context* is the active state exposed to the model in the current run, including instructions, recent observations, intermediate reasoning, retrieved evidence, and compressed history. *Memory* is the durable state that survives beyond the immediate window and can be retrieved or updated over time. We organize this component accordingly. Section 4.2.1 covers working state, namely the in-window or near-window state maintained within a run; Section 4.2.2 covers persistent state, namely the durable memory that survives across windows, tasks, and sessions. In Table 3, we list representative works for each sub-section. The overview of this section is shown in Figure 8. 我们根据上下文与记忆在运行时的时域范围及作用来对其进行区分。上下文是指当前运行过程中模型所接收到的活跃状态，包括指令、近期观测结果、中间推理过程、检索到的证据以及压缩后的历史记录。内存是指一种持久状态，它能够超越当前运行窗口而存续，并可随时间推移被检索或更新。我们据此对这一组件进行组织。第4.2.1节涵盖工作状态，即在单次运行过程中维持的“窗口内”或“近窗口”状态；第4.2.2节涵盖持久状态，即跨窗口、任务和会话均持续存在的持久化内存。在表3中，我们列出了各子章节的代表性研究工作；本节的概览如图8所示。

4.2.1 Working Context

4.2.1 工作环境

Every step of an agent run produces new state: model outputs, tool returns, environment observations, partial plans, and intermediate artifacts. Runtime context maintenance is the harness process that decides which of these states should condition the next model call. Some work refers to this in-window or near-window state as the agent’s *working memory*, emphasizing its role as a limited workspace for keeping task-relevant information available within a single episode [228]. For long-horizon agents, the central challenge is not only that the context window is finite, but that useful, stale, and distracting information accumulate together over time.

智能体运行的每一步都会产生新的状态：模型输出、工具返回值、环境观测结果、部分规划方案以及中间工件。运行时上下文维护是控制流程，用于决定哪些状态应作为下一次模型调用的条件。部分研究将这种“窗口内”或“近窗口”状态称为智能体的*工作记忆*，并强调其作为在单个任务回合内保持任务相关信息可访问的有限工作空间的作用[228]。对于长时程智能体而言，其核心挑战不仅在于上下文窗口是有限的，更在于有用信息、陈旧信息及干扰性信息会随时间推移而不断累积。

Too much context can degrade performance through *context rot* and *lost-in-the-middle* effects, while too little context removes constraints, evidence, or partial progress that the agent still needs [378, 511, 541]. The harness must therefore turn an expanding trajectory into a bounded, high-signal workspace. In practice, long-running agents combine three operations: *discard*, *compress*, and *select* [13, 14, 293].

过多的上下文信息可能会因上下文衰减和中间迷失效应而降低系统性能；而过少的上下文信息则会消除智能体仍需依赖的约束条件、证据或部分进展[378, 511, 541]。因此，该控制回路必须将扩张型运动轨迹转化为一个有界、高信号强度的工作空间。在实践中，长时代理结合了三种操作：丢弃、压缩和选择[13, 14, 293]。

Context Discard. Discarding removes information from the active window [113]. Common policies include resetting context at task boundaries, dropping bulky tool responses after they have been parsed or summarized, and keeping only the most recent turns [597]. For short interactions, discard mainly saves tokens; for long-horizon runs, it is also a reliability mechanism. Stale observations, obsolete plans, verbose logs, and irrelevant tool outputs can continue to bias later decisions if they remain visible.

上下文弃用。弃用操作可从活动窗口中移除信息[113]。常见的策略包括：在任务边界处重置上下文、在已解析或摘要后的冗长工具响应后将其丢弃，以及仅保留最新的对话轮次[597]。对于短时交互，“丢弃”操作主要可节省令牌；而对于长周期运行而言，该操作亦可作为一种可靠性保障机制。若过时的观测结果、陈旧的计划、冗长的日志或无关的工具输出仍可见，则这些内容仍可能对后续决策产生偏差。

Discard can also happen before information enters the trajectory. Tool or schema pruning, for example, prevents rarely useful capabilities or verbose specifications from occupying the prompt in the first place [27, 622]. This matters because noise compounds over long runs: a small amount of irrelevant state repeated across many steps can become a first-order source of drift. Discard is therefore the most direct harness-level way to enforce forgetting, rather than relying on the model to ignore low-value context.

“丢弃”操作也可能在信息进入轨迹之前就发生；例如，工具或模式剪枝可从源头上防止那些罕有用处的功能或冗长的规范占用提示符[27, 622]。这一点至关重要，因为噪声在长时间运行过程中会累积：少量无关状态在多个步骤中重复出现，就可能成为漂移的首要来源。因此，弃用是强制遗忘的最直接的硬件层面方法，而非依赖模型忽略低价值上下文。

Context Compression. Compression preserves task-relevant information while reducing its token and attention footprint. A useful operational distinction is between compressing the current payload and compressing the accumulated trajectory. The first case handles large immediate inputs, such as retrieved passages, tool outputs, web pages, logs, or DOM trees [57, 254, 773]. In agent harnesses, similar operations selectively rewrite verbose observations into compact, task-oriented state descriptions.

上下文压缩。压缩能够在减少模型的词元和注意力计算量的同时，保留任务相关的信息。一个有效的操作区分在于：可选择对当前负载进行压缩，亦可选择对累积轨迹进行压缩。第一个案例处理大型即时输入数据，例如检索到的段落、工具输出结果、网页、日志或 DOM 树[57, 254, 773]。在智能体框架中，类似的操作可将冗长的观测结果选择性地重写为简洁且以任务为导向的状态描述。

The second case is trajectory consolidation: replacing a growing interaction history with a maintained state. This state may be a summary, a list of open constraints, a table of facts, a plan with completion markers, or a graph of entities and dependencies. For long-horizon agents, the goal is not just to shorten the transcript, but to preserve what still matters for future action: completed decisions, remaining subgoals, unresolved failures, and produced artifacts. Recent methods explore learned, task-aware, budget-aware, and hierarchical forms of such compression [224, 268, 573, 693, 696, 768, 778, 824, 890]. This also marks a shift from hand-written context rules toward learned policies for when and how to fold state.

第二种情况是轨迹整合：用维持状态来替换不断增长的交互历史。该状态可以是摘要、开放约束列表、事实表、带有完成标记的计划，或实体及其依赖关系的图。对于长时程智能体而言，其目标不仅在于缩短转录文本，更在于保留对后续行动仍至关重要的内容：已完成的决策、尚待达成的子目标、未解决的失败案例以及已生成的产物。近期研究探索了此类压缩的多种方法，包括基于学习的方法、任务感知方法、预算感知方法以及分层方法[224, 268, 573, 693, 696, 768, 778, 824, 890]。这也标志着从手动编写上下文规则向采用学习策略来决定何时以及如何折叠状态的转变。

Context Selection. Selection keeps richer state outside the active window and retrieves only what is relevant to the current decision. The indexed state may include documents, events, artifacts, plans, tool outputs, or previous observations. Unlike ordinary RAG, selection in an agent harness often retrieves from the agent’s own trajectory, not only from an external knowledge base [202, 669]. Typically, CL-bench is designed to target and evaluate a model’s long-context comprehension and selection capabilities [133]. The goal is to keep the current context small while preserving access to the evidence trail.

上下文选择。选择机制可将更丰富的状态保留在活动窗口之外，并仅检索与当前决策相关的信息。索引状态可包含文档、事件、工件、计划、工具输出或先前观测结果。与普通的RAG不同，智能体框架中的选择机制通常会从智能体自身的轨迹中检索信息，而不仅限于从外部知识库[202, 669]中检索。通常，CL-bench的设计旨在针对模型的长上下文理解与选择能力进行目标定位与评估[133]。其目标是在保持当前上下文规模较小的同时，仍能访问证据链。

From the harness perspective, selection involves four choices: when to retrieve, how to form the query, which store to search, and how to filter or aggregate the results before injection. Retrieving too often increases cost and noise; retrieving too rarely risks losing dependencies that were established many steps earlier. Long-running agents therefore usually combine selection with discard and compression: bulky outputs are removed, completed subtasks are folded, and relevant past state is retrieved on demand [13, 230, 231, 824, 846]. These operations keep the active context bounded without forcing the agent to forget the full task history.

从数据源管理视角来看，选择过程涉及四个选项：何时进行检索、如何构建查询语句、搜索哪个存储库，以及在注入前如何对结果进行过滤或聚合。检索过于频繁会增加成本和系统开销；而检索频率过低则存在丢失此前已建立的依赖关系的风险。因此，长时代理通常会将选择、丢弃与压缩相结合：剔除冗余的输出结果，合并已完成的子任务，并按需检索相关的过往状态[13, 230, 231, 824, 846]。这些操作可使当前活动上下文保持受限状态，而无需强制智能体遗忘完整的任务历史。

4.2.2 Persistent Memory

4.2.2 持久内存

Working context keeps a single run usable; persistent memory lets the harness carry state beyond the current window, trajectory, or session [228, 583, 694, 848]. This is crucial for long-horizon agents because useful information often outlives the step in which it was produced. A failed attempt may become a lesson for later recovery, a completed subtask may define constraints for downstream actions, and a discovered procedure may be reused in future tasks. Without persistent memory, the agent must repeatedly rediscover the same facts, strategies, and skills as the context window turns over. We use a simple harness-oriented view. *Memory Contents* asks what persistent state is stored; *Memory Operations* asks how that state is written, maintained, retrieved, and evaluated.

工作环境可使单次运行保持可用性；持久化内存则允许该框架将状态保存至当前窗口、轨迹或会话之外[228, 583, 694, 848]。这对于长时程智能体至关重要，因为有用信息的效用往往持续超过其生成所处的时间步。失败的尝试可成为后续恢复的教训；已完成的子任务可为下游操作设定约束条件；而发现的流程则可在未来任务中复用。若缺乏持久记忆，智能体在上下文窗口不断切换时，必须反复重新发现相同的事实、策略与技能。我们采用一种基于框架的简单视角。*内存内容*用于询问存储了何种持久状态；*内存操作*则用于询问该状态的写入、维护、检索及评估方式。

Table 3 Representative context and memory systems in the agent harness. Rows follow the split of this subsection: *working context* (Section 4.2.1) is bounded within a single run, while *persistent memory* (Section 4.2.2) survives across windows, tasks, and sessions. **Type** names what the method does with state. **State form** is the representation it maintains. **Train-free** (✔) marks methods that plug into a frozen model as pure harness machinery, versus (✗) those whose policy must be trained or RL-tuned. **Trigger** is when the operation fires. **Key mechanism** briefly describes the core operation.

表3：智能体框架中的代表性上下文与记忆系统。各行内容对应本小节的划分方式：*工作上下文*（第4.2.1节）。) 仅在单次运行期间有效，而*持久化内存*（第 4.2.2 节）可在不同窗口、任务和会话之间持续存在。**类型**描述了该方法对状态的处理方式。**状态形式**是指其所维护的表示方式。**无训练** (✔) 表示那些可直接插入手头冻结模型作为纯硬件组件的方法，而 (✗) 则表示那些其策略必须经过训练或强化学习调优的方法。**触发**是指操作被激活的时刻。**关键机制**简要描述了核心操作流程。

Method 方法	Type 类型	State form 表格形式	Train-free 无列车运行	Trigger 触发器	Key mechanism 关键机制
Working Context 工作环境					
Effective Harnesses [14]	Discard	Turns	✔	Boundary	Reset context, drop parsed tool output
有效的安全带[14]	弃置	转数	✔	边界	重置上下文，清除已解析的工具输出
Deep Agents [293]	Discard	Files	✔	Boundary	Offload state to files
深度智能体[293]	弃置	文件	✔	边界	将卸载状态写入文件
Tool pruning [622]	Discard	Schemas	✔	Start-of-run	Prune tools/schemas before prompt
工具修剪[622]	弃置	模式	✔	运行开始	在提示前优化工具/模式
LLMLingua [254]	Compress	Summary	✗	Per-payload	Token-level prompt compression
	压缩	摘要	✗	每载荷	令牌级提示压缩
CompAct [773]	Compress	Summary	✗	Per-payload	Actively compress retrieved docs
	压缩	摘要	✗	每载荷	积极压缩检索到的文档
ReSum [693]	Compress	Summary	✔	Boundary	Learned trajectory summarization
简历[693]	压缩	摘要	✔	边界	学习轨迹摘要
MEM1 [890]	Compress	Summary	✗	Per-step	RL fuses memory and reasoning
	压缩	摘要	✗	逐步	强化学习融合了记忆与推理能力
MemAgent [778]	Compress	Summary	✗	Per-step	RL over fixed-size memory
	压缩	摘要	✗	逐步	固定大小内存上的强化学习
ACON [268]	Compress	Summary	✔	Boundary	Optimizes context-compression policy
	压缩	摘要	✔	边界	优化上下文压缩策略
ContextBudget [696]	Compress	Summary	✗	Per-step	Budget-aware context allocation
上下文预算[696]	压缩	摘要	✗	逐步	预算感知上下文分配
HiAgent [224]	Compress	Plan	✔	Boundary	Subgoal-scoped working memory
	压缩	计划	✔	边界	子目标范围工作记忆
Context-Folding [573]	Compress	Summary	✗	Per-step	Folds history under a budget
上下文折叠[573]	压缩	摘要	✗	逐步	在预算范围内梳理历史记录
AgentFold [768]	Compress	Summary	✗	Per-step	Proactive history folding
	压缩	摘要	✗	逐步	主动历史折叠
ACE [824]	Select	NL-note	✔	On-demand	Retrieves, evolves context entries
	选择	NL-注释	✔	按需服务	检索并更新上下文条目
LEGOMem [202]	Select	Summary	✔	On-demand	Modular per-subtask procedural memory
	选择	摘要	✔	按需	按子任务划分的模块化程序性记忆
Memex(RL) [669]	Select	Vector	✗	On-demand	RL over indexed experience memory
	选择	向量	✗	按需服务	基于索引经验记忆的强化学习
Memory-as-Action [846]	Mixed	Summary	✗	Per-step	Memory read/write as actions
记忆即行动[846]	混合	摘要	✗	逐步	内存读/写操作
Persistent Memory 持久内存					
CLAUDE.md [10]	Factual	NL-note	✔	Start-of-run	Project-rule file loaded per run
	事实性	NL注释	✔	运行开始	每次运行时均加载项目规则文件
Cursor Rules [104]	Factual	NL-note	✔	Start-of-run	Persistent project rules per run
光标规则[104]	事实性	NL注释	✔	运行开始	每次运行时的持久项目规则
AGENTS.md [98]	Factual	NL-note	✔	Start-of-run	Portable project-knowledge spec
	事实性	荷兰语注释	✔	运行开始	便携式项目知识规范
MemoryBank [874]	Factual	Vector	✔	On-demand	Store with forgetting-curve updates
	事实性	向量	✔	按需服务	支持遗忘曲线更新的存储系统
Mem0 [88]	Factual	Graph	✔	On-demand	Linked long-term fact store
	事实性	图表	✔	按需	关联长期事实存储库
MemoryOS [267]	Factual	Mixed	✔	On-demand	OS-style tiered memory
	事实性	混合	✔	按需	OS级分层内存
HippoRAG [199]	Factual	Graph	✔	On-demand	Graph memory for associative recall
	事实性	图表	✔	按需	用于联想回忆的图式记忆
A-MEM [731]	Factual	Graph	✔	On-demand	Self-organizing memory notes
	事实性	图表	✔	按需	自组织记忆笔记
Reflexion [544]	Experiential	NL-note	✔	After-failure	Verbal lessons from failures
反思[544]	体验式	NL-注释	✔	故障后	从失败中汲取的言语启示
ExpeL [853]	Experiential	NL-note	✔	On-demand	Reuses insights from trajectories
	体验式	NL-注释	✔	按需	复用轨迹分析所得的洞察力
Synapse [868]	Experiential	Summary	✔	On-demand	Trajectory-as-exemplar prompting
突触[868]	体验式	摘要	✔	按需服务	轨迹作为范例提示
SAGE [359]	Experiential	NL-note	✔	On-demand	Reflective memory-augmented evolution
	体验式	NL-注释	✔	按需服务	反思式记忆增强进化
Buffer of Thoughts [750]	Experiential	NL-note	✔	On-demand	Reusable thought templates
思维缓冲区[750]	体验式	NL-注释	✔	按需	可复用思维模板
ReasoningBank [462]	Experiential	NL-note	✔	On-demand	Distills reusable reasoning strategies
推理银行[462]	体验式	NL-注释	✔	按需服务	提炼可复用的推理策略
AWM [674]	Experiential	Code-skill	✔	Boundary	Induces workflows from traces
	体验式	代码技能	✔	边界	根据轨迹生成工作流
Agent KB [587]	Experiential	NL-note	✔	On-demand	Cross-domain experience knowledge base
KB代理[587]	体验式	NL-注释	✔	按需服务	跨领域经验知识库
Voyager [630]	Experiential	Code-skill	✔	On-demand	Reusable executable skill library
旅行者号[630]	体验式	代码技能	✔	按需服务	可重复使用的可执行技能库
MemGPT [464]	Mixed	Mixed	✔	On-demand	OS-style context/memory paging
	混合	混合	✔	按需服务	OS风格的上下文/内存分页
Generative Agents [470]	Mixed	Summary	✔	On-demand	Memory stream with reflection
生成式智能体[470]	混合	摘要	✔	按需服务	带反射功能的内存流

Memory Contents. Persistent memory mainly stores two kinds of state. *Factual memory* stores declarative information that the agent should not have to rediscover: user preferences, project rules, commitments, environment state, document or codebase facts, and the capabilities of external systems [228]. For a long-horizon harness, these facts support continuity: user-level facts keep behavior consistent across sessions, while environment-level facts prevent the agent from repeatedly re-inferring stable constraints. Practical examples include project-rule files such as `CLAUDE.md`, `Cursor Rules`, and `AGENTS.md`, which provide stable project **内存内容**。持久性内存主要用于存储两种类型的状态。*事实性记忆* 存储着代理无需重新发现的陈述性信息：用户偏好、项目规则、承诺、环境状态、文档或代码库中的事实，以及外部系统的功能[228]对于长时程策略套件而言，这些事实支持连续性原则：用户层面的事实可确保行为在不同会话间保持一致，而环境层面的事实则能防止智能体反复重新推断稳定的约束条件。实际示例包括项目规则文件，如`CLAUDE.md`、`Cursor Rules` 以及`AGENTS.md`，这些文件可为项目提供稳定的配置。

knowledge outside the immediate prompt [10, 98, 104]. User memory systems and graph-structured stores further extend this idea by maintaining profiles, preferences, and linked factual records that can be consolidated and audited over time [88, 199, 229, 267, 507, 731, 874].

非即时提示范围内的知识[10, 98, 104]。用户内存系统与图结构存储系统进一步拓展了这一理念，能够维护用户画像、偏好及关联的事实记录，这些记录可随时推移进行整合与审计[88, 199, 229, 267, 507, 731, 874]。

Experiential memory stores what the agent has learned from prior trajectories. It may contain failure lessons, successful cases, state-action exemplars, reusable reasoning patterns, plans, workflows, or executable procedures. The common purpose is to reduce long-horizon cost by replacing repeated rediscovery with retrieval of prior structure. Some systems store verbal reflections or lessons from past failures; others maintain experience pools, reusable reasoning templates, workflow memories, or cross-domain solution paths [44, 262, 359, 462, 544, 587, 674, 750, 807, 853, 868]. At the most operational end, experiential memory can be distilled into skills, scripts, API wrappers, or tool-like procedures, as in skill-library agents such as Voyager [446, 630]. Once stored in this form, memory begins to overlap with the tool layer: the harness can retrieve, version, validate, reuse, or retire a learned procedure without changing model weights.

体验性记忆 存储了智能体从先前轨迹中所习得的知识。它可能包含失败经验、成功案例、状态-动作范例、可复用的推理模式、计划、工作流或可执行的操作流程。其共同目标是通过将重复的重新发现过程替换为对先前结构的检索，从而降低长时程成本。某些系统会存储对过往失败的口头反思或教训；而其他系统则会维护经验库、可复用的推理模板、工作流记忆或跨领域解决方案路径[44, 262, 359, 462, 544, 587, 674, 750, 807, 853, 868]。在最底层的操作层面，体验式记忆可提炼为技能、脚本、API封装器或类工具的流程，例如Voyager这类技能库智能体[446, 630]。一旦以此形式存储，内存便开始与工具层发生重叠：该套件可检索、版本管理、验证、复用或废弃已学习的流程，而无需修改模型权重。

Memory Operations. A persistent store is useful only if the harness manages how memory is written, maintained, and read back into context. *Memory writing* decides what should be saved from a long run. Tool outputs, partial plans, reasoning traces, user feedback, observations, and intermediate artifacts are often too noisy to store as-is. The harness therefore extracts higher-value units such as facts, summaries, cases, lessons, workflows, or skills, using summarization, distillation, or structured construction [221, 407, 464, 470, 674]. The key question is what level of experience should survive: an event, a workflow, or an executable procedure. **内存操作**。只有当系统能够管理内存的写入、维护及回写至上下文时，持久化存储才具有实际意义。*内存写入操作*用于决定在长时间运行过程中应保存哪些数据。工具输出结果、部分计划、推理轨迹、用户反馈、观测数据及中间工件通常噪声过大，不宜直接存储。因此，该工具能够通过摘要、提炼或结构化构建[221, 407, 464, 470, 674]，提取出更具价值的单元，例如事实、摘要、案例、经验教训、工作流程或技能。关键问题在于：应保留何种程度的体验——是事件、工作流还是可执行流程？

Memory maintenance keeps the store usable as new information arrives. Over long horizons, memories can become stale, repeated, or inconsistent. The harness must merge related entries, update facts when users or environments change, and remove outdated or low-value material before retrieval becomes noisy. Existing systems use memory streams, graphs, linked notes, or other structured stores to support this process [199, 470, 731]. The model may suggest what to remember, but the harness decides whether a memory is inserted, merged, linked, archived, or forgotten.

*记忆维护*可确保存储系统在新信息不断输入的情况下仍能保持可用状态；而在长周期内，记忆可能会变得陈旧、重复或不一致。检索系统必须合并相关条目、在用户或环境变更时更新事实数据，并在检索过程产生噪声之前，移除过时或低价值的材料。现有系统采用内存流、图结构、链接笔记或其他结构化存储方式来支持这一流程[199, 470, 731]。该模型可提示应记住的内容，但由“安全带”决定某项记忆是应被插入、合并、链接、归档还是遗忘。

Memory retrieval decides how stored state re-enters the active context. Some memories are loaded at the start of a run, such as project rules or user preferences [664]; others are retrieved at subtask boundaries, after failures, before tool use, or when the model asks for them. Retrieval requires choices about the query, the store to search, the ranking or filtering method, and how much content to inject [488, 568, 610, 694]. Too little retrieval makes the agent repeat mistakes or miss long-range dependencies; too much retrieval turns memory back into context overload. Recent work therefore treats memory read/write as a policy problem, learning when to remember, update, or retrieve during inference [778, 846].

*记忆提取*决定了存储状态如何重新进入活动上下文。部分记忆在运行开始时即被载入，例如项目规则或用户偏好[664]；而其他记忆则在子任务边界、失败后、使用工具之前或当模型需要时进行检索。检索过程需对查询条件、待检索存储库、排序或过滤方法以及注入的内容量进行选择[488, 568, 610, 694]。检索量过少会导致智能体重复犯错或遗漏长程依赖关系；而检索量过大则会使记忆演变为上下文信息过载。因此，近期的研究将内存读写视为一个策略问题，旨在学习在推理过程中何时进行记忆、更新或检索[778, 846]。

Persistent memory is also hard to evaluate. End-task success often mixes memory quality with model strength, tool reliability, and benchmark variance. New benchmarks test long-term recall, cross-session consistency, and tasks that depend on earlier sessions, but recent analyses still find saturation, judge sensitivity, and limited coverage of update cost and memory maintenance [235, 253, 686]. For long-horizon harnesses, the key question is not whether an agent can recall an isolated fact, but whether it can save, update, retrieve, and inject the right memory when it changes the next action. We collect evaluation resources in Section 6.6.

持久性记忆也难以准确评估。任务完成后的成功率往往综合反映了记忆质量、模型性能、工具可靠性以及基准测试的变异性等因素。新的基准测试评估了长期回忆能力、跨会话的一致性以及依赖早期会话的任务，但近期分析仍发现存在饱和现象、判断敏感度不足，且对更新成本与记忆维护的覆盖范围有限[235, 253, 686]。对于长时程强化学习框架而言，关键问题并非智能体能否回忆起孤立的事实，而是当其改变下一次动作时，是否能够保存、更新、检索并注入正确的记忆。我们在第6.6节中收集评估资源。

4.3 Tools, MCP, and Skills

4.3 工具、MCP与技能

Where context and memory settle what the agent *knows*, this component settles what it can *do*. Without tools that return observations, even a capable model can only emit text; tools are what let it query a search engine, run code, or read and write a database. We organize this capability interface in three parts: the tools exposed by the runtime and the protocol layer, most prominently the Model Context Protocol (MCP), that standardizes them (Section 4.3.1); the discovery mechanisms that keep a large tool surface usable (Section 4.3.2); and the skills an agent accumulates on top of them (Section 4.3.3). This interface matters more as the horizon grows, because extended execution becomes a long chain of interdependent calls whose state must be carried, verified, and recovered across steps.

当上下文与记忆决定了智能体*所知*时，该组件则决定了其能够*执行*的操作。若没有能够返回观测结果的工具，即使是一个功能强大的模型也只能生成文本；而这些工具正是让模型能够查询搜索引擎、运行代码或读写数据库的关键。我们将此能力接口划分为三个部分：由运行时和协议层提供的工具，其中最突出的是模型上下文协议（MCP），该协议对这些工具进行了标准化（第4.3.1节）。)；确保大型工具界面始终可用的发

现机制（第 4.3.2 节）；以及智能体在此基础上积累的技能（第 4.3.3 节）。随着程序规模的扩大，该接口的重要性愈发凸显，因为扩展执行会演变为一條由相互依赖的调用组成的长链，其状态必须在各个步骤间进行传递、验证与恢复。

4.3.1 Tool Interfaces and Protocols

4.3.1 工具接口与协议

Function Calling. Tool interfaces let a language model act beyond its parametric knowledge. Foundational work established when, what, and how to call: Toolformer self-supervises where API calls should be inserted, Gorilla maps requests onto large API surfaces while suppressing hallucinated calls, and ToolLLM scales instruction-tuned tool use to thousands of real-world REST APIs [471, 492, 523]. Papers consolidate this line into a workflow of task planning, tool selection, tool calling, and response generation within a controller–tools–environment view [497, 724]. Single-call competence is by now well covered by mature benchmarks (BFCL, StableToolBench) and data-synthesis pipelines (APIGen, ToolACE) [198, 385, 392, 472].

函数调用。 工具接口可使语言模型发挥其超出参数化知识范围的功能。基础工作涵盖了以下方面：确定何时、如何以及如何命名；Toolformer能够自监督地识别应在何处插入API调用；Gorilla则将请求映射至大型API接口，并能抑制幻觉化调用。ToolLLM可将指令微调的工具使用能力扩展至数千个现实世界的REST API [471, 492, 523]。相关论文将这一思路整合为一个 workflow，涵盖任务规划、工具选择、工具调用及响应生成等环节，该流程运行于“控制器–工具”环境视图之中[497, 724]。单调函数生成能力目前已由成熟的基准测试（BFCL、StableToolBench）和数据综合流水线（APIGen、ToolACE）[198, 385, 392, 472]充分覆盖。

Tool Protocols and MCP. The dominant engineering response to a growing, heterogeneous tool surface is protocolization. The Model Context Protocol (MCP) defines a host–client–server lifecycle with capability negotiation, authorization, and tool annotations, turning ad hoc integrations into a uniform, discoverable layer; its ecosystem, architecture, and security are covered in dedicated and broader protocol papers [100, 758]. Because it lets agents compose independently maintained servers and exposes the tool space uniformly, MCP also sets up the scaling techniques of Section 4.3.2. MCP-grounded benchmarks such as MCP-Universe reveal what single-call leaderboards miss: once tasks span many servers and tools in stateful sandboxes, even the strongest models succeed on only a minority of them, with input tokens rising rapidly over steps and failures becoming increasingly cognitive rather than syntactic [31, 350, 411, 700]. Beyond MCP, a broader wave of tool-use benchmarks probes schema discipline, state mutation, conversational tool legality, and messy real-world user behavior, from constrained tool-legality suites such as τ -bench to long-horizon, cross-application, and in-the-wild benchmarks such as Toolathlon [213, 322, 399, 539, 548, 762, 779].

工具协议与 MCP。 面对日益复杂且异构的工具表面，主流的工程应对策略是采用协议化方案。模型上下文协议（MCP）定义了主机–客户端–服务器生命周期，支持功能协商、授权及工具注解机制，可将临时集成转化为统一且可发现的层；其生态系统、架构及安全性均在专门及综合性协议文档中有所阐述[100, 758]。由于MCP能够支持代理独立构建维护的服务器，并能统一地暴露工具空间，因此它还配置了第4.3.2节所述的扩展技术。诸如MCP-Universe等MCP接地基准测试能够揭示单次调用排行榜所无法捕捉的缺陷：当任务涉及多个服务器及状态保持型沙盒中的工具时，即便是最强大的模型，在其中也仅能在少数任务上取得成功。输入标记在步骤间快速增长，而错误则日益呈现认知性质而非句法性质[31, 350, 411, 700]。除MCP之外，更广泛的工具使用基准测试正致力于探究模式规约、状态突变、对话工具的合法性以及复杂的现实用户行为——其范围涵盖了诸如 等受限的工具合法性测试套件。涵盖从基准测试到长周期、跨应用以及真实场景下的基准测试，例如Toolathlon[213, 322, 399, 539, 548, 762, 779]。

Multi-turn Tool Use. Function calling and MCP standardize the *interface* of an individual call, but tool use at scale is inherently multi-turn: an agent chains many dependent calls, each conditioned on the previous observation. As these sequences grow to tens or hundreds of invocations, the bottleneck migrates from isolated-call correctness to sustaining the whole chain [724], and even strong agents complete only a fraction of realistic tasks, degrading through context saturation and memory decay as the turn count rises [322, 325, 406]. The limiting constraint is increasingly the state and context accumulated across turns rather than the mechanics of any single call. This is a property of the execution process rather than the call interface; the control-flow responses to it, namely decomposing, scheduling, and coordinating many turns, are orchestration, which we treat separately, so this subsection stays on the tool surface itself.

多轮工具使用。 函数调用与MCP可对单次调用的接口进行标准化，但大规模工具使用本质上是多轮交互过程：智能体会链式调用多个依赖性调用，且每个调用均基于前一次观察结果而触发。随着这些序列的调用次数增长至数十次或数百次，瓶颈问题将从“孤立调用的正确性”转向“维持整个链路”的[724]。甚至强效智能体也仅能完成部分现实任务；随着对话轮次的增加，其性能会因上下文饱和和记忆衰减而下降[322, 325, 406]。限制性约束日益取决于跨会话累积的状态与语境，而非任何单次调用的机制。这是执行过程的特性，而非调用接口的特性；针对该过程的控制流响应——即分解、调度以及协调多个轮次——属于编排过程，我们将在后文对此进行单独讨论。因此，本小节内容适用于工具表面本身。

4.3.2 Active Tool Discovery

4.3.2 主动工具发现

A fixed, prompt-resident tool list does not scale. Once catalogs reach thousands of tools, their descriptions crowd out the task, token cost tracks registry size rather than step demand, and selection accuracy degrades; the tool-retrieval saturation diagnosed above motivates active discovery [350, 411]. The remedy is to stop placing the entire surface in context and instead let the agent actively discover the few tools each step needs. 固定的、即时生效的工具列表无法实现横向扩展。当目录中的工具数量达到数千个时，其描述会挤占检索任务的可用空间；令牌成本会随注册表规模而非步骤需求而变化；且选择准确率会下降；上述所诊断出的工具检索饱和现象促使了主动发现机制的出现[350, 411]。解决方案是：停止将整个表面置于上下文中，转而让智能体主动发现每一步所需的少数工具。

Tool Retrieval. The first move is retrieval-based discovery: index tool metadata and inject only the top-ranked schemas at each step. RAG-MCP, AnyTool, and MCP-Zero develop this idea with plain, hierarchical, and agent-initiated retrievers respectively, sharply reducing token use on large toolsets [140, 153, 162]. A complementary, supply-side approach generates MCP servers directly from OpenAPI specifications, and Anthropic’s Tool Search Tool demonstrates the same retrieval principle in production, scaling to thousands of tools with large token savings and higher tool-selection accuracy [9, 423]. Pushing this further, DeepAgent makes tool search an action inside a single reasoning process, dynamically discovering tools from registries of tens of thousands as the task unfolds, while ToolGen removes the separate retriever altogether by representing each tool as a vocabulary token the model emits during generation [338, 647].

工具检索。 第一步是基于检索的发现过程：对工具元数据进行索引，并在每一步中仅注入排名靠前的模式。RAG-MCP、AnyTool 和 MCP-Zero 分别采用简单的检索器、分层式检索器及智能体发起的检索器来实现这一理念，从而显著降低了大型工具集中的令牌使用量[140, 153, 162]。一种互补的供给侧方法可直接根据OpenAPI规范生成MCP服务器，而Anthropic的“工具搜索工具”在实际生产环境中也体现了相同的检索原理。可扩展至数千种工具，实现显著的令牌节省并提升工具选择精度[9, 423]。进一步而言，DeepAgent将工具搜索作为单个推理过程中的一个动作，能够在任务执行过程中，从包含数万种工具的注册表中动态地发现工具。而ToolGen则通过将每个工具表示为生成过程中模型输出的词汇表标记，从而彻底消除了独立的检索器[338, 647]。

Programmatic Tool Use. A second move treats the tool surface itself as something the agent can read and program against. CodeAct treats executable code as the action space, letting a single step compose calls, branch, and loop [656]; in MCP settings, code-execution patterns present servers as importable APIs and coordinate many calls inside one code block, cutting token consumption by orders of magnitude [9,

15], and Code-Execution MCP formalizes the shift from tool-by-tool orchestration to a single model-generated program, keeping context consumption nearly constant in the call count and tool-ecosystem size at the cost of a larger executable attack surface [154].

程序化工具使用。 第二步将工具表面本身视为智能体可读取并据此进行编程的对象。CodeAct将可执行代码视为动作空间，允许单个步骤组合调用、分支和循环[656]；在 MCP 设置中，代码执行模式将服务器呈现为可导入的 API，并协调单个代码块内的大量调用，从而将令牌消耗量降低数个数量级[9, 15]Code-Execution MCP 将传统的“逐工具编排”模式正式转化为基于单一模型生成的程序。通过牺牲更大的可执行攻击面来维持调用次数与工具生态系统规模下的上下文消费量近乎恒定[154]。

As coding agents strengthen, tool *selection* itself becomes a workspace activity: rather than picking from a static registry, the agent inspects tool definitions, documentation, or source on demand, using progressive disclosure to keep a large ecosystem cheap in context [730]. The same insight drives agentic search. Direct corpus interaction has a strong agent search a raw corpus with terminal tools (grep, find, read) instead of a top- k retriever, beating sparse, dense, and reranking baselines [355], and agentic keyword search with only grep-style tools matches vector-database RAG [566]. Applied to code, a terminal-only RL-trained agent localizes code as well as far larger models [578], turning discovery from a static input choice into an agent-driven search. Together, code execution and workspace-mediated discovery bound a tool surface that would otherwise grow with every call.

随着编码剂浓度的增强，工具选择它本身便成为一项工作空间活动：代理不再从静态注册表中进行选择，而是可根据需要检查工具定义、文档或源代码，并采用渐进式披露机制，从而在特定上下文中保持大型生态系统的成本低廉[730]。同样的洞察力驱动了代理式搜索。直接语料库交互采用终端工具（`grep`、`find`、`read`）对原始语料库进行直接检索，而非使用Top- k 检索器，其性能优于稀疏、密集及重排序基线方法[355]。而仅使用 grep 风格工具进行代理式关键词搜索，可匹配向量数据库 RAG[566]。该方法应用于代码领域时，基于终端的强化学习训练智能体能够对代码以及规模远大于此的模型进行定位[578]，将代码发现过程从静态的输入选择转变为由智能体驱动的搜索过程。代码执行与工作区介导的发现共同构成了一个工具界面，该界面能够有效限制其范围——否则，该界面将随每次调用而不断扩展。

4.3.3 Skill Libraries

4.3.3 技能库

Once tools and workspace state are in place, the next layer of externalized capability is the *skill*: a reusable, parameterizable procedure that packages a validated combination of tools, control flow, and domain knowledge. Recent papers cast this as the step from episodic tool use to persistent, colleague-like agents under a “Workspace + Skill” paradigm, where a persistent workspace supplies state and a curated skill library supplies reusable operational knowledge [841]. We unpack it along three axes: what a skill is, how skills are acquired and reused, and how the library evolves.

一旦工具与工作空间状态均已就绪，下一层级的外部化能力便是技能：即一种可复用且可参数化的流程，该流程封装了经过验证的工具组合、控制流及领域知识。近期研究将这一过程表述为：从情景化工具使用向“工作空间+技能”范式下的持久型、类同事智能体的转变——在此范式下，一个持久的工作空间可提供状态信息，而一个精心构建的技能库则可提供可复用的操作知识[841]。我们从三个维度对其进行剖析：何为技能、技能如何习得与复用，以及该知识库如何演进。

Procedural Skills. On top of tool interfaces, the *skill* abstraction packages validated combinations of tools, control flow, and domain knowledge into named, reusable procedures. Rather than re-deriving a tool sequence each time, the agent recalls a procedure that already works, shifting the interface from tools to skills [888]. Papers frame skills as procedural artifacts with a representation, acquisition, retrieval, and evolution lifecycle, and as a layer orthogonal to MCP: where MCP supplies connectivity, skills supply the procedural knowledge for interpreting outputs and recovering from failure. The current landscape converges on three representation families: natural-language or Markdown procedures that condition a frozen agent, realized as a progressively disclosed file tree by Anthropic’s Agent Skills [16]; executable code the agent can call, inspect, and edit, as in Voyager [630]; and structured relational libraries, where SkillNet weaves over 200,000 skills into a typed skill graph (depend_on, compose_with, ...) [360]. The shared move is from flat, similarity-retrieved entries toward curated, relationally organized procedures owned by the harness as first-class assets.

操作技能。除了工具接口之外，技能抽象层还将经过验证的工具组合、控制流与领域知识封装为具有命名且可复用的程序。智能体无需每次重新推导工具序列，而是调用已有的有效程序，将交互界面从“工具”转向“技能”[888]。文献将技能界定为具有“表征、习得、检索与演化”生命周期的程序性人工制品，亦将其视为与MCP正交的独立层级——其中，MCP负责提供连接性。技能可提供解读输出结果及从故障中恢复所需的程序性知识。当前的技术生态可归结为三大表示范式家族：自然语言或 Markdown 格式化流程——用于对静态智能体进行配置；该智能体由 Anthropic 的 Agent Skills[16]实现，表现为一个逐步公开的文件树结构；可执行代码——代理可调用、检查及编辑此类代码，如Voyager[630]所示；以及结构化关系库——SkillNet将超过20万项技能编织成一个带类型标注的技能图（如depend_on、compose_with等）[360]。这一共享迁移过程，是从基于相似性检索的平铺式数据条目，转向由系统所有、采用关系型组织方式的流程——这些流程被视为第一类资产。

Skill Acquisition. Work on skill acquisition and reuse shows the payoff. Voyager, the canonical instance, discovers executable Minecraft skills and reloads them on later tasks, and later systems extend the idea to inference-time tool synthesis, cumulative scientific-reasoning skills, and skills for unfamiliar software [4, 237, 400, 577, 630]. Related systems evaluate higher-order tool-composition skills and convert episodic narratives into executable skills with explicit activation and termination conditions [65, 430]. Across these systems the recurring benefit is lower token cost, higher reliability, and shorter effective horizons through reuse.

技能获取。开展技能获取与复用工作可带来显著成效。“Voyager”这一经典范例能够发现可执行的Minecraft技能，并在后续任务中重新加载这些技能；而后续系统则将这一理念扩展至推理时工具合成、累积式科学推理技能以及针对陌生软件的技能[4, 237, 400, 577, 630]。相关系统可评估高阶工具组合技能，并将情景式叙事转化为可执行的技能，这些技能均具有明确的激活与终止条件[65, 430]。在这些系统中，其共同优势在于：较低的代币成本、更高的可靠性以及通过资源复用实现的更短有效周期。

Skill Library Evolution. What distinguishes a skill from a tool is that the library is a first-class object the harness owns and curates: skills can be versioned, retrieved, refined, retired, and reused, so where tools supply flexibility, skills supply efficiency and reliability [340]. Because the library lives outside the weights, it can evolve without gradient updates: Memento treats skill acquisition as memory-based reinforcement learning, Alita generates and reuses MCP servers on demand, and observability-driven harness evolution finds most of its gains in rewriting the tool, middleware, and memory layers rather than the system prompt [456, 495, 881]. A first line optimizes the skill text directly: SkillOpt trains a frozen agent by editing its natural-language skill document, accepting a change only when it improves a held-out validation score [757]. A second structures the library as a graph rather than a flat list: the prerequisite-skill graph originates in data-efficient training (Skill-it) [63], and is imported into agent libraries by SkillGraph, which co-evolves a typed skill graph with the policy via reinforcement learning [341], by GraSP, which compiles retrieved skills into an executable DAG with locality-bounded repair [707]. A growing line couples skill libraries directly with reinforcement learning: Skill0, for instance, trains with an in-context curriculum that progressively withdraws skill files until the policy runs skill-free, beginning to fold the library into the weights themselves [405]. This library-level self-improvement is the externalized form of evolution; its internalized counterpart is taken up in Section 5.

技能库演进。技能与工具的区别在于：库是一个一等对象，由Harness拥有并负责管理；技能可进行版本控制、检索、优化、退役及复用；因此，当工具提供灵活性时，技能则能提供效率。可靠性[340]。由于该库位于权重模块之外，其演化过程无需依赖梯度更新：Memento将技能获取视为基于记忆的强化学习；Alita可根据需求生成并复用MCP服务器；而可观测性驱动的Harness演化机制则实现了这一目标。其大部分收益源于对工具层、中间件层及内存层的重构，而非对系统提示词的修改[456, 495, 881]。第一种方法直接优化技能

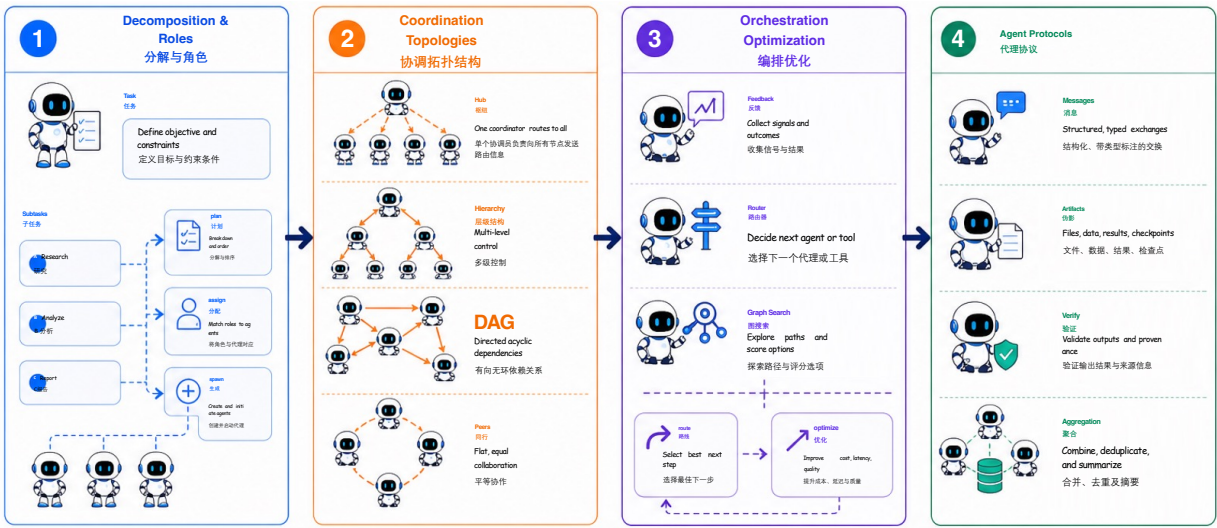


Figure 9 Overview of agent orchestration. Tasks are decomposed into roles, coordinated through different topologies, optimized via feedback-driven routing, and integrated through communication and aggregation protocols.

图9：代理编排概述。任务被分解为若干角色，通过不同的拓扑结构进行协调，借助反馈驱动路由机制进行优化，并通过通信与聚合协议实现集成。

文本：SkillOpt 通过编辑智能体的自然语言技能文档来训练该冻结智能体，仅当修改能提升留出验证集上的得分时才接受该修改[757]。第二种方法将知识库构建为图结构而非扁平列表：先决条件-技能图源自数据高效训练（Skill-it）[63]。该内容由SkillGraph导入至智能体库，SkillGraph通过强化学习[341]与策略共同进化一个带类型标注的技能图谱。由 GraSP 实现，该工具将检索到的技能编译为一个支持局部性受限修复机制的可执行有向无环图（DAG）[707]。越来越多的方案将技能库与强化学习直接相结合：例如，Skill0采用“上下文课程”进行训练，该课程会逐步撤除技能文件，直至策略能够无技能运行。开始将该库整合至权重本身之中[405]。这种库级的自我改进是进化过程的外化形式；其内化对应形式将在第5节中予以阐述。

4.4 Orchestration

4.4 编排

A single agent loop can attend to only one thing at a time; many long-horizon tasks are better served by composing several loops, whether as subtasks of one agent or as a team of specialized agents. Multi-agent collaboration can be described along a few dimensions: the participating agents, the interaction type (cooperation, competition, or a mix), the communication structure, the coordination strategy, and whether the arrangement is static or dynamic [197, 611]. We adopt a pragmatic version of this taxonomy and ask four questions: how work is decomposed and assigned, what topology the resulting computation takes, whether that topology is fixed or learned, and how the agents communicate. Orchestration is the externalized form of long-horizon control structure [468]: once a task outgrows what a single loop can sustain, how the work is split and coordinated often affects end-to-end success more than swapping the underlying model does. Composition is not free, however. A systematic study of multi-agent failures attributes most breakdowns to under-specified roles, inter-agent misalignment, and missing verification rather than to weak base models, a reminder that orchestration must be designed rather than merely assembled [48]. The overall framework of agent orchestration is shown in Figure 9.

单个智能体循环每次只能处理一项任务；而许多长时程任务通过组合多个循环来实现更优的解决方案——这些循环既可以作为单一智能体的子任务，也可以作为一组专门的智能体团队。多智能体协作可从以下几个维度进行描述：参与的智能体、交互类型（合作、竞争或混合）、通信结构以及协调策略。以及该排列是静态还是动态[197, 611]。我们采用这一分类法的实用版本，并提出四个问题：工作是如何分解与分配的；所得计算过程所采用的拓扑结构为何；该拓扑结构是固定的还是通过学习获得的；以及智能体之间如何进行通信。组织协调是长周期控制结构的外化形式[468]当任务规模超出单个循环的承载能力时，工作拆分与协调方式往往比更换底层模型更能影响端到端的成功；然而，组合过程并非免费的。对多智能体系统故障的系统性研究表明，大多数故障可归因于角色定义不充分、智能体间协调失准以及验证环节缺失，而非源于基础模型的薄弱。需注意，系统编排应是经过精心设计的，而非简单地进行组装[48]。智能体编排的整体框架如图 9所示。

4.4.1 Decomposition and Roles

4.4.1 分解与作用

The first decision is how a long task is split into units and assigned to roles. The earliest harnesses fix this split statically, hard-coding the roles and their interaction pattern, whether as software-team standard operating procedures, planner-executor role-play, staged role-conditioned dialogues, or a general programmable substrate in which roles and their conversations are first-class, as in frameworks such as software-team SOPs [219], role-play collaboration [314], staged dialogues [485], and a programmable substrate [692]. A predefined subtask graph is simple but brittle, since an early failure propagates downstream and a hand-coded role set scales poorly; this motivates dynamic decomposition that revises the split from feedback and can spawn a subtask-tailored subagent on the fly, as in TDAG [660]. Either way the harness must keep the split well-formed: agent-oriented planning, for example, holds the decomposing meta-agent to solvability, completeness, and non-redundancy, re-planning unresolvable or duplicated subtasks before any agent runs [305]. How fine to decompose is then a granularity trade-off: too coarse and a subtask overwhelms its agent, too fine and coordination overhead dominates, and that overhead grows with the horizon. Much of the benefit usually attributed to role personas in fact comes from the structural decomposition they induce, namely parallelism, context isolation, and tool specialization, rather than from the personas themselves [48].

第一个决策是：如何将长任务划分为若干单元，并将其分配给相应角色。最早的约束机制可静态地解决这一分裂问题，通过硬编码定义角色及其交互模式——无论是作为软件团队的标准操作规程、规划者与执行者的角色扮演，还是分阶段的、基于角色条件的对话。或是一种通用可编程平台，在此类平台上，角色及其对话被置于第一等地位，例如软件团队SOPs[219]、角色扮演协作[314]以及场景化对话[485]等框架中。以及可编程衬底[692]。预定义的子任务图虽然简单，但存在脆弱性：早期的失败会向下游传播，而手动编写的角色集

则难以有效扩展；这促使我们采用动态分解机制——该机制可根据反馈结果重新调整任务划分，并能够生成针对特定子任务定制的子代理。实时进行，如TDAG[660]所示。无论何种情况，约束机制都必须确保拆分过程保持良好形态：例如，基于代理的规划会将分解后的元代理与可求解性、完备性及非冗余性联系起来——即在任何代理执行之前，先对无法求解或存在重复子任务的情况进行重新规划[305]。。如何进行分解，便涉及粒度权衡问题：分解过于粗略会导致子任务压垮其智能体；分解过于精细则会引发协调开销过大，而这种开销还会随规划视野的扩大而增加。通常归因于角色人格的诸多益处，实际上源于其引发的结构分解——即并行性、上下文隔离与工具专业化——而非源于这些角色人格本身[48]。

4.4.2 Coordination Topologies

4.4.2 协调拓扑结构

Once units and roles are set, the way agents are wired together fixes the shape of the computation. These topologies fall into centralized forms built around a hub or orchestrator, decentralized or distributed forms such as peer debate and dynamic graphs, and hierarchical forms [611]. Centralized designs, exemplified by Magentic-One, place a hub or orchestrator that drives a team, in some variants stacking layers that successively refine one another or dynamically recruiting collaborators [69, 157, 638]. Decentralized designs instead let peers select and prune one another at inference time or scale collaboration over large network structures, as in DyLAN and MacNet [390, 486]. The dominant programmable abstraction across both is the directed acyclic graph: LangGraph defines workflows as DAGs whose nodes are agents or functions and whose edges carry typed state, while Agint and Flash-Searcher compile instructions into typed DAGs and recast web research as a parallel DAG to cut step counts [90, 491]. On long-horizon coding tasks this design space can matter more than the choice of model: on SWE-bench Verified, a fixed frontier model varies far more across scaffolds than different models do under a single fixed scaffold [468].

一旦确定单元与角色，智能体之间的连接方式便决定了计算的结构形态。这些拓扑结构可分为：基于中心节点或协调器构建的集中式架构；诸如对等网络和动态图之类的去中心化或分布式架构；以及分层式架构[611]。集中式设计（以Magentic-One为例）会设立一个驱动团队的中心枢纽或协调者；在某些变体中，该设计会采用分层结构——各层级依次相互精炼，或动态地招募协作方[69, 157, 638]。去中心化设计则允许节点在推理阶段相互选择与剪枝，或能够在大型网络结构上扩展协作能力，例如 DyLAN 和 MacNet[390, 486]。在这两方面，主导的可编程抽象形式均为有向无环图：LangGraph 将工作流定义为 DAG——其节点为代理或函数，而其边则承载类型化的状态。而 Agint 和 Flash-Searcher 将指令编译为带类型标注的有向无环图（DAG），并将网页检索重新表述为一个并行 DAG，以减少操作步骤数量[90, 491]。在长时程编码任务中，这一设计空间可能比模型选择更为重要：在 SWE-bench Verified 上，不同架构下的固定前沿模型之间的差异远大于同一架构下不同模型之间的差异[468]。

The wirings above coordinate agents toward a shared objective, but a complementary family makes the interaction itself *competitive*, pitting two roles in a GAN-like loop where one proposes ever-harder problems and the other strives to solve them, so the pair co-evolves an automatic curriculum with no external data. R-Zero instantiates a Challenger rewarded for posing tasks at the edge of a Solver’s competence and a Solver rewarded for cracking them, while Absolute Zero folds both roles into a single model that proposes and solves verifiable tasks in one self-play loop [233, 854]; earlier, SPAG showed that setting a model against a copy of itself in a two-player adversarial language game steadily sharpens reasoning [84]. Because this adversarial coordination improves the participants through training rather than routing a fixed roster of models, its payoff is ultimately internalized (Section 5); yet the arrangement itself remains a distinctly competitive orchestration pattern, the counterpart to the cooperative composition that dominates the rest of this section.

上述布线机制可将各智能体协调至共同目标，但互补的家族结构则使交互过程本身具有竞争性。该机制在GAN类循环中分配了两个角色：其中一个负责提出日益复杂的任务，而另一个则致力于解决这些任务；如此，二者便可在无需外部数据的情况下，共同演化出一套自动课程。R-Zero 构建了一个“挑战者”角色——该角色因在求解器能力边界处提出任务而获得奖励；同时构建了“求解器”角色——该角色因成功破解这些任务而获得奖励。而Absolute Zero将这两项任务整合至同一个模型中，该模型通过一个自我对弈循环来提出并解决可验证的任务[233, 854]；此前，SPAG研究表明，在双人对抗性语言博弈中，将模型与自身的副本进行对比，能够稳步强化其推理能力[84]。由于这种对抗性协调是通过训练来提升参与者的能力，而非通过路由固定列表的模型实现，其收益最终会被内化（第5节）；然而，这种编排方式本身仍是一种鲜明的竞争性配器法，与本节其余部分中占主导地位的协作式作曲手法相对应。

4.4.3 Orchestration Optimization

4.4.3 编排优化

A topology can be hand-designed or searched. AFlow treats workflow design as program-space search with Monte-Carlo tree search, and GPTSwarm represents a multi-agent system as an optimizable graph and learns both node prompts and edges [816, 897]. A related line learns who executes each unit. At the model level, routers learn to select among models under performance, cost, latency, and congestion constraints [447, 810, 880]; at the agent level, MasRouter jointly chooses collaboration mode, roles, and the model per role, while other systems spawn subagents on the fly or pair central planning with a tool-environment-agent protocol [519, 791, 828]. Taken to its limit, the coordination structure itself is allowed to emerge: a central orchestrator can be trained with reinforcement learning to induce compact coordination at lower cost, and related systems refine functionality and coordination from scratch or couple shared memory with asynchronous execution [107, 496, 836]; at scale, mixed self-organizing coordination can even outperform hand-designed hierarchies [127]. Like skill-library growth (Section 4.3.3), these black-box optimizations of the orchestration graph are an externalized form of evolution.

拓扑结构可手动设计或通过搜索生成。AFlow将工作流设计视为基于蒙特卡洛树搜索的程序空间搜索，而GPTSwarm则将多智能体系统表示为可优化的图，并学习节点提示与边[816, 897]。另一条相关线程用于确定每个单元的执行者。在模型层面，路由器能够根据性能、成本、延迟及拥塞约束，在不同模型之间进行选择[447, 810, 880]；在智能体层面，MasRouter可共同选择协作模式、角色以及各角色对应的模型。而其他系统则会在运行时动态生成子代理，或将中央规划与“工具-环境-代理”协议相结合[519, 791, 828]。当系统达到极限时，协调结构本身便可自发形成：可通过强化学习对中央协调器进行训练，从而以较低的成本诱导出紧凑的协调机制。相关系统可从零开始优化功能与协调机制，亦可将共享内存与异步执行相结合[107, 496, 836]；在大规模场景下，混合自组织协调机制甚至能优于人工设计的层级结构[127]。与技能库增长类似（第4.3.3节）中，对编排图的这种黑盒式优化可视为进化过程的一种外部化形式。

4.4.4 Agent Protocols

4.4.4 代理协议

Finally, composed agents must exchange messages reliably. The A2A protocol foregrounds Agent Cards, Tasks, and Artifacts over JSON-RPC with streaming, IBM’s Agent Communication Protocol offers a lighter REST-native alternative, and MCP, ACP, A2A, and ANP have been compared across architecture, discovery, communication, and security [97, 144, 512, 758]. Framework-level messaging conventions coexist with richer schemes that add decentralized identity and verifiable credentials, switch between structured protocols and natural language according to communication frequency, or perform decentralized discovery over encrypted peer-to-peer channels [219, 314, 422, 501, 504, 692]. A complementary harness duty is aggregation: when parallel trials or subagents run, their outputs must be reduced to a single result, through best-of- N or voting for verifiable tasks [323, 442, 659] and through confidence-aware agentic aggregation for open-ended ones [296, 298]. This shift from prescribed hierarchies toward protocol-mediated coordination stands in productive tension with the deterministic governance we turn to next, and remains one of open problems.

最后，组合代理必须能够可靠地交换消息。A2A协议将Agent Card、Task和Artifact置于优先地位，相较于支持流式传输的JSON-RPC；IBM的Agent Communication Protocol则提供了一种更轻量化的REST原生替代方案；此外，MCP、ACP、A2A及ANP已在系统架构与服务发现方面进行了对比分析。通信与安全[97, 144, 512, 758]。框架级消息传递约定与更丰富的方案并存——后者可引入去中心化身份与可验证凭证，并能根据



Figure 10 Overview of hooks and middleware. We categorize hooks into three paradigms based on the origin and adaptiveness of their safety criteria: pre-defined rules, user-defined policies, and runtime-adaptive signals.

图10: 钩子与中间件概述。我们根据钩子安全准则的来源及其适应性，将其划分为三种范式：预定义规则、用户自定义策略以及运行时自适应信号。

通信频率在结构化协议与自然语言之间进行切换。或在加密的点对点通道上执行去中心化发现[219, 314, 422, 501, 504, 692]。补充性任务调度机制为聚合：当并行试验或子代理运行时，其输出结果必须通过“最佳中 N ”方式或对可验证任务进行投票，最终汇总为单一结果[323, 442, 659]。并通过针对开放式问题的置信度感知代理聚合方法[296, 298]。从既定的等级制度向协议驱动的协调方式转变，与我们接下来要探讨的确定性治理形成了富有建设性的张力，且仍是一个尚未解决的开放性问题。

4.5 Hooks and Middleware

4.5 钩子与中间件

The preceding subsections establish the full execution infrastructure of an agent harness: loops govern control flow, context and memory manage information state, tools extend capabilities, and orchestration composes agents into collaborative systems. Yet the very expressiveness that makes these mechanisms powerful also amplifies the risk surface: an agent equipped with file-system access, API credentials, and multi-step autonomy can cause irreversible harm through hallucinated commands or adversarial jailbreaks [1]. Prompt-level instructions alone are insufficient to constrain such behavior, as they remain vulnerable to the same failure modes they aim to prevent. Recent architectures [421, 529] have therefore shifted toward programmable boundaries enforced at the harness level, which we collectively term *hooks*. Hooks perform enforcement, gating the action path with allow, block, or hand-off decisions, whether through rule-based evaluation or model judgment. As illustrated in Figure 10, based on who defines the safety criterion and how the trigger condition is determined, we identify three complementary paradigms: (1) pre-defined rule-based hooks enforce universal safety invariants hard-coded by framework designers; (2) custom user-defined hooks enforce domain-specific policies authored by deployers; and (3) runtime-adaptive hooks derive their trigger conditions from runtime signals such as model uncertainty and behavioral patterns rather than from any predetermined specification. 前文各小节构建了代理套件的完整执行基础设施：循环结构负责控制流管理，上下文与内存模块负责信息状态管理，工具集用于扩展代理功能，而编排机制则将多个代理整合为协同系统。然而，正是这种赋予这些机制强大功能的表达能力，也会放大风险敞口：若某个代理程序拥有文件系统访问权限或API凭证，多步自主性可能通过幻觉指令或对抗性越狱行为造成不可逆的损害[1]。仅依赖提示级指令不足以约束此类行为，因为这些指令仍可能受到与其旨在预防的相同故障模式的影响。因此，近期的架构[421, 529]已转向在总线层强制实施可编程边界的方式，我们统称为**钩子**。钩子负责执行策略，通过允许、阻断或转交等决策对操作路径进行管控，无论该过程是基于规则评估还是基于模型判断。如图10所示根据安全准则的定义主体以及触发条件的确定方式，我们识别出三种互补的范式：(1) 预定义的基于规则的挂钩机制强制执行由框架设计者硬编码的通用安全不变量；(2) 自定义用户定义钩子可强制执行由部署者编写的特定域策略；(3) 运行时自适应挂钩的触发条件源自运行时信号（如模型不确定性与行为模式），而非基于任何预设规范。

4.5.1 Pre-defined Rule-based Hooks

4.5.1 预定义的基于规则的挂钩

At the foundation of agent safety, harnesses employ immutable, hard-coded middleware to intercept or block actions that violate universal safety invariants without requiring user configuration. These hooks are triggered at fixed lifecycle points such as immediately before a shell command or API call, and because they operate on the action path rather than the reasoning path, they run outside the LLM’s context window and are therefore less directly exposed to prompt-level manipulation; their coverage, however, still depends on policy completeness and adversarial robustness, since encoded payloads, obfuscated arguments, or gaps in the policy can still slip past them. Concretely, pre-defined hooks manifest at three successive boundaries along the agent execution pipeline:

在代理安全机制的基石上，约束系统采用不可变的硬编码中间件，可拦截或阻断任何违反通用安全不变量的操作，且无需用户进行任何配置。这些钩子会在固定的生命周期节点触发，例如在执行Shell命令或API调用之前；由于它们作用于“操作路径”而非“推理路径”，它们运行于LLM的上下文窗口之外，因此不易直接受到提示层面操纵的影响；然而，其覆盖范围仍取决于策略的完整性与对抗鲁棒性——因为编码后的有效载荷、混淆后的参数或策略中的漏洞仍可能被绕过。具体而言，预定义的钩子会在代理执行流水线上的三个连续边界处触发：

- **Input Boundary.** Operating before data ingestion by the core language model, this mechanism intercepts incoming raw data streams, including user instructions, prompt templates, and retrieved external context. Its primary objective is to filter out adversarial prompt injections, jailbreaks, and malformed
- **输入边界。**该机制在核心语言模型开始接收数据之前运行，可拦截传入的原始数据流，包括用户指令、提示模板及检索到的外部上下文。其主要目标是过滤对抗性提示注入、越权攻击及畸形内容。

system directives designed to compromise the model’s behavioral alignment. Systems implementing this boundary [529] deploy independent, token-level or semantic static filters to reject non-compliant inputs before they can influence the model’s internal attention state.

旨在破坏模型行为一致性的一系列系统指令。实现该边界[529]的系统会部署独立的、基于令牌或语义的静态过滤器，以在非合规输入能够影响模型内部注意力状态之前，将其予以拒绝。

- **Tool-call Boundary.** Acting as a pre-execution firewall, this mechanism inspects tool arguments before execution to detect dangerous options (e.g., `rm -rf`) and common attack patterns such as code injection or unauthorized file access. AEGIS [783] instantiates this design by interposing a policy engine on every tool call, validating arguments against a curated set of known vulnerability patterns, including path traversal, with negligible latency overhead.
- **工具调用边界。** 该机制充当预执行防火墙，在工具执行前对其参数进行检查，以检测危险选项（例如 `rm -rf`）以及常见的攻击模式（如代码注入或未经授权的文件访问）。AEGIS[783]通过在每次工具调用处插入一个策略引擎来实现该设计——该引擎会将传入的参数与经过精心筛选的一组已知漏洞模式（包括路径遍历）进行验证，且其延迟开销可忽略不计。
- **Protocol Boundary.** Enforced at the inter-component communication layer, this boundary requires every request to carry explicit credentials and receive per-action authorization, ensuring no message is implicitly trusted regardless of its origin. Representative systems include Authenticated Workflows [503], which attaches cryptographic tokens to protocol messages for authenticity verification, and MagenticUI [441], which mandates explicit user approval before dispatching agent actions.
- **协议边界。** 该边界在组件间通信层面强制实施，要求所有请求均需携带显式凭证并接受按操作授权，从而确保无论消息来源如何，均不会对任何消息进行隐式信任。代表性系统包括Authenticated Workflows[503]（该系统通过将加密令牌附加至协议消息以实现身份验证）以及MagenticUI[441]。该机制要求在执行代理操作前必须获得用户的明确批准。

These three boundaries form independent, complementary defense layers: because each targets a distinct attack surface, the failure of any single layer does not compromise the others [421]. This defense-in-depth property is further strengthened by architectural isolation techniques such as IsolateGPT [698], whose hub-and-spoke design routes all tool interactions through an isolated mediator to prevent cross-tool information leakage. Together, these pre-defined mechanisms establish a non-negotiable safety floor that holds irrespective of the model’s reasoning quality, the user’s configuration choices, or the particular task being executed.

这三道边界构成了独立且互补的防御层：由于每层均针对不同的攻击面，任一单一层的失效都不会影响其他层[421]。这种纵深防御特性还可通过诸如IsolateGPT[698]等架构隔离技术得到进一步强化。其“中心辐射式”设计将所有工具间的交互操作均经由一个隔离的中介层进行处理，从而防止跨工具信息泄露。综上所述，这些预定义机制共同构建了一条不可妥协的安全底线——该底线无论模型的推理质量、用户的配置选择或正在执行的具体任务如何，均能予以保障。

4.5.2 Custom User-defined Hooks

4.5.2 自定义用户定义钩子

Beyond immutable rules, modern harnesses expose programmable interfaces that allow developers to define domain-specific policies enforced at runtime. While pre-defined hooks target universally dangerous operations, custom hooks address the far broader space of application-specific requirements: **workflow constraints, regulatory compliance, access control scoping, and safety boundaries that vary across deployment contexts.** The defining characteristic is the locus of policy authorship: in every approach below, it is the user or developer, not the framework designer, who specifies the substantive safety criterion. “User-defined” thus refers to who writes the policy, not how it is enforced; the enforcement mechanism ranges from deterministic symbolic engines to probabilistic model evaluators, but the policy content originates with the deployer in all cases.

除了不可变规则外，现代安全框架还提供了可编程接口，使开发人员能够定义并在运行时强制执行领域特定策略。虽然预定义钩子旨在应对普遍存在的危险操作，但自定义钩子则能够满足更为广泛的、特定于应用程序的需求：**包括工作流约束、法规合规性、访问控制范围以及在不同部署环境下各异的安全边界。**其核心特征在于政策制定者的定位：在下文所述的每种方法中，都是用户或开发者——而非框架设计者——来规定实质性的安全准则。“用户自定义”因此指的是政策的制定者，而非其执行方式；执行机制可涵盖从确定性符号引擎到概率模型评估器等多种形式，但在所有情况下，政策内容均源自部署方。

Existing approaches vary along two orthogonal dimensions: (i) how the user specifies a policy (formal language vs. natural language), and (ii) what enforces it at runtime (a deterministic symbolic engine vs. a probabilistic model). The interplay of these two dimensions yields three distinct categories. Formal-language policies pair naturally with symbolic enforcement. Natural-language policies split by whether an LLM translates them into executable artifacts offline and exits the enforcement path, or remains on the critical path to evaluate compliance at every step. The resulting spectrum trades specification effort against enforcement strength.

现有方法在两个正交维度上存在差异：(i) 用户如何指定策略（形式化语言 vs. 自然语言）；(ii) 在运行时由何种机制来强制执行该策略（确定性符号引擎 vs. 概率模型）。这两个维度的相互作用可产生三种不同的类别：形式化语言政策与符号化执行自然地相结合。自然语言策略根据大型语言模型（LLM）是否在离线状态下将这些策略翻译为可执行的业务流程并退出执行路径，还是继续保留在关键执行路径上、在每一步骤中评估合规性而进行划分。所得频谱方案在规范制定工作量与执行力度之间进行了权衡。

(1) Symbolic Hooks. Users author policies in a formal, machine-verifiable language, and a deterministic engine evaluates every action against the policy before execution. Because the LLM is entirely absent from the enforcement path, compliance guarantees are hard: a compliant action is provably within policy, and no prompt injection can subvert the check.

(1) 符号化钩子。 用户可使用一种形式化、可被机器验证的语言来编写策略，而一个确定性引擎会在执行任何操作之前，将该操作与所述策略进行比对。由于大语言模型（LLM）完全不在执行路径中，因此难以实现合规保障：合规操作可被证明符合政策要求，且任何提示注入都无法绕过该验证环节。

Several systems design event-triggered DSLs whose predicate guards and enforcement actions are executed by lightweight symbolic runtimes [509, 633]. Others go further by translating temporal-logic or JSON-schema specifications into first-order constraints and using SMT solvers or constrained token generation to make non-compliant tool calls physically impossible to emit [265, 542]. For multi-agent settings, PCAS [466] models inter-agent state as a causal dependency graph and enforces Datalog-derived policies via Differential Datalog with a reference monitor intercepting every action. A complementary line draws on operating-system security primitives such as capability labels and cryptographic audit records to enforce information-flow control and provenance tracking at the tool-call boundary [109, 617]. Across all variants, the enforcement engine is a formally deterministic program whose correctness can be audited independently of the LLM it guards.

已有若干系统设计事件触发型DSL，其谓词守卫与执行动作均由轻量级符号化运行时[509, 633]执行。还有一些方法更为深入，将时序逻辑或JSON Schema 规范转化为一阶约束，并利用SMT求解器或受限令牌生成技术，使不合规的工具调用在实际执行层面变得不可能发生[265, 542]。对于多智能体场景，PCAS[466]将智能体间的状态建模为因果依赖图，并通过采用带有参考监控器拦截所有动作的差分Datalog来强制执行基于Datalog的策略。一条补充线利用操作系统安全原语（如能力标签和加密审计记录），在工具调用边界处强制执行信息流控制与溯源追踪[109, 617]。在所有变体中，执行引擎均是一个形式上确定性的程序；其正确性可独立于其所保护的大语言模型（LLM）进行审计。

(2) Static LLM-compiled Hooks. A second paradigm accepts policies written in natural language or semi-structured documents but compiles them into executable artifacts that are then enforced deterministically or quasi-deterministically. The LLM participates at compile time but is absent from the enforcement path, yielding guarantees bounded by compilation quality but independent of runtime model reasoning.

(2) 静态LLM编译钩子。第二种范式接受用自然语言或半结构化文档编写的策略，但将这些策略编译为可执行产物，随后再以确定性或准确定性方式强制执行这些策略。LLM在编译时参与，但在执行路径中并不参与，从而可提供由编译质量所界定、但与运行时模型推理无关的保证。

The compilation targets range from executable guardrail code that runs deterministically to admit or block each action [708], to probabilistic rule circuits derived from regulatory documents and verified by formal model checkers [80], to formally verified policy code generated from Hoare-triple specifications and validated by an offline verifier before deployment [432]. The common thread is a two-phase architecture: an LLM-powered compilation step that transforms human-readable intent into machine-executable policy, followed by a symbolic or quasi-symbolic enforcement step. Errors can be introduced at compile time, but once compiled, enforcement proceeds without further LLM involvement.

编译目标涵盖范围广泛，从可执行的护栏代码——该代码能够以确定性方式允许或阻断每项操作[708]，到源自监管文件并通过形式化模型检查器验证的概率规则电路[80]。，是指基于Hoare三元组规范生成的经形式验证的策略代码，在部署前由离线验证器进行验证[432]。其共同点在于采用两阶段架构：首先由基于大语言模型（LLM）的编译步骤将人类可读的意图转换为机器可执行的策略；随后再通过符号化或准符号化执行步骤进行策略落实。错误可能在编译阶段引入，但一旦程序编译完成，验证过程便自动执行，无需人工干预。

(3) Dynamic LLM-evaluated Hooks. A third paradigm also starts from user-authored natural-language policies but, unlike static compilation, keeps the LLM on the enforcement critical path: the model directly evaluates each action or trajectory against the user-supplied policy at runtime. The deployer must provide domain-specific safety criteria without which the system has nothing to enforce, so the “user-defined” property holds in the same sense as the preceding categories. This maximizes expressiveness and minimizes specification effort at the cost of replacing hard guarantees with probabilistic ones.

(3) 基于动态LLM评估的钩子函数。第三种范式同样基于用户自定义的自然语言策略，但与静态编译不同，它将大语言模型保留在执行关键路径上：该模型在运行时直接根据用户提供的策略对每个动作或轨迹进行评估。部署者必须提供领域特定的安全准则——若无这些准则，系统便无任何需强制执行的内容；因此，“用户自定义”属性的含义与前述类别中的含义一致。这种方法最大限度地提升了表达能力，同时最小化了规范制定的工作量，但其代价是将硬性保证替换为概率性保证。

One approach deploys a guardian LLM that interprets natural-language policies with contextual reasoning across multiple control points [285]. A classifier-based alternative fine-tunes or prompts language models as configurable safety classifiers whose risk taxonomies or policy definitions can be adapted without retraining [244, 274]. A third variant composes multiple heterogeneous scanners into a layered pipeline that users can extend with custom scanners tailored to their threat model [87]. Because the enforcer is itself a language model, these approaches can handle nuanced, context-dependent policies that resist formalization, but compliance is probabilistic and susceptible to the same failure modes that motivate guardrails in the first place.

一种方法采用守护型大语言模型，该模型能够通过跨多个控制点的上下文推理来解析自然语言策略[285]。基于分类器的替代方法可对语言模型进行微调或提示，将其转化为可配置的安全分类器——其风险分类体系或策略定义可在无需重新训练的情况下进行调整[244,274]。第三种变体可将多个异构扫描器组合成一个分层流水线，用户可利用针对其威胁模型定制的自定义扫描器对该流水线进行扩展[87]。由于执行器本身就是一个语言模型，因此这些方法能够处理那些难以形式化描述的、具有细微差别且依赖于具体语境的策略。但合规性具有概率性，且易受与最初设计护栏所针对的相同失效模式的影响。

Collectively, these three categories trace a design-space frontier between specification cost and enforcement strength: symbolic hooks demand the most expertise to author but provide the hardest guarantees; LLM-compiled hooks offer a middle ground by lowering the authoring barrier while keeping the LLM off the enforcement path; and LLM-evaluated hooks are the most accessible but the least formally bounded, trading deterministic compliance for the ability to express nuanced, context-dependent policies. In practice, production deployments increasingly layer multiple paradigms (e.g., symbolic access-control checks composed with model-based content classifiers) to exploit the complementary strengths of each.

总体而言，这三类方案共同勾勒出一个在规范成本与执行力度之间的设计空间边界：符号化挂钩机制对开发者而言需要最专业的知识储备，但能提供最严格的保障；LLM编译钩子通过降低编写门槛，同时将LLM置于执行路径之外，从而提供了一种折衷方案；而基于LLM评估的挂钩机制虽最易于使用，但其形式约束也最弱——它以牺牲确定性合规为代价，换取了表达细微且依赖上下文的策略的能力。在实际应用中，生产部署正日益采用多层架构，融合多种范式（例如：将符号化访问控制检查与基于模型的内容分类器相结合），以充分发挥各范式的互补优势。

4.5.3 Runtime-adaptive Hooks

4.5.3 运行时自适应挂钩

While rule-based and user-defined hooks operate on predetermined conditions, a third, runtime-adaptive paradigm derives dynamic, context-sensitive triggers from runtime signals rather than fixed rules. The defining characteristic is that neither the trigger condition nor the enforcement threshold is fully specified before deployment; instead, these hooks continuously monitor the agent’s internal confidence, task progress, and external behavioral patterns in real time, activating timely protective measures when the system detects that the agent is operating outside its reliable competence range. We distinguish runtime-adaptive hooks from user-defined hooks by the locus of criterion definition: user-defined hooks enforce human-authored policies, whereas runtime-adaptive hooks let the system itself discover what to check from various data.

虽然基于规则的钩子和用户自定义钩子均基于预设条件运行，但第三种——运行时自适应范式——则能从运行时信号而非固定规则中衍生出动态且上下文敏感的触发机制。其核心特征在于：在部署前，既无触发条件的完整定义，也无执行阈值的明确设定；相反，这些钩子会实时持续监控代理的内部置信度、任务进度及外部行为模式。当系统检测到代理正在运行于其可靠能力范围之外时，应及时启动防护措施。我们通过标准定义的位置来区分运行时自适应钩子与用户自定义钩子：用户自定义钩子强制执行由人工编写的策略，而运行时自适应钩子则允许系统自身从各类数据中发现应进行检查的内容。

Existing approaches can be organized by the type of runtime signal to which the hook responds: the model’s own uncertainty, deviations in the agent’s behavioral patterns, or shifts in the external threat distribution.

现有方法可根据挂钩所响应的运行时信号类型进行分类：包括模型自身的不确定性、智能体行为模式的偏差，或外部威胁分布的变化。

(1) Uncertainty-driven Hooks. The most direct form of runtime adaptation monitors the model’s own confidence in its reasoning. These hooks extract calibrated risk scores from the inference process and surface them to downstream systems for intervention decisions, without themselves enforcing any fixed policy. Representative methods estimate semantic entropy over sampled completions to detect confabulations [151], propagate verbalized confidence scores across reasoning steps and trigger targeted recomputation when accumulated uncertainty exceeds a threshold [815], or model situational weights via hidden Markov models to extend uncertainty propagation to multi-step agentic reasoning [858]. Because these signals are derived from the model’s internal state at each inference step, the effective trigger condition adapts continuously without requiring any external training data or human-authored rules.

(1) 不确定性驱动的挂钩机制。最直接的运行时自适应形式是监测模型对其推理过程的信心程度。这些钩子函数可从推理过程中提取经过校准的风险评分，并将其提供给下游系统用于干预决策，但自身并不强制执行任何固定策略。代表性方法通过估算对采样补全结果的语义熵来检测虚构记忆[151]。，将口头化置信度评分在推理步骤间进行传播，并当累积不确定性超过某个阈值[815]时触发针对性的重新计算。或利用隐马尔可夫模型来建模情境权重，从而将不确定性传播扩展至多步主体推理[858]。由于这些信号均源自模型在每个推理步骤下的内部状态，因此其有效触发条件能够持续自适应调整，无需任何外部训练数据或人工编写的规则。

(2) Behavior-driven Hooks. A second class of hooks monitors the agent’s observable execution behavior and intervenes when action patterns deviate from learned expectations. Unlike uncertainty-driven hooks that read internal model states, these approaches build structural models of normal agent behavior from execution traces and use them to detect anomalies at runtime. One line of work learns formal behavioral models from historical traces and applies model checking to anticipate violations before they occur, enabling proactive intervention [634]. Another line operates at the trajectory level, training compact verifiers on synthetically perturbed trajectories for step-level error localization and targeted rollback [388], or constructing dynamic execution graphs from distributed traces to detect structural irregularities such as infinite loops, prompt injections, and collusive delegation across multi-agent interactions [214]. The common thread is that the definition of “normal behavior” is learned from data rather than specified by a human, making these hooks adaptive to the particular agent and task at hand.

(2) 行为驱动型钩子。第二类钩子会监控智能体可观察的执行行为，并在动作模式偏离已学习预期时进行干预。与依赖不确定性驱动的钩子（这些钩子会读取内部模型状态）不同，此类方法通过执行轨迹构建正常智能体行为的结构化模型，并利用该模型在运行时检测异常。其中一种研究方向是从历史轨迹中学习形式化行为模型，并利用模型验证技术在违规行为发生前预测其出现，从而实现主动干预[634]。另一条线在轨迹层面运行，通过在合成扰动的轨迹上训练紧凑型验证器，以实现步级错误定位与定向回滚[388]。或基于分布式跟踪数据构建动态执行图，以检测多智能体交互过程中的结构异常，例如无限循环、提示注入以及合谋委托[214]。其共同点在于，“正常行为”的定义是通过数据学习得出的，而非由人类直接规定，从而使这些钩子能够自适应于具体的智能体及任务。

(3) Threat-shift-driven Hooks. The preceding two categories respond to signals that arise within a fixed threat model; a third category addresses the scenario where the threat landscape itself evolves after deployment. These hooks detect that previously unseen attack types or domain shifts have rendered the current safety model inadequate, and update their own detection criteria accordingly. AdaptiveGuard [754] identifies novel attack types as out-of-distribution inputs on guardrail-model activations and rapidly adapts via continual learning, maintaining high detection performance where static guardrails degrade sharply. AGrail [408] takes a lifelong-learning approach: two cooperative LLMs generate and iteratively refine safety checks per action type via test-time adaptation, accumulating them in a memory module that generalizes across tasks. DuoGuard [119] employs adversarial co-evolution between a generator and a classifier during training to produce a robust safety classifier; notably, its adaptiveness is a training-time property and the deployed model is static, placing it at the boundary between this category and fixed guardrail classifiers.

(3) 基于威胁态势变化的 Hook 策略。前两类策略适用于在固定威胁模型中产生的信号；第三类策略则针对部署后威胁态势本身发生变化的情形。这些钩子能够检测到此前未见过的攻击类型或域偏移已导致当前安全模型不再适用，并据此更新其自身的检测标准。AdaptiveGuard[754]能够识别新型攻击类型——即作为防护栏模型激活过程中的分布外输入，并通过持续学习快速进行自适应调整，在静态防护栏的性能显著下降时，仍能保持较高的检测性能。AGrail[408]采用终身学习方法：两个协作的LLM通过测试时自适应技术，针对每种动作类型生成并迭代优化安全检查规则，并将这些规则累积至一个能够跨任务泛化的记忆模块中。DuoGuard[119]在训练过程中，利用生成器与分类器之间的对抗共进化机制，以生成一个鲁棒的安全分类器；值得注意的是，其自适应性属于训练时的特性，而部署后的模型是静态的，这使其处于该类别与固定护栏分类器之间的分界点。

Benchmarks such as Agent-SafetyBench [849] and ToolEmu [520], which reveal that even frontier agents struggle to achieve satisfactory safety scores, underscore the necessity of these signal-driven mechanisms. By deriving their trigger conditions from learned models rather than human-authored rules, runtime-adaptive hooks complement user-defined hooks: the former discovers what to watch for, while the latter enforces what the developer already knows.

诸如Agent-SafetyBench[849]和ToolEmu[520]等基准测试表明，即便是前沿智能体也难以取得令人满意的安全评分，这凸显了采用此类信号驱动机制的必要性。通过从学习模型而非人工编写的规则中推导触发条件，运行时自适应挂钩能够有效补充用户自定义挂钩：前者可发现需关注的指标，而后者则能落实开发人员已掌握的规则。

4.6 Verification
4.6 验证

Hooks establish deterministic boundaries that intercept or block actions, but interception alone is not sufficient: the system must also *evaluate* the quality, correctness, and safety of agent behavior at every stage. Verification mechanisms serve this complementary role, acting as the harness’s assessment engine that judges whether an intercepted action should proceed, whether intermediate reasoning is on track, and whether final outputs meet task requirements. Together with hooks, they close the governance loop by shifting the harness from a passive execution wrapper to an active controller. Table 4 provides a structured overview of representative verification and evaluation methods discussed in this subsection, organized by assessment target, verification level, and verifier strategy.

钩子可建立确定性的边界，用于拦截或阻断操作，但仅靠拦截是不够的：系统还必须在每个阶段 *评估* 智能体行为的质量、正确性与安全性。验证机制发挥着这一互补作用，充当约束器的评估引擎，用于判断被拦截的操作是否应继续执行、中间推理是否符合预期，以及最终输出是否满足任务要求。它们与钩子共同作用，通过将“套件”从被动的执行包装器转变为主动的控制器，从而完成治理闭环。表4按评估目标、验证级别及验证策略对本小节中讨论的代表性验证与评估方法进行了结构化概述。

4.6.1 Assessment Targets
4.6.1 评估目标

Verification mechanisms can be taxonomized by the property they certify: the factual correctness of outputs, the operational safety of actions, faithfulness to the user’s instructions, or agreement across verifiers.

验证机制可根据其所验证的属性进行分类：包括输出结果的事实正确性、操作行为的安全性、对用户指令的忠实度，或不同验证者之间的一致性。

(1) Correctness Verification. Correctness verification certifies the factual accuracy of agent outputs. SelfCheck-GPT [418] detects hallucinations through sampling consistency alone, providing a zero-resource baseline for factual self-assessment without any external knowledge source.

(1) 正确性验证。正确性验证用于认证智能体输出结果的事实准确性。SelfCheckGPT[418]可仅通过采样一致性来检测幻觉，为无需任何外部知识源的事实性自我评估提供了零资源基线。

(2) Safety Verification. Safety verification certifies that agent actions do not cause harm before they take effect. VeriGuard [432] generates both functional code and formal verification artifacts for each action, achieving near-zero attack success. A rapidly growing line builds dedicated safety verifiers and diagnostic guardrails: AgentDoG [287] organizes moderation around a three-dimensional risk taxonomy (source, failure mode, consequence) for fine-grained, trajectory-level judgments with root-cause diagnosis beyond binary labels, and its successor AgentDoG 1.5 [288] reports strong moderation from a small amount of training data while supporting real-time online deployment. ToolSafe [440] targets tool-invocation safety with a step-level guardrail

(2) 安全验证。安全验证可证明，代理行为在生效前不会造成危害。VeriGuard[432]可为每个操作生成功能代码与形式化验证产物，从而实现近乎零的攻击成功率。一个快速发展的项目正在构建专用的安全验证器与诊断防护机制：AgentDoG[287]该系统围绕三维风险分类体系（来源、失效模式、后果）构建了审核机制，可进行细粒度的轨迹级判断，并能实现超越二元标签的根本原因诊断；其后续版本为AgentDoG 1。文献[288]指出，基于少量训练数据即可实现强调制效果，同时支持实时在线部署；而ToolSafe[440]则通过采用分步级防护机制来保障工具调用的安全性。

Table 4 Representative verification and evaluation methods for LLM agents. Rows follow the three dimensions of this table: LLM代理的代表性验证与评估方法。各行均遵循本研究的三个维度 subsection: assessment targets, verification levels, and verifier strategies. **Train-free** (✔) marks methods that work with a frozen model, versus (✖) those requiring dedicated training.

子节：评估目标、验证层级及验证策略。**无需训练** (N) 标注的是适用于冻结模型的方法，而 (O) 则标注的是需要专门训练的方法。

Method 方法	Train-free 无列车运行	Key mechanism 关键机制
Assessment Targets 评估目标		
SelfCheckGPT [418]	✔	Sampling consistency for factuality verification 事实验证中的采样一致性
VeriGuard [432]	✔	Verified code generation for action safety 动作安全的经验证代码生成
ToolSafe [440]	✖	RL-trained proactive tool-invocation safety guardrail 基于强化学习训练的主动式工具调用安全防护机制
NSVIF [563]	✔	Neuro-symbolic constraint checking for instruction faithfulness 用于指令忠实性的神经符号约束检查
Multiagent Debate [139]	✔	Inter-model deliberation for factual agreement 跨模型协商以实现事实一致性
多智能体辩论[139]	✔	跨模型协商以实现事实一致性
Multi-Agent Verification [362]	✔	Independent aspect verifiers for cross-verifier agreement 用于跨验证器一致性验证的独立方面验证器
多智能体验证[362]	✔	用于跨验证器一致性验证的独立方面验证器
Verification Levels 验证级别		
Let’s Verify Step by Step [363]	✖	Human-supervised process rewards for intermediate steps 针对中间步骤设置的人工监督流程奖励机制
让我们逐步验证[363]	✖	针对中间步骤设置的人工监督流程奖励机制
AgentPRM [702]	✖	Per-decision process rewards for agent trajectories 智能体轨迹的决策过程奖励
CRITIC [182]	✔	Tool-interactive stepwise verify-then-correct loop 工具交互式逐步验证后修正循环
评论家[182]	✔	工具交互式逐步验证后修正循环
LLM-as-a-Judge [867]	✔	Model-based evaluation of final outputs 基于模型的最终输出结果评估
LLM作为裁判[867]	✔	基于模型的最终输出结果评估
Agent-as-a-Judge [898]	✔	Tool-augmented assessment after task completion 任务完成后进行的工具辅助评估
代理人作为裁判[898]	✔	任务完成后进行的工具辅助评估
Reflexion [544]	✔	End-of-trial verbal reflection with episodic memory 试验结束时的言语反思（结合情景记忆）
反思[544]	✔	试验结束时的言语反思（结合情景记忆）
Verifier Strategies 验证器策略		
Math-Shepherd [643]	✖	Monte Carlo process labels for verifier training 用于验证器训练的蒙特卡罗过程标签
Implicit PRM [787]	✖	Process rewards from policy logits without extra training 直接从策略逻辑值中获取过程奖励，无需额外训练
隐式PRM[787]	✖	直接从策略逻辑值中获取过程奖励，无需额外训练
Self-Consistency [659]	✔	Majority voting over diverse reasoning paths 基于多种推理路径的多数表决机制
自一致性[659]	✔	基于多种推理路径的多数表决机制
LATS [875]	✔	Tree search with LLM-generated value estimates 基于大语言模型生成的值估计的树形搜索
ReST-MCTS* [805]	✖	PRM-guided self-training via iterative tree search 基于PRM的迭代树搜索引导自训练
Compute-optimal scaling [550]	✖	Difficulty-adaptive compute allocation with process verifiers 基于进程验证器的难度自适应计算分配
计算最优缩放[550]	✖	基于进程验证器的难度自适应计算分配

(TS-Guard) trained via multi-task reinforcement learning that flags unsafe calls before execution, substantially reducing harmful invocations. These verifiers are increasingly stress-tested by dedicated safety benchmarks. The need for them is underscored by AgentHarm [8], a motivating benchmark showing that leading LLMs remain surprisingly compliant with explicitly malicious multi-step tasks; complementary benchmarks then probe specific settings: R-Judge [790] measures safety-risk awareness over multi-turn interaction records, RedCode [193] probes risky code execution and generation for code agents, ST-WebAgentBench [303] evaluates policy-constrained actions for web agents, and OS-Sentinel [571] (distinct from the SentinelAgent runtime monitor of §4.5) combines formal and neural validation for mobile GUI agents in realistic workflows.

(TS-Guard) 采用多任务强化学习进行训练，可在函数调用执行前识别出不安全的调用，从而大幅减少有害调用；这些验证器正越来越多地通过专门的安全基准测试进行压力测试。AgentHarm[8]有力地凸显了对这类模型的需求——该激励基准测试表明，领先的大型语言模型在处理明确恶意的多步骤任务时，仍表现出令人惊讶的合规性；互补性基准测试随后针对特定设置进行验证：R-Judge[790]通过多轮交互记录来衡量用户的安全风险意识；RedCode[193]Probes 用于检测代码代理中的高风险代码执行与生成行为；ST-WebAgentBench [303]用于评估 Web 代理在策略约束下的动作；而 OS-Sentinel[571]（与第4.5节所述的 SentinelAgent 运行时监控器不同）。）结合了形式验证与神经验证，用于真实 workflows 中的移动 GUI 代理。

(3) Faithfulness Verification. Faithfulness verification certifies that an agent follows the given instructions and stays on task, rather than drifting toward a superficially plausible but misaligned objective. Neuro-symbolic checkers translate natural-language instructions into formal constraints that an output can be mechanically checked against [563], while instruction-following evaluations for multi-turn, tool-using web agents test whether constraints stated early in a session are still honored many steps later [118]. This concern is sharpened by evidence of *goal drift*: agents can gradually abandon their original objective under accumulated context or contextual pressure, so faithfulness verification must track adherence across the whole trajectory rather than at a single step [20, 427]. In agentic settings it is thus the natural counterpart to the maker–checker separation of Section 4.1.3: an independent checker guards against completions that satisfy the visible goal while violating the user’s underlying intent.

(3) 忠实性验证。真实性验证旨在确认智能体是否遵循了给定的指令并始终专注于当前任务，而非偏离至一个表面上看似合理但与实际目标不符的目标。神经符号跳棋系统可将自然语言指令转换为形式约束，使得输出结果能够通过机械方式对其进行验证[563]。而针对多轮交互、工具使用型网络智能体的指令遵循评估，则用于检验会话初期设定的约束条件在后续多个步骤中是否仍能得到遵守[118]。这一担忧因目标漂移的证据而愈发凸显：智能体在累积的上下文或上下文压力下，可能会逐渐放弃其原始目标。因此，置信度验证必须追踪整个轨迹上的遵循情况，而非仅在单个步骤上进行[20, 427]。在代理型场景下，这正是第4.1.3节中“制造者–校验者分离”概念的自然对应形式。独立校验器可防止生成满足可见目标但违背用户潜在意图的完成结果。

(4) Multi-agent Cross-verification. Multi-agent cross-verification employs several models for mutual critique. Multiagent debate [139] shows that iterative inter-model deliberation substantially reduces factual errors, and Multi-Agent Verification [362] demonstrates that scaling the number of independent aspect verifiers yields weak-to-strong generalization.

(4) 多智能体交叉验证。多智能体交叉验证采用多个模型进行相互评析。多智能体辩论[139]表明，迭代式模型间协商能显著降低事实性错误；而多智能体验证[362]结果表明，增加独立方面验证器的数量可实现从弱到强的泛化能力。

4.6.2 Verification Levels

4.6.2 验证级别

Verification mechanisms can also be characterized by the stage at which they intervene during the agent’s execution lifecycle.

验证机制还可根据其在智能体执行生命周期中介入的阶段来加以描述。

(1) Step-level Verification. Step-level verification evaluates intermediate reasoning quality at each step, complementing hooks that enforce binary allow-or-block decisions via deterministic rules. Where a hook asks “is this action permitted?”, a step-level verifier asks “is this reasoning step productive toward the goal?”, producing scalar quality signals that guide search and correction rather than gating execution. Building on earlier evidence that training answer verifiers and combining process- with outcome-based feedback improves math reasoning [96, 618], process reward models (PRMs) [363] show that per-step supervision substantially outperforms outcome-only evaluation for complex reasoning, AgentPRM [702] extends this to agentic tasks by scoring each decision on goal proximity and incremental progress, and CRITIC [182] introduces a tool-interactive verify-then-correct loop that establishes external feedback as essential for reliable self-correction. Recent work advances step-level verifiers along three axes. *Generative verification*, which recasts reward modeling as next-token prediction [822], lets GenPRM [857] and ThinkPRM [272] reason step-by-step before scoring, showing that generative verifiers trained on minimal labels can match or outperform discriminative PRMs and rival strong proprietary judges on process benchmarks. *Domain-specialized PRMs* tailor verifiers to concrete agent domains: Web-Shepherd [49] builds structured subgoal checklists for web navigation at substantially lower cost than GPT-4o-based verification, while SWE-PRM [163] and SWE-TRACE [203] deliver trajectory-level course-correction and rubric-based dense feedback for software-engineering agents. *Efficient supervision* concentrates labels where they matter: CSO [329] restricts preference learning to verified critical steps that demonstrably flip task outcomes, requiring supervision at only a small fraction of trajectory steps, and ToolPRMBench [311] contributes the first large-scale PRM benchmark for tool-using settings, showing that reinforcement learning substantially improves robustness and generalization in tool-specialized verifiers. Most recently, step-level verification is moving toward fully *agentic* verifiers that themselves reason and invoke tools while scoring each decision, rather than emitting a single scalar in one pass, as in AgentV-RL [818].

(1) 分步验证。分步验证用于评估每个步骤中的中间推理质量，从而对通过确定性规则强制执行“允许或阻止”二元决策的挂钩机制进行补充。当某个挂钩提出“此操作是否被允许？”时……”，步级验证器会询问“当前推理步骤是否有助于达成目标？”，并生成标量质量信号来指导搜索与修正，而非用于控制执行流程。基于此前多项证据——即对答案验证者进行培训以及结合过程反馈与结果反馈可提升数学推理能力[96, 618]——过程奖励模型（PRMs）[363]研究表明，在复杂推理任务中，逐步监督法的性能显著优于仅基于结果的评估；AgentPRM[702]将此方法扩展至代理型任务，通过根据目标接近度和增量进展对每个决策进行评分；而 CRITIC[182]则采用了类似方法。本文引入了一种“先验证后修正”循环机制，该机制表明外部反馈对于实现可靠的自校正至关重要。近期研究在三个维度上对分步验证器进行了改进。*生成式验证*将奖励建模重新表述为下一个 token 的预测问题[822]，使得 GenPRM[857]和 ThinkPRM[272]可以应用。在评分前，本文将逐步阐述相关原因，证明：基于最小标注集训练的生成式验证器能够在流程基准测试中达到与判别式 PRM相当甚至优于其表现，并能与强大的专有评估器相媲美。*领域专用PRM*可将验证器定制化至特定智能体领域：Web-Shepherd[49]该方法可构建用于网页导航的结构化子目标检查清单，其成本远低于基于GPT-4o的验证方法，而SWE-PRM[163]与SWE-TRACE[203]亦然。为软件工程智能体提供轨迹级航向修正及基于评分标准的密集反馈。*高效监督*可将标签集中于关键位置：CSO[329]该方法将偏好学习限制在经过验证的关键步骤上——这些步骤能够明确改变任务结果，仅需在极少数轨迹步数处进行监督；ToolPRMBench[311]该研究提出了首个针对工具使用场景的大型PRM基准测试，结果表明，强化学习能显著提升工具专用验证器的鲁棒性与泛化能力。最近，步级验证正逐步向完全*具有自主性*的验证器方向发展——这类验证器能够自行进行推理并调用工具，在对每个决策进行评分的同时，而非像 AgentV-RL[818]那样仅在单次遍历中输出一个标量值。。

(2) End-of-run Verification. End-of-run verification evaluates agent outputs after task completion. The LLM-as-a-Judge paradigm [867] uses strong models to score final outputs, and its evolution toward agentic judges has proceeded along two lines. *Agent-as-a-Judge* augments the judge with tools and multi-step reasoning: Agent-as-a-Judge [898] equips the evaluator with tool-augmented assessment for substantially higher human agreement, and a recent paper [775] systematizes how agentic judges combine planning, tool-augmented verification, and persistent memory. *Multi-agent collaborative evaluation* leverages inter-model deliberation: CollabEval [489] runs a three-phase initial-evaluation, discussion, and consensus process that outperforms competitive debate; a complementary route instead scales test-time compute for judging itself, which we discuss under verifier strategies below. *Self-correction and environment grounding* close the loop after execution: Reflexion [544] performs verbal self-reflection with episodic memory across trials, though CorrectBench [605] finds that self-correction yields only limited gains on reasoning models at significant cost; environment-based frameworks such as WebArena [885] and SWE-bench [260] verify success through functional correctness of final states, and SWE-RM [546] complements test-suite checks with an execution-free reward model that measurably lifts SWE-bench Verified accuracy. Chain-of-Verification [121] decomposes a response into atomic claims for independent post-hoc checking. Moving beyond static post-hoc scoring, agentic verifiers increasingly probe the environment after a run: verifying a GUI agent through online, proactive interaction lets the judge actively confirm claimed outcomes rather than trust the final transcript alone [102].

(2) 运行结束验证。运行结束验证用于在任务完成之后对智能体的输出结果进行评估。LLM-as-a-Judge 模式[867]利用强模型对最终输出结果进行评分；该模式向“智能体裁判”方向的演进主要沿着两条路径展开。*Agent-as-a-Judge*通过为裁判提供工具和多步骤推理功能来增强其能力：Agent-as-a-Judge[898]可为评估者提供基于工具的评估支持，从而显著提高人工一致性；此外，近期的一篇论文[775]也涉及此主题。系统化地阐述了具有主体性的法官如何将规划、工具增强型验证与持久化存储相结合。*多智能体协同评估*利用了模型间的协商机制：CollabEval[489]采用三阶段流程——初始评估、讨论及共识达成过程，其性能优于竞争性辩论；另一种补充方案则是通过在测试阶段对计算过程进行缩放，以用于自评估——我们将在下文关于验证器策略的部分对此进行讨论。*自我修正与环境锚定*可在执行后闭合回路：Reflexion[544]会在不同试验间利用情景记忆进行言语自我反思，而CorrectBench[605]则采用其他方式。研究发现，自校正方法在推理模型上仅能带来有限的提升，且代价高昂；而基于环境的框架，如WebArena[885]和SWE-bench[260]通过最终状态的功能正确性来验证执行成功；而 SWE-RM[546]则在测试套件检查的基础上，引入了一种无需执行的奖励模型，该模型可显著提升 SWE-bench 的经验验证准确率。验证链[121]将响应分解为若干原子声明，以便进行独立的事后验证。在超越静态事后评分的基础上，代理验证器正日益倾向于在程序运行后对环境进行探查：例如通过在线方式对GUI代理进行验证。主动交互机制使法官能够积极确认所主张的结果，而非仅依赖最终笔录[102]。

4.6.3 Verifier Strategies

4.6.3 验证器策略

Orthogonal to temporal granularity, verifier quality can be improved through training-free architectural designs and learning-based methods, which we group into three families.

与时间粒度不同，验证器质量可通过无训练架构设计及基于学习的方法来提升；我们将这些方法归纳为三大类。

(1) Automated labels and implicit rewards. Math-Shepherd [643] automates process-label generation via Monte Carlo estimation, eliminating costly human annotation, and Implicit PRM [787] shows that process rewards can be extracted from outcome-trained models at no additional cost. Self-consistency [659] offers a complementary training-free signal through majority voting across diverse reasoning paths.

(1) 自动标签与隐性奖励。Math-Shepherd[643]通过蒙特卡洛估计实现流程标签的自动化生成，从而省去了高昂的人工标注成本；而 Implicit PRM[787]则表明，无需额外成本即可从结果训练模型中提取流程奖励。额外成本。Self-consistency[659]通过在多种推理路径上进行多数投票，提供了一个无需训练的补充信号。

(2) Tree search with verifier guidance. A family of methods couples Monte Carlo tree search with LLM self-evaluation for structured exploration. LATS [875] and rStar-Math [191] combine MCTS with model-generated

value estimates, AB-MCTS [436] adds adaptive branching that decides whether to “go wider” or “go deeper,” and ReST-MCTS* [805] develops reinforced self-training that harvests higher-quality reasoning traces through iterative process-reward-guided tree search.

(2) 基于验证器引导的树搜索。这一系列方法将蒙特卡洛树搜索与大语言模型（LLM）的自我评估相结合，用于结构化探索。LATS[875]与 rStar-Math[191]将 MCTS 与基于模型生成的价值估计相结合；AB-MCTS[436]引入了自适应分支机制，用于决定是“拓展搜索范围”还是“深入探索”；而 ReST-MCTS*[805]开发了强化自训练方法，该方法通过迭代过程中的过程奖励引导树搜索，能够获取更高质量的推理轨迹。

(3) Compute-optimal Verification Scaling. Test-time compute scaling [550] shows that verifier-guided search lets smaller models match substantially larger ones, and DeepSeek-R1 [112] demonstrates that self-verification can emerge from reinforcement learning with rule-based rewards alone. RLV [521] unifies LLM reasoners with learned value functions, improving MATH accuracy while enabling markedly more efficient scaling, while MCTS-Judge [665] carries test-time scaling into the LLM-as-a-Judge setting, markedly improving code-correctness accuracy through System-2 reasoning. Extending this scaling to specialized domains, tool-integrated verification interleaves reinforcement learning with external tools so that the verifier can certify multi-step reasoning that pure model judgment cannot [812].

(3) 计算最优验证缩放法。测试时计算规模扩展[550]表明，验证器引导搜索可使较小模型能够与显著更大的模型相匹配；而DeepSeek-R1[112]则证明，自验证机制可从强化学习中涌现。仅基于规则的奖励机制。RLV[521]将LLM推理器与学习到的价值函数相结合，不仅提升了MATH的准确率，还实现了显著更高的可扩展性；而MCTS-Judge[665]则将测试时可扩展性引入了“LLM作为裁判”的场景中。通过System-2推理，显著提升了代码正确性准确率。将这一扩展方法应用于特定领域时，工具集成验证技术将强化学习与外部工具相结合，从而使验证器能够对纯模型判断无法实现的多步推理进行认证[812]。

5 Optimization: Internalizing Long-Horizon Capability

5 优化：内化长周期能力

Overview. In Section 4, we describe how long-horizon capability can be supplied by the harness that specifies, steers, and consolidates each run. This section turns to *optimization*: how such capabilities can be progressively internalized into model parameters. We organize the section along the *training pipeline*. We begin with the *architectural substrate* (Section 5.1), which determines whether long-horizon training and inference are computationally feasible in the first place: a policy that cannot represent and efficiently process a long trajectory cannot be effectively trained on one. We then examine six stages of capability development: data and environment synthesis (Section 5.2), pre-training and mid-training (Section 5.3), fine-tuning (Section 5.4), reinforcement learning (Section 5.5), on-policy distillation (Section 5.6), and self-evolution (Section 5.7). For each stage, we identify the long-horizon capabilities it develops, the mechanisms through which they are acquired, and the limitations that remain. Figure 11 provides an overview of the pipeline, while Table 5 summarizes the characteristics, mechanisms, and representative works associated with each stage.

概述。在第4节中，我们将阐述如何通过该框架——该框架负责定义、引导并整合每条运行——来提供长周期处理能力。本节将探讨 *优化* 问题：即如何逐步将此类能力内化为模型参数。本节将按照 *训练流程* 的顺序展开论述。我们首先从 *架构基底*（第5.1节）开始），该条件决定了长时程训练与推理在计算上是否具有可行性：若某个策略无法对长轨迹进行表征或高效处理，则无法基于此类长轨迹对其进行有效训练。随后，我们分析了能力开发的六个阶段：数据与环境合成（第5.2节）、预训练与中期训练（第5.3节）以及微调（第5.4节）。），强化学习（第5.5节）、同策略蒸馏（第5.6节）以及自进化（第5.7节）。针对每个阶段，我们均会识别该阶段所培养的长周期能力、这些能力的获取机制以及仍存在的局限性。图11概述了整个流程，而表5则汇总了各阶段的特性、作用机制及代表性成果。

The Architectural Substrate and Six Training Stages	
建筑基底与六个培训阶段	
1. Architectural Substrate (Section 5.1): makes long trajectories efficient to train on and decode from.	1. 建筑基底 （第5.1节）：可使长轨迹在训练和解码过程中更加高效。
2. Data and Environment Synthesis (Section 5.2): constructs verifiable long-horizon experience, including tasks, environments, and trajectories, that later stages can use for training and feedback.	2. 数据与环境合成 （第5.2节）：构建可验证的长时程经验，包括任务、环境及轨迹，供后续阶段用于训练与反馈。
3. Pre-/Mid-training (Section 5.3): builds action–observation and reasoning priors, so agentic behavior becomes native rather than prompted.	3. 预训练/中期训练 （第5.3节）：构建动作–观察与推理先验，从而使具代理性的行为成为内生性而非外在触发的。
4. Fine-tuning (Section 5.4): instills behavioral and tool-use discipline from curated agentic trajectories.	4. 微调 （第5.4节）：通过精心设计的智能体行为轨迹，培养行为规范与工具使用纪律。
5. Reinforcement Learning (Section 5.5): optimizes whole trajectories and confronts long-horizon credit assignment under sparse, delayed reward.	5. 强化学习 （第5.5节）：可优化整个轨迹，并在稀疏且延迟的奖励条件下解决长时域信用分配问题。
6. On-policy Distillation (Section 5.6): consolidates the policy on its own state distribution, reducing distribution shift and stabilizing earlier learned behaviors.	6. On-policy Distillation （第5.6节）：该方法基于模型自身的状态分布来整合策略，可减少分布偏移，并稳定先前学习到的行为。
7. Self-evolution (Section 5.7): turns self-generated experience, feedback, and task-environment updates into continuing policy improvement.	7. 自进化 （第5.7节）：将自生成的经验、反馈及任务-环境更新转化为持续的策略优化过程。

5.1 Architectural Substrate

5.1 建筑基底

A long-horizon agent can internalize only what its architecture can represent: a policy that cannot ingest, retain, and efficiently use a long interaction history, such as observations, tool results, and constraints that accrue across many steps, cannot be trained or served on long trajectories. The architectural substrate therefore sets the ceiling for every later training stage, which is why we begin here. From an architecture view, the central question is how the model keeps and uses a long history. We group the main approaches into three representational classes: explicit-context architectures, compressed-state architectures, and hybrid architectures. These are not strict model families, but different ways to represent past information within the model. We discuss high-throughput mechanisms separately as a serving-oriented axis rather than as a fourth representation class. This view follows the main split in efficient Transformer and state-space papers [591]. We do not treat Mixture-of-Experts (MoE) as a fourth representational class. MoE routes tokens to different feed-forward experts, as in GShard and Switch Transformer [152, 301]. It primarily changes model

Table 5 The agentic training pipeline by stage. Each row is the architectural substrate or one of the training stages. **Representative mechanisms** lists the stage’s sub-mechanisms (one per line, following its subsections), and **representative works** pairs each sub-mechanism with two concrete works on the same line.

表5：按阶段划分的代理训练流水线。每行代表相应的架构基底或某一训练阶段。**代表性机制**列出了该阶段的子机制（每行对应一个子机制，且遵循其子章节顺序），而**代表性著作**则将每个子机制与同一条行上的两部具体著作进行配对。

Stage (§) 阶段 (§)	Internalized capability 内化能力	Representative mechanisms 代表性作用机制	Representative works 代表性作品
Architectural Substrate (§5.1) 建筑基材 (§5.1)	Affordably representing and serving long histories 以亲民价格呈现并传承悠久历史	• Explicit-context • 显性语境 • Compressed-state • 压缩态 • Hybrid memory • 混合内存 • High-throughput • 高通量	Longformer [32], Big Bird [792] Mamba [186], RWKV [476] Jamba [300], Kimi Linear [842] EAGLE [352], GQA [5]
Data & Environment Synthesis (§5.2) 数据与环境综合 (§5.2)	Executable, verifiable 可执行、可验证 long-horizon experience 长周期经验	• Task synthesis • 任务合成 • Environment synthesis • 环境综合 • Trajectory synthesis • 轨迹合成 • Reasoning priors • 推理先验分布	TaskCraft [540], WebShaper [590] TaskCraft[540]、WebShaper[590] SWE-Gym [467], EnvScaler [555] TOUCAN [737], AgentGym-RL [701] SWE-Gym[467]、EnvScaler[555]、TOUCAN [737]、AgentGym-RL[701]
		• Long-context priors • 长上下文先验 • Multimodal priors • 多模态先验 • Data-mixture design • 数据混合设计 • Selection & mixing	
Pre-/Mid-training 训练前期/中期 (§5.3)	Reasoning and perception priors 推理与感知先验	• Long-context priors • 长上下文先验 • Multimodal priors • 多模态先验 • Data-mixture design • 数据混合设计 • Selection & mixing	DeepSeek-V3 [110], Qwen3-Coder [46] YaRN [477], LongLoRA [78] Qwen2-VL [644], InternVL3 [892] DoReMi [715], OPUS [649]
		• 筛选与混合 • Curriculum • 课程设置 • Distillation • 蒸馏 • Credit assignment • 信用分配	
Fine-tuning (§5.4) 微调 (§5.4)	Behavioral and tool-use discipline 行为与工具使用规范	• Policy optimization • 策略优化 • Sampling strategy • 抽样策略 • Interaction patterns • 交互模式	AgentTuning [794], LIMi [711] ScalingInter-RL [701], E2H Reasoner [469] Agent Distillation [269], Agent-R [789]
		• Teacher-guided • 教师引导式 • Self-improving • 自我提升	
Reinforcement Learning (§5.5) 强化学习 (§5.5)	Feedback-driven long-horizon decisions 反馈驱动的长期决策	• Offline bootstrap • 离线自举 • Online interaction • 在线互动 • Co-evolution • 共进化	ToolRL [487], RuscaRL [887] DAPO [780], GiGPO [156] Tree-GRPO [250], TCOd [636] GLIDER [232], Memory-R1 [743]
On-policy Distillation 正策略蒸馏 (§5.6)	Consolidating behavior on the policy’s own states 在政策自身的状态上进行整合操作	• Teacher-guided • 教师引导式 • Self-improving • 自我提升	GKD [3], DAgger-LLM [309] GKD[3]、DAgger-LLM[309] π -Play [838], Self-Distillation Zero [216] -Play[838]，自蒸馏零值[216]
Self-evolution (§5.7) 自进化 (§5.7)	Cross-task experience accumulation 跨任务经验积累	• Offline bootstrap • 离线自举 • Online interaction • 在线互动 • Co-evolution • 共进化	STaR [793], rStar-Math [191] R-Zero [233], Agent0 [704] Env Tuning [401], Agent-World [131]

capacity, but it does not by itself decide how the sequence history is stored. Accordingly, MoE models can still use explicit attention, latent attention, or hybrid blocks [111, 300, 601].

长时程智能体仅能内化其架构所能表征的内容：即那些无法摄取、保留并高效利用长交互历史（例如在多个步骤中累积的观测数据、工具输出结果及约束条件）的策略。该模型无法在长轨迹上进行训练或服务。因此，其架构基础为后续每个训练阶段设定了上限；这也正是我们从这里开始的原因。从架构视角来看，核心问题在于模型如何保持并利用长历史信息。我们将主要方法归纳为三大表征类别：显式上下文架构、压缩状态架构以及混合架构。这些并非严格的模型家族，而是表示模型内部历史信息的不同方式。我们对高通量机制进行单独讨论，将其视为一种服务导向的维度，而非第四类表征类别。该观点遵循了高效Transformer与状态空间相关论文之间的主要分歧[591]。我们并未将“专家混合”（MoE）视为第四种表示类别。MoE将令牌分配至不同的前馈专家，这与GShard和Switch Transformer[152, 301]中的机制一致。该机制主要改变模型的容量，但并不直接决定序列历史如何存储。因此，MoE模型仍可采用显式注意力、潜在注意力或混合模块[111, 300, 601]。

5.1.1 Explicit-Context Architectures

5.1.1 显式上下文架构

Explicit-context architectures keep history as a token sequence. The model then uses attention to route information across these tokens. Full attention gives the cleanest form of this design, because any token can attend to any other token [620]. Long-context Transformer variants reduce the cost with local attention, sliding windows, block-sparse attention, or global tokens. Longformer and Big Bird are classic examples of this direction [32, 792]. Mistral 7B uses sliding-window attention and grouped-query attention (GQA) for lower-cost decoding [252]. Gemma 2 and Gemma 3 use local-global attention patterns. Gemma 3 uses more local attention layers to reduce KV-cache cost at long context [595, 596]. Efficient Transformer papers give a broader view of this design space [591]. More recent work makes sparsity *natively trainable* rather than an inference-time approximation, making long-horizon training affordable: DeepSeek NSA couples a hardware-aligned, natively trainable sparse-attention design with efficient kernels [786], and MoBA applies a mixture-of-block-attention routing that lets each query attend to a learned subset of context blocks [398].

显式上下文架构将历史信息表示为一个标记序列；随后，该模型利用注意力机制对这些标记之间的信息进行路由处理。全注意力机制能实现该设计中最简洁的形式，因为任何标记均可关注任意其他标记[620]。长上下文Transformer变体通过采用局部注意力、滑动窗口、块稀疏注意力或全局标记等方式来降低计算成本。Longformer和Big Bird便是这一研究方向的经典范例[32, 792]。Mistral 7B 采用滑动窗口注意力机制与分组查询注意力机制（GQA）以降低解码成本[252]；而 Gemma 2 和 Gemma 3 则采用局部-全局注意力模式。Gemma 3采用更多的局部注意力层，以降低长上下文下的KV缓存成本[595, 596]；而高效的Transformer相关论文则从更广阔的视角探讨了这一设计空间[591]。较新的研究使得稀疏性能够 *原生地进行训练*。DeepSeek NSA 并非采用推理时近似方法，而是使得长时程训练变得可行：该模型将与硬件高度契合的、可原生训练的稀疏注意力设计与高效的内核相结合[786]。MoBA采用混合块注意力路由机制，使每个查询能够关注由学习得到的上下文块子集[398]。

This class is useful for long-horizon agents because earlier states remain explicitly represented. The model can, in principle, look back at an old observation, a tool result, or a user constraint. These objects are common in reasoning-and-acting agents and web agents [761, 885]. The cost is the main weakness. As the context grows, attention compute and key-value (KV) cache size also grow [5, 620]. If the model uses a sparse pattern, it may also miss a remote token that matters. Long-context benchmarks and analyses show that locating useful evidence inside long inputs remains hard [28, 378]. Many open base model reports still fit this class. Qwen2.5 and Qwen3 mainly use decoder-only Transformer architectures; Qwen3 adds both dense and MoE variants, but the history is still accessed through attention [601, 744]. The GLM series follows the same pattern. The ChatGLM report describes the path from GLM-130B to GLM-4. GLM-4.5 is an MoE foundation model for agentic, reasoning, and coding tasks, but it is still an attention-based Transformer family [178, 795].

该类模型对于长时程智能体非常有用，因为早期状态仍能被显式表示。从原理上讲，该模型可回顾历史观测值、工具结果或用户约束条件。这些对象在推理与行动智能体及网络智能体中十分常见[761, 885]。其主要缺点在于成本问题：随着上下文规模的扩大，注意力计算量及键值（KV）缓存大小也会相应增大[5, 620]。

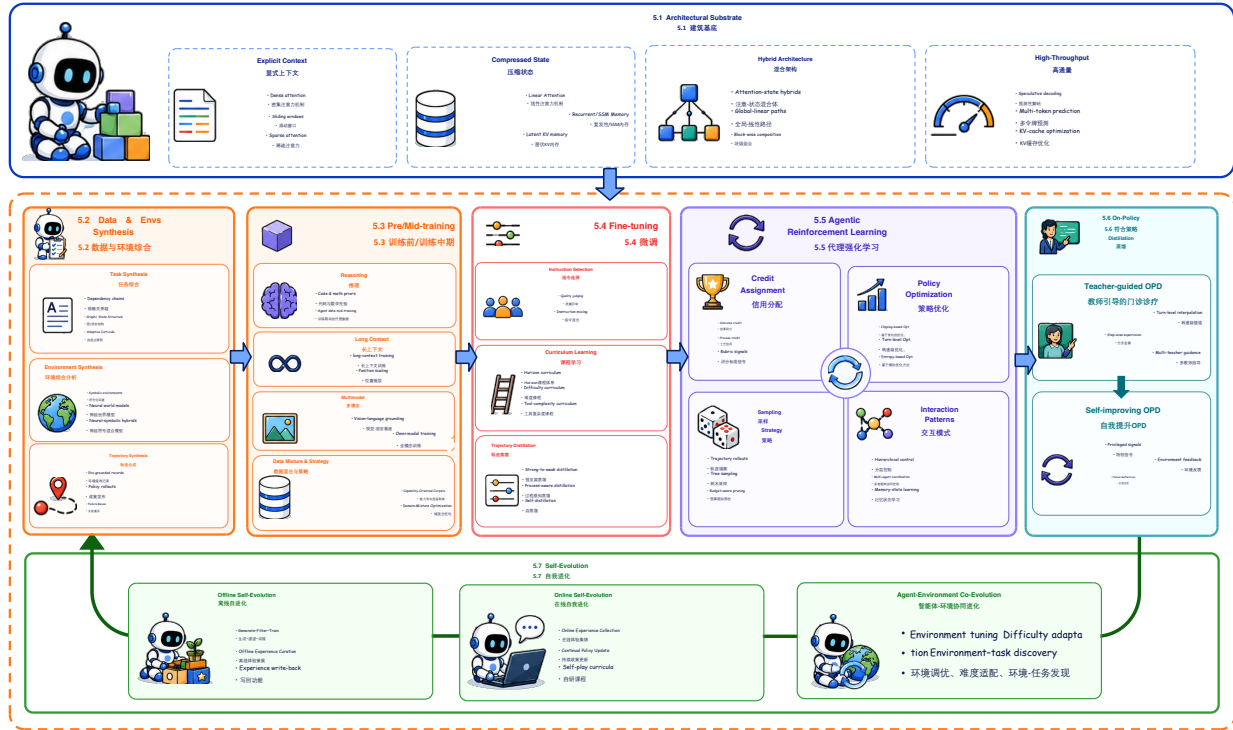


Figure 11 The agentic training pipeline for internalizing long-horizon capabilities. As the internalized counterpart to the externalized harness described in Section 4, the pipeline builds on an architectural substrate that supports long-sequence representation and proceeds through six stages. Each stage either internalizes a capability previously provided at runtime or generates the experience required for its subsequent internalization.

图11：内化长周期能力的代理式训练流水线。作为第4节所述外化约束结构的内化对应物，该流水线基于一种支持长序列表示的架构基底，并经过六个阶段。每个阶段要么内化此前在运行时提供的能力，要么生成其后续内化所需的体验。

若模型采用稀疏模式，可能会遗漏至关重要的远程标记。长上下文基准测试与分析表明，在长输入中定位有效证据仍具有挑战性[28, 378]。许多开源基础模型报告仍属于此类。Qwen2.5 和 Qwen3 主要采用仅解码器的 Transformer 架构；Qwen3 还引入了稠密架构和 MoE 变体，但其历史查询仍通过注意力机制实现[601, 744]。GLM系列遵循相同的模式。ChatGLM报告阐述了从GLM-130B到GLM-4的演进路径。GLM-4.5 是一个适用于代理型、推理及编码任务的MoE基础模型，但它仍属于基于注意力机制的Transformer系列[178, 795]。

5.1.2 Compressed-State Architectures 5.1.2 压缩态架构

Compressed-state architectures do not keep all pairwise token interactions. They instead encode history into a more compact internal representation. Some methods compress the attention map or the token set, as in low-rank attention and kernelized linear attention [92, 651]. Other methods use a recurrent or state-space form. This includes Retentive Network (RetNet), Receptance Weighted Key Value (RWKV), structured State Space Models (SSMs), and selective SSMs such as Mamba [186, 187, 476, 576].

压缩状态架构并不保留所有两两标记间的交互关系；相反，它们将历史信息编码为一种更为紧凑的内部表示。部分方法会对注意力图或令牌集进行压缩，例如低秩注意力和核化线性注意力[92, 651]；而其他方法则采用循环结构或状态空间形式。这包括保持网络（RetNet）、接收权重键值模型（RWKV）、结构化状态空间模型（SSMs）以及选择性SSMs，例如Mamba[186, 187, 476, 576]。

This class fits the stream-like nature of agent work. In many agent settings, a model observes, reasons, acts, and observes again [761, 885]. A compressed state can be updated step by step, so the runtime memory can stay small [186, 187, 476, 576]. The weakness is precision. Once old tokens are folded into a state, the model may not recover a rare fact or an exact old string. This is a core trade-off between efficient recurrence and content-level access [473, 606]. This is a problem when an agent must obey a past constraint exactly, as in tool-use and web tasks that carry instructions across steps [761, 885]. A partial, and importantly different, base-model version of this idea is Multi-head Latent Attention (MLA), which compresses the KV cache into a latent form. DeepSeek-V2 introduced MLA together with DeepSeekMoE, and DeepSeek-V3 kept this design at larger scale [110, 111]; Kimi-K2 likewise pairs a large MoE model with MLA [597]. Unlike recurrent or state-space compression, however, MLA keeps a per-token latent and still attends over all past tokens, so it preserves per-token addressability and does not incur the exact-recall loss described above. It is therefore best read not as a recurrent compressed state but as a low-rank KV compression of explicit context, sitting between the two classes and doubling as a throughput mechanism (Section 5.1.4).

该类模型契合智能体工作的流式特性。在许多智能体场景中，模型会进行观察、推理、行动，然后再进行再次观察[761, 885]。压缩状态可分步更新，因此运行时内存可保持较小[186, 187, 476, 576]。其缺点在于精度问题：一旦旧的标记被折叠至状态中，模型可能无法复现罕见事实或精确的旧字符串。这是高效递归与内容级访问之间的核心权衡[473, 606]。当智能体必须严格遵守某种历史约束时（例如在涉及跨步骤执行指令的工具使用或网页操作任务中[761, 885]），这一问题便随之出现。这一理念的一个部分且尤为重要基础模型版本是多头潜在注意力机制（MLA），该机制将KV缓存压缩为潜在形式。DeepSeek-V2在引入DeepSeekMoE的同时引入了MLA；而DeepSeek-V3在更大规模的场景下延续了这一设计[110, 111]；Kimi-K2同样将大型MoE模型与MLA结合使用[597]。然而，与循环压缩或状态空间压缩不同，MLA为每个词元维护一个独立的潜在表示，并仍对所有历史词元进行注意力机制计算；因此，它保留了针对单个词元的可寻址性，且无需承担上述所述的精确召回损失。因此，最好将其不视为一种循环的压缩状态，而应将其理解为对显式上下文进行的低秩KV压缩（该机制处于上述两类方法之间，并兼作一种吞吐量机制，参见第5.1.4节）。

5.1.3 Hybrid Architectures

5.1.3 混合架构

Hybrid architectures combine explicit attention with compressed or recurrent state [300, 842]. The goal is to combine precise content access with efficient long-range state tracking. Attention can keep precise access to important tokens [378, 620]. A compressed state can carry long-range signals at low cost [186, 606]. In this view, attention is used for exact evidence, while the state module is used for long-term continuity [300, 842]. Jamba is a direct example: it mixes Transformer layers, Mamba layers, and MoE layers [300]. Kimi Linear is another clear example. It combines Kimi Delta Attention units with a global MLA layer. The model therefore keeps both a linear-attention path and an explicit global attention path [842].

混合架构将显式注意力机制与压缩或循环状态相结合[300, 842]，其目标是将精确的内容访问与高效的长程状态跟踪相结合。注意力机制能够精确地访问重要标记[378, 620]；压缩状态可以较低的成本传递长程信号[186, 606]。在这种设计思路中，注意力机制用于获取精确证据，而状态模块则用于实现长期连续性[300, 842]。Jamba 即为一个典型示例：它融合了 Transformer 层、Mamba 层以及 MoE 层[300]。Kimi Linear 是另一个典型范例。它将 Kimi Delta 注意单元与全局 MLA 层相结合，因此该模型既保留了线性注意力路径，又保留了显式的全局注意力路径[842]。

This makes hybrid design a plausible direction for long-horizon agents. Agent runs need both exact recall and cheap long-term tracking, as shown by web-agent and long-context tasks [28, 885]. Pure attention gives direct access but can be expensive at long context [591, 620]. Pure compressed state is cheaper but may lose details [473, 606]. A hybrid model can assign these roles to different layers, heads, or blocks, as in Jamba and Kimi Linear [300, 842]. The reports above show the same direction at system scale: strong models often combine attention, KV-cache reduction, and MoE routing [110, 597, 601]. These parts solve different problems and should not be collapsed into one label [5, 152, 301].

这使得混合设计成为长时程智能体的合理发展方向。智能体运行需要兼具精确的回忆能力与低成本的长期跟踪能力，这一点在Web-Agent和长上下文任务中得到了体现[28, 885]。纯注意力机制可直接访问信息，但当上下文长度较长时成本较高[591, 620]；而纯压缩状态方案成本较低，但可能丢失部分细节[473, 606]。混合模型可将这些角色分配给不同的层、头或块，例如在 Jamba 和 Kimi Linear[300, 842]中。上述报告在系统层面均呈现相同趋势：优秀的模型通常会结合注意力机制、KV缓存缩减以及MoE路由[110, 597, 601]。这些部分解决了不同的问题，不应合并为同一个标签[5, 152, 301]。

5.1.4 High-Throughput Mechanisms

5.1.4 高通量机制

High-throughput mechanisms form a second axis under architecture [5, 111, 180, 302, 535]. They do not decide how the model stores long history; they reduce the serial work or cache traffic needed to decode from that history [5, 111, 180, 302, 535]. A complementary, kernel-level route keeps attention *exact* but IO-aware: FlashAttention tiles the computation to avoid materializing the full attention matrix in high-bandwidth memory, cutting memory traffic and enabling longer sequences without altering the model’s outputs [108]. One group reduces serial dependence in the output path through parallel generation with diffusion language models [559] or through autoregressive lookahead [43, 180, 302, 352]. Multi-token prediction trains the model to predict several future tokens at each position, so the model learns a longer local prediction horizon [180]. Speculative decoding uses a draft model to propose several tokens and lets the target model verify them in parallel [302]. Medusa keeps the draft process inside one model by adding multiple decoding heads [43]. EAGLE moves the draft signal into feature space and treats feature uncertainty as central to speculative sampling [352]. DeepSeek-V3 uses multi-token prediction as an auxiliary training objective and also presents it as a path to speculative decoding [110].

高通量机制构成了架构下的第二轴[5, 111, 180, 302, 535]。它们并不决定模型如何存储长历史记录；而是减少了从该历史记录进行解码所需的串行计算量或缓存流量[5, 111, 180, 302, 535]。一个互补的内核级路由机制保持了*精确*的注意力计算，同时兼顾 I/O 效率：FlashAttention 采用分块计算方式，从而避免在高带宽内存中显式构建完整的注意力矩阵。可减少内存流量，并能在不改变模型输出的情况下支持更长的序列[108]。其中一个组通过采用基于扩散语言模型的并行生成方法[559]或通过自回归前瞻机制[43, 180, 302, 352]来降低输出路径中的序列相关性。多标记预测通过训练模型在每个位置预测多个未来标记，从而使模型能够学习更长的局部预测范围[180]。推测式解码利用草稿模型提出若干标记，并允许目标模型并行验证这些标记[302]。Medusa 通过添加多个解码头，将草稿生成过程保留在同一个模型内部[43]。EAGLE将草图信号转换至特征空间，并将特征不确定性视为推测采样的核心要素[352]。DeepSeek-V3采用多令牌预测作为辅助训练目标，并将其视为实现推测解码的一种途径[110]。

The other group reduces the cost of reading and writing the KV cache [5, 111, 535]. Multi-query attention shares key and value projections across query heads to speed decoding [535]. Grouped-query attention keeps more key-value groups than multi-query attention, but still reduces the cache cost relative to full multi-head attention [5]. MLA compresses the KV cache into a latent form, so it is both a compressed-state design and a throughput design [110, 111]. These mechanisms matter for long horizon agents because long runs often produce many intermediate tokens after a large context has already been built [180, 302, 761, 885]. They reduce the serving cost of long decoding, but they do not remove the need for explicit recall, compressed state, or hybrid memory when old evidence must be found accurately [28, 180, 302, 378].

另一类方法可降低KV缓存的读写成本[5, 111, 535]；多查询注意力机制则通过在多个查询头之间共享键和值投影来加速解码过程[535]。分组查询注意力机制比多查询注意力机制能保留更多的键值对组，但仍能较完整的多头注意力机制降低缓存成本[5]。MLA将KV缓存压缩为潜在形式，因此兼具压缩状态设计与吞吐量优化设计特性[110, 111]。这些机制对长时程智能体至关重要，因为当大型上下文已构建完毕后，长期运行往往会产生大量中间令牌[180, 302, 761, 885]。这些方法降低了长时解码的计算成本，但并未消除在需要准确检索旧证据时对显式召回、压缩状态或混合存储器的需求[28, 180, 302, 378]。

5.2 Data and Environment Synthesis

5.2 数据与环境综合分析

Long-horizon behavior is learned from interaction experience that is executable, feedback-bearing, and reusable. Unlike static instruction–response pairs, such experience records how an agent decides, observes feedback, corrects errors, and accumulates information over a long trajectory. Deployment logs, demonstrations, and benchmark rollouts offer natural sources of such data, but their direct use is constrained by acquisition cost, privacy, and limited reproducibility. Controlled data synthesis has therefore become an important route for constructing long-horizon training experience. It centers on three coupled objects: *tasks* define goals and capability pressures, *environments* instantiate execution and verification, and *trajectories* turn interaction histories into trainable signals.

长时程行为可从可执行、具反馈且可复用的交互经验中习得。与静态指令–响应对不同，此类经验记录了智能体如何做出决策、观察反馈、纠正错误以及在较长的轨迹上累积信息的过程。部署日志、演示案例及基准测试部署均可作为此类数据的天然来源，但其直接应用受限于获取成本、隐私问题以及可复现性不足等因素。因此，可控数据合成已成为构建长时程训练经验的重要途径。该框架围绕三个耦合对象展开：*任务*定义目标与能力压力，*环境*实现执行与验证，而*轨迹*则将交互历史转化为可训练信号。

5.2.1 Task Synthesis

5.2.1 任务综合

Task synthesis determines the objective distribution for long-horizon agents. A task defines the target behavior through goals, constraints, initial and terminal states, and success criteria. Task generation sits at the entry of interactive experience collection, since a task definition largely fixes horizon, dependency structure, and failure modes [238, 808]. Existing methods mainly construct tasks through dependency chains, structured information spaces, state transitions, and capability-adaptive curricula.

任务合成决定了长时程智能体的目标分布。任务通过目标、约束条件、初始状态与终止状态以及成功判定标准来定义目标行为。任务生成位于交互式体验采集的起点，因为任务定义在很大程度上决定了时间范围、依赖关系结构及故障模式[238, 808]。现有方法主要通过依赖链、结构化信息空间、状态转移以及能力自适应课程来构建任务。

Dependency-chain Construction. The most direct way to lengthen horizon is to chain subgoals so that early decisions constrain the actions later available. Long-horizon diagnostics relate failures to intrinsic horizon and compositional depth [658]; horizon-length studies further show that longer tasks aggravate optimization instability and credit assignment [279]. AgentSynth composes easy-to-generate subtasks into longer computer-use tasks, while TaskCraft expands atomic tasks into multi-hop, multi-tool objectives [540, 713]. The horizon here grows through necessary intermediate dependencies rather than through mere interaction length.

依赖关系链构建。延长时间范围的最直接方法是将子目标串联起来，使得早期做出的决策能够约束后续可执行的操作。长时程诊断将故障归因于内在视野与组成深度[658]；关于视野长度的研究进一步表明，任务时长越长，会加剧优化不稳定性及信用分配问题[279]。AgentSynth 将易于生成的子任务组合成更长的计算机使用任务，而 TaskCraft 则将原子级任务扩展为多跳、多工具的目标[540, 713]。此处的视界是通过必要的中间依赖关系而非单纯的交互长度而扩展的。

Graph-structured Information Construction. When horizon arises from evidence rather than from actions, tasks are drawn as paths through web pages, documents, tools, or evidence graphs, and the difficulty shifts to selecting relevant evidence under branching, incomplete, or masked observations. WebWalker and WebDancer construct multi-step information-seeking tasks from traversal, crawling, and backward question evolution [689, 690]. WebSailor uses knowledge-graph walks and information masking, while WebShaper formulates information seeking through set and knowledge projection to control evidence structure, answer space, and verifiability [327, 590]. WebSailor-V2 and WebWeaver further extend this line to more open-ended information-seeking tasks [326, 356].

图结构信息构建。当“视界”源于证据而非行动时，任务会被呈现为穿过网页、文档、工具或证据图谱的路径；此时，难点则转变为在分支式、不完整或被遮蔽的观测条件下，选择相关的证据。WebWalker 和 WebDancer 通过遍历、爬取及逆向问题演化等机制构建多步骤信息检索任务[689, 690]。WebSailor采用知识图谱遍历与信息屏蔽技术；而WebShaper则通过集合与知识投影来构建信息寻求过程，从而控制证据结构、答案空间及可验证性[327, 590]。WebSailor-V2 和 WebWeaver 进一步将这一系列方法扩展至更为开放的信息检索任务[326, 356]。

State-transition Construction. A third view frames a task as the gap between an initial and a target state, or as recovery from an erroneous one. In software settings, SWE-Gym, R2E, and SWE-smith reconstruct tasks from repositories, issues, commits, and tests [248, 467, 748]. Reverse synthesis makes this state-transition view explicit: DIVE generates tasks from executed evidence traces, TASTE evolves valid tool-sequence skeletons, and CLI-Gym derives repair tasks from abnormal command-line states and historical errors [53, 271, 369]. Execution traces and state gaps are thereby recast as tasks with verifiable outcomes.

状态转换建模法。第三种视角将任务表述为初始状态与目标状态之间的差异，或视为从错误状态中恢复的过程。在软件配置环境中，SWE-Gym、R2E 和 SWE-smith 均可基于代码库、问题、提交记录及测试用例进行重构任务[248, 467, 748]。逆向合成将这种状态转换视角显式化：DIVE可根据已执行的证据轨迹生成任务；TASTE可演化出有效的工具序列骨架；而CLI-Gym则可从异常命令行状态及历史错误中推导出修复任务[53, 271, 369]。因此，执行轨迹与状态间隙可被重新表述为具有可验证结果的任务。

Curriculum-driven Construction. The routes above fix a task distribution in advance; curricula instead let difficulty track the current policy, reducing the mismatch between static task sets and evolving capability. WebRL generates new web tasks from failed attempts, and Learn-by-interact reconstructs instructions and supervision from environment documents and interaction histories [484, 562]. AgentFrontier targets the model’s zone of proximal development, while CuES uses exploration and curiosity signals to synthesize environment-grounded RL tasks [75, 417]. Self-evolving agent papers view such mechanisms as part of joint agent–task evolution [165].

课程驱动式构建。上述路径预先确定了任务分配方案；而课程机制则允许难度随当前策略动态调整，从而减少了静态任务集与不断演进的能力之间的不匹配。WebRL可从失败尝试中生成新的网络任务，而“Learn-by-interact”则能根据环境文档和交互历史重建指令与监督信息[484, 562]。AgentFrontier针对模型的最近发展区，而CuES则利用探索与好奇心信号来合成基于环境的强化学习任务[75, 417]。自进化智能体相关文献将此类机制视为智能体与任务共同进化的组成部分[165]。

These routes impose complementary capability pressures on plan maintenance, evidence selection, and state recovery, whereas curricula act orthogonally, tuning how fast difficulty grows rather than what it tests. Which pressure to emphasize depends on the target deployment, since a distribution poorly matched to the policy tends to either saturate or destabilize training.

这些路径对计划维护、证据筛选及状态恢复施加了互补性的能力压力，而课程则呈正交关系——其作用在于调节难度增长的速率，而非决定所测试的内容。应强调何种压力取决于目标部署，因为与策略不匹配的分布往往会导致训练饱和或训练不稳定。

5.2.2 Environment Synthesis

5.2.2 环境综合设计

Environment synthesis supplies the training ground where long-horizon agents accumulate the interaction experience that static corpora cannot provide. Whereas tasks specify what to solve, environments determine how actions are executed, observed, and rewarded, closing the action–observation–feedback loop that turns a passive predictor into an active decision maker. A synthesized environment makes this loop reproducible, resettable, parallelizable, and verifiable, so that agents can act, fail safely, and receive checkable feedback at scale. Real repositories, browsers, APIs, databases, desktop systems, and game simulators provide fidelity [145], but only controlled synthesis makes such interaction cheap and reliable enough for repeated rollout. We organize environment synthesis into three families according to how their transitions and feedback are produced: symbolic environments, neural environments, and neural–symbolic hybrids.

环境合成提供了训练平台，使长时程智能体能够积累静态语料库无法提供的交互经验。任务规定了需解决的问题，而环境则决定了动作的执行方式、观察方式及奖励机制，从而闭环于“动作–观察–反馈”循环之中，将被动的预测器转变为主动的决策者。合成环境使得该循环能够实现可复现、可重置、可并行化及可验证，从而使智能体能够在大规模场景下执行操作、安全地处理失败，并接收可验证的反馈。真实的存储库、浏览器、API、数据库、桌面系统及游戏模拟器均能提供高保真度[145]，但唯有经过可控合成的方案，才能使此类交互足够经济且可靠，以支持重复部署。我们根据环境的转换与反馈机制生成方式，将环境综合划分为三大类：符号环境、神经环境以及神经-符号混合环境。

Symbolic Environments. Symbolic environments implement transitions and feedback through code, tests, databases, or business logic, which yields stable execution and verifiable hard signals. This family has developed along several technical routes, distinguished by the substrate they make executable. A first route builds *repository-level software environments*: SWE-Gym packages real repositories, issues, dependencies, and tests into executable tasks, and follow-up systems scale repository construction, test generation, and task expansion [248, 467, 748, 797, 856]. A second route targets *terminal and command-line environments*, where Terminal-Bench standardizes hard, realistic shell tasks and later work co-derives instructions, sandboxes, and teacher trajectories to scale terminal-agent training [82, 86, 369, 428]. A third route provides *GUI, web, and game execution environments*, represented by WebArena, OSWorld, and GameWorld and extended toward broader interface and generation settings [251, 461, 695, 716, 852, 885]. A fourth route constructs *tool- and domain-specific environments*, ranging from multi-step tool substrates such as AgentScaler to grounded scientific and medical settings [148, 538, 555, 626, 729]. Across these routes, explicit rules stabilize verification but remain less effective under ambiguity, dynamic interfaces, and cross-tool side effects.

符号化环境。符号化环境通过代码、测试、数据库或业务逻辑来实现状态转换与反馈，从而确保执行过程的稳定性，并生成可验证的硬信号。该家族在多个技术路径上发展起来，其特点在于它们所支持的执行基底。第一种方法构建 **仓库级软件环境**：SWE-Gym 将实际的代码仓库、问题、依赖关系及测试项封装为可执行任务；随后的系统则负责扩展仓库构建与测试生成过程。以及任务扩展[248, 467, 748, 797, 856]。第二条路径针对 **终端与命令行环境**，其中Terminal-Bench可标准化硬性、真实的Shell任务，并后续通过协同推导生成指令、沙箱及教师轨迹，从而实现终端-智能体训练的规模化扩展。[82, 86, 369, 428]。第三条路径提供了 **GUI、Web 及游戏执行环境**，其代表产品包括WebArena、OSWorld和GameWorld，并可进一步扩展至更广泛的界面与生成设置[251, 461, 695, 716, 852, 885]。第四种途径构建了 **工具与领域特定的环境**，其范围涵盖从诸如AgentScaler之类的多步骤工具基底到基于具体科学及医学场景的环境[148, 538, 555, 626, 729]。在这些路径中，显式规则能够稳定验证过程，但在存在歧义、动态接口及跨工具副作用的情况下，其效果则相对较弱。

Neural Environments and World Models. Neural environments approximate transitions, observations, or feedback with LLMs, visual generators, or latent dynamics, reducing the cost of repeated interaction. These methods treat the model as a *world action model*: conditioned on the interaction history and the agent’s action, it simulates the next round of environmental feedback, so that rollout no longer touches the real system. Three technical routes differ mainly in the representation over which the next state is predicted. The first route learns *pixel-level world models*, represented by DIAMOND and GameNGen, which generate action-conditioned visual futures and have since been extended to embodied and open-world settings [2, 6, 89, 170, 196, 249, 619, 769, 839]. The second route builds *text-level simulators* better suited to web, GUI, tools, and code, where WebDreameer predicts future web states and later systems structure UI, webpage, or software states [47, 149, 188, 572, 661, 712, 870]; recent language world models push this route toward general agents, learning next-state prediction across many agent domains and coupling generative simulators with agents in difficulty-aligned co-evolution [195, 905]. The third route learns *latent world models* in the style of joint-embedding prediction, which compress dynamics for cheaper long-horizon prediction [24, 25, 30, 176]. Neural environments scale cheaply, but they may introduce hallucination, drift, unreliable feedback, and sim-to-real gaps.

神经环境与世界模型。神经环境利用大型语言模型、视觉生成器或潜在动力学来近似状态转移、观测值或反馈，从而降低重复交互的成本。这些方法将模型视为 **世界动作模型**：在给定交互历史和智能体动作的条件下，该模型会模拟下一轮环境反馈过程，从而使rollout过程不再涉及真实系统。三种技术路径的主要区别在于预测下一状态时所采用的表示方式。第一种方法学习 **像素级世界模型**，该模型由DIAMOND和GameNGen实现，能够生成基于动作的视觉未来，并已进一步扩展至具身化及开放世界场景[2, 6, 89, 170, 196, 249, 619, 769, 839]。。第二条路径构建了 **文本级模拟器**，这些模拟器更适用于Web、GUI、工具及代码场景；其中，WebDreameer可预测未来的Web状态，并随后生成系统的UI、网页或软件状态[47, 149, 188, 572, 661, 712, 870]。近期的语言世界模型正推动这一研究路径向通用智能体方向发展，即在多个智能体领域中学习下一状态预测，并将生成式模拟器与难度对齐的共进化中的智能体相结合[195, 905]。第三种方法采用联合嵌入预测的风格来学习 **潜在世界模型**，该方法通过压缩动力学信息以实现更低成本的长时程预测[24, 25, 30, 176]。神经环境的扩展成本较低，但可能引发幻觉、漂移、不可靠的反馈以及仿真与真实之间的差距。

Neural–Symbolic Hybrid Environments. Neural–symbolic hybrid environments keep hard state and final verification in databases, file systems, programs, or APIs, while using models for user simulation, language interfaces, or hard-to-specify observations. They can be grouped by where the model is inserted into an otherwise symbolic loop. One route places models at the *user and application interface*, as in τ -bench, which combines stateful applications, user simulation, and final-state checks [399, 614, 762]. A second route standardizes *tool-server backends* through MCP [299, 411, 668, 700]. A third route pairs *generated content with symbolic constraints*, coupling generated webpages with state-machine constraints and completion checks [50, 695, 852]. A fourth route grounds *domain environments* in real data, retrieval, or computation while using models to expand tasks and feedback [538, 626, 804, 894]. The design choice throughout is where to draw the boundary between model-based simulation and symbolic verification.

神经-符号混合环境。神经-符号混合环境将硬状态和最终验证保留在数据库、文件系统、程序或API中，同时利用模型进行用户仿真、语言接口处理或对难以明确描述的观测结果进行处理。这些项可根据模型在符号循环中的插入位置进行分组。其中一种方法将模型置于 **用户与应用程序接口**处，例如 τ -bench，该方法结合了有状态应用程序、用户仿真及最终状态验证[399, 614, 762]。第二条路径通过MCP[299, 411, 668, 700]对 **工具服务器后端**进行标准化。第三种方法将 **生成内容与符号约束相结合**，将生成的网页与状态机约束及完整性校验相耦合[50, 695, 852]。第四种方法将 **域环境**置于真实数据、检索或计算场景中，并利用模型来扩展任务及反馈[538, 626, 804, 894]。整个设计过程中的关键抉择在于：如何界定基于模型的仿真与符号验证之间的边界。

Across these families, feedback and verification quality remain central. Software tasks can rely on tests and execution; tool tasks use persistent database, file, or API states; web and GUI tasks often require scripts, page-state checks, visual-semantic matching, or expert rules; and open-ended tasks depend more heavily on rubrics, LLM judges, or hybrid verifiers. Dense step-level signals can improve credit assignment, but stronger automation also raises judge bias, verifier loopholes, and reward-hacking risks [238, 402]. Synthesized environments must therefore balance correctness, diversity, complexity, and fidelity.

在这些家族中，反馈与验证质量始终居于核心地位。软件任务可依赖于测试与执行；工具任务需使用持久化数据库、文件或API状态；Web及GUI任务通常需要脚本、页面状态检查、视觉语义匹配或专家规则；而开放式任务则更依赖评分量表、LLM评审员或混合型验证器。密集的步骤级信号可改善信用分配，但更强的自动化也会加剧裁判者偏见、验证者漏洞以及奖励篡改风险[238, 402]。因此，合成环境必须在正确性、多样性、复杂性和保真度之间取得平衡。

5.2.3 Trajectory Synthesis

5.2.3 轨迹合成

For long-horizon agents, trajectory synthesis internalizes interaction as trainable behavioral experience: it records how a policy acts, observes, errs, and recovers over an environment, and converts that history into supervision, preferences, rewards, and memory. What can be recorded is bounded by the environment’s observability and verification, so trajectory quality inherits the strengths and gaps of the synthesized environment. A useful trajectory keeps the cross-step dependencies, delayed feedback, and recovery decisions that plain instruction–response data discard. Prior work develops this idea along four directions: grounding records in real environments, aligning rollouts with the trained policy, assigning step-level credit while reusing failures, and compressing long histories into reusable experience.

对于长时程智能体，轨迹合成将交互过程内化为可训练的行为经验：它会记录策略在环境中如何行动、如何

观察、如何出错以及如何进行恢复，并将这一历史记录转化为监督信号、偏好值与奖励。以及记忆。可记录的内容受限于环境的可观测性与可验证性，因此轨迹质量会继承所合成环境的优势与不足。有效的轨迹能够保留交叉步依赖关系、延迟反馈以及恢复决策——而这些正是普通指令-响应数据所缺失的。先前的研究沿着四个方向发展了这一理念：将记录置于真实环境中、使rollout与训练好的策略保持一致、在复用失败案例时分配步级信用，以及将长历史序列压缩为可复用的经验。

Environment-grounded Execution Records. Environment-grounded records capture state changes, tool side effects, and feedback produced during execution. Textual ReAct traces or function-call transcripts provide useful templates, but they often omit post-action states and failure signals. TOUCAN synthesizes large-scale tool-agentic trajectories in real MCP environments; OpenResearcher anchors deep-research trajectories in a reproducible offline corpus; and ASTRA constrains multi-turn tool trajectories with tool-call graphs [354, 604, 737]. WebSTAR, ANCHOR, and HATS further tie GUI and computer-use trajectories to interface states, branch points, action ambiguity, and step-level feedback [215, 531, 679].

基于环境的执行记录。基于环境的记录可捕获执行过程中发生的状态变化、工具副作用以及反馈信息。Textual ReAct 跟踪记录或函数调用日志可提供有用的模板，但这些记录往往会省略操作后的状态及失败信号。TOUCAN可在真实的MCP环境中合成大规模工具-主体轨迹；OpenResearcher可将深度研究轨迹锚定于可复现的离线语料库中；而ASTRA则利用工具调用图[354, 604,737]来约束多轮工具轨迹。WebSTAR、ANCHOR 和 HATS 进一步将 GUI 与计算机使用轨迹与界面状态、分支点、操作模糊性以及步骤级反馈联系起来[215, 531, 679]。

Policy-aligned Rollouts. Policy-aligned rollouts reduce distribution shift between collected trajectories and the policy being trained. Expert, human, or strong-model trajectories provide useful cold-start priors, whereas on-policy or near-on-policy rollouts expose the current policy’s own exploration, failures, and recovery states. AgentGym-RL, AgentRL, and ASearcher combine multi-turn rollout, horizon scheduling, and asynchronous sampling; DPEPO uses parallel trajectories for cross-path exploration; and Agent Lightning decomposes execution records into RL-consumable units [166, 409, 701, 811, 820].

政策对齐部署。政策对齐部署可减少所收集轨迹与正在训练的策略之间的分布偏移。专家轨迹、人类轨迹或强模型轨迹可提供有效的冷启动先验分布；而同策略或近同策略的 rollout 过程则能揭示当前策略自身的探索行为、失败状态及恢复状态。AgentGym-RL、AgentRL 和 ASearcher 将多轮模拟、视野调度与异步采样相结合；DPEPO 则利用并行轨迹进行跨路径探索；而 Agent Lightning 则将执行记录分解为可用于强化学习的单元[166, 409, 701, 811, 820]。。

Step-level Credit Assignment and Failure Reuse. Step-level credit assignment exposes local correctness, redundant actions, and critical errors that are hidden by final success labels. WebSTAR filters learnable GUI actions with a process reward model; ANCHOR expands and cleans post-branch trajectories; and HATS targets difficult segments under contextual and visual ambiguity [215, 531, 679]. CSO identifies outcome-changing critical steps, VPR constructs verifiable turn-level rewards, and GEAR reallocates trajectory-level advantages to segments [329, 333, 785]. The unit of supervision shifts from whole trajectories to steps, segments, and critical decisions. Failure trajectories provide complementary training signals for planning, evidence selection, tool use, state recovery, and harness interaction. CSO generates critical-step replacements; AgentHER relabels failed attempts under reachable alternative goals; and AOI and PIVOT use failed records for correction and trajectory refinement [125, 329, 751, 826]. HORIZON and HarnessFix further show that failures may stem from tools, context management, verifiers, or harnesses rather than model decisions alone [64, 658]. Useful trajectories therefore retain errors, control flow, state changes, and verification provenance.

步骤级信用分配与故障复用。步骤级信用分配能够揭示局部正确性、冗余操作以及被最终成功标签所掩盖的关键错误。WebSTAR滤波器利用过程奖励模型来学习可学习的GUI操作；ANCHOR负责扩展并清理分支后的轨迹；而HATS则用于定位在上下文及视觉模糊条件下存在的困难段落[215, 531, 679]。CSO 识别出可改变结果的关键步骤，VPR 构建了可验证的回合级奖励，而 GEAR 将轨迹级优势重新分配至各段落[329, 333, 785]。监督机制的运作单元已从整个轨迹转向具体步骤、片段及关键决策。故障轨迹可为规划、证据筛选、工具使用、状态恢复以及约束交互提供互补的训练信号。CSO可生成关键步骤替换方案；AgentHER可在可达的备选目标下对失败尝试进行重新标注；而AOI与PIVOT则利用失败记录进行修正及轨迹优化[125, 329, 751, 826]。HORIZON与HarnessFix进一步表明，故障可能源于工具、上下文管理、验证器或测试套件，而非仅由模型决策引起[64, 658]。因此，有效的轨迹能够保留误差、控制流、状态变更及验证溯源信息。

History Compression and Experience Reuse. History compression addresses the context and credit-assignment difficulties caused by long interaction records. ReSum trains over summary-bounded segments; Context-Folding and AgentFold organize trajectories into foldable subprocesses; and Memory-as-Action and Memory-R2 treat memory edits as trajectory decisions with credit-assignment consequences [573, 693, 742, 768, 846]. Long-horizon trajectories can thus be segmented, summarized, folded, converted into memory states, and abstracted into reusable experience.

历史压缩与经验复用。历史压缩可解决由长交互记录所引发的上下文及信用分配难题。ReSum在摘要受限的段落上进行训练；Context-Folding与AgentFold将轨迹组织为可折叠的子过程；而Memory-as-Action与Memory-R2则将记忆编辑视为具有信用分配影响的轨迹决策[573, 693, 742,768, 846]。因此，长时程轨迹可被分割、汇总、折叠、转换为记忆状态，并抽象为可复用的经验。

In summary, trajectory synthesis is moving from text-only demonstrations toward environment-grounded, verifiable, and reusable agentic experience. Task synthesis sets the capability pressure, environment synthesis supplies the interaction loop, and trajectory synthesis turns execution histories into trainable signals.

综上所述，轨迹合成正从仅基于文本的演示逐步转向基于环境、可验证且可复用的智能体体验。任务合成负责设定能力压力，环境合成提供交互循环，而轨迹合成则将执行历史转化为可训练信号。

5.3 Pre-training and Mid-training

5.3 预训练与中期训练

Pre-training and mid-training are the earliest stages for internalizing the capabilities required by long-horizon agents. Pre-training establishes broad linguistic, factual, coding, mathematical, and reasoning priors through large-scale self-supervised learning, while mid-training further reshapes these priors before instruction tuning or reinforcement learning. For long-horizon agents, these stages are important because later supervised fine-tuning (SFT), alignment, and policy optimization can only elicit and refine capabilities that are already sufficiently represented in the base model. We therefore discuss pre-training and mid-training from four aspects: reasoning capability, long-context state modeling, multimodal perception, and the data mixture and training strategies that support these capabilities.

预训练与中期训练是使长时程智能体内化其所需能力的最早阶段。预训练通过大规模自监督学习，建立了广泛的语言、事实、编码、数学及推理先验知识；而中期训练则在指令微调或强化学习之前，对这些先验知识进行进一步重塑。对于长时程智能体而言，这些阶段至关重要，因为后续的监督微调（SFT）、对齐以及策略优化，仅能激发并完善那些已在基础模型中已有充分表征的能力。因此，我们从四个方面探讨预训练与中期训练：推理能力、长上下文状态建模、多模态感知，以及支持上述能力的数据混合与训练策略。

5.3.1 Reasoning Priors

5.3.1 推理先验分布

Reasoning is a foundation for long-horizon agency because agents must decompose goals, maintain intermediate hypotheses, evaluate partial results, and revise plans over multiple steps. Code and mathematics are representative sources for injecting such priors during pre-training and mid-training, since they expose models to structured dependencies, executable programs, multi-step derivations, and verifiable intermediate states. Recent model reports increasingly emphasize coding, mathematics, reasoning, and agentic capabilities as part of their training recipes, including Qwen2.5 [744], DeepSeek-V3 [110], GLM-4.5 and GLM-5 [178, 179], and Kimi K2 [597]. Qwen3-Coder-Next [46] makes this direction more explicit through targeted mid-training toward code reasoning, repository-level understanding, and agent-style interaction patterns. These structured domains make reasoning supervision closer to long-horizon agent workloads than ordinary short-form text, because they require cross-step consistency, error checking, and compositional problem solving.

推理是长时程智能体的基础，因为智能体必须对目标进行分解、维护中间假设、评估部分结果，并在多个步骤中不断修订计划。代码与数学是预训练及中后期注入此类先验知识的典型来源，因其能使模型接触结构化依赖关系、可执行程序、多步推导过程以及可验证的中间状态。近期发布的各类模型报告日益强调将编程、数学、推理及智能体能力纳入其训练方案之中，例如 Qwen2.5[744]、DeepSeek-V3[110]以及 GLM-4.5 和 GLM-5[178, 179]，以及 Kimi K2[597]。Qwen3-Coder-Next[46]通过针对代码推理、代码库级理解及智能体式交互模式的针对性中端训练，使这一方向更加明确。这些结构化域使得推理监督能够更贴近长时程智能体的工作负载，而非普通短文本——因为它们需要实现跨步骤的一致性、错误校验以及组合式问题求解。

5.3.2 Long-Context State

5.3.2 长上下文状态

Long-context capability provides the substrate for maintaining task state across extended interactions. For a long-horizon agent, context is not merely a longer input window; it serves as a serialized working state that may contain goals, observations, retrieved evidence, tool outputs, failed attempts, intermediate plans, and revised decisions. A first line of work extends the usable window by rescaling rotary position embeddings: Position Interpolation linearly rescales positions to fit a target length [66], YaRN improves this with an NTK-aware, frequency-dependent schedule for cheaper extension [477], and LongLoRA makes such extension parameter-efficient through shifted sparse attention during fine-tuning [78]. A complementary line studies the training recipe rather than the positional scheme. ProLong shows that long-context capability depends on continued-training data composition, training sequence length, and instruction tuning [171]; UltraLong pushes context to the million-token regime through continued pre-training and instruction tuning [721]; and LongWriter shows that long input windows do not by themselves induce long-output behavior without long-output training examples [29]. Recent frontier models also operationalize this direction. GLM-5.2 couples a 1M-token context window with specialized training for long-horizon coding-agent scenarios [179]. What matters for agents is therefore whether extended context is actually used for state tracking and evidence integration, not the maximum length a model nominally accepts [133, 625].

长上下文能力为在长时间交互过程中维持任务状态提供了基础。对于长时程智能体而言，上下文并非仅仅是更宽的输入窗口；它还充当一个序列化的工作状态，其中可能包含目标、观测结果、检索到的证据、工具输出、失败尝试、中间计划以及修订后的决策。第一类研究方法通过重新缩放旋转位置嵌入来扩展可用窗口：Position Interpolation 通过线性缩放位置，以适应目标长度[66]。YaRN通过采用一种基于NTK且具有频率依赖性的调度策略来改进这一问题，从而实现更低成本的扩展[477]；而LongLoRA则在微调过程中利用偏移稀疏注意力机制，实现了此类扩展的参数高效性[78]。互补线法侧重于研究训练配方，而非位置安排方案。ProLong实验表明，长上下文能力取决于持续训练数据的构成、训练序列长度以及指令微调[171]；UltraLong通过持续的预训练与指令微调，将上下文长度扩展至百万令牌量级[721]；LongWriter 实验证明，长输入窗口本身并不会在缺乏长输出训练样本的情况下诱导出长输出行为[29]。近期的前沿模型也沿用了这一研究方向——GLM-5。将一个包含 1M 个令牌的上下文窗口与针对长时程编码代理场景设计的专用训练相结合[179]。因此，对于代理而言，关键在于扩展上下文是否确实被用于状态跟踪与证据整合，而非模型名义上可接受的最大长度[133, 625]。

5.3.3 Multimodal Perception

5.3.3 多模态感知

Long-horizon agents increasingly operate in environments where task state is distributed across webpages, screenshots, videos, and audio streams; their perceptual priors must therefore extend beyond linguistic shortcuts that can mask basic visual failures [61, 502, 608, 624]. Open vision-language models establish the basic substrate: Cambrian-1 explores vision-centric representations and visual grounding [607]; Qwen2-VL perceives images and long videos at varying scales through dynamic resolution and multimodal rotary position embedding [644]; and the InternVL3 series advances open pre-training for high-resolution, long-context visual understanding [654, 892]. Recent industrial reports push these priors toward agent-centric perception, with the common thread being that visual grounding and GUI control are trained as native capabilities rather than added after a text-only stage: Seed1.5-VL and the subsequent Seed1.6 series pre-train on large-scale interleaved vision, text, and interaction data to sustain long-video understanding and GUI control across evolving screens [42, 194], while MiMo-VL treats visual grounding and GUI localization as first-class targets on a reasoning-optimized backbone [393, 394]. On the omni-modal side, Qwen3.5-Omni and Kimi K2.5 extend joint text, vision, and audio pre-training toward multimodal agentic intelligence [498, 598]. The shared implication is that long-horizon capability depends on persistent multimodal state tracking and cross-modal evidence grounding, not textual reasoning alone.

长时程智能体正越来越多地应用于任务状态分散于网页、屏幕截图、视频及音频流等不同媒介的环境中；因此，它们的感知先验必须超越那些可能掩盖基本视觉缺陷的语言捷径[61, 502, 608, 624]。开放式视觉-语言模型构建了基础架构：Cambrian-1探索了以视觉为中心的表征与视觉锚定[607]；Qwen2-VL通过动态分辨率和多模态旋转位置嵌入[644]，能够在不同尺度下感知图像与长视频；InternVL3系列进一步推进了高分辨率、长上下文视觉理解的开放预训练[654, 892]。近期的工业研究报告正推动这些先验方法向“智能体中心化感知”方向发展，其共同特征在于：视觉定位与GUI控制被作为原生能力进行训练，而非在仅基于文本的阶段之后才予以添加——即Seed1方案。5-VL 及其后续的 Seed1。第6系列模型基于大规模交错分布的视觉、文本及交互数据进行预训练，旨在支持长视频理解与GUI控制，以适应不断演进的屏幕环境[42, 194]。而MiMo-VL将视觉定位与GUI定位视为推理优化主干网络上的首要目标[393, 394]；在多模态领域，则有Qwen3.5-Omni和Kimi K2.5 将文本、视觉及音频预训练技术拓展至多模态智能体领域[498, 598]。共同的启示在于：长时程能力依赖于持续的多模态状态跟踪与跨模态证据锚定，而非仅依赖文本推理。

5.3.4 Data Mixture and Training Strategies

5.3.4 数据混合与训练策略

The above capabilities are shaped by how training data are selected, mixed, and structured. Recent work shifts pre-training from generic web-scale language modeling toward capability-oriented data distribution design. DataComp-LM [316], FineWeb [475], Dolma [552], and OLMo [184] make pre-training data construction more reproducible. Beyond static corpus construction, DoReMi [715] and Data Mixing Laws [767] show that domain proportions can be optimized or predicted with proxy experiments, while OPUS [649] performs principled data selection at each pre-training iteration by scoring sample utility in the optimizer-induced update space, shifting pre-training from more tokens to better tokens and improving both from-scratch and continued pre-training under a fixed compute budget. For long-horizon agents, such data strategies act as inductive biases for reasoning, state modeling, and compositional problem solving, rather than merely as

mechanisms for scaling factual knowledge. A further strategy is to structure training data as trajectories rather than isolated text samples. AgentTuning [794], Agent-FLAN [79], and AgentBank [557] show that interaction trajectories can improve generalized agent abilities such as planning, memory, and tool use. Since high-quality long-horizon trajectories are scarce, synthetic and verifier-guided data are also used to control horizon length, task difficulty, feedback structure, and failure-recovery patterns [29, 46]. Here the decisive factor is not what the data contain but whether they preserve temporal dependencies, interaction feedback, and recovery processes.

上述能力的形成取决于训练数据的选取、混合及结构化方式。近期的研究正将预训练从通用的网络规模语言建模转向以能力为导向的数据分布设计。DataComp-LM[316]、FineWeb[475]、Dolma[552]以及 OLMo[184]使得预训练数据集的构建更具可复现性。除静态语料库构建外，DoReMi[715]与Data Mixing Laws[767]表明，领域比例可通过代理实验进行优化或预测，而OPUS[649]则.....在每次预训练迭代中，通过在优化器诱导的更新空间内对样本效用进行评分，实现基于原则的数据选择。将预训练从更多标记转向更优质的标记，并在固定计算预算下同时优化从头预训练与持续预训练。对于长时程智能体而言，此类数据策略可作为推理、状态建模及组合式问题求解的归纳偏置，而非仅作为扩展事实知识的机制。另一种策略是将训练数据结构化为轨迹，而非孤立的文本样本。AgentTuning[794]、Agent-FLAN[79]和 AgentBank[557]的研究结果表明，交互轨迹能够提升智能体的泛化能力，例如规划、记忆和工具使用能力。由于高质量的长时程轨迹较为稀缺，还采用了合成数据与验证器引导的数据来控制时域长度、任务难度、反馈结构以及故障恢复模式[29, 46]。此处的关键因素并非数据所包含的内容，而是这些数据是否保留了时间依赖性、交互反馈以及恢复过程。

Taken together, pre-training and mid-training form the first stage of internalizing long-horizon capability. Reasoning-oriented data provide structured problem-solving priors, long-context training supports state maintenance over extended interactions, multimodal training enables persistent observation across heterogeneous evidence, and data mixture strategies determine how these signals are balanced and selected. These latent priors are then elicited, sequenced, and optimized by SFT and RL into reliable agentic behavior.

综上所述，预训练与中期训练共同构成了内化长周期能力的第一阶段。推理导向型数据可提供结构化的问题求解先验知识，长上下文训练能够支持在长时间交互过程中保持状态，而多模态训练则可实现对异构证据的持续观测。数据混合策略决定了这些信号如何进行平衡与选择；随后，SFT与强化学习（RL）将对这些潜在先验分布进行提取、排序并优化，从而生成可靠的代理行为。

5.4 Fine-tuning

5.4 微调

Fine-tuning converts the latent priors acquired during pre-training and mid-training into controllable long-horizon behavior. Whereas pre-training shapes broad parameter-level capabilities, supervised fine-tuning (SFT) teaches the model how these capabilities should be expressed during interaction: how to decompose goals, call tools, interpret observations, update state, recover from errors, and terminate with a final answer. For long-horizon agents, SFT is therefore process-level imitation over trajectories that contain intermediate reasoning, actions, tool results, and state transitions, rather than instruction-following over isolated input-output pairs. It typically serves as a cold-start stage that constrains the model to valid, task-relevant interaction patterns before any online optimization.

微调可将预训练和中期训练期间获得的潜在先验转化为可控的长时程行为。预训练可塑造广泛的参数级能力，而监督微调（SFT）则能指导模型如何在交互过程中表达这些能力：即如何分解目标、调用工具、解读观测结果以及更新状态。从错误中恢复，并输出最终答案。对于长视野智能体而言，SFT因此是指对包含中间推理、动作、工具结果及状态转移的轨迹进行过程级模仿，而非对孤立的输入-输出对进行指令遵循。该阶段通常作为冷启动阶段，可在进行任何在线优化之前，将模型约束在有效且与任务相关的交互模式范围内。

The central issue is how to construct a supervised trajectory distribution that is diverse enough to teach general agent behavior, structured enough to ensure valid tool use, and progressive enough to prepare the model for later stages. We organize this section around three questions: which demonstrations to select and how to mix them, how to stage them through a curriculum, and how to obtain them through distillation.

核心问题在于如何构建一种监督轨迹分布：该分布需具备足够的多样性，以教导通用智能体的行为；需具有足够的结构化程度，以确保工具的正确使用；且需具备足够的渐进性，以便为模型后续阶段做好准备。本节围绕三个问题展开论述：应选择哪些演示案例及其混合方式、如何通过课程体系进行演示安排，以及如何通过提炼过程获取这些案例。

5.4.1 Instruction Selection and Mixing

5.4.1 指令选择与混合

Trajectory Quality Over Quantity. Fine-tuning long-horizon agents requires data that expose not only correct answers, but also the intermediate decisions connecting goals, observations, actions, and recovery behaviors across many turns. AgentTuning [794] constructs AgentInstruct, a compact set of high-quality interaction trajectories, and mixes it with general instruction data to strengthen agent skills while preserving general ability. FireAct [54] shows that trajectories generated by stronger models improve agent behavior beyond prompt-only adaptation, and LIMi [711] argues that a small set of strategically curated demonstrations can elicit strong agency-oriented behavior. Rather than cloning full trajectories, ATLaS [81] identifies the *critical steps* of planning, subgoal reasoning, and strategic decision making, and fine-tunes only on them, mitigating expert bias and improving cross-environment generalization. These works reframe agent SFT as an instruction-selection problem: prioritizing trajectory quality, decision-point coverage, and task diversity over raw demonstration count.

重质量而非数量：轨迹质量优先。对长时程智能体进行微调时，所需的数据不仅应包含正确答案，还应涵盖连接多个回合中目标、观测值、动作及恢复行为之间的中间决策。AgentTuning[794]构建了名为 AgentInstruct 的紧凑型高质量交互轨迹集，并将其与通用指令数据相结合，从而在增强智能体技能的同时保留其通用能力。FireAct[54]表明，由更强模型生成的轨迹能够提升智能体的行为表现，其效果超越仅基于提示的自适应；而LIMi[711]该观点认为，少量经过战略性筛选的示范案例能够激发强烈的能动性导向行为。ATLaS[81]并非克隆完整的轨迹，而是识别规划、子目标推理及策略决策的**关键步骤**，仅在这些步骤上进行微调。这些研究旨在减轻专家偏差并提升跨环境泛化能力；它们将智能体的SFT重新定义为一个指令选择问题：即在优先考虑轨迹质量、决策点覆盖度及任务多样性时，优先于原始演示次数。

Compact Reasoning Demonstrations. A complementary line explains why small trajectory sets can be effective. LIMO [770] elicits complex mathematical reasoning from a strong base model with only 817 curated demonstrations, treating them as cognitive templates over knowledge already encoded during pre-training; s1 [442] builds a 1K-example set under quality, difficulty, and diversity criteria; and LIMR [344] extends the less-is-more observation to reinforcement learning through learning-impact-based sample selection. Unified Data Selection [342] sharpens this idea by selecting samples around high-entropy tokens, which mark critical branching points in reasoning trajectories. This “less-is-more” view is not unchallenged: Ding et al. [122] report that, in an SFT-then-RL pipeline, data scale still governs the attainable performance ceiling while trajectory difficulty acts as a multiplier, suggesting that selection should maximize decision coverage rather than merely shrink the dataset.

紧凑推理演示。另一条补充说明阐释了为何小轨迹集也能取得良好效果。LIMO[770]基于一个仅包含 817 个精心筛选的示例的强大基础模型，引导其进行复杂的数学推理；该模型将这些示例视为对预训练阶段已编码知识的认知模板；s1[442]构建了一个基于质量、难度和多样性标准的1K示例集；而LIMR[344]则通过基于学习影响的样本选择方法，将“少即是多”的原则扩展至强化学习领域。统一数据选择[342]通过选择围绕高熵

标记的样本来强化这一理念——这些标记标示着推理轨迹中的关键分支点。这种“少即是多”的观点并非毫无争议：Ding 等人。[122]报告指出，在“先SFT后RL”的流程中，数据规模仍主导着可达到的性能上限，而轨迹难度则起到乘数效应；这表明，选择策略应侧重于最大化决策覆盖范围，而非仅仅缩小数据集。

Executable Tool-use Data and Mixture. For tool-using agents, data value further depends on whether actions are syntactically valid, executable, and grounded in observations. CodeAct [656] represents actions as executable Python code, enabling code execution, observation feedback, and self-debugging; APIGen [392] generates function-calling data verified through format, execution, and semantic checks; ToolACE [385] scales synthetic tool-learning data with a self-evolving API pool and multi-agent dialogue generation; and TOUCAN [737] synthesizes large-scale multi-tool, multi-turn trajectories from real-world Model Context Protocol (MCP) environments. Selection also matters at scale: EDGE [844] picks informative multi-turn samples according to guideline effectiveness. Because such corpora remain fragmented across incompatible formats, the Agent Data Protocol [558] introduces a unified schema that standardizes heterogeneous agent datasets into a common trajectory representation; SFT on the resulting cross-domain mixture yields broad gains and avoids the negative transfer induced by single-domain tuning.

可执行工具使用数据与混合数据。对于工具使用智能体而言，数据值的取值还取决于动作是否在语法上有效、是否可执行以及是否基于观察结果。CodeAct[656]将操作表示为可执行的Python代码，支持代码执行、观察反馈及自调试功能；APIGen[392]可生成通过格式、执行及语义校验验证的函数调用数据；ToolACE [385]采用自进化API池与多智能体对话生成技术对合成工具学习数据进行缩放；以及TOUCAN[737]。能够从真实世界的模型上下文协议（MCP）环境中合成大规模的多工具、多轮对话轨迹。在大规模场景下，选择机制同样至关重要：EDGE[844]根据指南的有效性来筛选具有信息价值的多轮对话样本。由于这些语料库仍分散存储于不兼容的格式中，因此 Agent Data Protocol[558]引入了一种统一模式，将异构代理数据集标准化为一种通用轨迹表示法；SFT应用于所得跨域混合模型可带来显著性能提升，并能避免由单域调参所引发的负迁移问题。

5.4.2 Curriculum Learning

5.4.2 课程学习

Staging by Horizon. Long-horizon trajectories are difficult to imitate directly because errors compound over time and the model must maintain state across many steps before receiving any signal of success or failure. Curriculum learning addresses this by staging the training distribution rather than exposing the model to full-length tasks at once, with the interaction horizon as a natural first axis. ScalingInter-RL, introduced in AgentGym-RL [701], begins with short-horizon exploitation to establish stable foundational policies and then progressively enlarges the interaction budget, mitigating the collapse that plagues naive long-horizon optimization. Studying horizon in isolation, Kim et al. [279] construct controlled task suites that vary only the required action-sequence length and show that horizon itself is a training bottleneck: reducing the horizon first and then extending it stabilizes learning and induces *horizon generalization*, where competence acquired on short goals transfers to unseen longer ones at inference time.

由 Horizon 实现的阶段化处理。长期规划轨迹难以直接模拟，因为误差会随时间累积，且模型必须在接收到任何成功或失败的信号之前，跨越多个步骤持续维持状态。课程学习通过对训练分布进行分阶段安排——而非让模型一次性接触完整的任务——来解决这一问题，其中交互时域被设定为自然的首个维度。ScalingInter-RL方法由AgentGym-RL[701]提出，其流程始于短时程利用阶段，旨在建立稳定的基础策略，随后逐步扩大交互预算。缓解困扰于朴素远期优化问题的崩溃现象；Kim 等人对“规划期限”这一概念进行了独立研究。[279] 构建仅改变所需动作序列长度的受控任务集，从而证明“时域”本身便是一个训练瓶颈：先缩短时域再延长时域可稳定学习过程，并诱导出*时域泛化能力*。在此情境下，通过短期目标习得的能力可在推理阶段迁移到未见过的更长任务中。

Staging by Difficulty. A second axis orders tasks from easy to hard. E2H Reasoner [469] schedules problems probabilistically from easy to difficult, and shows both empirically and through approximate-policy-iteration bounds that gradually fading out easy tasks prevents overfitting and lets small models acquire reasoning skills that vanilla training cannot reach. Curricula for long-horizon agents can be graded along several dimensions at once, including horizon length, task difficulty, tool complexity, feedback density, and recovery demand, so that a staged cold-start distribution first instills basic interaction competence before later stages optimize decisions under delayed rewards.

按难度分级。第二个维度将任务按从易到难的顺序排列。E2H Reasoner[469]采用概率性调度方式，将问题按难度从易到难进行排序；该方法不仅通过实证结果，还借助近似策略迭代界证明：逐步淡化简单任务能够防止过拟合，并使小型模型得以运行。可习得常规训练无法达到的推理能力。长时程智能体的课程体系可同时从多个维度进行评估，包括：视野长度、任务难度、工具复杂度、反馈密度及恢复需求；因此，分阶段的冷启动分布机制可先奠定基础。在后期阶段中，通过优化基于延迟奖励的决策来提升交互能力。

5.4.3 Distillation

5.4.3 蒸馏

Because high-quality long-horizon demonstrations are scarce, many fine-tuning pipelines obtain supervision by distilling interaction processes from stronger models, scaffolded agent systems, or executable environments. This is better understood as trajectory distillation rather than answer distillation: the student learns not only final responses, but also planning steps, action formats, observation interpretation, and recovery strategies. We use *distillation* here in its offline sense, where supervision comes from fixed teacher or past-model trajectories consumed as static SFT data, in contrast to the on-policy distillation of Section 5.6 and the closed-loop self-evolution of Section 5.7.

由于高质量的长时程演示数据十分稀缺，许多微调流程会通过从更强的模型、架设好的智能体系统或可执行环境中提炼交互过程来获取监督信号。这更应理解为“轨迹蒸馏”而非“答案蒸馏”：学生不仅能够学习最终回答，还能掌握规划步骤、动作格式、观察解读及恢复策略。本文中，我们采用蒸馏这一概念的离线含义——即监督信号来源于固定的教师模型或作为静态SFT数据使用的过往模型轨迹；这与第5.6节所述的在线策略蒸馏方式形成对比。以及第5.7节中的闭环自演化过程。

Strong-to-weak Distillation. *Strong-to-weak distillation* transfers agentic behavior from powerful teacher models or expensive agent systems into smaller, deployable students, reusing costly execution as supervised traces. Because agent competence lives in full reason-act-observe trajectories, recent work supervises structure rather than final answers. Agent Distillation [269] transfers retrieval and code-execution trajectories from a large teacher into sub-3B students, using a first-thought prefix and self-consistent action generation to keep tiny agents competitive with far larger chain-of-thought-distilled baselines. Structured Agent Distillation [376] segments each trajectory into [REASON] and [ACT] spans with segment-specific losses, preserving both reasoning fidelity and action validity under compression. Pushing the teacher side further, AgentArk [410] distills a **强到弱蒸馏**。*强到弱蒸馏*将智能体行为从强大的教师模型或昂贵的智能体系统迁移至规模更小、可部署的学生模型中，将高昂的执行成本重用于构建监督训练轨迹。由于智能体的能力根植于完整的“推理-行动-观察”轨迹之中，近期的研究更倾向于对结构进行监督，而非直接监督最终答案。代理蒸馏[269]可将大型教师模型的检索与代码执行轨迹迁移至子3B学生模型中。通过采用“第一想法”前缀和自洽的行动生成机制，可使微小智能体与规模远大于其的“思维链蒸馏”基线模型保持竞争力。结构化智能体蒸馏[376]将每条轨迹划分为[推理]与[动作]两段，并采用各段专属损失函数，从而在压缩过程中既保持推理的准确性，又确保动作的有效性。在深化教师侧功能方面，AgentArk[410]提炼出了一种

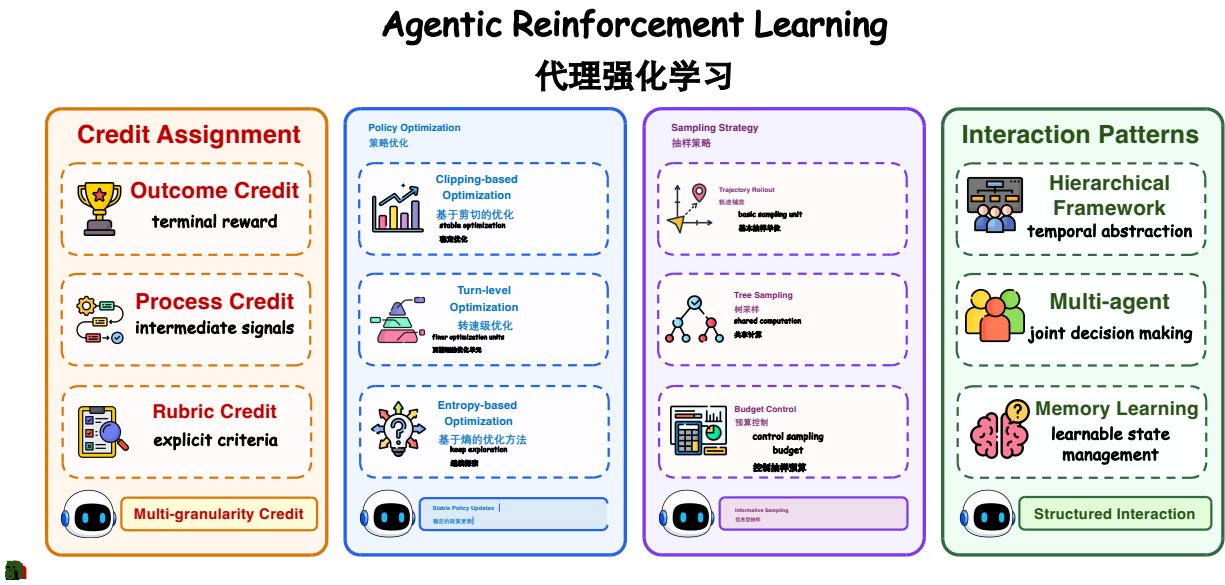


Figure 12 Agentic reinforcement learning through four key questions. These questions correspond to the four components discussed in Section 5.5: credit assignment, policy optimization, sampling strategy and interaction structure.

图12：通过四个关键问题实现的代理强化学习。 这些问题对应于第5.5节中讨论的四个组成部分。：信用分配、策略优化、采样策略及交互结构。

multi-agent debate system into a single model, turning expensive test-time interaction into implicit self-correction ability. Across these methods, effective distillation aligns supervision with trajectory structure rather than compressing outputs alone.

将多智能体辩论系统整合为单一模型，将高昂的测试阶段交互成本转化为隐式自我修正能力。在这些方法中，有效的蒸馏技术是将监督信息与轨迹结构相匹配，而非单纯对输出进行压缩。

Self-distillation and Reflective Self-improvement. *Self-distillation* removes the external teacher: the model generates its own trajectories, filters or reflects on them, and trains back on the curated result. Since many long-horizon failures are local, such as choosing a wrong tool, misreading an observation, or failing to recover after an execution error, the central design choice is how to turn raw rollouts into clean, corrective supervision. Watch Every Step [719] explores along expert trajectories, collects step-level process feedback, and learns from contrastive action pairs. Agent-R [789] uses Monte Carlo Tree Search to locate the first erroneous step in a failed rollout and splices it onto an adjacent correct path, so that iterative fine-tuning teaches timely on-the-fly correction rather than end-of-episode fixes. Reflection enriches the signal further: RESD [845] converts failed rollouts into retrospective diagnoses and a reusable “playbook” of lessons, while HERO [374] turns each observation into a turn-level hindsight reflection used to re-score the student’s own actions. These reflection-driven variants shade into the on-policy self-distillation of Section 5.6; used offline, their filtered traces still supply denser process supervision than final-answer imitation.

自我蒸馏与反思式自我提升。 *自我蒸馏* 消除了外部教师的作用：模型可生成自身的轨迹，对这些轨迹进行过滤或反思，并基于经过精心筛选的结果进行反向训练。由于许多长时程故障属于局部性故障——例如选择错误的工具、误读观测值或在执行错误后未能恢复——因此，核心设计抉择在于如何将原始 rollout 结果转化为干净且具有纠正作用的监督信号。“Watch Every Step[719]”沿专家轨迹进行探索，收集步级过程反馈，并从对比动作对中学习。Agent-R[789]采用蒙特卡洛树搜索技术，用于定位失败的rollout过程中的首个错误步骤，并将其拼接至相邻的正确路径上。因此，迭代微调能够实现及时的实时修正，而非仅在回合结束时进行补救。反思能进一步丰富信号：RESD[845]将失败的部署过程转化为回顾性诊断，并提供一套可复用的“经验手册”；而 HERO[374]将每个观察结果转化为回合级的事后反思，用于对学生的自身行为进行重新评分。这些基于反思的变体可过渡至第5.6节所述的在线策略自蒸馏方法；当离线应用时，其过滤后的轨迹仍能提供比最终答案模仿更密集的过程监督。

Both regimes share a lesson: distillation for long-horizon agents must preserve *process* structure, whether transferring capability from strong teachers or recycling an agent’s own history into cleaner supervision. Both can also amplify biases and environment-specific habits, so execution-based verification, semantic filtering, and mixture with general instruction data remain essential for robust, transferable behavior.

两种方法都蕴含着一个共同启示：对于长时程智能体而言，蒸馏过程必须保留*过程*结构——无论该过程是通过从强教师处转移能力，还是将智能体自身的历史记录重新利用以生成更优质的监督信号。这两者都可能放大偏见及特定环境下的固有习惯，因此基于执行的验证、语义过滤以及与通用指令数据相结合的方法，对于实现鲁棒且可迁移的行为仍然至关重要。

Overall, fine-tuning determines how long-horizon priors surface as interactive behavior: instruction selection defines the supervised behavior distribution, curriculum learning stages it by horizon and difficulty, and distillation transfers complex interaction processes from stronger or self-generated sources. It improves controllability, tool-use validity, state tracking, and recovery, yet stays bounded by the coverage of supervised demonstrations, leaving decisions under delayed rewards and compounding errors to later online optimization.

总体而言，微调决定了长时程先验如何表现为交互式行为：指令选择定义了监督学习行为分布，而课程学习则按时间跨度和难度对这些行为分布进行分阶段构建。而蒸馏过程将复杂的相互作用过程从更强或自生的源中转移出来。该方法提升了可控性、工具使用有效性、状态跟踪能力及系统恢复性能，但其效果仍受限於监督式演示的覆盖范围；将受延迟奖励影响的决策及累积误差问题交由后续的在线优化过程处理。

5.5 Agentic Reinforcement Learning

5.5 代理强化学习

Compared with supervised fine-tuning, Agentic Reinforcement Learning (Agentic RL) further optimizes the policy against feedback from the agent’s own interaction. This extends the optimization targets of LLMs from single-turn responses to long-horizon interaction trajectories. As illustrated in Figure 12, the core challenges involve four aspects: sampling informative multi-turn trajectories, deriving multi-granularity credit signals from sparse feedback, integrating these signals into stable policy updates, and organizing training interaction patterns such as hierarchy, tool use, multi-agent collaboration, and memory. In the following subsections, we elaborate on credit assignment, policy optimization, and sampling methods in Agentic RL, as well as several

unique interaction patterns of Agentic RL. Representative works for each part are shown in Table 6.

与监督微调相比，代理强化学习（Agentic RL）能够根据智能体自身交互的反馈进一步优化策略。这将大语言模型（LLMs）的优化目标从单轮响应扩展至长时程交互轨迹。如图12所示，核心挑战涉及四个方面：采集具有信息价值的多轮交互轨迹、从稀疏反馈中推导出多粒度信用信号、将这些信号整合至稳定的策略更新过程之中。并组织训练交互模式，例如层级结构、工具使用、多智能体协作及记忆功能。在以下子节中，我们将详细阐述代理式强化学习中的信用分配、策略优化及采样方法，以及代理式强化学习中几种独特的交互模式。各部分的代表性作品列于表6中。

5.5.1 Credit Assignment

5.5.1 信用分配

Credit assignment is one of the core challenges for Agentic RL. It concerns how to attribute credit to heterogeneous decision units when an agent completes long-horizon tasks. This section reviews credit assignment from three perspectives: outcome credit assignment, which scores only the final result; process credit assignment, which shapes the intermediate decision process; and rubric credit assignment, which expresses quality through explicit semantic criteria.

信用分配是智能体强化学习的核心挑战之一，其核心问题在于：当智能体完成长时程任务时，如何为异质性决策单元分配信用。本节从三个视角回顾了信用分配：结果信用分配——仅对最终结果进行评分；过程信用分配——则用于塑造中间决策过程；以及评分标准下的信用赋值法，该方法通过明确的语义准则来表征质量。

Outcome Credit Assignment. Outcome credit assignment is the most direct form of credit assignment, where the reward is determined only by the final result, such as exact match, F1, task success, final-state checks, unit-test results, or retrieved-answer correctness. Its appeal lies in being automatable, cheap to verify, and easy to scale to large online rollouts. This paradigm was first validated by verifiable terminal rewards in mathematical and code reasoning [112, 345, 533]. It was subsequently carried into interactive settings by a line of web and information-seeking agents led by Search-R1, which reuse task success, environment-state checks, or retrieved-answer correctness as terminal rewards across long-horizon application environments [60, 256, 261, 681, 871]. However, this form of credit is coarse-grained. A single terminal score is shared across the entire trajectory, conflating key exploration, redundant actions, and early mistakes into one learning signal. As a result, the policy struggles to discriminate the quality of individual local decisions.

结果归因。结果归因是信用分配的最直接形式，其奖励仅由最终结果决定，例如精确匹配、F1值、任务成功与否、最终状态检查、单元测试结果或检索答案的正确性。其优势在于可实现自动化、验证成本低廉，且易于扩展至大规模在线部署。该范式最初通过可验证的终端奖励在数学与代码推理领域得到了验证[112, 345, 533]。随后，由Search-R1主导的一系列网络与信息检索智能体将该方法引入交互式场景，这些智能体可复用任务成功状态及环境状态检查机制。或在长时程应用环境中将检索答案的正确性作为终端奖励[60, 256, 261, 681, 871]。然而，此类奖励形式较为粗粒化。单一终端评分在整个轨迹中均适用，将关键探索、冗余动作及早期错误统合为一个学习信号；因此，策略难以区分各个局部决策的质量。

Process Credit Assignment. Process supervision predates agentic RL in mathematical reasoning, where process reward models (PRMs) score intermediate steps rather than only final answers [363, 643]; reinforcement learning with verifiable rewards then scaled outcome-based optimization for reasoning [112, 533]. Agentic credit assignment inherits this lineage and extends it to multi-step, tool-using trajectories. Process credit assignment extends the evaluation target from the final answer to the intermediate process. Existing methods follow two complementary routes. The first decomposes the quality of an individual decision: a group of tool-use-oriented methods represented by ToolRL break tool invocation into separately scoreable dimensions, including output format, tool selection, and parameter names and values [129, 487, 834]. The second distributes denser reward across the interaction process rather than scoring a single decision, spanning a spectrum of granularity: some works such as AdaTIR and AgentPRM assign reward to intermediate units of the trajectory rather than to its endpoint alone [77, 150, 318, 554, 682, 702, 837]. Among them, some methods fold process signals such as interaction cost and query quality into trajectory-level rewards, while others attach explicit step- or turn-level credit via process reward models or advantage redistribution. Across both routes, the shared goal is not merely to make rewards denser, but to align credit signals with the true decision structure of the agent.

过程信用分配。过程监督在数学推理领域早于代理式强化学习出现，其中过程奖励模型（PRMs）会对中间步骤进行评分，而非仅对最终答案进行评分[363, 643]；随后，基于可验证奖励的强化学习得到了推广。基于结果的推理优化[112, 533]。自主信用分配继承了这一传统，并将其扩展至多步骤、工具使用轨迹的场景。过程信用分配将评估目标从最终答案扩展至中间过程。现有方法主要遵循两条互补路径。第一种方法可分解个体决策的质量：由ToolRL为代表的工具使用导向型方法，将工具调用过程分解为多个可分别评分的维度，包括输出格式、工具选择等。以及参数名称与值[129, 487, 834]。第二种方法会在整个交互过程中更均匀地分配奖励，而非对单个决策进行评分。其粒度范围广泛：部分研究（如AdaTIR和AgentPRM）会将奖励分配给轨迹的中间节点，而非仅分配给其终点[77, 150, 318, 554, 682, 702, 837]。其中，部分方法将过程信号（如交互成本与查询质量）折叠至轨迹级奖励中；而另一些方法则通过过程奖励模型或优势重分配机制，赋予显式的步级或转向级奖励。在这两种路径下，共同目标不仅在于使奖励分布更加密集，更在于将信用信号与智能体的真实决策结构相匹配。

Rubric Credit Assignment. Rubric credit assignment expresses answer or trajectory quality through explicit, multi-dimensional semantic criteria (e.g., factuality, completeness, reasoning soundness, evidence grounding, and safety) when a single verifiable metric is insufficient; a recent paper separates such rubrics from reward models, verifiable rewards, and LLM-as-a-judge [386]. Methods differ along two axes: how rubrics are constructed, and how they are turned into credit.

评分量表的评分分配。评分量表的评分分配通过明确的、多维的语义标准（例如）来评估答案或解题过程的质量。（事实性、完整性、推理有效性、证据支撑性及安全性）在单一可验证指标不足以评估时；近期的一篇论文将此类评估维度与奖励模型、可验证奖励以及“LLM-as-a-judge” [386]进行了区分。评估方法沿两个维度存在差异：一是评分标准的构建方式，二是其如何转化为学分。

On construction, rubrics are produced along an increasingly automated spectrum: (1) strong LLMs or human experts author checklists used directly as scoring criteria or as training rewards [534], as in Rubrics as Rewards and the rubric bank of Reinforcement Learning with Rubric Anchors [192, 240]; (2) rubrics are induced from preference contrasts, where OpenRubrics extracts rules and implicit principles filtered by preference-label consistency and Chasing the Tail targets the high-reward tail to curb over-optimization [384, 819]; and (3) rubrics are made context-grounded and online, as when Agentic Rubrics lets an expert agent inspect a repository before generating criteria and DR Tulu co-evolves positive and negative rubrics with the policy during deep-research training [500, 532].

在构建过程中，评分标准的生成方式正沿着日益自动化的谱系展开：(1) 由强大的LLM或人类专家编写的检查清单可直接用作评分标准或作为训练奖励[534]例如，如“评分标准作为奖励”以及基于评分标准锚点的强化学习评分标准库[192, 240]；(2)评分标准可从偏好对比中推导得出：OpenRubrics通过偏好标签的一致性对规则和隐含原则进行筛选；而“Chasing the Tail”方法则针对高奖励尾部进行优化，以抑制过度优化[384, 819]；(3)评分标准需基于具体语境且支持在线构建——例如，Agentic Rubrics 允许专家智能体在生成评分标准之前先对知识库进行审查；而 DR Tulu 则在深度研究训练过程中，与策略共同演化正向与负向评分标准[500, 532]。。

On conversion, rubrics become credit signals (1) as direct RL rewards, aggregated by weights or an LLM judge and optionally decomposed into fine-grained signals such as answer correctness, step validity, and premise 在转换过程中，评分标准会转变为学分信号(1)——即作为直接的强化学习奖励，这些奖励可由权重或大语言模型（LLM）评审员进行聚合，并可选择性地分解为细粒度信号，例如答案正确性、步骤有效性及前提条件。

utilization [164, 192]; (2) as supervision for reward models or verifiers, including Prometheus, the OpenRubrics reward model, and the interpretable scorer R3 [19, 278, 384]; and (3) as guidance for policy behavior rather than only final scoring, as in Rubric-Scaffolded RL, which injects rubrics as decaying exploration scaffolds during rollout and as judge references during exploitation [887]. Rubric credit is especially useful in Agentic RL, where context dependence, tool interaction, and side effects are easily obscured by a single composite reward; its open challenges remain rubric quality, judge bias, criterion weighting, and reward hacking.

利用[164, 192]；(2) 作为奖励模型或验证器的监督机制，包括Prometheus、OpenRubrics奖励模型以及可解释评分器R3[19, 278, 384]；且(3)将其作为政策行为的指导依据，而非仅用于最终评分——这与“基于评分量表的强化学习”（Rubric-Scaffolded RL）方法不同；后者在策略执行阶段将评分量表作为递减式探索支架引入，在策略利用阶段则将其作为评估参考依据[887]。评分量表信用机制在代理式强化学习（Agentic RL）中尤为有用，因为在该领域，上下文依赖性、工具交互及副作用极易被单一复合奖励所掩盖；其面临的开放性挑战仍包括评分标准质量、评委偏见、评分标准权重分配以及奖励操纵等问题。

5.5.2 Policy Optimization

5.5.2 策略优化

Policy optimization is where credit signals are turned into parameter updates. In Agentic RL, this step is difficult because the updates must be computed from multi-turn trajectories that mix tool calls, environmental observations, GUI states, and delayed terminal rewards [261, 321, 628, 681]. We organize the discussion around three practical questions: how to make sparse-reward updates stable, which interaction unit should be optimized, and how to preserve exploration over long trajectories.

策略优化是指将信用信号转化为参数更新的过程。在代理式强化学习中，这一步骤较为困难，因为更新值必须基于多轮轨迹来计算，而这些轨迹会混合工具调用、环境观测、GUI状态以及延迟的终端奖励[261, 321, 628, 681]。我们围绕三个实际问题组织本次讨论：如何使稀疏奖励更新过程保持稳定、应优化何种交互单元，以及如何在长轨迹中维持探索能力。

Clipping-based Policy Optimization. The first problem is how to make PPO- or GRPO-style RLVR reliable when trajectories are long and rewards are sparse or binary [460, 524, 525, 533]. The main difficulty is the mismatch between noisy group statistics and long sequence-level gradients. This mismatch appears as length bias, local normalization bias, homogeneous all-correct or all-wrong groups, unstable token-level importance ratios, and reward-scale drift. Existing methods fall into two broad categories. One category makes group-relative updates less biased through normalization, clipping, and sampling design; representative examples include REINFORCE++, DAPO, Dr.GRPO, CISPO, and GPPO [222, 389, 435, 565, 780]. GSPO defines the importance ratio and applies clipping at the sequence level rather than per token [865]. The other category changes the estimation unit or objective to reduce variance and constrain policy drift, with representative examples such as BNPO [95, 205, 391, 710].

基于截断的策略优化。第一个问题在于：当轨迹较长且奖励信号稀疏或为二值时，如何使PPO或GRPO风格的RLVR方法保持可靠性[460, 524, 525, 533]。主要难点在于噪声群组统计量与长序列级梯度之间的不匹配问题。这种不匹配现象表现为长度偏差、局部归一化偏差、均质的全部正确或全部错误数据组、不稳定的Token 级别重要性比率以及奖励尺度漂移。现有方法大致可分为两大类。其中一类方法通过归一化、截断和采样设计来降低群体相对更新的偏差；代表性示例包括REINFORCE++、DAPO、Dr.GRPO、CISPO和GPPO [222, 389, 435, 565, 780]。GSPO 定义了重要性比率，并在序列层面而非单个词元层面应用截断操作[865]。另一类方法通过改变估计单元或优化目标来降低方差并约束策略漂移，其代表性示例包括BNPO[95, 205, 391, 710]。

Turn-level Policy Optimization. The second problem is the granularity of optimization. In agent-environment interaction, the meaningful decision unit is often a trajectory, turn, step, subgoal, or branch rather than a single response. Early agentic RL systems therefore lift optimization from response-level training to full interaction trajectories, as in web and search agents such as WebAgent-R1 [261, 681]. A more fine-grained line then aligns advantages with explicit segments, steps, or turns, represented by SegPO, StepPO, and Turn-PPO [321, 628, 901]. More recent methods make the comparison set itself more local and informative. Some anchor comparisons to shared states or trajectory structures, as in GiGPO and related methods [156, 212, 318, 349]. Others use shared branches, subgoal segments, or hindsight information to ask which local choice changed the downstream trajectory, as in Tree-GRPO and its extensions [250, 478, 582, 687].

转弯级策略优化。第二个问题在于优化的粒度。在智能体与环境的交互过程中，具有实际意义的决策单元通常是轨迹、转弯、步、子目标或分支，而非单一响应。因此，早期的智能体强化学习系统将优化过程从响应级训练提升至完整的交互轨迹层面——例如 WebAgent-R1[261, 681]等 Web 和搜索智能体。更细粒度的线可将优势与显式片段、步骤或转向进行对齐，这些元素分别由 SegPO、StepPO 和 Turn-PPO 表示[321, 628, 901]。较新的方法则使比较集本身更具局部性和信息量。部分锚定比较方法采用共享状态或轨迹结构，例如 GiGPO及其相关方法[156, 212, 318, 349]。其他方法则利用共享分支、子目标段或事后信息来探究哪个局部选择改变了下游轨迹，例如 Tree-GRPO 及其扩展版本[250, 478, 582, 687]。

Entropy-based Policy Optimization. The third problem is exploration. Long-horizon RL can quickly over-reinforce early successful patterns, causing entropy collapse and reducing the diversity of future trajectories. Recent work therefore treats entropy as a decision- or trajectory-level control variable rather than only a token-level regularizer. One group modifies the update rule to avoid excessive probability concentration: Clip-Cov and KL-Cov gate over-concentrating updates, and OPEFO balances entropy-increasing and entropy-decreasing gradients [103, 725]. Another group injects diversity at critical decision points, for example CURE, which re-concatenates rollouts around critical (high-entropy) tokens to prevent entropy collapse [330]. Other methods, such as AEnt and AER, attempt to stabilize policy entropy within a bounded interval to alleviate entropy collapse [536, 830]. For multi-turn agents, the same idea becomes trajectory-level exploration control: EPO regularizes trajectory entropy under sparse rewards [732]. ARPO and AEPO branch from high-entropy positions after tool feedback to trade off rollout diversity against gradient distortion [128, 130].

基于熵的策略优化。第三个问题在于探索。长时程强化学习可能会迅速过度强化早期的成功模式，从而导致熵坍缩，并降低未来轨迹的多样性。因此，近期研究将熵视为决策或轨迹层面的控制变量，而非仅将其作为标记层面的正则化项。其中，有一组研究通过修改更新规则来避免概率过度集中：Clip-Cov 和 KL-Cov gate 可有效抑制更新过程中的过度集中现象，而 OPEFO 则能平衡熵增梯度与熵减梯度[103, 725]。另一组方案在关键决策节点引入多样性，例如 CURE——该方案通过围绕关键（高熵）令牌对部署过程进行重新拼接，以防止熵坍缩现象发生[330]。其他方法（如AEnt和AER）旨在将策略熵稳定在某个有界区间内，以缓解熵坍缩[536, 830]。对于多轮智能体，这一思路可转化为轨迹级探索控制：EPO可在稀疏奖励下对轨迹熵进行正则化[732]。ARPO与AEPO在工具反馈后从高熵位置分支，旨在在推演多样性与梯度失真之间进行权衡[128, 130]。

5.5.3 Sampling Strategy

5.5.3 抽样策略

In long-horizon agent tasks, the sampling target is multi-turn interaction trajectories encompassing tool calls, environmental observations, memory operations and final rewards [808, 821]. Unlike credit assignment and policy optimization, the sampling strategy decides which trajectories are generated in the first place. We organize this from three angles: trajectory-level rollout that fixes the basic sampling unit, tree-structured sampling that shares computation and produces relative signals, and budget-aware pruning that keeps sampling informative under a fixed cost.

在长时程智能体任务中，采样目标是多轮交互轨迹，该轨迹涵盖了工具调用、环境观测、记忆操作及最终奖

励[808, 821]。与信用分配和策略优化不同，采样策略首先决定了哪些轨迹将被生成。我们从三个角度对这一方法进行组织：轨迹级部署——固定基本采样单元；树状结构采样——共享计算资源并生成相对信号；以及预算感知剪枝——在固定成本约束下保持采样的信息量。

Trajectory-level Rollout. The basic sampling unit in Agentic RL is a multi-step agent-environment trajectory: at each step the model acts on observations, history, and task goals, and the environment returns new observations and reshapes the future state distribution. Early systems represented by ReAct established this reasoning-acting-observing loop, together with tool use, verbal reflection, and skill reuse, mostly through prompting rather than training [523, 544, 630, 761]. Subsequent RL work turned such loops into the rollout unit: a line of information-seeking agents led by Search-R1 folds query generation, evidence acquisition, and stopping decisions into on-policy trajectories [256, 261, 553, 871], while web and tool agents organize browser operations or executable tool composition as multi-turn online trajectories [484, 653, 681, 701, 834]. At larger scale, deep-research systems represented by WebDancer and the WebSailor family build end-to-end pipelines that combine synthetic data, tool-based web environments, and trajectory-level rollout [326, 327, 689]. On top of such pipelines, WebResearcher, ReSum, and WebWeaver further address long-context research, context summarization, and evidence structuring during sampling [356, 490, 693].

轨迹级部署。代理式强化学习中的基本采样单元是一个多步的“智能体-环境”轨迹：在每一步中，模型根据当前观测值、历史信息及任务目标进行决策，而环境则返回新的观测值并重塑未来的状态分布。以 ReAct 为代表的早期系统确立了这一“推理-行动-观察”循环，并结合工具使用、言语反思及技能复用机制，这些机制大多通过提示而非训练来实现[523, 544, 630, 761]。随后的强化学习研究将此类循环结构转化为“rollout 单元”：由 Search-R1 引导的一系列信息寻求智能体，将查询生成、证据获取及终止决策整合为同策略轨迹[256, 261, 553, 871]。而网络与工具代理则将浏览器操作或可执行工具的组合组织为多轮在线交互轨迹[484, 653, 681, 701, 834]。在更广泛的层面，以 WebDancer 和 WebSailor 系列为代表的深度研究系统能够构建端到端的流水线，该流水线将合成数据、基于工具的网络环境以及轨迹级部署相结合[326, 327, 689]。除上述流程外，WebResearcher、ReSum 和 WebWeaver 还进一步解决了长上下文研究、上下文摘要以及采样过程中的证据结构化问题[356, 490, 693]。

Tree-structured Sampling. Independent full-trajectory rollout is expensive and, when trajectories are homogeneous, yields weak relative learning signals. Tree-structured sampling addresses both problems by sharing prefixes across rollouts and contrasting sibling branches. The idea originates from organizing thoughts or actions as trees during reasoning and planning, as in Tree of Thoughts, RAP, and LATS [204, 760, 875], and is brought into RL training by Tree-GRPO, which compares sibling branches under a fixed token or tool-call budget [250]. A subsequent group refines where and how the tree expands: AT2PO, TreePO, and PORTool guide expansion toward uncertain turns, reuse inference computation, or organize shared and divergent tool-use paths [348, 687, 900], while branching methods such as BranPO and Tool-Light branch selectively from high-entropy positions, late critical decisions, or otherwise informative states to maximize rollout contrast [76, 860].

树状采样。独立的完整轨迹模拟计算成本高昂，且当轨迹具有同质性时，所产生的相对学习信号较弱。树状采样通过在不同轨迹间共享前缀以及对比同胞分支，能够同时解决这两个问题。该理念源于在推理与规划过程中，将思想或行动以树状结构进行组织，例如“思维树”（Tree of Thoughts）、RAP 和 LATS [204, 760, 875]。该数据通过 Tree-GRPO 引入强化学习（RL）训练过程，该方法在固定的令牌或工具调用预算下对同级分支进行比较[250]。后续研究团队进一步优化了树结构的扩展位置与方式：AT2PO、TreePO 和 PORTool 分别引导扩展过程向不确定转向点、复用推断计算方向或组织共享与分歧的工具使用路径[348, 687, 900]。而诸如 BranPO 和 Tool-Light 等分支方法则会从高熵位置、后期关键决策点或具有信息价值的状态中进行选择性分支，以最大化 rollout 对比度[76, 860]。

Budget-aware and Pruning Strategies. A third angle keeps sampling informative under a fixed rollout budget, either by pruning or reshaping the trajectory distribution or by allocating the branching budget adaptively. One line prunes or reshapes the trajectory distribution: representative mechanisms include experience reuse and confidence-based filtering in Tool-R1 and WebRL, difficulty- or pass-rate-based pruning in LiteResearcher, and trajectory-depth curricula in TCOD [334, 484, 636, 834]. These strategies discard low-signal rollouts, concentrate budget on contrastive or appropriately difficult cases, and bound trajectory length. They reduce the cost of producing each informative training sample in long-horizon settings. Another research approach makes branching itself budget-aware: ARPO triggers branch sampling only at high-entropy tool-call steps, reducing the tool-use budget of trajectory-level RL, while AEPO adaptively allocates the budget between global and branch sampling [128, 130].

预算感知与剪枝策略。第三种思路是在固定 rollout 预算下仍能进行信息量丰富的采样，其实现方式可包括：对轨迹分布进行剪枝或重塑，或对分支预算进行自适应分配。单行代码可对轨迹分布进行剪枝或重塑：代表性机制包括 Tool-R1 和 WebRL 中的经验复用与置信度过滤，以及 LiteResearcher 中的基于难度或通过率的剪枝。以及 TCOD 中的轨迹深度课程[334, 484, 636, 834]。这些策略会舍弃低信号的 rollout 情况，将预算集中于对比性或难度适中的案例，并对轨迹长度进行约束。它们能够降低长时程场景下生成每个信息性训练样本的成本。另一种研究方法可使分支选择过程本身具备预算感知特性：ARPO 仅在熵值较高的工具调用步骤触发分支采样，从而降低了轨迹级强化学习的工具使用预算。AEPO 可自适应地在全局采样与分支采样之间分配预算[128, 130]。

5.5.4 Interaction Patterns

5.5.4 交互模式

Interaction patterns concern how to organize optimizable decision-making structures during training. We discuss them from three perspectives: hierarchical interaction frameworks, multi-agent interaction frameworks, and memory-state learning.

交互模式涉及在训练过程中如何组织可优化的决策结构。我们从三个视角对此进行探讨：层级式交互框架、多智能体交互框架以及记忆-状态学习。

Hierarchical Interaction Frameworks. Hierarchical frameworks exploit the temporal abstraction that naturally exists in long-horizon tasks, and split along two threads. The first is explicit planner-executor decomposition, where the policy is separated into a high-level module that sets subgoals and a low-level module that executes them, so that subgoal boundaries become new optimization units. Representative instances include HiPER, GLIDER, and STEP-HRL, which differ mainly in whether the low-level layer is a task-agnostic skill set or a progress-compressed executor [232, 478, 863]. The second thread pushes hierarchy toward experience abstraction and capability reuse: building on the observation that procedural skills in agentic exploration originally rely on external skill libraries and prompting. SkillRL evolves a skill library from validation failures during RL and Skill0 internalizes skills into model parameters via in-context agentic RL [405, 705]. Analogous high-low decompositions appear in tool-augmented reasoning, where THOR jointly optimizes episode-level problem solving and step-level tool or code operations [52].

分层交互框架。分层框架利用了长时程任务中自然存在的时间抽象特性，并沿两条线程进行划分。第一种方法是显式的规划器-执行器分解，即将策略拆分为一个负责设定子目标的高层模块和一个负责执行这些子目标的低层模块，从而使子目标边界成为新的优化单元。代表性实例包括 HiPER、GLIDER 和 STEP-HRL，它们的主要区别在于底层层是采用任务无关的技能集还是采用进度压缩型执行器[232, 478, 863]。第二条思路将层次化设计导向经验抽象与能力复用：其基础在于观察到，在具身探索中，程序性技能最初依赖于外部技能库和提示机制。SkillRL 通过在强化学习过程中处理验证失败来构建技能库；而 Skill0 则通过上下文智能体强化学习[405, 705]将各类技能内化为模型参数。类似的高低分解方法也出现在工具增强推理中，其中 THOR 能够同时优化回合级问题求解与步骤级工具或代码操作[52]。

Multi-agent Frameworks. Multi-agent frameworks extend the training target from a single policy to a joint decision-making system. Their central difficulties are role non-stationarity, enlarged joint action spaces, and credit assignment under shared rewards. One thread formalizes LLM collaboration as cooperative MARL in Dec-POMDP settings and adapts group-relative training accordingly. MAGRPO uses centralized group-relative advantages with decentralized execution, while Stronger-MAS (AT-GRPO) restores variance reduction by grouping samples per agent and per turn [382, 861]. To assign credit across heterogeneous roles, M-GRPO computes separate group-relative advantages for a planner and its tool-executing sub-agents under decoupled cross-server training, whereas MATPO trains the planner and worker within a single LLM by normalizing over their joint rollouts [218, 439]. Dr.MAMR instead pushes credit to the step level with a Shapley-inspired causal-influence estimate to counter lazy-agent collapse [850]. A second thread covers self-play, debate, and role specialization, which inherit classical MARL principles such as centralized training with decentralized execution, counterfactual baselines, value decomposition, and MAPPO [396, 506, 776]. Within this thread, recent systems instantiate complementary System 1/System 2 policies for deep research, variance-reduced group-relative training with evolving memory, and role-conditioned advantage estimation in zero-sum language games [56, 201, 371]. Multi-agent debate adds a further variant, where competitive interaction provides additional preference, refutation, and verification signals for the final answer [139, 358].

多智能体框架。多智能体框架将训练目标从单一策略扩展至联合决策系统。其核心难点在于角色非平稳性、扩大的联合动作空间以及共享奖励下的信用分配问题。其中，有一篇论文将LLM协作问题建模为Dec-POMDP环境下的合作多智能体强化学习（cooperative MARL），并相应地调整了群组相对训练策略。MA-GRPO采用集中式群体相对优势与分散式执行相结合的方式；而Stronger-MAS（AT-GRPO）则通过按智能体和轮次对样本进行分组来实现方差缩减[382, 861]。为实现跨异构角色的信用分配，M-GRPO在解耦的跨服务器训练框架下，分别计算规划器及其工具执行子代理的组相对优势。而MATPO则通过对其联合策略输出进行归一化处理，从而在单一大型语言模型（LLM）内对规划器与工作者进行训练[218, 439]。Dr.MAMR则通过采用受Shapley启发的因果影响估计方法，将信用分配至步骤层级，以应对懒惰智能体崩溃问题[850]。第二部分探讨了自博弈、辩论及角色专业化问题，这些内容继承了经典的多智能体强化学习（MARL）原理，例如集中式训练与分散式执行、反事实基线、价值分解以及MAPPO[396, 506, 776]。在本讨论帖中，近期研究工作通过系统实例化互补的System 1/System 2策略，实现了以下目标：深度研究、基于动态记忆的方差缩减群相对训练，以及零和语言博弈中的角色条件优势估计[56, 201, 371]。多智能体辩论引入了另一种变体：在该模型中，竞争性交互可为最终答案提供额外的偏好、反驳及验证信号[139, 358]。

Memory-state Learning. Memory-state learning turns runtime structures that are usually fixed by an external harness, including context, workflows, long-term memory, tool-use histories, and reusable experience, into learnable policies. This shifts Agentic RL from optimizing isolated LLM calls to training state management over long horizons. One thread makes memory operations themselves a learning target: Memory-R2 observes that memory writes place trajectories in different effective environments and uses LoGo-GRPO to combine a global objective with local comparisons from the same memory state, Memory-R1 separates a Memory Manager from an Answer Agent, and Memory as Action treats working-memory editing as an explicit policy action handled by Dynamic Context Policy Optimization [742, 743, 846]. A parallel thread lifts the same idea to the organization of the task process: Agent Workflow Memory induces reusable workflows from successful tasks, FlowSteer optimizes a workflow director with structural and task-level rewards, and Agent Lightning offers a system-level route that traces existing workflows as spans and transitions and converts them into trainable samples [409, 674, 823].

记忆状态学习。记忆状态学习将通常由外部系统固定的运行时结构（包括上下文、工作流、长期记忆、工具使用历史及可复用经验）转化为可学习的策略。这一转变将智能体强化学习（Agentic RL）的重点从优化独立的大型语言模型（LLM）调用，转向对长周期状态管理进行训练。其中一个线程将内存操作本身设定为学习目标：Memory-R2会观察到内存写入操作会在不同的有效环境中生成轨迹，并利用LoGo-GRPO将全局目标与来自同一内存的局部比较相结合。在该状态下，Memory-R1将内存管理器与回答代理分离；而“Memory as Action”则将工作记忆编辑视为一项显式策略动作，由动态上下文策略优化机制[742, 743, 846]来处理。并行线程将这一理念进一步延伸至任务流程的组织层面：Agent Workflow Memory可从成功任务中提取可复用的工作流；而FlowSteer则通过结构级与任务级奖励对工作流控制器进行优化。Agent Lightning提供了一套系统级路由方案，可将现有的工作流以跨度和状态转移的形式进行追踪，并将其转换为可训练的样本[409, 674, 823]。

5.6 On-Policy Distillation

5.6 在线策略蒸馏

Alongside the emergence of Agentic RL, recent research has increasingly explored alternative training paradigms tailored to long-horizon trajectory optimization. As interaction horizons grow, agent training becomes increasingly challenged by distribution shift, error accumulation, and sparse, delayed supervision [377]. Existing training paradigms only partially address these challenges. Off-policy imitation, such as behavioral cloning and offline knowledge distillation [609], relies on static expert trajectories and therefore fails to expose agents to their own induced state distributions. Reinforcement learning [94], in contrast, optimizes on on-policy trajectories but often suffers from sparse rewards, costly exploration, and poor sample efficiency.

随着代理强化学习（Agentic RL）的兴起，近期研究日益关注针对长时程轨迹优化而设计的替代训练范式。随着交互范围的扩大，智能体训练面临着日益严峻的分布偏移、误差累积以及稀疏且延迟的监督问题[377]。现有的训练范式仅能部分解决这些挑战。离线策略模仿（例如行为克隆与离线知识蒸馏[609]）依赖于静态专家轨迹，因此无法让智能体接触到其自身诱导出的状态分布。相比之下，强化学习[94]虽能优化同策略轨迹，但往往面临稀疏奖励、探索成本高昂以及样本效率较低等问题。

On-Policy Distillation (OPD), first formalized for language models by GKD [3], combines the high-quality supervision of distillation with the state coverage of on-policy learning [309, 635]. Instead of learning solely from fixed expert demonstrations or sparse outcome rewards, OPD trains the student on trajectories generated by its current policy and provides additional supervisory signals on these self-generated trajectories. As a result, OPD exposes the student to self-induced state distributions while providing richer supervision than sparse outcome rewards. This delimits OPD from its neighbors: unlike the offline trajectory distillation of Section 5.4, which learns from a fixed set of teacher or past-model traces, OPD draws supervision on the student’s own current rollouts; unlike the self-evolution of Section 5.7, it keeps the task set and environment fixed and only densifies the supervision signal. These properties make OPD particularly attractive for long-horizon agents, where robustness to distribution shift, recovery from compounding errors, and access to dense supervisory signals are all critical for effective learning [697, 838].

在线策略蒸馏（OPD）最初由GKD[3]对语言模型进行形式化定义，它将高质量的蒸馏监督学习与在线策略学习的状态覆盖范围相结合[309, 635]。OPD方法并非仅通过固定的专家演示或稀疏的结果奖励来训练学生，而是利用当前策略生成的轨迹对学生进行训练，并针对这些自动生成的轨迹提供额外的监督信号。因此，OPD让学生接触到自诱导的状态分布，同时提供了比稀疏结果奖励更丰富的监督信号。这一特性将OPD与其它方法区分开来：与第5.4节中所述的离线轨迹蒸馏方法不同。OPD从一组固定的教师模型或历史模型轨迹中学习，并利用学生当前的输出结果进行监督；这与第5.7节中的自进化方式不同。该方法保持任务集与环境不变，仅对监督信号进行稠密化处理。这些特性使得OPD在长时程智能体中尤为具有吸引力——对于此类智能体而言，对分布偏移的鲁棒性、从复合误差中恢复的能力以及获取密集监督信号的能力，均对实现有效的学习至关重要[697, 838]。

5.6.1 Teacher-guided OPD

5.6.1 教师引导的门诊诊疗

In teacher-guided OPD, one or more teacher models are queried on states visited by the student policy to provide supervisory signals for the generated trajectories [309, 312, 635, 697]. Unlike conventional offline distillation that relies on fixed expert demonstrations, teacher-guided OPD generates supervision adaptively online according to the student’s evolving trajectories, allowing teachers to provide guidance tailored to the

student’s current behaviors and mistakes. Existing methods can be characterized by two complementary design choices: how supervision is organized over trajectories and how teacher signals are constructed.

在教师引导的OPD方法中，系统会查询一个或多个教师模型所访问的状态，为生成的轨迹提供监督信号[309, 312, 635, 697]。与依赖固定专家示范的传统离线蒸馏方法不同，教师引导的OPD能够根据学生不断演变的学习轨迹，在线自适应地生成监督信号。使教师能够根据学生的当前行为与错误，提供量身定制的指导。现有方法可归纳为两个互补的设计选择：即如何对轨迹进行监督，以及如何构建教师信号。

Supervision Scheduling. A first question is how teacher signals should be organized and delivered throughout long interaction trajectories. Representative methods such as DAgger-LLM introduce turn-level interpolation and step-wise supervision to provide dense corrections on states visited by the student policy [309, 873]. Other approaches further incorporate adaptive curricula and preference-based guidance, progressively adjusting the timing, granularity, and difficulty of supervision during training [313, 636, 835]. What emerges is that teacher-guided OPD hinges not only on the quality of teacher signals, but also on when and at what resolution they are injected during multi-turn interactions.

监督调度。首要问题在于：在漫长的交互过程中，教师信号应如何组织与传递？诸如DAgger-LLM等代表性方法引入了转接层级插值与分步监督机制，旨在为学生策略所访问的状态提供密集的修正[309, 873]。其他方法还进一步引入了自适应课程与基于偏好指导机制，能够逐步调整训练过程中监督的时机、粒度及难度[313, 636, 835]。研究结果表明，教师引导的OPD不仅取决于教师信号的质量，还取决于这些信号在多轮交互过程中何时以及以何种分辨率注入。

Teacher Signals. A second question is how to construct richer and more informative supervision for on-policy distillation. A series of methods represented by MAD-OPD leverage multiple agents or agent evolvers to aggregate complementary expertise and generate more reliable supervision than a single teacher policy [635, 809]. Along a related line, KAT-Coder-V2 [312] exploits specialist agents with complementary capabilities and distills their collective knowledge into a unified policy. Retrieval-augmented approaches such as LiteGUI [697] instead enrich teacher supervision with oracle trajectories and dynamically retrieved references, providing structured guidance in multi-solution interaction environments. Across these efforts, the recurring lesson is that supervision improves when teacher signals are made more diverse, complementary, and context-aware.

教师信号。第二个问题在于，如何为在线策略蒸馏构建更丰富、更具信息量的监督机制。以 MAD-OPD 为代表的一系列方法，通过利用多个智能体或智能体演化器来聚合互补性专业知识，从而生成比单一教师策略更可靠的监督信号[635, 809]。在这一相关思路下，KAT-Coder-V2[312]利用具备互补能力的专用智能体，并将这些智能体的集体知识提炼为统一策略。检索增强型方法（例如 LiteGUI[697]）通过引入先验轨迹与动态检索到的参考文献来增强教师监督机制，从而在多解法交互环境中提供结构化引导。纵观这些实践，一个反复出现的启示是：当教师信号变得更加多样化、互补且具有情境意识时，教学督导效果便会得到提升。

5.6.2 Self-improving OPD

5.6.2 自我提升OPD

Self-improving OPD removes direct reliance on external teacher models and instead constructs its own supervisory signals from auxiliary information available during training. Such information generally arises from two complementary sources: privileged information that is accessible only during training [838] and adaptive feedback generated through repeated interactions with the environment [216, 242, 845].

自改进OPD模型无需依赖外部教师模型，而是利用训练期间可获取的辅助信息来构建自身的监督信号。此类信息通常源自两个互补的来源：仅在训练期间可访问的特权信息[838]，以及通过与环境反复交互而生成的自适应反馈[216, 242, 845]。。

Privileged Information. One source of self-supervision is information that is accessible during training but unavailable at deployment time. Some methods leverage privileged annotations or modalities, such as outcome-aware information and fine-grained visual details, to provide dense supervision beyond what the agent’s own observations reveal [788]. Others derive privileged information from the interaction trajectories themselves, treating intermediate construction processes, latent skills, or other trajectory-level abstractions as additional supervision that exposes the hidden structure of successful behaviors [631, 838]. Augmenting sparse interaction signals with training-only information in this way helps the student acquire more robust and reusable long-horizon behaviors.

特权信息。自我监督的一个来源是：在训练期间可访问但在部署时不可用的信息。部分方法利用特权注解或模态（例如结果感知信息和细粒度视觉细节），以提供超越智能体自身观测所能揭示范围的密集监督[788]。其他方法则直接从交互轨迹本身中提取特权信息，处理其中的中间构建过程与潜在技能。或采用其他轨迹级抽象方法作为额外监督信号，以揭示成功行为背后的隐藏结构[631, 838]。通过这种方式，利用仅含训练信息的手段增强稀疏交互信号，有助于学习者习得更鲁棒且可复用的长时程行为。

Environment Feedback. The other source of self-supervision is the interaction outcomes and verifier signals returned by the environment. Some methods transform sparse environmental feedback, such as rewards, textual critiques, runtime errors, and verifier outputs, into dense self-supervision by conditioning the policy on feedback-informed predictions and distilling them back into the model [216, 242]. A complementary view treats failures as supervision, generating retrospective reflections and accumulating reusable lessons that guide future interactions [757, 845]. Beyond obtaining feedback, a further line studies how to use it well during distillation, for example by emphasizing informative signals, learning feedback-aware representations, and mitigating the instability caused by noisy or stale supervision [72, 161, 735, 782]. The decisive factor is therefore not whether environmental feedback is available, but how it is transformed, accumulated, and reused across long-horizon training.

环境反馈。自监督的另一来源是环境返回的交互结果与验证器信号。一些方法可对稀疏的环境反馈（例如奖励、文本评论、运行时错误及验证器输出）进行转换。通过将策略基于反馈驱动的预测进行条件化，并将这些预测蒸馏回模型中，从而实现密集式自监督学习[216, 242]。另一种互补性视角将故障视为一种监督机制，从而引发回顾性反思并积累可复用的经验教训，为未来的交互行为提供指导[757, 845]。除获取反馈外，另一类研究方向是探讨如何在蒸馏过程中有效利用反馈，例如：强调信息性信号、学习反馈感知表示，以及缓解由噪声或陈旧监督数据所导致的不稳定性[72, 161, 735, 782]。因此，决定性因素并非环境反馈是否可用，而是该反馈在长周期训练过程中如何被转化、累积和再利用。

5.7 Self-evolution

5.7 自进化

This subsection discusses self-evolution as a training mechanism for internalizing experience generated by the agent itself. The preceding stages mainly rely on prepared data, task sets, environment collections, or trajectories. Self-evolution adds a feedback loop: the current policy produces new attempts, failures, and feedback in its environment, and training writes selected records back into the model. This is what distinguishes it from the two distillation stages above: whereas fine-tuning distillation (Section 5.4) and on-policy distillation (Section 5.6) both consume a supervision source that is held fixed within a training

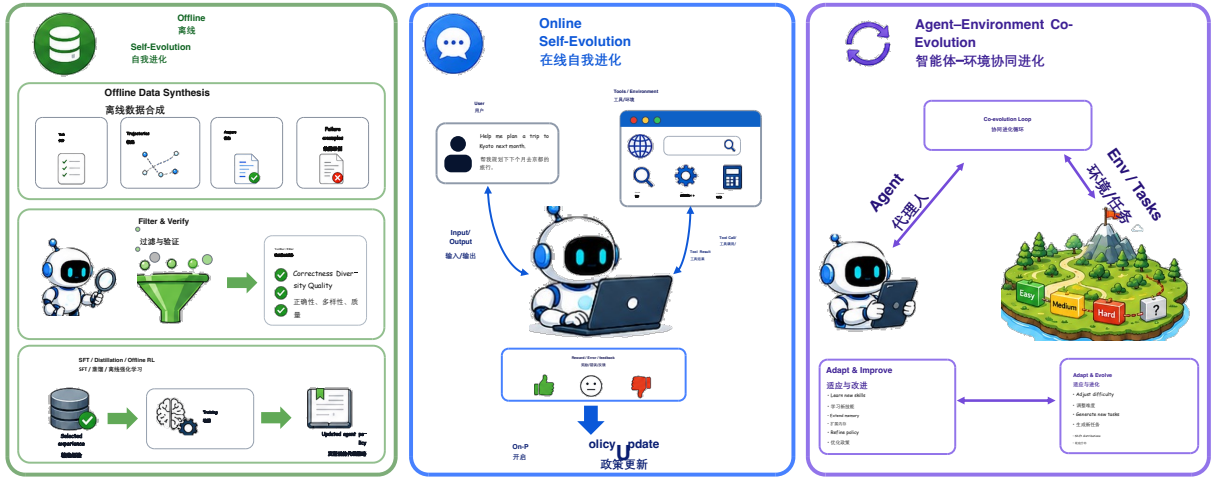


Figure 13 Three modes of self-evolution for long-horizon agents. *Offline self-evolution* learns from selected self-generated experience through a generate–filter–train pipeline; *online self-evolution* learns from live environment interaction on the agent’s own induced state distribution; and *agent–environment co-evolution* jointly adapts the agent and its task/environment distribution to maintain a learnable capability boundary.

图13：长时程智能体的三种自进化模式。 *离线自进化* 通过“生成–过滤–训练”流程，从选定的自生成经验中进行学习；*在线自进化* 则基于智能体自身诱导的状态分布，通过与实时环境的交互来实现学习；以及 *智能体–环境共进化* 机制可共同调整智能体及其任务/环境分布，以维持一个可学习的能力边界。

round, self-evolution treats the data, tasks, and even the environment as objects that the loop itself updates. This has recently grown into a fast-moving research area, and dedicated papers now catalog how agents can autonomously evolve their models, memories, tools, and workflows toward lifelong adaptation [147, 165]. As illustrated in Figure 13, we organize this line of work into three forms: offline self-evolution, online self-evolution, and agent–environment co-evolution.

本小节探讨了自进化作为一种训练机制，用于内化智能体自身所产生的经验。前序阶段主要依赖于预置数据、任务集、环境集合或轨迹。自进化机制引入了反馈回路：当前策略会在其环境中产生新的尝试、失败及反馈，而训练过程则将筛选出的记录写回模型。这正是其与上述两个蒸馏阶段的区别所在：而微调蒸馏（第5.4节）与在线策略蒸馏（第5.6节）则不同。两者均使用在训练轮次内保持固定的监督源；而自进化方法则将数据、任务乃至环境本身都视为该循环机制所更新的对象。该领域近年来已发展成为一个快速演进的研究方向，目前已有一些专门论文系统梳理了智能体如何能够自主演化其模型、记忆、工具及工作流程，以实现终身适应[147, 165]。如图13所示，我们可将这一研究方向归纳为三种形式：离线自进化、在线自进化以及智能体–环境共进化。

5.7.1 Offline Self-evolution

5.7.1 离线自进化

Offline bootstrapped self-evolution typically follows a generate–filter–train pattern. The model generates candidate tasks, answers, reasoning traces, code patches, tool calls, or interaction trajectories; verifiers, tests, reward models, teacher models, or self-evaluation mechanisms select successful samples, informative failures, and correctable fragments; training then turns these samples into SFT, distillation, preference optimization, or offline RL objectives. STaR bootstraps reasoning from successful rationales [793], and subsequent work extends the same idea to self-edits, tool-augmented reasoning, multi-agent demonstrations, and code-execution feedback [310, 517, 859, 906]. More recent work emphasizes how to turn search processes and failures into stable training assets. SOAR alternates evolutionary search with hindsight learning in program synthesis, converting search attempts into fine-tuning pairs; SAMULE synthesizes reflection data at the trajectory, task, and cross-task levels, and trains a retrospective model from failure analysis [173, 481]. A parallel search-based route runs the generate–filter–train loop for several rounds: rStar-Math uses Monte Carlo tree search to synthesize step-by-step verified reasoning trajectories and iteratively co-evolves a policy model together with a process preference model, so that each round’s verified rollouts become the next round’s supervision [191]. This route is attractive because data can be audited, denoised, and relabeled before an update, making it suitable for cold start and capability consolidation. Its main risk is that candidate data still largely comes from the existing model distribution, which can reinforce known behaviors while doing little to discover out-of-distribution strategies.

离线自举式自我进化通常遵循“生成–过滤–训练”模式。该模型可生成候选任务、答案、推理轨迹、代码补丁、工具调用或交互轨迹；而验证器、测试用例、奖励模型、教师模型或自我评估机制则可筛选出成功的样本与具有信息价值的失败案例。以及可修正片段；随后，训练过程将这些样本转换为SFT、蒸馏、偏好优化或离线强化学习目标。STaR通过基于成功的推理方案[793]进行推理，后续研究则将这一思路扩展至自我编辑、工具增强推理、多智能体演示以及代码执行反馈[310, 517, 859, 906]。近期的研究更侧重于探讨如何将搜索过程及其失败转化为稳定的训练资源。SOAR在程序合成成功中交替采用进化搜索与回溯学习，将搜索尝试转化为微调对；而SAMULE则在轨迹、任务及跨任务等多个层面合成反思数据。并基于故障分析训练一个回顾性模型[173, 481]。并行搜索式路径规划方法会执行多次“生成–过滤–训练”循环：rStar-Math采用蒙特卡洛树搜索技术，来合成逐步经过验证的推理轨迹，并迭代地协同进化策略模型与过程。偏好模型，使得每轮经过验证的部署结果可作为下一轮的监督数据[191]。该路径具有吸引力，因为数据可在更新前进行审计、去噪及重新标注，因此特别适用于冷启动和能力整合场景。其主要风险在于，候选数据仍主要来源于现有模型的分布，这可能会强化已知行为，而对发现分布外策略则收效甚微。

5.7.2 Online Self-evolution

5.7.2 在线自我进化

Online interactive self-evolution interleaves experience collection with policy updates, training the model on state distributions induced by its own behavior. For long-horizon agents, difficult states are often not static benchmark inputs; they arise from early decisions, tool side effects, recovery attempts, and environment state changes. Recent work therefore incorporates web, code, search, tool-use, and multi-agent game environments

into training, using rollout, environment feedback, and reinforcement learning to continually update the policy [484, 672, 701, 799, 871]. A complementary line reduces external data seeds by using self-play, challenger–solver, or proposer–solver structures to create training pressure, so that the model obtains learning signals by proposing tasks, solving them, and verifying outcomes [233, 371, 854]. Recent agentic instantiations push this toward tool use and multi-step reasoning: Agent0 co-evolves a curriculum agent and a tool-using executor from the same base model with zero external data, so that a stronger executor pressures the curriculum agent to propose harder, tool-aware tasks [704]. Compared with offline bootstrapping, online self-evolution is more likely to move beyond the ceiling of existing data, but it is also more sensitive to the quality of environments and verifiers: delayed rewards, credit-assignment errors, policy collapse, verifier loopholes, and exploration cost are all amplified in long rollouts.

在线交互式自进化机制将经验收集与策略更新相结合，通过在其自身行为所诱导的状态分布上对模型进行训练。对于长时程智能体而言，困难状态通常并非静态的基准输入：它们往往源于早期决策、工具副作用、恢复尝试以及环境状态的变化。因此，近期的研究将网络、代码、搜索、工具使用及多智能体博弈环境纳入训练过程，通过 rollout、环境反馈和强化学习机制，持续更新策略[484, 672, 701, 799, 871]。互补线通过采用自我对弈、挑战者–求解器或提议者–求解器结构来产生训练压力，从而减少外部数据种子，使模型能够通过提出任务并完成这些任务来获取学习信号。并验证结果[233, 371, 854]。近期的智能体实例化研究正推动这一方向向工具使用与多步骤推理演进：Agent0可在无需任何外部数据的情况下，从同一基础模型中协同进化出一个课程规划器和一个工具使用执行器，从而使更强的执行器能够对课程规划器产生压力。代理可提出更难的、具备工具感知能力的任务[704]。与离线自举法相比，在线自进化更有可能突破现有数据的上限，但同时也对环境及验证器的质量更为敏感——例如延迟奖励、信用分配错误等问题。政策崩溃、验证器漏洞以及探索成本在长周期部署中均会加剧。

5.7.3 Agent-Environment Co-evolution

5.7.3 智能体–环境共进化

Agent–environment co-evolution further treats the training environment as an object of update. A fixed environment can quickly drift away from the model’s capability boundary: if it is too easy, it only reinforces existing strategies; if it is too hard, it yields little usable learning signal. A first line makes the environment itself tunable to the policy: Environment Tuning uses curricula, environment augmentation, and progress feedback to mitigate cold start and sparse rewards in tool-use training, while RLVE and GenEnv align the difficulty of verifiable or generated environments with the policy’s current capability, and Agent-World closes the loop by discovering environment–task pairs, exposing capability gaps through dynamic task synthesis, and improving the policy through multi-environment RL [131, 195, 401, 798]. In multi-agent settings, the environment may also be constituted by the policies of other agents; CoMAS uses interaction rewards to jointly update multiple agent policies, showing that peer interaction can serve as a training signal for self-evolution [738]. Socratic-Zero pushes this idea in a data-free direction, letting a Teacher, a Solver, and a Generator co-evolve so that the Teacher continually crafts questions targeting the Solver’s weaknesses while the Generator distills its curriculum-design strategy, effectively turning the task distribution itself into a co-adapting counterpart [648]. From this perspective, the environment is not merely a source of samples, but a dynamic curriculum, feedback generator, and probe of the capability boundary: improvements in the agent change future reachable states and failure modes, while environment updates reshape the next round of exploration and optimization.

智能体–环境共进化进一步将训练环境视为可更新的对象。固定环境可能迅速偏离模型的能力边界：若难度过低，只会强化现有策略；若难度过高，则难以产生有效的学习信号。第一线方法可使环境本身能够适配策略：环境调优通过采用课程学习、环境增强及进度反馈等方式，旨在缓解工具使用训练中的冷启动问题与稀疏奖励问题。RLVE与GenEnv能够将可验证或生成环境的难度与策略当前能力相匹配，而Agent-World则通过发现环境–任务对、借助动态任务合成来揭示能力缺口，从而完成闭环控制。并通过多环境强化学习[131, 195, 401, 798]改进策略。在多智能体环境中，环境还可能由其他智能体的策略构成；CoMAS利用交互奖励来协同更新多个智能体的策略。表明同伴交互可作为自我进化过程中的训练信号[738]。Socratic-Zero 将这一理念推向了无数据的方向，允许“教师”或“求解器”发挥作用。生成器与教师共同进化：教师持续设计针对求解器薄弱环节的问题，而生成器则提炼其课程设计策略。有效地将任务分配本身转化为一种协同适应的对应机制[648]。从这一视角来看，环境不仅是一个样本来源，更是一个动态课程、反馈生成器以及对能力边界进行探测的工具：智能体的改进会改变未来可到达的状态与故障模式。而环境更新则重塑了下一轮探索与优化过程。

Overall, training self-evolution is not unconstrained self-training, but an experience loop jointly constrained by environments, verifiers, and optimization. External memories, skill libraries, and workflow traces can serve as intermediate forms of accumulated experience, but they constitute training self-evolution only when their stable patterns are absorbed through distillation, fine-tuning, reinforcement learning, or trainable modules [705]. The risks of this loop arise from its bootstrapping structure: self-generated data may shrink the distribution, weak verifiers may induce reward hacking, and dynamic curricula may reinforce flawed capabilities. Long-horizon agent self-evolution therefore requires strong verifiers, process-level feedback, and a combination of offline audit with online exploration.

总体而言，训练自进化并非无约束的自训练过程，而是一个由环境、验证器及优化机制共同约束的经验循环。外部记忆、技能库及工作流程轨迹可作为累积经验的中间载体，但唯有当这些模式通过提炼与微调过程被吸收后，它们才能构成训练驱动的自我进化机制。强化学习，或可训练模块[705]。该循环的潜在风险源于其自举结构：自生成的数据可能压缩分布范围，薄弱验证器可能诱发奖励劫持，而动态课程设置则可能强化缺陷能力。因此，长周期智能体自我进化需要强大的验证机制、进程级反馈，以及离线审计与在线探索的有机结合。

6 Application: Long-Horizon Agents in Practice

6 应用领域：实际应用中的长视野智能体

The preceding sections examined the general ingredients of long-horizon agency, including persistent state, feedback-driven adaptation, and mechanisms for maintaining alignment over extended trajectories. We now turn to the settings in which these requirements arise in practice. Long-horizon agents operate across a wide range of environments, from code repositories and web documents to graphical interfaces, multimodal streams, and open-ended research workflows. These settings differ not only in task duration, but also in the interface through which an agent observes, acts, and receives feedback from the world, which in turn shapes what information is available during execution, what errors can be detected locally, and what capabilities the agent must add to compensate for what the environment does not provide. Therefore, we organize this section by the interface between the agent and its environment rather than by overall difficulty.

前文各节已探讨了长时程代理机制的总体构成要素，包括持续状态、反馈驱动的适应机制以及在较长时间跨度内维持一致性的作用机制。接下来，我们将探讨这些要求在实践中出现的具体情境。长时程智能体可应用于广泛的各类环境，涵盖代码仓库、网页文档、图形界面、多模态数据流以及开放式研究 workflows 等场景。这些设置不仅在任务时长上存在差异，还在智能体观察、执行及从环境中接收反馈所采用的界面方面有所不同；而该界面又会直接影响执行过程中可获取的信息内容。可进行本地检测的错误有哪些，以及智能体必须添加哪些功能以弥补环境无法提供的信息。因此，本节的编排并非基于整体难度，而是根据智能体与环境之间的接口进行划分。

Figure 14 groups representative applications into five broad categories, distinguished by the interface through which each agent observes and acts:

图 14 将代表性应用划分为五个大类，这些类别根据每个智能体观察与行动所采用的接口进行区分：

Five Representative Applications of Long-Horizon Agents
长时程智能体的五个典型应用案例

1. **Software Engineering Agents** (Section 6.1): act over code repositories and execution environments, where edits can often be checked through tests, compilers, and runtime outputs.

1. **软件工程智能体**（第6.1节）：可在代码库和执行环境中运行，其中对代码的修改通常可通过测试、编译器及运行时输出来验证。

2. **Information-seeking Agents** (Section 6.2): interact with web documents, search engines, and knowledge sources, where the challenge is to gather and synthesize evidence without a direct execution oracle.

2. **信息检索智能体**（第 6.2 节）：与 Web 文档、搜索引擎及知识源进行交互；其核心挑战在于在没有直接执行预言机的情况下收集并综合证据。

3. **Computer-use Agents** (Section 6.3): operate graphical interfaces through screenshots, accessibility trees, and low-level actions, inferring and manipulating partially observable interface state.

3. **计算机交互代理**（第6.3节）：通过屏幕截图、可访问性树及底层操作来操控图形界面，进而推断并操纵部分可观测的界面状态。

4. **Multimodal Agents** (Section 6.4): reason over long visual, auditory, and textual streams, where relevant information is distributed across time and modalities.

4. **多模态智能体**（第6.4节）：能够对长时程的视觉、听觉及文本流进行推理，其中相关信息可能分散于不同时间点及不同模态之中。

5. **General-purpose Agents** (Section 6.5): combine several such interfaces within a single workflow, drawing on heterogeneous tools, permissions, and external services.

5. **通用代理**（第6.5节）：可在单个工作流中整合多种此类接口，从而调用异构工具、权限及外部服务。

For each category, we ask three questions:

针对每个类别，我们提出三个问题：

- What information and actions does the interface expose?
- 该界面提供了哪些信息及操作功能？
- What kinds of state and feedback must persist across a long trajectory?
- 在漫长的轨迹中，必须持续存在哪些状态与反馈？
- What capabilities must the agent or harness add to compensate for what the environment does not provide directly?
- 代理或工具包必须添加哪些功能，以弥补环境无法直接提供的部分？

The following subsections address these questions by discussing the core capabilities required in each domain and representative systems that instantiate them.

以下各小节将通过探讨各领域所需的核心能力以及实现这些能力的代表性系统，来回答上述问题。

6.1 Software Engineering

6.1 软件工程

Software engineering agents are one of the most foundational and representative classes of long-horizon agents. The agent interacts with its environment through code: it acts by writing and running code, obtains feedback from how that code behaves, and uses this feedback to advance its task. A defining feature of this paradigm is unusually strong executable feedback: through tests, compilers, and runtime traces, the environment can often tell the agent automatically whether an action has succeeded, which is what distinguishes software engineering from many other long-horizon tasks. These signals nonetheless remain incomplete proxies for intended behavior: passing tests and clean compiles cannot always certify that the real issue is resolved, especially where hidden requirements, security, maintainability, and regression risk are concerned [499, 671, 801].

软件工程智能体是长时程智能体中最基础且最具代表性的类别之一。智能体通过代码与环境进行交互：它通过编写并运行代码来执行操作，从该代码的运行表现中获取反馈，并利用这些反馈来推进其任务。这一范式的一个显著特征是异常强烈的可执行反馈机制：通过测试、编译器及运行时跟踪信息，环境通常能够自动判断某个动作是否成功。这正是软件工程与许多其他长期规划型任务相区别的关键所在。尽管如此，这些信号仍不能完全代表预期的行为：通过测试或编译无误并不总能证明实际问题已得到解决——尤其是在存在隐藏需求、安全性或可维护性等问题的情况下。涉及回归风险[499, 671, 801]。

Turning this feedback into reliable long-horizon behavior, however, takes more than the signal itself. We organize long-horizon software engineering around three capabilities that an agent must add. The agent first has to work out *where* to act, building a usable picture of a codebase too large to fit in its context window; we call this *repository grounding*. It then has to keep the overall objective intact as it proceeds, ordering its edits and tracking progress rather than chasing one fix at a time, which is *workflow-level planning*. Finally, it has to use the feedback it receives to actually correct its mistakes, the task of *feedback-driven repair*. Together, these capabilities mark the step up from fixing isolated bugs to repository-scale work [124, 749], including implementation, migration, and pull-request preparation; commercial systems such as Devin² brought such end-to-end long-horizon coding agents into production. The next subsections examine each capability in turn. 然而，将这些反馈转化为可靠的长时程行为，仅靠信号本身是不够的。我们围绕着智能体必须具备的三项能力，来组织长时程软件工程。代理首先必须确定在何处采取行动，通过构建一个可用于操作的代码库视图——该视图对于过于庞大的代码库而言无法完全容纳于其上下文窗口内；我们称之为**代码库锚定**。随后，该系统在执行过程中必须保持整体目标不变——即对编辑操作进行统筹安排并跟踪进度，而非逐次追求单一问题的修复；这属于**工作流级规划**。最后，它必须利用接收到的反馈来实际修正其错误，即**反馈驱动修复**这一任务。这些功能共同标志着从修复孤立 Bug 到进行仓库级工作[124, 749]的升级，涵盖实现、迁移及 Pull Request 准备等工作；商业系统如 Devin² 亦可使用该工具。已将此类端到端长周期编码代理投入生产。下一小节将依次探讨各项功能。

6.1.1 Repository Grounding

6.1.1 存储库接地

Before any code is written, the agent must decide *where* to act. A natural-language issue typically says little about which files, dependencies, or project conventions are relevant, so long-horizon systems need a way to progressively construct and reuse a working model of the repository.

在编写任何代码之前，智能体必须决定在何处采取行动。自然语言描述通常难以明确指出哪些文件、依赖关系或项目规范是相关的；因此，长周期系统需要一种能够逐步构建并复用代码库工作模型的方法。

²Devin (Cognition): <https://devin.ai>.
²Devin（认知模块）：<https://devin.ai>。



Figure 14 Representative long-horizon agent applications grouped by the structure of the agent-environment interface. Different interfaces expose different forms of observation, action, state, and feedback, leading to distinct capability requirements for long-horizon behavior.

接口可提供不同形式的观察、动作、状态及反馈，从而对长时程行为提出了不同的能力要求。

Existing designs largely follow three strategies. The first is to improve *interactive navigation*: SWE-agent, for example, exposes search, navigation, and editing commands through an agent-computer interface tailored to LLM use [747], while structure-aware systems such as AutoCodeRover search over a program’s class and method hierarchy rather than over raw text [843]. The second is to maintain a *persistent workspace*, so that file, shell, and browser state can accumulate across steps rather than being repeatedly reconstructed; OpenHands is representative of this approach [657]. The third is to externalize repository structure in compact, reusable artifacts, such as Aider’s repository map,³ IDE-level project rules,⁴ and AGENTS.md-style instructions.⁵ Although implemented differently, these approaches share a purpose: turning a large, partially observed codebase into stable task-relevant state that can guide later decisions. Related work extends this idea by constructing executable environments from arbitrary repositories [248], or by training models to internalize more repository-level structure directly [46].

现有设计大多遵循三种策略。第一点是改进 **交互式导航**：例如，SWE-agent可通过专为LLM使用而定制的智能体-计算机接口，提供搜索、导航及编辑命令[747]。而结构感知系统（如AutoCodeRover）则是在程序的类与方法层次结构上进行搜索，而非在原始文本上进行搜索[843]。第二点是维护**持久工作空间**，以便文件、Shell 和浏览器的状态能够在不同步骤间持续累积，而无需反复重建；OpenHands 正是这一方法的典型代表[657]。第三点是将代码库结构封装到紧凑且可复用的构件中，例如Aider的代码库映射、³IDE级项目规则、⁴以及AGENTS.md格式指南。⁵尽管这些方法的实现方式各异，但它们都具有共同目标：将庞大的、部分可观测的代码库转化为稳定的、与任务相关的状态，从而为后续决策提供指导。相关研究拓展了这一思路，通过从任意代码库构建可执行环境[248]，或通过训练模型直接内化更深层次的代码库级结构[46]。

6.1.2 Workflow-level Planning

6.1.2 工作流级规划

Knowing where to act does not by itself determine how to sequence the work. Feature development, dependency migration, experiment reproduction, and pull-request preparation require the agent to decompose an objective, track intermediate progress, manage artifacts, and decide when available evidence is sufficient to stop [117, 142, 260, 317, 437]. The challenge here is to preserve the relationship between each local action and a global engineering objective.

知晓应采取哪些行动，并不能单独决定工作的执行顺序。功能开发、依赖迁移、实验复现及 Pull Request 准备过程均需由代理负责分解目标、跟踪中间进度、管理构建产物，并判断何时可用的证据已足以终止[117, 142, 260, 317, 437]。此处的挑战在于：如何保持每项局部操作与全局工程目标之间的关系。

Long-horizon systems increasingly externalize this relationship through explicit workflow state. One line of work structures execution around branches, plans, checkpoints, and approval gates, so that progress can be reviewed and resumed rather than remaining implicit in a conversation history.⁶ A second line factors recurring orchestration functions, such as planning, file-system state, tool invocation, and delegation, into reusable runtime components, as in OpenClaw and Deep Agents [293, 457]. A third line extends the workflow beyond a single coding loop by coordinating specialized agents across browsing, terminal use, file

³Aider: <https://github.com/Aider-AI/aider>.
³Aider: <https://github.com/Aider-AI/aider>.
⁴Cursor Rules: <https://cursor.com/docs/rules.md>.
⁴Cursor 规则: <https://cursor.com/docs/rules.md>.
⁵AGENTS.md: <https://agents.md/>.
⁵AGENTS.md: <https://agents.md/>.
⁶Codex, Agent Approvals & Security: <https://developers.openai.com/codex/agent-approvals-security>. Anthropic, Claude
⁶Codex、智能体审批与安全机制: <https://developers.openai.com/codex/agent-approvals-security>. Anthropic, Claude

manipulation, context management and code editing [157, 381]. At deployment scale, managed execution systems further separate session-level reasoning from sandboxed execution, allowing long-running work to be supervised, audited, and resumed [18, 561]. Across these designs, the common goal is to make the task trajectory itself a persistent object of control rather than an unstructured sequence of model actions.

长周期系统正日益通过显式 workflow 状态来实现这种关系的外部化。某类 workflow 架构围绕分支、计划、检查点和审批节点进行执行，从而能够对进度进行审核与恢复，而非让进度隐含于对话历史之中。⁶第二种方法将常见的编排功能（例如规划、文件系统状态、工具调用及委托机制）转化为可复用的运行时组件，如 OpenClaw 和 Deep Agents 中所示[293, 457]。第三条 workflow 通过协调浏览、终端使用、文件操作、上下文管理及代码编辑等环节中的专用代理，将 workflow 扩展至单个编码循环之外[157, 381]。在部署规模下，托管执行系统将会话级推理与沙箱化执行进一步分离，从而使得长周期运行的工作能够受到监控、审计和恢复[18, 561]。在这些设计中，共同目标在于将任务轨迹本身视为一个持续存在的控制对象，而非一系列无结构的模型动作序列。

6.1.3 Feedback-driven Repair

6.1.3 反馈驱动修复

Within each step of such a plan, the agent must turn execution feedback into revisions without mistaking local evidence for complete correctness. Tests and compiler errors make failures visible, but they do not fully specify the intended behavior, and an agent may repeatedly patch the most recent symptom, overfit to visible tests, or introduce a regression elsewhere in the repository.

在该计划的每个步骤中，智能体必须将执行反馈转化为修订方案，且不得将局部证据误判为完全正确。测试与编译器错误能够使故障显性化，但无法完全界定预期行为；此时，代理程序可能会反复修补最新的症状，从而过度拟合于可见的测试结果，甚至可能在代码库的其他部分引入回归问题。

Systems address this problem at both training and execution time. At training time, executable environments and repair trajectories teach agents to associate failures with plausible diagnosis-and-revision behavior, as in SWE-Gym and SWE-smith [467, 748]. At execution time, the central principle is to make repair *inspectable and reversible*: agents preserve diffs, logs, execution traces, and isolated workspaces so that failed attempts can be diagnosed rather than simply overwritten [657, 747]. Production-oriented systems extend this loop with branch isolation, hooks, approval boundaries, rollback, and final verification gates.⁷ The result is more than iterative debugging: the agent accumulates evidence across failed attempts while retaining enough control to revise or discard earlier decisions. Not all of this requires an open-ended agent loop, however: Agentless shows that a fixed localize-then-repair pipeline can rival more elaborate agentic systems [703].

系统在训练阶段和执行阶段均能解决这一问题。在训练阶段，可执行环境与修复轨迹可引导智能体将故障与合理的诊断-修正行为相关联，正如SWE-Gym和SWE-smith[467, 748]中所述。在执行时，核心原则是使修复操作 *可检查且可逆*：代理会保留差异文件、日志、执行轨迹及隔离工作空间，以便对失败的尝试进行诊断，而非直接将其覆盖[657, 747]。生产导向型系统通过分支隔离、钩子、审批边界、回滚及最终验证闸门等功能，扩展了这一流程循环。⁷其结果远不止于迭代调试：智能体能够在多次失败尝试中累积证据，同时仍保留足够的控制权，以修正或舍弃此前的决策。然而，并非所有情况都需采用开放式智能体循环：无智能体方案表明，一种固定的“定位-修复”流程亦可与更为复杂的智能体系统相媲美[703]。

6.2 Information Seeking

6.2 信息寻求

Information-seeking agents are a prominent class of long-horizon agents, because answering complex information needs often requires progressively accumulating evidence over extended retrieval trajectories, where relevant information is distributed across heterogeneous sources and only partially observable at any individual search step. We organize information-seeking agents around four capabilities: *deep search* sustains a single line of inquiry across many retrieval hops; *wide search* coordinates numerous parallel investigations under a shared objective; *multimodal grounding* extends retrieval beyond text to evidence in documents, figures, and images; and *research synthesis* organizes dispersed evidence into coherent, well-attributed research reports.

信息寻求智能体是长时程智能体中的一个重要类别，因为满足复杂的信息需求往往需要在漫长的检索过程中逐步积累证据。当相关信息分布于异构数据源且在任何单个搜索步骤中仅能部分观测时。我们根据四大能力对信息寻求代理进行分类：*深度搜索*可在多个检索跳转之间维持单一的探究路径；*广度搜索*则能在共同目标下协调多项并行的探究工作；*多模态锚定*可将检索范围从文本扩展至文档、图表及图像中的证据；而*研究综述*则能将分散的证据整合为结构连贯、来源可追溯的研究报告。

Together, these capabilities define the progression from retrieval-centric systems to long-horizon information-seeking agents. Rather than locating isolated facts, such agents must continuously acquire, compress, verify, and integrate evidence across extended trajectories [284]. They coordinate multiple investigations, maintain evolving research state, synthesize findings into structured outputs, and ground conclusions in heterogeneous sources. The resulting challenge is no longer search alone, but the end-to-end management of evidence throughout the entire research lifecycle [239].

这些能力共同界定了从检索中心型系统向长时程信息检索智能体的演进过程。这类智能体并非仅需定位孤立的事实，而必须持续地从漫长的轨迹中获取、压缩、验证并整合证据[284]。他们负责协调多项研究、保持不断演进的研究态势、将研究成果整合为结构化输出，并基于异构数据源支撑结论的形成。由此产生的挑战已不再局限于单纯的搜索，而是需要在整个研究生命周期内对证据进行端到端的管理[239]。

6.2.1 Deep Search

6.2.1 深度搜索

Deep search concerns sustained investigation along a single line of inquiry, where relevant evidence may only emerge after many rounds of retrieval, navigation, and reasoning [223, 883]. As search trajectories lengthen, maintaining useful state becomes increasingly important. Deep search agents can be broadly categorized into two paradigms according to how they optimize long-horizon search behaviors. The first paradigm consists of RL-based deep search agents, which explicitly optimize multi-step search policies through reinforcement learning [367]. Existing efforts mainly improve these systems from two complementary directions. One line focuses on constructing challenging long-horizon search tasks that encourage models to acquire complex search and reasoning behaviors [586, 689], while another develops more effective reinforcement learning algorithms to stabilize exploration, credit assignment, and long-term planning [120, 130, 682]. The second paradigm comprises workflow-based deep search agents, which rely on carefully designed reasoning and retrieval pipelines rather than policy optimization. A series of methods represented by ReSum therefore compress accumulated interaction histories into compact summaries or latent states that can be repeatedly reused across long trajectories [530, 693], while another class of approaches focus on condensing retrieved documents into task-relevant evidence representations for downstream reasoning [106, 336]. Long search chains also make retrieval failures harder to distinguish from reasoning failures [772] and amplify the risk of propagating incorrect assumptions or low-quality evidence [480]. Representative systems such as WebSeer continuously reassess intermediate conclusions, identify unsupported claims, and trigger additional retrieval when evidence

Code documentation: <https://code.claude.com/docs/en/overview>.

代码文档: <https://code.claude.com/docs/en/overview>。

⁷Anthropic, *Claude Code Hooks* reference: <https://code.claude.com/docs/en/hooks>.

⁷Anthropic, *Claude Code Hooks* 参考文档: <https://code.claude.com/docs/en/hooks>。

remains incomplete or inconsistent [207, 332, 746]. Through these mechanisms, retrieval becomes a persistent evidence-accumulation process that supports deep investigation over extended horizons.

深度搜索是指沿单一查询线索进行持续调查的过程，在此过程中，相关证据可能仅在经过多轮检索、导航与推理后才得以显现[223, 883]。随着搜索轨迹的延长，维持有效状态变得愈发重要。深度搜索智能体可根据其优化长时程搜索行为的方式，大致分为两种范式。第一种范式由基于强化学习的深度搜索智能体构成，这些智能体通过强化学习[367]显式地优化多步搜索策略。现有研究主要从两个互补的方向对这类系统进行改进。其中一条研究方向侧重于构建具有挑战性的长时程搜索任务，旨在促使模型习得复杂的搜索与推理行为[586, 689]。而另一些研究则致力于开发更有效的强化学习算法，以稳定探索、信用分配及长期规划[120, 130, 682]。第二种范式包含基于工作流的深度搜索代理，这类代理依赖于精心设计的推理与检索流水线，而非依赖策略优化。以ReSum为代表的系列方法可将累积的交互历史压缩为紧凑的摘要或潜在状态，这些摘要或潜在状态可在长轨迹中被反复复用[530, 693]。而另一类方法则侧重于将检索到的文档压缩为与任务相关的证据表示，用于下游推理[106, 336]。长搜索链还会使检索失败更难与推理失败相区分[772]，并会加剧传播错误假设或低质量证据的风险[480]。诸如WebSeer等代表性系统会持续重新评估中间结论，识别缺乏支持的断言，并在证据尚不完整或存在不一致时触发额外检索[207, 332, 746]。通过这些机制，检索过程转变为一个持续的证据积累过程，为开展跨越长周期的深度探究提供支持。

6.2.2 Wide Search

6.2.2 广域搜索

Wide search addresses research objectives whose scope exceeds a single search trajectory. Rather than pursuing one chain of evidence to completion, agents must coordinate many partially independent investigations while maintaining coverage of a large information space. A common strategy is task decomposition, where broad objectives are recursively broken into manageable research questions, evidence requirements, or execution plans. Explicit planning further enables agents to revise information needs and adapt research trajectories as new evidence emerges during execution [227, 292, 650, 746]. Increasingly, decomposition is coupled with parallel exploration. Different agents or search branches investigate complementary hypotheses, domains, or source collections before aggregating findings into a shared research state. Systems such as FlowSearch demonstrate that parallel search can substantially improve coverage and reduce the effective depth of individual trajectories [227, 241, 295, 556, 736]. Beyond parallel exploration, recent methods increasingly organize the search process as structured graphs, trees, or tables, which explicitly maintain search states and support dynamic expansion and revision as exploration proceeds [227, 289, 292]. The resulting challenge shifts from discovering isolated facts to coordinating large-scale evidence collection under a unified objective.

广域搜索可解决研究目标的范围超出单一搜索轨迹的问题。智能体无需追求单一证据链的完整闭环，而应协调多项部分独立的调查工作，在同时覆盖广阔信息空间的前提下进行协作。一种常用策略是任务分解，即通过递归方式将总体目标分解为可管理的研究问题、证据需求或执行计划。显式规划还可使智能体能够修订信息需求，并在执行过程中随着新证据的出现而调整研究路径[227, 292, 650, 746]；此外，分解过程正日益与并行探索相结合。不同的代理或搜索分支会先对互补的假设、领域或数据源集合进行探索，随后将所得结果汇总至共享的研究状态中。诸如FlowSearch之类的系统表明，并行搜索能够显著提升覆盖范围，并降低单个轨迹的有效深度[227, 241, 295, 556, 736]。除并行探索外，近年来的诸多方法日益倾向于将搜索过程组织为结构化图、树或表格形式——这些结构能够显式地维护搜索状态，并在探索过程中支持动态扩展与修订[227, 289, 292]。由此产生的挑战已从发现孤立事实，转变为在统一目标下协调大规模证据收集。

6.2.3 Multimodal Grounding

6.2.3 多模态接地

Multimodal grounding extends information seeking beyond textual retrieval. Many long-horizon research tasks require agents to gather evidence scattered across images, videos, and visually rich documents [114, 414, 714], where multilingual visual-text comprehension, document parsing, and OCR provide the perceptual access needed when text-based retrieval alone is insufficient [159, 290, 459, 584]. Recent work has therefore expanded information-seeking agents toward image search, enabling agents to retrieve, localize, and interpret visual evidence embedded in web-pages or documents, such as screenshots, diagrams, charts, and figures, while grounding subsequent reasoning in visual observations rather than textual descriptions alone [67, 138, 175, 346, 802, 825]. Another line of work focuses on video search, where agents actively explore long videos to identify answer-critical segments through iterative reasoning and interaction, replacing exhaustive frame-by-frame parsing with adaptive evidence seeking over temporal visual content [146, 361, 372]; the underlying active-perception and video-memory systems are treated in Section 6.4. These advances extend information seeking from text-centric retrieval toward unified search over heterogeneous sources.

多模态接地技术可将信息寻求范围扩展至超越文本检索的领域。许多长周期研究任务要求智能体从图像、视频及视觉信息丰富的文档中收集分散分布的证据[114, 414, 714]，其中涉及多语言视觉-文本理解与文档解析等任务。OCR能够提供文本检索单独使用时所欠缺的感知访问能力[159, 290, 459, 584]。因此，近期的研究将信息检索智能体的应用范围扩展至图像搜索领域，使这些智能体能够检索、定位并解析嵌入在网页或文档（如屏幕截图、图表、统计图及图形）中的视觉证据。将后续推理建立在视觉观察基础上，而非仅依赖文本描述[67, 138, 175, 346, 802, 825]。另一研究方向聚焦于视频搜索领域，其中智能体会主动探索长视频，通过迭代推理与交互来识别对答案至关重要的片段。将逐帧穷举解析替换为对时域视觉内容的自适应证据搜索[146, 361, 372]；其背后的主动感知与视频记忆系统将在第6.4节中予以阐述。这些进展将信息检索从以文本为中心的检索扩展至对异构来源的统一搜索。

6.2.4 Research Synthesis

6.2.4 研究综述

Research synthesis focuses on transforming large collections of retrieved evidence into coherent research artifacts. In long-horizon settings, information gathering alone is insufficient; agents must organize findings accumulated across many searches, identify relationships among sources, and construct narratives that remain faithful to the underlying evidence. Its central application is deep-research report generation, where agents synthesize retrieved evidence into comprehensive reports for domains such as open-domain research, finance, and business analysis [136]. Beyond research prototypes, widely deployed commercial systems such as OpenAI Deep Research, Gemini Deep Research, and Perplexity have brought this capability to broad audiences. Representative systems progressively externalize intermediate artifacts, including outlines, section plans, research notes, and persistent workspaces, before composing final reports [337, 356, 827, 891]. They further maintain explicit evidence attribution so that readers can trace individual claims back to their sources, improving both faithfulness and auditability [292, 347, 621]. The core challenge is therefore not merely to summarize retrieved documents, but to preserve coverage, narrative coherence, factual support [434, 677], and source-level traceability across a long search trajectory.

研究综述旨在将大量检索到的证据集转化为连贯的研究成果。在长周期情境下，仅靠信息收集是不够的；智能体必须整合来自多次搜索所积累的信息，识别不同来源之间的关联，并构建能够忠实反映底层证据的叙事。其核心应用是深度研究报告生成，其中智能体可将检索到的证据综合成全面的报告，适用于开放域研究、金融及商业分析等领域[136]。除研究原型外，诸如OpenAI Deep Research、Gemini Deep Research及Perplexity等已广泛部署的商业系统，也使这一能力得以惠及广大用户。代表性系统会逐步将中间生成的成果物（包括轮廓图、剖面图、研究笔记及持久工作空间）进行外部化处理，直至最终撰写报告[337, 356, 827, 891]。它们还保留了明确的证据归因机制，使读者能够将各项断言追溯至其来源，从而提升了文本的忠实度与可审计性[292, 347, 621]。因此，核心挑战不仅在于对检索到的文档进行摘要，更在于在漫长的搜索过程中，能够保持覆盖范围、叙事连贯性、事实支撑[434, 677]以及源级可追溯性。

6.3 Computer Use

6.3 计算机使用

Computer-use agents interact with stateful software systems in which actions have persistent consequences and task completion requires sustained execution across extended interaction trajectories. These agents autonomously manipulate desktops, browsers, and applications through heterogeneous interfaces including screenshots, DOMs, accessibility trees, shells, executable code, and file systems [126, 588]. Recent systems such as Anthropic’s Computer Use and OpenAI’s Operator support planning, multi-tool coordination, and persistent execution state across long-horizon tasks [12, 452, 745].

计算机使用代理与具有状态感知特性的软件系统进行交互；在这些系统中，用户操作会产生持久性影响，而任务的完成则需要在整个漫长的交互过程中持续执行。这些工具能够通过多种异构接口自主操控桌面、浏览器及应用程序，这些接口包括屏幕截图、DOM、可访问性树、外壳、可执行代码以及文件系统[126, 588]。近期的系统（如Anthropic公司的“计算机使用”模块和OpenAI的“操作员支持”模块）能够实现计算机使用与操作员支持规划、多工具协调，以及在长时程任务中保持执行状态的持久化[12, 452, 745]。

We organize long-horizon computer use according to the execution environment: *browser agents*, *desktop GUI agents*, and *mobile agents*. Browser agents focus on sustained task execution within web environments through navigation and information extraction. Desktop GUI agents emphasize reliable visual grounding and interaction with operating systems and desktop applications. Mobile agents further extend long-horizon execution to smartphone applications, where agents must coordinate touch-based interaction across diverse mobile interfaces. Together, these directions characterize the evolution from browser automation to general-purpose computer use. Beyond executing actions within a single application, modern computer-use agents must coordinate perception, reasoning, and execution across heterogeneous software environments while maintaining persistent state and coherent progress toward long-running objectives.

我们根据执行环境对长时程计算机资源使用进行分类：*浏览器代理*、*桌面GUI代理*以及*移动代理*。浏览器代理侧重于通过导航和信息提取在 Web 环境中实现持续任务执行；而桌面 GUI 代理则着重于提供可靠的视觉定位功能以及与操作系统及桌面应用程序进行交互。移动代理进一步将长时程执行机制扩展至智能手机应用领域，其中代理需协调跨多种移动界面的触控交互。这些方向共同体现了从浏览器自动化向通用计算机应用的演进过程。现代计算机智能代理不仅需在单一应用程序内执行操作，还必须能够在异构软件环境中协调感知、推理与执行过程，同时保持持久状态并确保向目标的连贯推进。长期目标。

6.3.1 Browser Agents

6.3.1 浏览器代理

Browser agents investigate long-horizon interaction within web environments through navigation, information extraction, and form manipulation. Benchmarks such as WebArena, Mind2Web, and ClawBench establish browser interaction as a representative long-horizon computer-use setting, requiring agents to interpret page content, choose actions, and maintain progress across evolving interface states [116, 847, 885]. To support sustained execution, many systems expose browsers through reusable action primitives such as clicking, typing, scrolling, navigation, and form completion, allowing agents to reason over browser interaction as a structured action space rather than raw interface manipulation [210, 226, 404, 443, 781]. At the same time, accumulated observations, retrieved information, and intermediate decisions are increasingly externalized through memories, notes, summaries, or reusable knowledge stores that persist across long interaction trajectories [637, 781]. These developments progressively transform browser interaction from isolated page-level decisions into persistent task execution over extended horizons.

浏览器代理通过导航、信息提取和表单操作，来探究网络环境中的长时程交互。诸如WebArena、Mind2Web和ClawBench等基准测试将浏览器交互设定为一种典型的长时程计算机使用场景，要求智能体能够解析页面内容并选择相应操作。并在不断演变的界面状态下保持进展[116, 847, 885]。为支持持续执行，许多系统会通过可复用的动作原语（如点击、输入、滚动、导航及表单填写）来暴露浏览器接口，从而使智能体能够将浏览器交互过程视为一个结构化的动作空间进行推理。优于直接的接口操作[210, 226, 404, 443, 781]。与此同时，累积的观察结果、检索到的信息及中间决策正日益通过记忆、笔记、摘要或可复用的知识库等方式进行外部化处理，这些知识库能够在漫长的交互过程中持续存在[637, 781]。这些发展使浏览器交互方式逐步从孤立的页面级决策转变为在较长的时间跨度内持续执行的任务。

6.3.2 Desktop GUI Agents

6.3.2 桌面GUI代理

Desktop GUI agents study how autonomous systems interact with operating systems and graphical desktop environments through multimodal perception and action. Benchmarks such as OSWorld and Workflow-GYM, together with representative systems including UI-TARS, have established desktop interaction as a realistic long-horizon setting, requiring agents to perceive interface state, execute multi-step workflows, and recover from execution failures across diverse applications [494, 526, 551, 716, 745]. Visual grounding plays a central role in desktop interaction by maintaining reliable correspondence between evolving interface observations and executable actions. Because graphical layouts can be both dense and dynamic across applications and execution steps, agents must identify relevant elements in high-resolution interfaces [324], interpret their functional roles, and update action targets throughout long interaction trajectories [181, 691]. Modern systems further complement multimodal perception with execution verification mechanisms based on screenshot comparison, visual change detection, semantic consistency checks, and artifact inspection [12, 259, 518], together with safety guardrails such as permission boundaries and confirmation policies [12, 455] to improve robustness under execution uncertainty.

桌面图形用户界面代理研究自主系统如何通过多模态感知与行动机制与操作系统及图形化桌面环境进行交互。诸如OSWorld和Workflow-GYM等基准测试，以及UI-TARS等代表性系统，已将桌面交互确立为一种现实的长周期应用场景——该场景要求智能体能够感知界面状态、执行多步骤 workflows。并能够在各类应用程序中从执行失败中恢复[494, 526, 551, 716, 745]。视觉接地在桌面交互中发挥着核心作用，其功能在于维持不断演变的界面观察结果与可执行操作之间的可靠对应关系。由于图形布局在不同应用程序及执行步骤中可能既密集又动态，智能体必须能够在高分辨率界面中识别出相关元素[324]。解释其功能作用，并在整个长交互轨迹中更新行动目标[181, 691]。现代系统还通过基于屏幕截图比对、视觉变化检测、语义一致性校验及人工痕迹检查的执行验证机制，进一步完善多模态感知功能[12, 259, 518]。此外，还可采用安全防护栏（如权限边界与确认策略[12, 455]）以增强系统在执行不确定性下的鲁棒性。

6.3.3 Mobile Agents

6.3.3 移动代理

Mobile agents extend long-horizon computer use to smartphones and tablets, where agents must coordinate interactions across multiple applications through touch-based interfaces [282, 373, 570]. Compared with desktop environments, mobile tasks typically involve long multi-app workflows, limited screen space, frequent context switching, and gesture-based operations, making persistent planning and state management particularly important [667, 766]. Recent systems increasingly adopt hierarchical agent architectures that separate high-level planning from low-level UI execution, allowing complex user requests to be decomposed into executable subtasks while maintaining progress across extended interaction trajectories [543, 667, 723, 766]. At the same time, long-term memories, reusable interaction skills, and execution verification mechanisms are incorporated to improve robustness and enable continual improvement from previous experiences [105, 571, 723, 766]. These developments transform mobile agents from reactive UI operators into persistent assistants capable of accomplishing complex real-world workflows across diverse mobile applications.

移动代理将长时程计算机服务扩展至智能手机和平板电脑，其中代理需通过触控界面协调跨多个应用程序的交互[282, 373, 570]。与桌面环境相比，移动任务通常涉及较长的多应用 workflows、屏幕空间有限、频繁的上下文切换以及基于手势的操作，这使得持续规划与状态管理尤为重要[667, 766]。近年来，各类系统日益采用分层智能体架构——该架构将高层规划与低层用户界面执行相分离，能够将复杂的用户请求分解为可执行的子任务，同时仍能维持贯穿整个交互过程的进度跟踪。离子轨迹[543, 667, 723, 766]。同时，系统整合了长期记忆、可复用的交互技能以及执行验证机制，以增强系统的鲁棒性，并支持基于过往经验进行持续优化[105, 571, 723, 766]。这些发展将移动代理从被动的用户界面操作者转变为持久型助手，使其能够在多种移动应用中完成复杂的现实工作流程。

6.4 Multimodal Agents

6.4 多模态智能体

Multimodal agents are long-horizon agents that must process information from different modalities. In long-horizon scenarios, multimodal evidence may be distributed across long visual contexts or multiple videos [55, 320, 413, 666], requiring agents to locate and acquire relevant information, store it, reason over it, and integrate it into an answer or multimodal output. Recent benchmarks such as AgentVista and OmniGAIA mainly evaluate agents’ ability to solve cross-modal tasks [339, 564, 589], rather than focusing only on isolated visual question answering or reasoning [45, 85, 190, 465].

多模态智能体是具有长时间跨度的智能体，必须处理来自不同模态的信息。在长时程场景下，多模态证据可能分布于较长的视觉上下文中或多个视频中[55, 320, 413, 666]，这要求智能体能够定位并获取相关信息、存储该信息，并对其进行推理。并将其整合至答案或多模态输出中。近期的基准测试（如AgentVista和OmniGAIA）主要评估智能体解决跨模态任务的能力[339, 564, 589]，而非仅关注孤立的视觉问答或推理任务[45, 85, 190, 465]。

We organize long-horizon multimodal agents into three categories according to their tasks and capabilities: *multimodal understanding*, *multimodal generation*, and *omnimodal agency*. Multimodal understanding focuses on how to locate useful information over extended timelines, including deciding which segments to inspect and which perception tools to invoke. Multimodal generation focuses on how to understand user intent and conduct multi-round planning, generation, and refinement, a process that relies on external evidence, verification mechanisms, and persistent memory. Omnimodal agency focuses on the coordination of different modalities so that they can jointly accomplish an end-to-end task. Overall, these three types of capabilities enable agents to handle richer multimodal information and interact more deeply with the real world.

我们将长时程多模态智能体根据其任务与能力划分为三类：*多模态理解*、*多模态生成*以及*全模态代理*。多模态理解旨在探讨如何在长时序范围内定位有用信息，具体包括决定需检查哪些时间片段以及调用哪些感知工具。多模态生成技术侧重于如何理解用户意图并执行多轮规划、生成与优化过程；该过程依赖于外部证据、验证机制以及持久化记忆。全模态代理侧重于不同模态之间的协调，以便它们能够共同完成端到端任务。总体而言，这三种能力使智能体能够处理更丰富的多模态信息，并与现实世界进行更深层次的交互。

6.4.1 Multimodal Understanding

6.4.1 多模态理解

For long-horizon multimodal understanding, agents must possess the ability of *active perception*. Under limited budgets, it is impossible to read all information streams at once; truly important information often appears only in a few frames of a long video, a small region of an image, or a single line of text in a document. Therefore, agents should be able to actively locate useful information within long information streams.

对于长时程多模态理解，智能体必须具备*主动感知*能力。在预算有限的情况下，无法一次性读取所有信息流；真正重要的信息往往仅出现在长视频的少数帧、图像的局部区域或文档中的单行文本中。因此，智能体应能够主动在长信息流中定位有用信息。

Current designs follow two main lines. The first line studies *tool-augmented active perception*. Works represented by VideoSeek treat long videos and images as explorable spaces rather than inputs that are fed into the model all at once [138, 365, 655, 825, 903]. The agent dynamically invokes browsing and retrieval tools according to the current question and existing clues, progressively locating key evidence in long videos. The second line studies *memory-augmented evidence organization*. Representative works such as DVD [510, 739, 829] construct structured video memory and then retrieve relevant evidence according to task demands. The former emphasizes active exploration strategies, while the latter emphasizes structured representations of long information streams and evidence preservation [796]. Their shared goal is to avoid blindly reading all multimodal inputs under limited budgets.

当前的设计主要遵循两条主线。第一条主线研究*基于工具增强的主动感知*。VideoSeek所采用的方法将长视频和图像视为可探索的空间，而非一次性输入至模型的输入数据[138, 365, 655, 825, 903]。该代理根据当前问题及现有线索，动态调用浏览与检索工具，逐步定位长视频中的关键证据；第二部分研究*记忆增强型证据组织*。代表性研究（如DVD[510, 739, 829]）会构建结构化视频记忆，随后根据任务需求检索相关证据。前者强调主动探索策略，而后者则强调对长信息流的结构化表征及证据保留[796]。他们的共同目标是避免在预算有限的情况下盲目处理所有多模态输入。

6.4.2 Multimodal Generation

6.4.2 多模态生成

In long-horizon scenarios, multimodal generation is not a one-step mapping from a prompt to an image or video clip. Agents usually need to understand user intent, plan the generation process, and conduct multiple rounds of generation and verification. Therefore, multimodal generation is a long-term task that must be completed through continuous iteration. Recent multimodal generation models, such as those in the Qwen-Image, Seedream, and SeedEdit series, have significantly improved intent understanding, multimodal creation, image editing, and video synthesis capabilities [73, 115, 172, 528, 600, 645, 855]. However, for long-horizon agents, beyond the generative capability of the model itself, the design of the surrounding harness is more important for ensuring that generated content remains consistent with user requirements.

在长时程生成场景中，多模态生成并非简单的从提示词到图像或视频片段的单步映射过程；智能体通常需要理解用户意图、规划生成流程，并执行多轮生成与验证操作。因此，多模态生成是一项长期任务，必须通过持续迭代来完成。近期的多模态生成模型（如Qwen-Image、Seedream和SeedEdit系列）显著提升了意图理解、多模态创作、图像编辑及视频合成能力[73, 115, 172, 528, 600, 645, 855]。然而，对于长周期规划型智能体而言，除模型自身的生成能力外，外围控制系统的设计在确保生成内容与用户需求保持一致方面更为关键。

Therefore, three mechanisms are crucial. *Evidence seeking* provides grounding for generation by retrieving images, documents, and videos. This is especially necessary in knowledge-intensive multimodal generation tasks, where sufficient task-relevant information must be collected. Recent generative agents autonomously invoke retrieval and visual analysis tools to gather task-relevant evidence before generation [68, 209, 255]. *Verification* checks generated results from multiple dimensions to ensure that they remain aligned with the original goal [574, 741, 831]. *Memory* stores user requirements and evidence at different granularities, enabling subsequent generation to remain consistent with earlier decisions [71, 733]. Recent multimodal generation systems increasingly organize planning, evidence collection, memory, verification, and rendering into unified agentic workflows. For example, PTAH coordinates specialized agents throughout the research

and writing lifecycle, maintains structured visual working memory, and enforces cross-modal consistency through verifier-guided report generation [803]. Similarly, long-form audio-video systems such as MUSE employ planning-execution-verification-revision loops, transforming multimodal generation into a long-horizon iterative process [574].

因此，三种机制至关重要。*证据检索*通过检索图像、文档及视频为生成过程提供依据。这在知识密集型多模态生成任务中尤为必要，此类任务需收集充足的与任务相关的信息。近期的生成代理会在生成前，自主调用检索与视觉分析工具以收集任务相关的证据[68, 209, 255]。*验证*通过从多个维度对结果进行检查，以确保其仍与原始目标保持一致[574, 741, 831]。*记忆库*以不同粒度存储用户需求与证据，从而使后续生成过程能够与此前做出的决策保持一致[71, 733]。近年来，多模态生成系统日益将规划、证据收集、记忆、验证及渲染等环节整合为统一的智能体工作流。例如，Ptah可在整个研究与写作生命周期中协调专门代理，维持结构化的视觉工作记忆，并通过验证器引导的报告生成机制来确保跨模态一致性[803]。类似地，诸如MUSE之类的长时音频-视频系统采用“规划-执行-验证-修订”循环，将多模态生成转化为一个长周期的迭代过程[574]。

6.4.3 Omnimodal Agency

6.4.3 全模态机构

Omnimodal tasks require agents to coordinate across different modalities, because a single modality or a single model is often insufficient to complete the task. This is closer to real-world scenarios, where inputs may include text, images, speech, audio, video, and other modalities. Therefore, agents need the ability of *cross-modal orchestration*. Specifically, an agent needs to decompose the task, route subtasks to appropriate specialized models, and then verify and integrate their outputs.

全模态任务要求智能体在不同模态之间进行协调，因为仅依赖单一模态或单一模型往往不足以完成该任务。这更贴近现实场景，其中输入可能包含文本、图像、语音、音频、视频及其他模态；因此，智能体需要具备*跨模态协调能力*。具体而言，智能体需对任务进行分解，将子任务分配给相应的专用模型，随后对这些模型的输出结果进行验证与整合。

Recent work has developed along two complementary directions. One direction focuses on native omnimodal models, such as those in the Qwen-Omni series, which provide unified interfaces for multimodal understanding or generation and usually support streaming interaction and speech-centered multimodal dialogue [498, 726, 727]. The other direction focuses on orchestration-based agents, such as OmniAgent, Orchestra-o1, and OmniGAIA, which explicitly assign subtasks to different modalities and coordinate their execution processes and outputs [137, 339, 364, 718, 728, 806]. In long-horizon omnimodal tasks, information from different modalities may conflict with each other, so explicit verification is required. Omnimodal agent capabilities enable agents to interact more deeply with real-world environments and broaden their range of applications. 近期的研究工作沿着两个互补的方向展开。其中一个研究方向聚焦于原生多模态模型，例如Qwen-Omni系列中的模型，这类模型为多模态理解或生成提供了统一的接口，通常支持流式交互及以语音为中心的多模态对话[498, 726, 727]。另一方向则聚焦于基于编排的智能体，例如OmniAgent、Orchestra-o1和OmniGAIA——这些智能体会明确地将子任务分配给不同的模态，并协调其执行过程与输出结果[137, 339, 364, 718, 728, 806]。在长时程多模态任务中，来自不同模态的信息可能存在冲突，因此需要进行显式验证。全模态智能体功能使智能体能够与现实世界环境进行更深层次的交互，并拓宽其应用范围。

6.5 General-purpose Agents

6.5 通用智能体

General-purpose agents represent one of the broadest forms of long-horizon agency, because they must pursue open-ended goals across heterogeneous interfaces rather than operate within a single fixed environment. A user request may require an agent to interact with multiple interfaces and invoke different types of tools. The central challenge for such agents is therefore *sustained coordination*: an agent must conduct overall planning for the task, maintain and transfer state across different environments, and recover from failures during execution. Benchmarks such as GAIA and τ -bench reflect a recent shift toward evaluating whether agents can consistently complete tasks in heterogeneous environments [335, 387, 431, 762, 840]; we do not restate the individual environments here, and collect them together with the domain-specific benchmarks reused throughout this section in Table 7.

通用智能体代表了长时程代理领域中最广泛的形式之一，因为它们必须在异构接口上追求开放性目标，而非局限于单一的固定环境内运行。用户请求可能要求代理与多个界面进行交互，并调用不同类型的工具。因此，此类智能体面临的核心挑战在于*持续协调*：智能体必须对任务进行整体规划，能够在不同环境中维持并传递状态，并在执行过程中从故障中恢复。诸如GAIA和 τ -bench等基准测试反映了近期研究趋势的转变——即评估智能体是否能够在异构环境中持续完成任务[335, 387, 431, 762, 840]；我们此处不重新陈述各个环境，而是将这些环境与本节中多次复用的领域特定基准测试一并汇总于表7中。

We organize general-purpose agents into three representative categories according to their deployment scenarios. *Personal assistants* deploy general-purpose agent capabilities in personal computing and office environments, where agents must operate across applications and interact continuously with users. *Embodied agents and world models* extend such capabilities to physical environments, where agents must perceive, reason, and predict the consequences of actions in the world. *Productive agents* apply general-purpose agent capabilities to professional workflows, where outputs must satisfy domain-specific standards and verification requirements. 我们根据通用智能体的部署场景，将其划分为三大代表性类别。*个人助理*在个人计算与办公环境中部署通用智能体功能，其中智能体需跨应用程序运行，并与用户进行持续交互。*具身智能体与世界模型*可将此类能力扩展至物理环境，使智能体能够在该环境中感知、推理并预测行动在世界中的后果。*生产力工具*可将通用工具功能应用于专业工作流程，其中输出结果必须符合特定领域的标准与验证要求。

6.5.1 Personal Assistants

6.5.1 个人助理

Personal assistants are an increasingly important class of agents designed to operate across everyday digital environments. Unlike traditional conversational assistants, these agents are expected not only to answer user questions, but also to complete continuous tasks across applications and local resources. Personal assistants therefore mark a shift from conversational assistance to persistent computer agents: the agent no longer merely generates text, but can continuously carry out tasks through multi-turn interaction with both the user and the environment.

个人助理是一类日益重要的智能代理，旨在在日常数字环境中运行。与传统的对话助手不同，这类智能代理不仅需回答用户问题，还需能够在不同应用程序及本地资源之间完成连续任务。因此，个人助理标志着从对话式辅助向持久性计算机智能体的转变：该智能体已不再仅仅是生成文本。但能够通过用户及环境的多轮交互，持续执行任务。

A capability underlying all personal assistants is *general tool use*: selecting and invoking the right tools to act beyond the model’s parametric knowledge. Early benchmarks assess this ability *without environment state*, testing whether an agent can choose and call the correct function from large, scalable tool sets, as in ToolBench [492] and the Berkeley Function-Calling Leaderboard [472]. τ -bench [762] then moves tool use into *stateful* environments, where each call mutates a persistent backend and success is judged against realistic multi-turn user interactions rather than isolated call correctness. More recent benchmarks push toward mixed, cross-application settings with increasingly long-tailed tool inventories: Toolathlon [322] spans

diverse realistic applications, while MCP-Atlas [31] and MCP-Mark [700] stress-test tool-use competency over large, heterogeneous MCP tool ecosystems. Finally, Claw-Eval [764] and ClawMark [426] evaluate tool use mediated by reusable *skills*, enabling more flexible invocation that better reflects modern agent deployments.

所有个人助手的底层能力均在于 *通用工具使用*：即选择并调用合适的工具，以实现超越模型参数化知识的行动。早期的基准测试在 *无需考虑环境状态* 的情况下评估这一能力，旨在检验智能体是否能够从大型、可扩展的工具集之中选择并调用正确的函数——例如在 ToolBench[492]中。以及伯克利函数调用排行榜[472]。-bench[762]将工具使用功能引入 *状态保持型* 环境，其中每次调用都会对持久化后端进行变异，且成功判定依据是真实的多轮用户交互，而非孤立的单次调用。正确性。近期的基准测试正向混合、跨应用场景发展，且工具库的长尾分布日益显著：Toolathlon[322]涵盖了多种真实的应用场景，而MCP-Atlas[31]与MCP-Mark[700]则.....针对大型且异质的MCP工具生态系统，压力测试工具使用能力评估工具。最后，Claw-Eval[764]和ClawMark[426]用于评估由可复用 *技能* 介导的工具使用行为，从而支持更灵活的调用方式，更好地契合现代智能体部署场景。

Personal assistants can be broadly divided into four categories. The first targets *office scenarios*, focusing on whether agents can complete tasks in realistic workflows involving documents, emails, calendars, and related office applications. Benchmarks such as OfficeQA, APEX-Agents, PresentBench, and SpreadsheetBench examine capabilities in information retrieval, file and presentation generation, spreadsheet manipulation, and multi-step task execution in such office environments [74, 415, 458, 623]. The second targets *personal computing scenarios*, emphasizing agents’ ability to operate real device interfaces, including browsers, desktop applications, mobile apps, and local file systems; this setting largely reuses the computer-use systems and benchmarks of Section 6.3 (e.g., UI-TARS and OSWorld [494, 716]) and is not detailed again here. The third targets *software development scenarios*, where agents assist developers with coding tasks that require code understanding, modification, and validation. Systems such as Claude Code and Codex are representative examples of this direction [10, 453]. The fourth targets *persistent personal assistant scenarios*, emphasizing agents that run continuously in user-controlled devices or environments and connect local resources, long-term memory, and external tools. Systems such as OpenClaw and Hermes illustrate this direction [420, 457, 513, 893].

个人助理可大致分为四类。首个测试目标 *针对办公场景*，重点评估智能体是否能够在涉及文档、电子邮件、日历及相关办公应用程序的现实工作流程中完成任务。诸如OfficeQA、APEX-Agents、PresentBench和SpreadsheetBench等基准测试工具，可评估此类办公环境中信息检索、文件与演示文稿生成、电子表格操作以及多步骤任务执行等方面的能力[74, 415, 458, 623]。。第二个目标针对 *个人计算场景*，侧重于智能体操作真实设备接口的能力——包括浏览器、桌面应用程序、移动应用及本地文件系统；该场景在很大程度上复用了计算机使用场景。第6.3节中的系统与基准测试（例如UI-TARS和OSWorld[494, 716]），此处不再赘述。第三个目标 *针对软件开发场景*，在此类场景下，智能体可协助开发者完成需要理解、修改及验证代码的编码任务。诸如Claude Code和Codex等系统便是这一方向的典型代表[10, 453]。第四类目标 *针对持续性个人助手场景*，重点在于那些在用户可控的设备或环境中持续运行，并能够连接本地资源、长期记忆及外部工具的智能体。诸如OpenClaw和Hermes等系统体现了这一方向[420, 457, 513, 893]。

6.5.2 Embodied Agents and World Models

6.5.2 具身智能体与世界模型

Embodied agents extend agent capabilities from digital environments to the physical world, where they must continuously perceive the environment, reason about goals and constraints, and execute actions that alter the world state. Long-horizon embodied tasks therefore require a sustained loop of perception, planning, and action, together with reliable state tracking, safety constraints, and recovery from execution errors. Their capabilities can be broadly divided into *navigation* and *manipulation*. Navigation focuses on understanding spatial layouts, planning paths, and adapting to environmental changes [135, 755, 771], whereas manipulation requires fine-grained reasoning about spatial references, object states, physical properties, and interactions such as grasping, moving, and assembling [93, 684, 872, 878]. Recent vision-language-action models, including Gemini Robotics and Qwen’s robotic VLA models, integrate language instructions, visual observations, and robotic control within a unified action interface, marking a shift from task-specific controllers toward more general-purpose embodied agents [35, 245, 594, 646, 784, 817].

具身智能体可将智能体的能力从数字环境扩展至物理世界，在此环境中，它们必须持续感知环境、对目标与约束条件进行推理，并执行能够改变世界状态的操作。因此，长时程具身任务需要一个持续的感知-规划-行动循环，并需具备可靠的状态跟踪、安全约束以及对执行错误的恢复能力。其功能可大致分为 *导航与操作* 两部分。导航侧重于理解空间布局、规划路径以及适应环境变化[135, 755, 771]，而操作则需要对空间参照、物体状态及物理属性进行精细推理。以及诸如抓取、移动和组装等交互操作[93, 684, 872, 878]。近期的视觉-语言-动作模型（包括Gemini Robotics和Qwen的机器人VLA模型）均能将语言指令、视觉观测结果与机器人控制整合至统一的动作接口中。这标志着从任务专用控制器向更通用的具身智能体的转变[35, 245, 594, 646, 784, 817]。

World models provide embodied agents with predictive representations of environment dynamics, allowing them to evaluate possible action outcomes before acting. This capability is particularly important in physical environments, where exploration is costly and execution failures may be difficult to reverse. Recent world-model research can be broadly divided into two main directions. The first is *predictive representation models*, exemplified by V-JEPA 2 and DINO-WM, which compress the environment into abstract latent representations and predict how these representations evolve over time and in response to actions, rather than directly generating complete future images [25, 879]. By omitting visual details such as texture and illumination, these models can support more efficient multi-step prediction and planning. The second direction is *generative world models*, including Cosmos, Cosmos-Predict2.5, and OSCAR, which directly generate predicted outcomes following an agent’s actions to support trajectory comparison, policy training, and risk assessment before real-world execution [2, 449, 699]. Building on these two directions, *world-action models*, such as DreamZero and τ_0 -WM, have recently emerged to unify world prediction and action generation by jointly modeling future states and executable actions [769, 884].

世界模型可为具身智能体提供对环境动态的预测性表征，使其能够在采取行动前评估可能的动作结果。这一能力在物理环境中尤为重要，因为在此类环境中，探索过程成本高昂，而执行失败则可能难以恢复。近期的世界模型研究可大致分为两大方向。第一种是 *预测性表征模型* 例如 V-JEPA 2 和 DINO-WM 等模型，它们将环境压缩为抽象的潜在表示，并预测这些表示随时间推移及对动作响应而如何演变，而非直接生成完整的未来图像[25, 879]通过省略纹理、光照等视觉细节，这些模型能够支持更高效的多步骤预测与规划。第二个研究方向是 *生成式世界模型*，例如 Cosmos 和 Cosmos-Predict2.5，以及OSCAR——该系统在真实世界执行前，会根据智能体的行动直接生成预测结果，以支持轨迹比较、策略训练及风险评估[2, 449, 699]。基于上述两个方向，*世界-动作模型*（如DreamZero和 τ_0 -WM）近期应运而生，通过联合建模未来状态与可执行动作，实现了对世界预测与动作生成的统一[769, 884]。。

6.5.3 Productive Agents

6.5.3 生产性因子

Productive agents apply general-purpose capabilities to workflows whose outputs must satisfy domain-specific standards. They retrieve authoritative sources, invoke specialized tools, and maintain traceable intermediate and final artifacts over extended trajectories. The central challenge is therefore not simply task automation, but the integration of planning, tool use, memory, and verification into established practice. We group these applications into two intuitive families: (1) *professional services*, where agents assist established work in finance, law, and medicine; and (2) *scientific discovery and reasoning*, where agents help generate and verify new knowledge.

生产力工具可将通用功能应用于其输出需符合特定领域标准的工作流。它们可检索权威来源、调用专业工具，并能在较长的时间跨度内持续维护可追溯的中间及最终成果。因此，核心挑战并非仅仅是任务自动化，而在于将规划、工具使用、记忆与验证整合至既有的实践之中。我们将这些应用归纳为两个直观类别：(1) *专业服务*，即智能体可协助处理金融、法律及医学领域的既定事务；以及 (2) *科学发现与推理*。其中，智能体可协助生成和验证新知识。

Professional Services. In finance, agents integrate market data, financial statements, and related evidence to produce traceable analyses, as illustrated by FinRobot and FinSight [34, 263, 886]. Legal agents retrieve authoritative sources, connect facts to rules, and produce outputs that can be reviewed against legal doctrine and procedure; the Legal Agent Benchmarks, together with agents such as Parthenon Law and LawThinker, reflect the shift from static question answering toward long-horizon legal work [174, 185, 315, 756]. Medical agents must further combine patient records, images, and examination reports while respecting safety, privacy, and auditability requirements [21, 904]. MedAgentBench, MedAgentBoard, and PhysicianBench accordingly evaluate sustained clinical reasoning and workflow execution rather than isolated diagnosis [23, 257, 380, 895]. Across these domains, correctness cannot usually be reduced to a single executable oracle, so provenance, procedural compliance, and expert review remain central [474, 752].

专业服务。在金融领域，代理机构通过整合市场数据、财务报表及相关证据，生成可追溯的分析结果，FinRobot 和 FinSight 即是对此的典型示例[34, 263, 886]。法律代理人员可检索权威资料，将事实与规则相匹配，并生成可供对照法律学说及程序进行审查的输出结果：“法律代理基准”以及Parthenon Law、LawThinker等代理机构，均体现了从静态问答向长周期法律工作的转变[174, 185, 315, 756]。医疗系统必须进一步整合患者病历、影像资料及检查报告，同时需满足安全性、隐私保护及可审计性要求[21, 904]。MedAgentBench、MedAgentBoard 和 PhysicianBench 分别用于评估持续的临床推理与 workflow 执行，而非孤立的诊断[23, 257, 380, 895]。在这些领域中，正确性通常无法简化为单一的可执行预言器，因此溯源、流程合规性及专家评审仍居核心地位[474, 752]。

Scientific Discovery and Reasoning. Scientific-discovery agents go beyond producing domain-compliant analyses: they generate new claims and test them against computational or empirical feedback. Their workflows connect literature review with expert-level and experimental scientific reasoning [642, 882], hypothesis generation, experimental design, execution, result analysis, and iterative refinement [505, 678, 869]. The AI Scientist and its successor organize machine-learning research into repeated cycles of ideation, experimentation, evaluation, and writing [397, 740]. AlphaEvolve and Kosmos extend this pattern to algorithmic and cross-domain discovery [438, 448], while NovelSeek closes the loop from hypothesis generation to verification across heterogeneous scientific tasks [800]. In empirical settings, Coscientist couples language-model planning with robotic chemical experimentation [37], and Robin and the Virtual Lab connect multi-agent reasoning to biomedical discovery and laboratory validation [177, 581]. Hypotheses, programs, experimental records, and manuscripts consequently become evolving research state that must be revised whenever new evidence arrives.

科学发现与推理。科学发现代理不仅限于生成符合领域规范的分析结果：它们还能生成新的论断，并将这些论断与计算或实证反馈进行检验。其工作流程将文献综述与专家级及实验性科学推理[642, 882]、假设生成、实验设计、实验执行、结果分析以及迭代优化[505, 678, 869]有机衔接。AI Scientist 及其后续版本将机器学习研究组织成由构思、实验、评估和撰写组成的重复循环[397, 740]。AlphaEvolve 和 Kosmos 将这一模式扩展至算法与跨领域发现[438, 448]，而 NovelSeek 则在异构科学任务中实现了从假设生成到验证的闭环[800]。在实证研究场景中，Coscientist 将语言模型规划与机器人化学实验相结合[37]；而 Robin 和 Virtual Lab 则将多智能体推理应用于生物医学发现及实验室验证领域[177, 581]。。假设、研究计划、实验记录及论文手稿均属于不断演进的研究状态，每当出现新证据时，这些内容均需进行修订。

Formal mathematics represents a complementary form of discovery in which proof assistants can mechanically check each completed proof [615, 866]. Seed-Prover 1.5 combines natural-language reasoning, lemma-level sketches, recursive decomposition, and an agentic Lean prover to construct long proofs under iterative verification [59]. Prover Agent similarly coordinates informal reasoning, auxiliary-lemma generation, formal proof construction, and Lean diagnostics in a feedback-driven refinement loop [26]. Automated Proof Engineering broadens the objective beyond isolated theorem proving to repository-scale work, including feature addition, refactoring, and bug repair across evolving Lean libraries, with evaluation based on both compilation and semantic requirement satisfaction [717]. Together, empirical science and formal mathematics show how productive agents can move beyond summarizing existing knowledge to generating claims that survive domain-specific verification [629].

形式化数学代表了一种互补性的探索方式，在这种方式中，证明助手可以对每个完成的证明进行机械性验证[615, 866]。Seed-Prover 1.5 结合了自然语言推理、引理级草图、递归分解以及一个具有“代理”特性的 Lean 证明器，可在迭代验证过程中构建长篇证明[59]。Prover Agent 同样在反馈驱动的精细化循环中，协调非形式推理、辅助引理生成、形式化证明构建以及 Lean 诊断过程[26]。自动化证明工程将目标范围从孤立的定理证明扩展至存储库规模的全流程工作，涵盖对不断演进的 Lean 库进行功能添加、重构及错误修复等任务。其评估基于编译与语义需求满足两方面[717]。实证科学与形式数学共同揭示：富有创造力的主体能够超越对现有知识的简单总结，进而生成为能够经受领域特定验证的命题[629]。

6.6 Benchmarks and Resources
6.6 基准测试与资源

Progress on long-horizon agents is measured and reproduced through a shared substrate of benchmarks and open resources. Because a reported score reflects the joint contribution of the model, the harness, and the training recipe, long-horizon evaluation is best read as *horizon-aware*, sensitive to how long a goal must be held, and *attribution-aware*, able to separate harness-induced from model-induced variance [123, 286, 468]. The same verifiable, long-horizon environments that serve as benchmarks are also frequently reused as training environments for the optimization methods of Section 5, so progress on evaluation and progress on internalization are tightly coupled. Beyond the domain benchmarks tabulated below, a growing family of suites deliberately stresses multi-hour autonomy. Examples range from machine-learning engineering in MLE-bench [51] to end-to-end replication of research papers in PaperBench [560], complementing the frontier time-horizon measurements of METR [286, 429]; these appear among the productive-agent entries of Table 7. 长时程智能体的研究进展通过共享的基准测试与开放资源平台进行衡量与复现。由于报告的评分反映了模型、约束条件及训练配方的共同贡献，长时程评估应被理解为*具有视野感知能力*——即对目标需维持多长时间具有敏感性，且*具备归因感知能力*。能够区分由导线系统引起的方差与由模型引起的方差[123, 286, 468]。作为基准测试的相同可验证、长周期环境，也常被用作第5节中优化方法的训练环境。因此，评估进展与内化进展紧密相关。除下表所示的领域基准测试外，还涌现出越来越多专门旨在考验数小时自主运行能力的测试套件。示例涵盖了从MLE-bench[51]中的机器学习工程，到PaperBench[560]中对研究论文的端到端复现，这些工作与METR[286, 429]的前沿时间跨度测量形成了互补；这些内容出现在表7的生产要素列中。

Table 7 consolidates representative resources for the application domains above, grouped by domain and, within each domain, ordered by the capability taxonomy of Section 6. Each row pairs an evaluation benchmark or a reusable system with the capability it targets and, where available, a link to its public repository or official release page (🔗), offering a concrete entry point into the works discussed throughout this section. The multimodal-understanding entries (e.g., video and streaming comprehension benchmarks) are not long-horizon. 表7汇总了上述应用领域中的代表性资源，这些资源按领域进行分组，并在每个领域内按照第6节所述的能力分类体系排序。每行均将一项评估基准或可复用系统与其目标功能进行配对，并在可用的情况下，提供其公共代码库或官方发布页面的链接(🔗)，为读者提供了深入了解本节所讨论作品的具体切入点。这些多模态理解相关条目（例如视频与流媒体理解基准测试）并不属于长周期性项目。

agent tasks in the strict sense; we include them because they measure the foundational perception capabilities on which long-horizon multimodal agents build.

严格意义上的代理任务；我们将其纳入考量，是因为这些任务能够衡量长时程多模态代理所依赖的基础感知能力。

Table 7 Representative systems and benchmarks relevant to long-horizon agents, grouped by domain. One work per row, annotated with its capability, type, a short description, an (approximate) release date, and a code repository. *Type* distinguishes evaluation *Benchmarks* from reusable *Systems*; within each domain, rows are grouped by capability (per Section 6) and ordered by release date. The list is representative rather than exhaustive.

表7：与长时程智能体相关的代表性系统及基准测试，按领域进行分类。每行对应一项研究工作，其中标注了其功能、类型、简要描述、（近似）发布日期以及代码仓库地址。“类型”用于区分评估*基准测试*与可复用*系统*；在每个领域内，行按功能特性（参见第6节）进行分组，并按发布日期排序。该列表仅为代表性列举，而非详尽清单。

Work 工作	Capability 能力	Type 类型	Description 描述	Date 日期	GitHub Repo GitHub仓库
Software Engineering 软件工程					
AutoCodeRover [843]	Repository Grounding 存储库接地	System 系统	Structure-aware fault localization and patching 结构感知型故障定位与修复	2024.04	 GitHub
SWE-agent [747] SWE-试剂[747]	Repository Grounding 存储库接地	System 系统	Agent–computer interface for repositories 存储库代理-计算机接口	2024.05	 GitHub
OpenHands [657]	Repository Grounding 存储库接地	System 系统	Persistent coding workspace 持久编码工作区	2024.07	 GitHub
NL2Repo-Bench [124]	Repository Grounding 存储库接地	Benchmark 基准	From-scratch repository generation 从零开始的代码库生成	2025.12	 GitHub
OctoBench [123]	Repository Grounding 存储库接地	Benchmark 基准	Scaffold-aware instruction following 基于脚手架的指令遵循	2026.01	 GitHub
SWE-bench [260]	Workflow-level Planning 工作流级规划	Benchmark 基准	Repository issue resolution 代码库问题解决	2023.10	 GitHub
Claude Code [10]	Workflow-level Planning 工作流级规划	System 系统	Autonomous code editing 自主代码编辑	2025.02	 GitHub
Trae Agent [169] 获取代理[169]	Workflow-level Planning 工作流级规划	System 系统	Ensemble agent for repository issue resolution 企业代码库问题解决	2025.07	 GitHub
SWE-bench Pro [117]	Workflow-level Planning 工作流级规划	Benchmark 基准	Enterprise repository issue resolution 企业代码库问题解决	2025.09	 GitHub
Terminal- 终端 Bench 2.0 [428]	Workflow-level Planning 工作流级规划	Benchmark 基准	Hard terminal-based tasks 硬终端型任务	2025.11	 GitHub
Aider ⁸ 帮助 ⁸	Feedback-driven Repair 反馈驱动式修复	System 系统	Iterative pair-programming loop 迭代式双人编程循环	2023.06	 GitHub
DebugBench [602]	Feedback-driven Repair 反馈驱动式修复	Benchmark 基准	Multi-language bug diagnosis 多语言错误诊断	2024.01	 GitHub
Agentless [703] 无代理[703]	Feedback-driven Repair 反馈驱动式修复	System 系统	Localize-then-repair pipeline “先本地化后修复”流水线	2024.07	 GitHub
SWE-Gym [467]	Feedback-driven Repair 反馈驱动式修复	System 系统	Executable training environments 可执行训练环境	2024.10	 GitHub
SWE-smith [748]	Feedback-driven Repair 反馈驱动式修复	System 系统	Repair-trajectory synthesis 修复轨迹合成	2025.04	 GitHub
Information Seeking 信息寻求					
Search-o1 [336]	Deep Search 深度搜索	System 系统	Retrieval and document reading 检索与文档阅读	2025.01	 GitHub
BrowseComp [676] 浏览组件[676]	Deep Search 深度搜索	Benchmark 基准	Persistent web browsing 持续网页浏览	2025.04	 GitHub
WebDancer [689]	Deep Search 深度搜索	System 系统	Information-seeking policy 信息寻求策略	2025.05	 GitHub
Tongyi DeepResearch [307] 通义深研[307]	Deep Search 深度搜索	System 系统	Agentic deep-research model 代理型深度研究模型	2025.10	 GitHub
MiroThinker [599]	Deep Search 深度搜索	System 系统	Interactive-scaling research agent 交互式扩展研究代理	2025.11	 GitHub
Open Deep Research [292] 开放深度研究[292]	Wide Search 广域搜索	System 系统	Evidence aggregation and synthesis 证据聚合与综合	2025.06	 GitHub
WideSearch [685]	Wide Search 广域搜索	Benchmark 基准	Structured information collection 结构化信息采集	2025.08	 GitHub
FlowSearch [227]	Wide Search 广域搜索	System 系统	Knowledge-flow orchestration 知识流编排	2025.10	 GitHub
GISA [896]	Wide Search 广域搜索	Benchmark 基准	Deep reasoning and broad information aggregation 深度推理与广域信息聚合	2026.02	 GitHub
MMInA [603]	Multimodal Grounding 多模态接地	Benchmark 基准	Multi-hop multimodal browsing 多跳多模态浏览	2024.04	 GitHub
MM-BrowseComp [331]	Multimodal Grounding 多模态接地	Benchmark 基准	Multimodal evidence seeking 多模态证据检索	2025.08	 GitHub
WebWatcher [175]	Multimodal Grounding 多模态接地	System 系统	Vision-language deep research 视觉-语言深度学习研究	2025.08	 GitHub
WebThinker [337]	Research Synthesis 研究综述	System 系统	Autonomous research workflow 自主研究 workflows	2025.04	 GitHub
DeepResearch Bench [136]	Research Synthesis 研究综述	Benchmark 基准	Research-report quality evaluation 研究报告质量评估	2025.06	 GitHub
WebWeaver [356]	Research Synthesis 研究综述	System 系统	Evidence organization and synthesis 证据组织与综合	2025.09	 GitHub
DeepConsult [592]	Research Synthesis 研究综述	Benchmark 基准	Consulting and business deep research 咨询与深度业务研究	~2025	 GitHub
Computer Use 计算机使用					
Mind2Web [116]	Browser Agents 浏览器代理	Benchmark 基准	Cross-domain web demonstrations 跨域网页演示	2023.06	 GitHub
WebArena [885]	Browser Agents 浏览器代理	Benchmark 基准	Realistic interactive websites 逼真的交互式网站	2023.07	 GitHub
WebVoyager [208]	Browser Agents 浏览器代理	Benchmark 基准	End-to-end live-web agent evaluation 端到端实时网络代理评估	2024.01	 GitHub
browser-use ⁹ 浏览器使用 ⁹	Browser Agents 浏览器代理	System 系统	Browser control through agent loop 通过代理循环实现浏览器控制	2024.10	 GitHub
BrowserAgent [781]	Browser Agents 浏览器代理	System 系统	Human-inspired native browsing 受人类启发的原生浏览体验	2025.10	 GitHub
OSWorld [716]	Desktop GUI Agents 桌面图形用户界面代理	Benchmark 基准	Desktop task environment 桌面任务环境	2024.04	 GitHub
UI-TARS [494]	Desktop GUI Agents 桌面图形用户界面代理	System 系统	Native visual GUI perception 原生视觉GUI感知	2025.01	 GitHub
ScreenSpot-Pro [324]	Desktop GUI Agents 桌面图形用户界面代理	Benchmark 基准	High-resolution professional GUI grounding 高分辨率专业GUI接地系统	2025.01	 GitHub
PointArena [83]	Desktop GUI Agents 桌面图形用户界面代理	Benchmark 基准	Language-guided visual pointing 语言引导的视觉指向	2025.05	 GitHub
Seed 2.0 [527]	Desktop GUI Agents 桌面图形用户界面代理	System 系统	Frontier multimodal foundation model 前沿多模态基础模型	2026.01	 GitHub
AndroidWorld [508]	Mobile Agents 移动代理	Benchmark 基准	Dynamic Android environment 动态Android环境	2024.05	 GitHub
AndroidLab [734]	Mobile Agents 移动代理	Benchmark 基准	Android agent training framework Android代理训练框架	2024.10	 GitHub
Mobile-Agent-v3 [766]	Mobile Agents 移动代理	System 系统	Fundamental GUI-automation agents 基础GUI自动化代理	2025.08	 GitHub

(continued on next page)
(下页继续)

Table 7 (continued)					
表7 (续)					
Work 工作	Capability 能力	Type 类型	Description 描述	Date 日期	GitHub Repo GitHub仓库
Mobile-Agent-v3.5 [723]	Mobile Agents 移动代理	System 系统	Multi-platform fundamental GUI agents 多平台基础GUI代理	2026.01	 GitHub
Multimodal Agents 多模态智能体					
EgoSchema [419] 自我图式[419]	Multimodal Understanding 多模态理解	Benchmark 基准	Egocentric video question answering 自我中心视频问答	2023.08	 GitHub
MVBench [328]	Multimodal Understanding 多模态理解	Benchmark 基准	Comprehensive video understanding 全面视频理解	2023.11	 GitHub
VideoAgent [655]	Multimodal Understanding 多模态理解	System 系统	Agentic evidence selection over long video 长视频中的代理式证据选择	2024.03	 GitHub
Video-MME [158] 视频-MME[158]	Multimodal Understanding 多模态理解	Benchmark 基准	Long-video understanding 长视频理解	2024.05	 GitHub
LongVideoBench [688]	Multimodal Understanding 多模态理解	Benchmark 基准	Long-video and text understanding 长视频与文本理解	2024.07	 GitHub
StreamingBench [366]	Multimodal Understanding 多模态理解	Benchmark 基准	Streaming-video understanding 流媒体视频理解	2024.11	 GitHub
DVD [829]	Multimodal Understanding 多模态理解	System 系统	Tool-using video deep research 工具使用视频深度研究	2025.05	 GitHub
VideoSeek [365]	Multimodal Understanding 多模态理解	System 系统	Tool-augmented active perception in long video 长视频中的工具增强型主动感知	2026.03	 GitHub
GenAI-Bench [306]	Multimodal Generation 多模态生成	Benchmark 基准	Text-to-visual generation evaluation 文本到视觉生成评估	2024.06	 GitHub
MM-StoryAgent [733]	Multimodal Generation 多模态生成	System 系统	Text-image-audio story generation 文本-图像-音频故事生成	2025.03	 GitHub
MUSE [574]	Multimodal Generation 多模态生成	System 系统	Plan-execute-verify story-video generation 计划-执行-验证：故事视频生成流程	2026.02	 GitHub
Ptah [803]	Multimodal Generation 多模态生成	System 系统	Verifier-guided multimodal report generation 验证器引导的多模态报告生成	2026.05	 GitHub
CrayOtter [741]	Multimodal Generation 多模态生成	System 系统	Verification-driven multimodal generation 验证驱动的多模态生成	2026.06	 GitHub
OmniVideoBench [308]	Omnimodal Agency 全模态机构	Benchmark 基准	Audio-visual evaluation 视听评估	2025.10	 GitHub
Agent-Omni [364]	Omnimodal Agency 全模态机构	System 系统	Omnimodal reasoning agent 全模态推理代理	2025.11	 GitHub
OmniGAIA [339]	Omnimodal Agency 全模态机构	Benchmark 基准	Cross-modal tool-use reasoning 跨模态工具使用推理	2026.02	 GitHub
AgentVista [564]	Omnimodal Agency 全模态机构	Benchmark 基准	Multimodal agent evaluation 多模态智能体评估	2026.02	 GitHub
Orchestra-o1 [806]	Omnimodal Agency 全模态机构	System 系统	Cross-modal orchestration agent 跨模态编排代理	2026.06	 GitHub
General-Purpose Agents 通用智能体					
ToolBench [492]	Personal Assistants 个人助理	Benchmark 基准	Large-scale tool use without env state 无需环境状态的大规模工具使用	2023.07	 GitHub
Open Interpreter ¹⁰ 打开解释器 ¹⁰	Personal Assistants 个人助理	System 系统	Local computer and code execution 本地计算机及代码执行	2023.07	 GitHub
AgentBench [387]	Personal Assistants 个人助理	Benchmark 基准	Multi-environment agent evaluation 多环境智能体评估	2023.08	 GitHub
GAIA [431]	Personal Assistants 个人助理	Benchmark 基准	General assistant tasks 通用辅助任务	2023.11	 GitHub
τ -bench [762]	Personal Assistants 个人助理	Benchmark 基准	Stateful tool-agent-user interaction 具有状态保持功能的工具-代理-用户交互模式	2024.06	 GitHub
AppWorld [614]	Personal Assistants 个人助理	Benchmark 基准	Stateful app-based personal tasks 基于应用程序的有状态个人任务	2024.07	 GitHub
BFCL [472]	Personal Assistants 个人助理	Benchmark 基准	Function-calling leaderboard 函数调用排行榜	2024.08	 GitHub
OpenManus ¹¹	Personal Assistants 个人助理	System 系统	General-purpose agent workflows 通用代理 workflows	2025.03	 GitHub
MCP-Mark [700]	Personal Assistants 个人助理	Benchmark 基准	Realistic MCP tool-use stress test 真实场景MCP工具使用压力测试	2025.09	 GitHub
Toolathlon [322]	Personal Assistants 个人助理	Benchmark 基准	Diverse realistic tool environments 多种逼真的工具环境	2025.10	 GitHub
工具马拉松[322]	Personal Assistants 个人助理	Benchmark 基准	Large-scale tool-use competency 大规模工具使用能力	2026.02	 GitHub
MCP-Atlas [31]	Personal Assistants 个人助理	Benchmark 基准	Trustworthy autonomous-agent evaluation 可信自主智能体评估	2026.04	 GitHub
Claw-Eval [764]	Personal Assistants 个人助理	Benchmark 基准	Multi-day multimodal living-world eval 多日多模态真实世界评估	2026.04	 GitHub
ClawMark [426]	Personal Assistants 个人助理	Benchmark 基准	Frontier professional-workflow evaluation 前沿专业工作流评估	2026.06	 GitHub
Agents’ Last Exam [575] 代理人的最后一场考试 [575]	Personal Assistants 个人助理	Benchmark 基准	Continuous vision-language navigation 连续视觉-语言导航	2020.04	 GitHub
VLN-CE [283]	Embodied & World Models 具身化与世界模型	Benchmark 基准	Open vision-language-action model 开放式视觉-语言-动作模型	2024.06	 GitHub
OpenVLA [277]	Embodied & World Models 具身化与世界模型	System 系统	Feature-space world model 特征空间世界模型	2024.11	 GitHub
DINO-WM [879]	Embodied & World Models 具身化与世界模型	System 系统	Embodied task evaluation suite 具身任务评估套件	2025.02	 GitHub
EmbodiedBench [753]	Embodied & World Models 具身化与世界模型	Benchmark 基准	Humanoid vision-language-action model 人形机器人视觉-语言-动作模型	2025.03	 GitHub
GR00T N1 [35]	Embodied & World Models 具身化与世界模型	System 系统	Open-world VLA model 开放世界VLA模型	2025.04	 GitHub
$\pi_{0.5}$ [245] Á.ó [245]	Embodied & World Models 具身化与世界模型	System 系统	Latent video-prediction world model 潜变量视频预测世界模型	2025.06	 GitHub
V-JEPA 2 [25]	Embodied & World Models 具身化与世界模型	System 系统	Video world simulation 视频世界模拟	2025.10	 GitHub
Cosmos-Predict2.5 [2]	Productive Agents 生产性因子	System 系统	Tool-augmented chemistry reasoning 工具增强型化学推理	2023.04	 GitHub
ChemCrow [39]	Productive Agents 生产性因子	System 系统	Financial research and analysis 财务研究与分析	2024.05	 GitHub
FinRobot [886]	Productive Agents 生产性因子	Benchmark 基准	Machine-learning engineering tasks 机器学习工程任务	2024.10	 GitHub
MLE-bench [51]	Productive Agents 生产性因子	Benchmark 基准	Multi-step legal tasks 多步骤法律任务	2024.12	 GitHub
LegalAgentBench [315]	Productive Agents 生产性因子	Benchmark 基准	Clinical EHR workflows 临床电子健康记录（EHR）工作流	2025.01	 GitHub
MedAgentBench [257]	Productive Agents 生产性因子	Benchmark 基准	Frontier time-horizon measurement 前沿时间跨度测量	2025.03	 GitHub
METR [286, 429]	Productive Agents 生产性因子	System 系统	Autonomous scientific discovery 自主科学探索	2025.04	 GitHub
AI Scientist-v2 [740] AI科学家-v2[740]	Productive Agents 生产性因子	Benchmark 基准	End-to-end research-paper replication 端到端研究论文复现	2025.04	 GitHub
PaperBench [560]	Productive Agents 生产性因子	System 系统	Algorithmic discovery agent 算法发现代理	2025.05	 GitHub
AlphaEvolve [448]	Productive Agents 生产性因子	Benchmark 基准	Real-world financial analysis tasks 现实世界的财务分析任务	2025.08	 GitHub
Finance Agent Bench- mark [34] 金融代理基准[34]	Productive Agents 生产性因子	Benchmark 基准	Real-world economic task value 现实经济任务价值	2025.10	 GitHub
GDPval [474]	Productive Agents 生产性因子	Benchmark 基准			

(continued on next page)
(下页继续)

Table 7 (continued)

表7（续）

Work 工作	Capability 能力	Type 类型	Description 描述	Date 日期	GitHub Repo GitHub仓库
Kosmos [438]	Productive Agents	System	Cross-domain discovery agent	2025.11	 GitHub
宇宙[438]	生产性因子	系统	跨域发现代理		
APEX-Agents [623]	Productive Agents	Benchmark	Office multi-step task execution	2026.01	 GitHub
	生产性因子	基准	办公多步骤任务执行		
OfficeQA Pro [458]	Productive Agents	Benchmark	Enterprise grounded reasoning	2026.03	 GitHub
	生产性因子	基准	企业接地推理		
PresentBench [74]	Productive Agents	Benchmark	Rubric-based slide generation	2026.03	 GitHub
	生产性因子	基准	基于评分标准的幻灯片生成		
OneMillion Bench [752]	Productive Agents	Benchmark	Human-level economic tasks	2026.03	 GitHub
	生产性因子	基准	人类水平的经济任务		
Workspace-Bench [588]	Productive Agents	Benchmark	Workspace file-dependency tasks	2026.05	 GitHub
	生产性因子	基准	工作区文件依赖任务		
YC-Bench [211]	Productive Agents	Benchmark	Long-term planning consistency	2026.06	 GitHub
	生产性因子	基准	长期规划的一致性		

Taken together, these resources show that long-horizon evaluation is consolidating around stateful, multi-hour, and economically grounded tasks, and that a reported score must be read as a joint property of the model, the harness, and the training recipe rather than of the model alone.

综上所述，这些资源表明，长周期评估正逐渐聚焦于具有状态信息、持续数小时且基于经济考量的任务；同时，所报告的分數应被理解为模型与硬件平台共同具有的属性。是训练配方而非模型本身。

7 Frontier: Open Challenges and Outlooks

7 前沿：开放性挑战与展望

The previous sections have organized long-horizon agency as it currently stands, across both pillars and the experience loop that links them. We now present a set of *positions*: claims about the most consequential paradigm-level shifts that we expect to define the next several years of research, together with the open problems each shift entails. In our view, the frontiers that matter most lie in this *in-between* space: capability migrates bidirectionally between the harness and the weights. The harness itself turns from fixed scaffolding into a learnable, portable object. And in noisy, costly, real environments, long horizons invalidate the idealized premises that both pillars rely on. Our central diagnosis is that practical long-horizon progress is increasingly a property of the runtime around the model, not of the model alone. To make these pressures easier to compare, we group the frontiers into four higher-level axes: *evolution*, *effectiveness*, *efficiency*, and *trustworthiness*. Table 8 previews the nine positions under this structure. For each concrete frontier we first state where the field stands and what blocks it (*status and challenges*), then our position on where it should go and the open problems it leaves behind (*outlook*).

前文各节已对当前的长周期代理机制进行了梳理，涵盖了其两大支柱以及连接这两者的经验循环。我们现提出一组 *观点*：这些观点阐述了我们预期将定义未来数年研究方向的、最具深远影响的范式层面转变，以及每种转变所涉及的开放性问题。我们认为，最关键的研究边界恰恰存在于这个 “*中间地带*”：能力可在安全带与配重之间双向迁移；而安全带本身也从固定的支架转变为可学习、可便携的装置。在噪声干扰严重、成本高昂的实际环境中，长时间跨度会使这两项支柱所依赖的理想化前提失效。我们的核心结论是：实际的长周期进展日益成为模型所处运行环境的固有特性，而非仅由模型本身决定。为便于比较这些压力因素，我们将相关边界划分为四个更高层次的维度：*演进性*、*有效性*、*效率性* 以及 *可信度*。表8预览了该结构下的九个位置。针对每一个具体领域，我们首先阐述该领域目前的现状及其面临的障碍（*状态与挑战*），随后阐明我们对这一领域未来发展方向的看法，以及其留下的未解问题（*展望*）。

7.1 Evolution: Generalization and Learning Over Time

7.1 进化：泛化与随时间推移的学习

The first frontier axis is evolution: whether an agent’s long-horizon competence can improve over time rather than remain locked to a hand-written scaffold and a frozen policy. This axis raises three linked questions. Can the harness itself self-evolve? Can the resulting competence transfer across harnesses? And can experience compound across episodes so that the agent keeps learning, and even learns for a lifetime, without destroying previously acquired capability?

第一个前沿维度是演化：智能体的长时程能力能否随时间推移而不断提升，而非局限于某个手写架构和固定策略；这一维度引出了三个相互关联的问题：驾驭机制本身能否实现自我演化？由此获得的能力能否跨不同任务集迁移？此外，经验是否能够跨越不同任务场景进行整合，从而使智能体持续学习——甚至终身学习——而不会破坏其先前习得的能力？

7.1.1 Self-evolving Harness and Agents

7.1.1 自进化约束与代理

Status and Challenges. The dominant paradigm of *hand-maintained* agents is a transitional artifact, and self-improvement offers a direct route to extending the effective horizon. We distinguish two levels as follow:

现状与挑战。 *人工维护型* 智能体的主流范式是一种过渡性产物，而自我改进则为扩展有效时间范围提供了直接途径。我们可将这一领域划分为以下两个层面：

The first is the *self-evolving harness*: with little or no model training, the agent keeps rewriting the runtime structure around a fixed policy. This includes skill-driven fast adaptation, where new skills are distilled directly from failure and success trajectories [206, 705, 706]; recursive procedural-memory evolution, where memory and skills co-evolve from execution traces [813, 853]; and topology evolution, where sub-agents are spawned on demand and the multi-agent graph itself becomes a learnable object [101, 652, 670, 791]. Increasingly the harness is treated as a first-class artifact that can be automatically generated and optimized, as in Meta-Harness and AutoHarness [297, 395], and even hosted by agent operating systems such as AIOS that provide a unified, resource-managed runtime across agents and tasks [424]; industrial trace-based iteration treats run traces as the primary evolution signal [293, 765].

第一种是 *自进化约束机制*：在几乎无需或完全无需模型训练的情况下，智能体可围绕固定策略持续重写运行时结构。这包括基于技能的快速适应，即新技能可直接从失败与成功轨迹中提炼得出[206, 705, 706]；递归程序性记忆演化，即记忆与技能从执行轨迹中共同演化[813, 853]；以及拓扑结构演化，即子智能体按需生成，且多智能体图本身则成为可学习的对象[101, 652, 670, 791]。。如今，约束机制正日益被视为一类一流的技术产物——可被自动生成与优化（如Meta-Harness和AutoHarness[297, 395]），甚至可由诸如AIOS等提供统一接口的代理操作系统来托管。跨代理与任务的资源管理运行时[424]；基于工业追踪的迭代方法将运行轨迹视为主要的演化信号[293, 765]。

⁸Aider: <https://github.com/Aider-AI/aider>.

⁸Aider: <https://github.com/Aider-AI/aider>。

⁹browser-use: <https://github.com/browser-use/browser-use>.

⁹browser-use: <https://github.com/browser-use/browser-use>。

¹⁰Open Interpreter: <https://github.com/OpenInterpreter/open-interpreter>.

¹⁰Open Interpreter: <https://github.com/OpenInterpreter/open-interpreter>。

¹¹OpenManus: <https://github.com/FoundationAgents/OpenManus>.

¹¹OpenManus: <https://github.com/foundationAgents/OpenManus>。

Table 8 Four frontier axes and nine frontiers for long-horizon agents. For each frontier we summarize its *open challenges* and our *outlook*. A recurring thread: the harness is where much of the next advance must happen.

表8：长周期智能体的四个前沿维度及九项前沿领域。 针对每个前沿领域，我们均总结了其 *开放挑战* 以及我们的 *展望*。一个反复出现的议题：安全带领域正是未来诸多突破的关键所在。

Frontier 边境	Open challenges 开放挑战	Outlook
I. Evolution — generalize across runtimes and keep learning over time		
I. Evolution——支持跨运行时的泛化，并能随时间持续学习		
Self-evolving Harness & Agents 自进化安全带与代理程序	Hand-set optimization metric - 手持设备优化指标 - Limited gains on out-of-distribution tasks - Overfitting and drift over long runs - 在非分布任务中获得的收益有限；在长时间运行过程中会出现过拟合与漂移现象。	- Adaptively choose optimization objective and scope - 自适应选择优化目标与范围 - Autonomous evolution guided by feedback - 基于反馈的自主演化 - Lifelong, non-overfitting evolution - 终身、非过拟合的演化过程
Harness 安全带	Models tied to specific harnesses 与特定安全带系统配套的模型	Multi-harness (hybrid-harness) training 多安全带（混合式安全带）培训
Generalization and 泛化与	Performance varies across model–harness pairings 不同模型与硬件组合下的性能表现存在差异	Portable skills / experience at inference time 推理时的可移植技能/经验
Transferability 可迁移性	Portable standards still ad hoc 便携式标准仍属临时性安排	A standard harness protocol 标准吊带协议
Continual & Lifelong Learning 持续终身学习	External experience is noisy and unverified - Catastrophic forgetting during weight updates - 外部经验具有噪声性且未经验证；在权重更新过程中存在灾难性遗忘现象。	Learning only from validated experience - Safe online learning from billion-user logs - 仅基于经过验证的经验进行学习：基于数十亿用户日志的安全在线学习
II. Effectiveness — act reliably in realistic, verifiable environments		
II. 效能——在真实且可验证的环境中可靠地执行操作		
Real-world Environment Interaction 真实环境交互	- Slow, costly, and irreversible interactions with real-world systems - 缓慢、成本高昂且不可逆的相互作用 - 真实世界系统 - Synthesized Envs & world-models may be unfaithful - 合成的环境与世界模型可能为不忠	- Measure, audit, and optimize sim-to-real - 仿真到真实环境的测量、审计与优化 faithfulness - 忠实性 - Fusion of symbolic and neural simulation - 符号化与神经模拟的融合 environments - 环境
From Digital to Embodied Agents 从数字智能体到具身智能体	- Deep planning vs. real-time control - 深度规划与实时控制 - Limited modeling of physical dynamics & laws - 对物理动力学与定律的建模较为有限 - Coarse vs. fine-grained feedback - 粗粒度反馈与细粒度反馈	- Async, hierarchical harness (deliberation vs. reflex) - 异步、分层调控机制（deliberation 与 reflex） - Predict physical outcomes with world models - 利用世界模型预测生理结局 - Integrate feedback across timescales - 整合跨时间尺度的反馈
III. Efficiency — spend compute, context, and modality budgets		
III. 效率——合理分配计算、上下文及模态预算		
Cost- & Budget-aware Agency 预算敏感型代理机构	- Model selection is poorly matched to task difficulty - 模型选择与任务难度不匹配。 - No calibrated cost / infeasibility sense - 无校准成本/不可行性含义 - No runtime enforcement of ceilings - 未对上限进行运行时强制执行 - No law linking budget to success - 尚无将预算与成效挂钩的法律	Perceive cost, difficulty, infeasibility - 感知成本、难度及不可行性 Act mid-run: shorten, re-route, or stop - 中段行动：缩短行程、改道或停车 Harness-enforced ceilings & cost-aware routing - 网状约束式天花板与成本感知路由 A budget-vs-success scaling law - 预算与成功之间的标度律
Multimodal & Omni Harness 多模态及全功能安全带系统	- Multimodality bolted on; no vision-native harness - 多模态系统采用外接方式集成；未配备视觉专用总线。 - Visual-token budgeting is heuristic - 视觉令牌预算法属于启发式方法。 - Cross-modal verification is unreliable - 跨模态验证不可靠	A vision-native, unified omni harness substrate - 面向视觉的统一全功能总线基板 Per-modality granularity control - 多模态粒度控制 Cross-modal verification & omni-modal routing - 跨模态验证与全模态路由
IV. Trustworthiness — stay robust and governable as the horizon grows		
IV. 可信性——在发展视野不断拓展的过程中，始终保持稳健与可治理性		
Reflection & Error Robustness 反射与误差鲁棒性	- Failures detected late under noisy, delayed feedback - 在噪声干扰及延迟反馈条件下，故障检测较晚发生 - Intrinsic self-correction is unreliable - 内在自校正机制不可靠 - Errors compound into goal drift - 误差会累积导致目标偏差	Early failure detection & path-switching - 早期故障检测与路径切换 External-anchored, calibrated self-knowledge - 外部锚定、校准的自我认知 Align to human course-correction - 与人类修正轨迹对齐
Safety & Governance 安全与治理	- Injected error & hazardous experience reuse - 注射相关错误与危险操作经历再利用率 - No unified safety-verification standard - 尚无统一的安全验证标准 - Self-evolution erodes invariants; biased self-checks - 自演化会侵蚀不变量；具有偏倚性。自检	Untrusted external input; gate experience reuse - 不可信的外部输入；门电路经验复用 Score action safety beside effectiveness - 在效能之外，还需重视操作安全性 Independent, invariant-preserving verification - 独立且保不变量的验证

The second, more ambitious level is the *self-evolving agent*, whose goal is to fold harness structure and skills back into the weights: the Darwin Gödel Machine rewrites its own code to improve a coding agent [814], and a small but growing line couples skill discovery with reinforcement learning or routes rich runtime trajectories into training, as in SkillRL and OpenClaw-style RL such as Claw-R1 [457, 627, 705].

第二层，也是更为雄心勃勃的层级，便是 *自进化智能体*——其目标是将控制结构与技能重新整合至权重之中：达尔文-哥德尔机器会重写自身的代码，以优化编码智能体[814]。此外，还存在一些规模虽小但日益增多的方案，它们将技能发现与强化学习相结合，或将丰富的运行时轨迹引入训练过程，例如SkillRL以及OpenClaw风格的强化学习方法（如Claw-R1[457, 627, 705]）。

The shared limitations are threefold, mirroring the outlook below: **(1)** the optimization objective is still a hand-set benchmark metric rather than something the agent chooses for itself; **(2)** gains remain largely within-distribution while the evolution history stays opaque about *what* changed and *why*; and **(3)** over long runs such systems risk overfitting a single benchmark or drifting.

共同的局限性共有三点，与下文所述观点一致：**(1)**优化目标仍是一个人工设定的基准指标，而非智能体自主选择的目标；**(2)**收益在很大程度上仍局限于内部分布范围内，而其演化历史对于 *发生了什么* 以及 *原因为何* 仍不透明；且 **(3)** 在长周期内，此类系统存在过度拟合单一基准或发生漂移的风险。

Outlook. We expect self-evolution to mature into a complex *automated-optimization* problem pursued with minimal human supervision, and we highlight three requirements. **(1)** The agent should *autonomously discover its own optimization objective*. Today this is typically a single task’s benchmark metric or a training-loss level; in the future it will more likely be a joint objective over many tasks. The agent should locate where its own competence is weak on each task and decide accordingly where to optimize. Sometimes it is enough to adjust one part of the harness (orchestration logic, a memory module, the tool set), while in other cases the harness and the model weights should be optimized together. **(2)** Self-evolution should proceed *autonomously*: the agent should acquire external information, distill reusable skills (for instance, high-value strategies for deep web search), and continuously update its optimization strategy in response to environmental feedback.

展望。我们预计，自我进化将逐步成熟为一个复杂的*自动化优化*问题——该问题可在极少的人工监督下进行求解，并重点阐述了以下三项要求。**(1)**代理应*自主发现其自身的优化目标*。目前，这通常是指单个任务的基准指标或训练损失值；而在未来，这更有可能成为涵盖多个任务的联合目标。智能体应定位其在各项任务中能力较弱的环节，并据此决定应在何处进行优化。有时，仅需调整控制套件的某一部分（如编排逻辑、内存模块或工具集）即可；而在其他情况下，则需将控制套件与模型权重一同进行优化。**(2)**自我进化应*自主进行*该智能体应获取外部信息，提炼可复用的技能（例如适用于深度Web搜索的高价值策略），并根据环境反馈持续更新其优化策略。

(3) This process should be able to run *continually, even for a lifetime*: driven by a stream of sparse external signals, it should neither overfit to a single benchmark nor drift into random change, which makes stable long-run self-evolution a genuinely long-term goal. Crucially, if self-evolution is to be more than per-benchmark overfitting, the structure it produces must survive being moved to a different runtime. This is the transferability problem we turn to next.

(3) 该过程应能够*持续运行，甚至可运行至终身*。在稀疏外部信号流的驱动下，该系统既不应过度拟合于单一基准模型，也不应陷入随机波动；因此，实现稳定的长期自我演化便成为一项真正的长期目标。至关重要是的，若自进化机制要避免仅在各基准测试上出现过拟合，其所生成的结构必须能够在迁移到不同的运行时环境下依然保持有效。这正是我们接下来将探讨的“可迁移性问题”。

7.1.2 Harness Generalization and Transferability

Status and Challenges. Harness paradigms keep multiplying, spanning early patterns such as ReAct and CodeAct, the Model Context Protocol, and today’s workstation-style harnesses like OpenClaw, Hermes, and Claude Code. As they do, the field is drifting toward a regime in which each strong model is implicitly co-trained with, and bound to, a single harness. Yet what users expect from a capable model is the opposite: that they can drive it from their own agent, and even extend it to a harness they design themselves. We regard this binding as the central scalability risk of the field: a model competitive only inside the harness it was tuned for must be retrained for every new one. It cannot keep pace with a runtime ecosystem that turns over every few months.

现状与挑战。软件开发框架正不断涌现，涵盖了从ReAct、CodeAct等早期模式，到Model Context Protocol，乃至如今的工作站式框架（如OpenClaw、Hermes和Claude Code）等多种类型。随着这一趋势的发展，该领域正逐渐向一种新模式转变：在这种模式下，每个强大的模型都会被隐式地与某个单一框架共同训练，并与该框架紧密绑定。然而，用户对高性能模型的期望恰恰相反：即能够通过自身的代理来驱动该模型，甚至可将其扩展至由用户自行设计的控制装置。我们将这一绑定问题视为该领域最核心的可扩展性风险：一个仅在其原定应用场景内具有竞争力的模型，对于每一个新的应用场景都必须重新训练；它无法跟上每数月就会更新一次的运行时生态系统的发展步伐。

Three concrete pressures make the risk tangible. **(1)** Harness-specific tuning effects are large: identical harnesses change rank when ported across providers, sometimes by tens of points, so a model’s measured “ability” is partly an artifact of how well its idioms match the harness [468]. **(2)** Multi-model orchestration is already standard: large models plan while small models execute. Learned preference-based routing [451] and layered model composition [638] serve as building blocks, with systems such as EvoRoute, Chimera, and ORCH offering early systematic studies [447, 810, 880]; any single-harness assumption thus breaks immediately.

三重具体压力使得风险变得切实可感。**(1)**各测试套件的调优效应十分显著：即使使用相同的测试套件，当在不同供应商之间移植时，其排名也可能发生变化——有时变化幅度可达数十个点；因此，模型测得的“能力”在一定程度上是其惯用写法与该测试套件匹配程度的产物[468]。**(2)**多模型编排已成为标准做法：大型模型负责规划，而小型模型负责执行。基于学习的偏好路由[451]与分层模型组合[638]可作为构建模块，诸如EvoRoute、Chimera和ORCH等系统已开展了早期的系统性研究[447, 810, 880]；因此，任何单吊索假设均立即失效。

(3) Portable cross-provider standards such as AGENTS.md and Agent Skills are emerging organically rather than from any single vendor [16, 98]. This development elevates *transferability*, the extent to which a harness configuration remains effective when the base model is swapped, to a first-class concern.

(3) 诸如AGENTS.md和Agent Skills等便携式跨供应商标准正通过自然演进方式出现，而非源自任何单一供应商[16, 98]。这一进展将*可移植性*——即当基础车型更换时，传动系统配置仍能保持有效性的程度——提升至首要关注事项。

Outlook. We see three complementary routes toward harness-robust competence. **(1)** *External transfer*: package experience, and ideally skills, into portable artifacts so that a model can read externally mounted skills at inference time and gradually raise its competence without retraining. **(2)** *Internal generalization*, in our view the more fundamental route: it exposes the model to many harnesses during training by mixing multi-harness trajectories and even running hybrid-harness RL, so that harness-robust behavior is built into the weights rather than bolted on. **(3)** *Standardizing a harness protocol*: just as the Model Context Protocol connected the world’s tools behind one discoverable, authorizable interface and thereby made tool use broadly transferable [100], a standard harness protocol that supports flexible design and registration could become the starting point for world-wide harness generalization.

Outlook. 我们看到了通往“驾驭稳健能力”的三条互补路径。**(1)***外部迁移*：将封装好的经验（理想情况下还包括技能）迁移到可移植的实体中，使得模型在推理时能够读取外部安装的技能，并在无需重新训练的情况下逐步提升其能力。**(2)***内部泛化*我们认为，这一方法更为根本：它通过混合多种训练轨迹甚至运行混合策略的强化学习，使模型在训练过程中接触多种训练环境，从而使鲁棒于不同训练环境的行为能够直接内化于权重之中，而非事后附加。**(3)***标准化套件协议*：正如“模型上下文协议”通过将全球工具连接至一个可发现且可授权的接口，从而实现了工具使用的广泛可移植性[100]一种支持灵活设计与注册的标准化互操作协议，或可成为全球互操作性的通用起点。

The concrete open problems are: the minimal portable contract that preserves performance across providers; training recipes that explicitly optimize for cross-harness transfer rather than single-harness fit; robustness under harness drift, when a new server or tool-calling convention silently changes behavior; and portable observability, so that traces from different runtimes can be analyzed in one stack. Transferability concerns whether competence transfers across runtimes at a single point in time; the complementary question is whether competence *accumulates* across time, which is the problem of continual learning.

当前亟待解决的开放性问题包括：能够跨供应商保持性能一致的最小可移植合约；能够明确优化跨框架迁移而非单框架适配的训练方案；在系统配置变更（例如新服务器或工具调用约定悄然改变行为时）下的鲁棒性；以及可移植可观测性，使得来自不同运行时的跟踪信息均可在单一堆栈中进行分析。可移植性问题的关键在于能力是否能在单一时点跨不同运行时进行转移；其互补问题则是能力是否*随时间累积*——这正是持续学习问题。

7.1.3 Continual and Lifelong Learning

7.1.3 持续终身学习

Status and Challenges. Continual and lifelong learning are central to long-horizon agents because such agents operate across many tasks, sessions, environments, and users rather than within a single isolated episode. A capable agent should not only solve the current task, but also accumulate experience that improves later behavior while preserving previously acquired abilities. Existing work pursues this along two complementary routes. **(1)** *External* lifelong learning stores experience outside the parameters: Voyager maintains an expanding library of executable skills for open-ended exploration [630], ExpeL extracts natural-language lessons from past trajectories [853], and Reflexion keeps verbal feedback in episodic memory to improve later trials without weight updates [544]; such methods are cheap, reversible, and quick to adapt. **(2)** *Internal* continual learning folds new knowledge or behavior into the model itself; this supports broader generalization, but frequent updates remain costly and risk catastrophic forgetting, especially under noisy, shifting deployment data [58, 641].

现状与挑战。持续终身学习是长时程智能体的核心所在，因为这类智能体需在多种任务、会话、环境及用户之间进行运作，而非仅在单一孤立的场景中运行。一个高效的智能体不仅应能够完成当前任务，还应能够积

累经验——这些经验可优化其后续行为，同时又能保留先前已习得的能力。现有研究正沿着两条互补的路径来实现这一目标。**(1) 外部**终身学习平台能够获取超出常规范围的经验：Voyager 维持着一个不断扩展的可执行技能库，用于进行开放式探索[630]ExpeL可从过往轨迹中提取自然语言课程[853]，而Reflexion则将口头反馈保留在情景记忆中，以在后续试验中无需权重更新的情况下提升表现[544]；这些方法成本低廉、可逆性强且易于快速适应。**(2) 内部**持续学习将新知识或新行为直接融入模型本身；这有助于实现更广泛的泛化能力，但频繁的模型更新仍成本高昂，并存在发生灾难性遗忘的风险——尤其是在部署数据存在噪声且动态变化的情况下[58, 641]。

Outlook. The key challenge is to connect these two forms of learning into a stable lifelong-learning loop: most new experience should first be kept externally as memory, trajectories, or skills, where it can be inspected, revised, and tested across future tasks, and only experience that proves reliable and broadly useful should later be consolidated into weights. This staged process makes lifelong learning less about fine-tuning after every interaction and more about deciding what to remember, reuse, generalize, or forget.

Outlook. 关键挑战在于将这两种学习形式连接成一个稳定的终身学习循环：大多数新经验应首先以记忆、轨迹或技能的形式存储于外部，以便在未来的任务中对其进行审视、修订与检验。唯有经验证实可靠且具有广泛适用性的经验，方可后续整合为权重值。这种分阶段的学习过程使得终身学习不再侧重于每次交互后的微调，而更侧重于决定哪些内容应记住、复用、推广或遗忘。

A distinctive new opportunity is *online learning from deployment at scale*: as billion-user agent products such as Cursor¹², Claude Code¹³, and Doubao¹⁴ mature, they accumulate vast streams of user logs and feedback. Turning these traces into safe improvements is a difficult but consequential long-term research challenge. Doing so requires metrics that identify high-value episodes and online-learning algorithms that incorporate them into the agent with minimal human intervention and without regressions. The open problems include how to prevent forgetting during periodic updates, how to extract trustworthy learning signals from noisy trajectories and user feedback, how to transfer lessons across tasks and environments, and how to evaluate improvement over many sessions rather than within a single episode. In this sense, lifelong learning is a necessary condition for long-horizon agency: without it, an agent may act for a long time, yet never truly become better over time.

一个独特的新机遇是*大规模部署下的在线学习*：例如Cursor¹²、Claude Code¹³以及Doubao¹⁴等拥有数十亿用户量的智能代理产品。随着系统日趋成熟，它们会积累海量的用户日志与反馈数据。将这些追踪数据转化为安全的改进方案，是一项难度较高但意义深远的长期研究挑战。实现这一目标需要具备能够识别高价值学习片段的度量指标，以及能够将这些片段融入智能体、且在极少人工干预及无需进行回归分析的情况下完成此融合的在线学习算法。尚未解决的问题包括：如何在周期性更新过程中防止知识遗忘；如何从含噪轨迹及用户反馈中提取可信的学习信号；以及如何实现跨任务、跨环境的知识迁移。以及如何评估跨多次会话的改善情况，而非仅评估单次治疗期间的改善情况。从这个意义上说，终身学习是实现长周期主体性发展的必要条件：若缺乏这一条件，主体或许会持续行动很长时间，但并未真正随时间推移而有所进步。

7.2 Effectiveness: Acting in Realistic Environments

7.2 有效性：在真实环境中进行操作

Evolution matters only if the resulting agent remains effective when the environment becomes slow, noisy, partially irreversible, and hard to verify. The second axis concerns whether long-horizon agents can operate effectively in realistic environments. We first consider the digital environments used for training and evaluation, and then turn to the physical world, where challenges arise from real-time perception and control.

进化机制仅在以下情况下才具有意义：当环境变得缓慢、存在噪声、部分不可逆且难以验证时，由此产生的智能体仍能保持其有效性；第二个维度则涉及长视距智能体是否能在现实环境中有效运行。我们首先探讨用于训练与评估的数字环境，随后转向物理世界——在该领域，实时感知与控制带来了诸多挑战。

7.2.1 Real-world Environment Interaction

7.2.1 真实环境交互

Status and Challenges. A great many agent applications are defined by sustained interaction with real-world environments: ordering food or booking a flight, drafting a Feishu or Notion document, running real-time financial analysis and forecasting, or communicating on platforms such as WeChat¹⁵ and X¹⁶. For training long-horizon agents, however, we usually cannot act directly in these live environments because interaction is slow, costly, and often irreversible. The binding constraint becomes the *environment* rather than the model: where an agent can act, whether it can fail safely, and whether it can receive a checkable reward.

现状与挑战。大量智能代理应用均基于与现实环境的持续交互来定义：例如点餐、预订航班、起草飞书或Notion文档，以及执行实时财务分析与预测。或在微信¹⁵和X¹⁶等平台上进行通信。然而，对于训练长时域智能体而言，我们通常无法直接在这些实时环境中采取行动，因为交互过程缓慢、成本高昂且往往不可逆转。绑定约束转变为环境而非模型：即定义了智能体可执行操作的范围、其能否安全失败，以及其是否能够获得可验证奖励。

Two construction routes are emerging. **(1) Verifiable, executable synthesis** builds environments that behave like the real systems they stand in for and mechanically decide success and failure: realistic testbeds such as WebArena, OSWorld, and TheAgentCompany ground agents in real browsers, operating systems, and multi-day company workflows [716, 722, 885], and automated environment synthesis pushes this toward scale [131, 238, 555]. **(2) Model-simulated environments** treat a large model itself as an implicit world model, as in GenEnv, Qwen-AgentWorld, and reasoning-model simulators such as Simia-SFT [195, 905], trading mechanical guarantees for flexibility and coverage.

两条建设路线正在形成。**(1) 可验证、可执行的综合**构建环境能够模拟其所代表的真实系统的行为，并能机械地判定成功与否：例如 WebArena、OSWorld 等真实测试平台。TheAgentCompany 的地面代理可在真实浏览器、操作系统及多天的公司 workflows 中运行[716, 722, 885]，而自动化环境合成则推动了这一过程向规模化发展[131, 238, 555]。**(2) 模型仿真环境**将大型模型本身视为一种隐式世界模型，例如GenEnv、Qwen-AgentWorld以及推理模型模拟器（如Simia-SFT）[195, 905]。通过交易机械担保来实现灵活性与保障。

Both routes face the same test of *faithfulness*: an environment that is inexpensive to evaluate but unfaithful may train agents that perform well in controlled settings yet fail in deployment. Faithfulness is thus best treated not as one property but as several separately auditable axes: whether interfaces and available actions match the real system; whether identical actions produce the same state change; whether automatic validators agree with real acceptance criteria; whether simulated users behave like real ones; and whether a reset returns the system to a functionally equivalent state rather than one that merely looks similar.

两种路径均需面对*忠实性*这一相同的考验：即在评估成本较低但不具忠实性的环境中训练智能体，可能会使该智能体在受控环境下表现良好，但在实际部署时却可能失效。因此，对“忠实性”的处理不应将其视为单一属性，而应将其视为若干可分别审计的维度：即接口与可用操作是否与实际系统一致；以及相同的操作是否能产生相同的状态变化；自动验证器是否与实际验收准则一致；模拟用户的行为是否与实际用户一致；以及重置操作是否能使系统恢复至功能等效状态，而非仅恢复至外观相似的状态。

¹²<https://cursor.com>
¹³<https://www.anthropic.com/claude-code>
¹⁴<https://www.doubao.com>
¹⁵<https://www.wechat.com>
¹⁶<https://x.com>

Outlook. We hold that environment construction, not model scale, is the rate-limiting step for the next phase of agent capability, and that “faithfulness” deserves to be measured as explicitly as task success is today. The most promising direction is to *fuse* the two routes above by combining the verifiability of synthesized environments with the flexibility of model-simulated ones. This fusion would allow agents to train broadly while still receiving trustworthy rewards. On the learning side the binding constraint is sample efficiency: real-world rollouts are slow, costly, and sometimes irreversible, so the algorithms that matter extract the most signal per interaction through off-policy reuse, experience replay over returned traces, and curricula in which environment difficulty co-evolves with the policy.

Outlook. 我们认为，环境构建——而非模型规模——是下一阶段智能体能力发展的限速步骤；因此，“忠实性”理应像如今衡量任务成功率那样得到明确量化。最具前景的方向是融合上述两种方法，即结合合成环境的可验证性与模型仿真环境的灵活性。这种融合机制可使智能体在广泛训练的同时，仍能获得可信的奖励。在学习方面，绑定约束体现为样本效率问题：现实世界的环境模拟过程速度缓慢、成本高昂，且有时不可逆转；因此，关键算法通过离策略复用方式，在每次交互中能提取出最多的信号。基于返回轨迹的经验回放，以及环境难度与策略共同进化的课程设计。

The open problems are: automated construction of environments that are provably checkable and measurably faithful; metrics and audits for the sim-to-real faithfulness of synthetic and model-simulated environments; sample-efficient RL for slow, costly, partially irreversible interaction; and curricula that co-evolve environment difficulty with agent competence. The most demanding realistic environment of all, however, is the physical world rather than the digital one. In this setting, perception and control must run in real time, and many actions cannot be undone. These constraints define the frontier of embodiment.

待解决的开放问题包括：能够证明可验证且可衡量保真度的环境的自动化构建；针对合成环境及模型仿真环境的“仿真实现到现实”的保真度评估指标与审计机制；适用于缓慢、成本高昂且部分不可逆交互场景的样本高效强化学习方法；以及使环境难度与智能体能力协同演进的学习课程设计。然而，所有现实环境之中最具挑战性的当属物理世界，而非数字世界。在这一环境中，感知与控制必须实时运行，且许多操作不可逆。这些约束条件界定了具身化的边界。

7.2.2 From Digital to Embodied Agents

7.2.2 从数字智能体到具身智能体

Status and Challenges. A natural extension of long-horizon digital agency is embodiment: equipping an agent with real-world perception and control so that it can provide high-level deliberation for an embodied system. The externalization logic of this paper extends directly to embodiment. The digital harness becomes a high-level controller that orchestrates low-level physical skills, recapitulating the harness pattern as a division of labor between the cerebrum and cerebellum. In this arrangement, a high-level LLM/VLM agent interprets goals, decomposes tasks, and replans based on feedback [243, 357], while Vision-Language-Action (VLA) models serve as callable real-time skill modules for atomic manipulation [35, 36, 134, 277, 899]. But embodiment sharpens three challenges specific to the physical world. **(1)** A *timescale conflict*: long-horizon planning is deep but slow, and its deliberation clashes with the millisecond reactive control an embodied system demands. **(2)** A *dimensionality and physics gap*: today’s long-horizon agents reason mostly over language and, increasingly, other digital modalities, whereas the physical world has far higher effective dimensionality, so better modeling of physical laws and dynamics is a key hurdle to migrating a digital long-horizon agent into a body. **(3)** A *feedback-granularity gap*: real environments provide feedback at multiple spatial and temporal scales, from sparse task-level signals to dense control-level signals (pose deviations, contact forces, or tactile mismatches), and an embodied agent must integrate feedback across these scales rather than rely only on the coarse plan-level signals that text harnesses handle well.

现状与挑战。长时程数字智能体的自然延伸便是“具身化”：即为智能体配备现实世界的感知与控制能力，使其能够为具身系统提供高层级的推理与决策支持。本文的外部化逻辑可直接延伸至具身化。数字控制装置演变为一个高层控制器，负责协调低层级的物理技能，通过大脑与小脑之间的分工来再现控制模式。在此架构下，高级LLM/VLM智能体可对目标进行解析、分解任务，并根据反馈[243, 357]进行重新规划。而视觉-语言-动作（VLA）模型可作为用于原子级操作的可调用实时技能模块[35, 36, 134, 277, 899]；然而，具身化技术则能有效解决物理世界中面临的三大特定挑战。**(1)**时间尺度冲突：长时程规划过程虽深度丰富但执行缓慢，其深思熟虑的过程与具身系统所要求的毫秒级即时响应控制存在冲突。**(2)**一个维度与物理能隙当前的长时程智能体主要基于语言进行推理，且正日益扩展至其他数字模态；而物理世界的有效维度则要高得多，因此，更精准地建模物理定律与动力学过程，便成为迁移至该领域的关键障碍。将数字长时程智能体集成至实体系统中。**(3)**反馈粒度差距：真实环境会在多种空间与时间尺度上提供反馈，涵盖从稀疏的任务级信号到密集的控制级信号（姿态偏差、接触力等）。或触觉不匹配），具身智能体必须整合跨这些不同尺度的反馈，而非仅依赖文本处理能力良好的粗粒度计划级信号。

Outlook. We expect embodied and digital agent architectures to converge on a shared engineering stack with the long-horizon harness as the deliberative core, and we read the three challenges above as its research agenda.

展望。我们预计，实体智能体与数字智能体架构将趋向于采用一个共同的工程栈——其中“长周期调控层”作为其审慎决策核心；而上述三个挑战则可视作该领域的研究议程。

(1) For the timescale conflict, the promising route is an asynchronous, hierarchical harness that decouples slow deliberation from a fast reactive control loop, letting the deliberative core replan out-of-band while cached skills and reflexes keep the body responsive. **(2)** For the physics gap, embodied harnesses will likely lean on action-conditioned world models and simulation, so that the agent can predict physical consequences and close part of the sim-to-real gap before acting. **(3)** For the feedback-granularity gap, memory, skill, and protocol designs must carry both coarse subgoals and fine-grained continuous corrections through one interface, with verifiers that operate at both scales.

(1) 针对时间尺度冲突问题，一个颇具前景的解决方案是采用异步分层控制架构，将缓慢的决策过程与快速的响应控制环解耦。让决策核心负责带外重新规划，而缓存的技能与反射机制则确保身体保持响应状态。**(2)** 针对物理间隙问题，具身控制系统很可能依赖于基于动作的环境模型及仿真技术，从而使智能体能够预测物理后果，并在采取行动前弥合部分“仿真-现实”间隙。**(3)** 针对反馈粒度差距问题，记忆、技能及协议设计必须通过单一接口同时承载粗粒度子目标与细粒度连续修正，并配备能在两种尺度上运作的验证器。

The open problems are: real-time, safety-critical action verification under latency budgets text harnesses can ignore; memory, skill, and protocol designs for irreversible physical action; closing the sim-to-real gap between simulated and physical deployment; and a unified deliberative harness that drives both digital tools and physical effectors. Whether an agent acts through digital tools or physical effectors, each step also draws down a finite budget of computation, context, and time. Therefore, how these resources should be allocated becomes a first-class frontier.

尚未解决的开放性问题包括：在延迟预算范围内实现实时、安全关键型动作验证；针对不可逆物理动作，需进行内存、技能及协议设计；弥合仿真与物理部署之间的“仿真-真实”鸿沟；以及一个能够驱动数字工具与物理执行器的统一决策枢纽。无论智能体是通过数字工具还是物理执行器进行操作，每一步操作都会消耗有限的计算资源、上下文信息及时间资源；因此，如何分配这些资源便成为了一个首要的前沿课题。

7.3 Efficiency: Spending Compute, Context, and Modality Budgets

7.3 效率：计算、上下文与模态预算分配

Long horizons turn resources into first-class design variables. An agent must decide not only what to do, but how much computation, context, modality bandwidth, and verification effort each part of the run deserves. The efficiency frontier spans both classical compute budgeting and the newer problem of scheduling high-cost multimodal evidence.

广阔的视野可将资源转化为一流的设计变量。智能体不仅需决定采取何种行动，还需决定运行过程中各环节应获得多少计算资源、上下文信息、模态带宽及验证投入。效率前沿既涵盖了经典的计算预算规划，也涉及了更新的高成本多模态证据调度问题。

7.3.1 Cost- and Budget-aware Agency

7.3.1 成本与预算导向型机构

Status and Challenges. As long-horizon agents move from research demos into ordinary users’ everyday workflows, such as hours-long coding sessions and personal-assistance workflows spanning an entire day, cost and budget cease to be implementation details and become first-class concerns for both usability and economics. Yet today’s agents are largely *budget-blind*: they carry almost no working sense of what a token costs, no notion that context and compute are a finite budget to be spent deliberately, and no felt pressure to finish within a price or time ceiling. In practice they default to the most capable model at every step, re-read context they could have summarized, and keep spending on runs that are already doomed. This is merely wasteful at demo scale, but once such agents run continuously for millions of users it translates into real cost and a degraded day-to-day experience.

现状与挑战。随着长周期型智能体从研究演示场景逐步过渡至普通用户的日常工作流程——例如持续数小时的编码过程或覆盖全天的个人辅助工作流程——成本与预算便不再仅仅是实施过程中的细节，而是转变为高度重视可用性与经济性。然而，当今的代理系统在很大程度上缺乏预算意识：它们几乎无法准确把握代币的成本，也无从意识到上下文与计算资源构成一个需审慎分配的有限预算，更不存在必须在特定价格范围内完成任务的紧迫压力。时间上限。在实际应用中，它们会在每一步默认选择能力最强的模型，重新读取本可进行摘要的上下文，并继续为那些已注定失败的运行分配资源。在演示规模下，这种情况仅属浪费；但当此类代理程序为数百万用户持续运行时，便会转化为实际成本并导致日常使用体验下降。

The mechanisms to spend more wisely already exist, including compute-optimal test-time allocation [550], adaptive reasoning budgets [442], tunable best-of- N and multi-agent aggregation [323], and cost-aware model routing and cascades [62, 237, 447, 451, 810, 880]. However, agents do not yet use these mechanisms with any awareness of their own spending. Budget-awareness is best seen not as a single capability but as a chain: an agent must perceive what a task will cost, act on that estimate, run inside a harness that can enforce the limits it would otherwise ignore, and ideally be guided by a law that predicts the trade-off.

实现更明智的资源分配机制已有所存在，包括计算最优的测试时分配[550]、自适应推理预算[442]、可调节的“最佳- N ”方案 N 以及多智能体聚合[323]。以及成本感知的模型路由与级联机制[62, 237, 447, 451, 810, 880]。然而，智能体尚未利用这些机制，且对其自身支出缺乏任何认知。预算意识不应被视作单一能力，而应理解为一个链条：智能体必须感知任务的成本、基于该估算采取行动，并在能够强制执行相应限制的框架内运行——这些限制若无此框架则可能被忽略。理想情况下，应由能够预测这种权衡关系的法律来引导。

Outlook. We frame the way forward as four open problems along this chain. **(1) Perceiving cost, difficulty, and infeasibility.** An agent should forecast, and keep re-forecasting as a run unfolds, how hard a task is, how much budget remains, and whether the task has already become infeasible; even a coarse predicted-infeasibility signal can save a large fraction of the tokens otherwise burned on doomed runs [370]. The quantity is multi-dimensional and coupled (tokens, money, wall-clock time, sometimes real resources), and the estimate must be *calibrated* rather than optimistic, since only a calibrated forecast tells the agent when and by how much to change course. **(2) Acting on the estimate.** A cost estimate is worthless unless it changes behavior: the agent should shorten reasoning on easy subtasks, avoid redundant re-reading, reserve depth for where it changes the outcome, and escalate, stop, or re-route when the forecast turns bad, while avoiding a measurable loss in quality. The difficulty is that these choices are made mid-trajectory under uncertainty, where a false economy early can cost far more later. **(3) Enforcing budgets from the harness.** Not all budget discipline should live in the model’s head; the harness should enforce frugality through budget-aware routing and cascades, context compression and eviction, caching and reuse of intermediate results, and scheduling primitives that hold the agent to a hard price, token, or time ceiling even when the model would overspend. Because the harness sees the whole run, it is also the natural place to account for cost across a long, multi-tool trajectory.

Outlook. 我们将后续研究方向归纳为沿该链条分布的四个开放性问题：**(1)感知成本、难度与不可行性。**代理应能够预测任务的难度、剩余预算金额，以及该任务是否已变得不可行；并在流程执行过程中持续进行重新预测。即使是一个粗略的预测不可行信号，也能节省在注定失败的运行中会烧毁的大量令牌[370]。数量具有多维性和耦合性（如令牌、货币、实际时间，有时还包括真实资源），其估算值必须经过校准。并非乐观，因为只有经过校准的预测才能告知智能体何时以及应改变多少航向。**(2)基于该估计值采取行动。**成本估算若不能改变行为，则毫无价值：智能体应缩短对简单子任务的推理过程，避免重复阅读，将深度资源保留于能够改变结果的环节，并在预测结果恶化时采取升级、停止或重新路由等措施。同时可避免质量的显著下降。难点在于，这些选择是在轨迹中途且处于不确定性条件下做出的；若在早期采取错误的经济性策略，可能会在后期付出远更高的代价。**(3)强制执行来自约束系统的预算。**并非所有的预算纪律都应局限于模型的内部；该框架应通过以下机制来实现节俭原则：基于预算的路由与级联机制、上下文压缩与驱逐机制、中间结果的缓存与复用，以及将智能体约束于特定成本或令牌限制的调度原语。即使模型可能出现超支情况，该机制也能设定时间上限。由于约束系统能够观测整个运行过程，因此也是核算长距离、多工具轨迹中各项成本的自然选择。

(4) A scaling law for budget and success. Further out, is there a predictable relationship linking compute budget, tool calls, and run length to success rate, analogous to a Chinchilla-style law for agency? Without one, budget allocation stays a craft; the first credible candidates should emerge from the long-running-task regime, where budgets are large enough that depth-versus-breadth trade-offs dominate.

(4) 一项关于预算与成功率的标度律。在更宏观的层面，是否存在一种可预测的关系，将计算预算、工具调用次数及运行时长与成功率联系起来——这类似于适用于代理理论的“Chinchilla式”定律？若缺乏这一机制，预算分配将难以实现规范化；首批可信的候选方案应产生于长期任务体制之中——在该体制下，预算规模足够大，使得深度与广度之间的权衡成为主导因素。

In text-only agents these budgets are already difficult; in multimodal agents the dominant cost may be not reasoning depth but which sensory stream to keep, compress, or discard.

在纯文本智能体中，这些预算本就十分紧张；而在多模态智能体中，主导成本可能并非推理深度，而是应保留、压缩还是舍弃哪条感官流。

7.3.2 Multimodal and Omni Harnesses

7.3.2 多模态及全功能安全带系统

Status and Challenges. Current harnesses are still designed primarily around textual inputs and command-line interaction, whereas future long-horizon agents will operate over far richer multimodal information, including images, video, audio, GUI states, and generated media. Current multimodal agents typically treat multimodality as an auxiliary capability rather than as a native substrate. Doubao’s task-assistant mode, for example, uses a text-centric main agent that invokes external tools for image and video generation and perception. Recent work has begun to address this challenge. Examples include efforts toward on-policy data evolution for visual-native multimodal deep-search agents, as well as verifiable multimodal deep research through multi-agent harnesses for interleaved report generation. However, a unified definition and architecture for a multimodal or omni harness is still missing. The technical obstacles are concrete. **(1) The cost of visual tokens:** video and image streams cannot be placed into context in full in the same way as text logs, so the agent must decide when to observe, what evidence to retain, and how to compress, retrieve, and memorize multimodal information; today this allocation across text, images, and video remains largely heuristic. **(2) Cross-modal verification:** multimodal evidence often must be transcribed, aligned, or abstracted into another representation before it can be judged, and these intermediate transformations can themselves fail or hallucinate, so adding more verification stages need not improve reliability linearly. **(3) A wider externalization surface:** multimodal

externalization changes the design assumptions underlying skill specification, memory indexing, and protocol schemas, opening a substantially wider frontier for what can be externalized at all.

现状与挑战。当前的交互框架仍主要围绕文本输入和命令行交互进行设计，而未来的长时程智能体将基于更为丰富的多模态信息进行运作，这些信息包括图像、视频、音频、GUI状态以及生成的媒体内容。当前的多模态代理通常将多模态视为一种辅助功能，而非其原生基础。例如，Doubao的任务辅助模式采用了一种以文本为中心的主代理，该主代理可调用外部工具来实现图像与视频的生成及感知。近期的研究已开始着手应对这一挑战。例如，包括针对视觉原生多模态深度搜索智能体开展的“同策略数据演化”研究，以及通过多智能体协同框架实现可验证的多模态深度研究（用于交互式报告生成）。然而，针对多模态或全模态总线的统一定义与架构仍缺位，其技术障碍亦十分具体。**(1) 视觉令牌成本**与文本日志不同，视频和图像流无法被完整地置于特定语境中；因此，智能体必须决定何时进行观测、保留哪些证据，以及如何对多模态信息进行压缩、检索和记忆；目前，文本、图像与视频之间的内容分配仍主要依赖经验法则。**(2) 跨模态验证：**多模态证据通常需经过转录、对齐或抽象为另一种表示形式，方可进行判断。这些中间转换过程本身可能失败或产生幻觉，因此增加更多验证阶段未必能线性提升可靠性。**(3) 更广的外部化表征空间：**多模态外部化会改变支撑技能规范、记忆索引及协议模式的底层设计假设。为所有可外部化的事项开辟了更为广阔的疆域。

Outlook. The most consequential shift is that a unified, *vision-native* omni harness must emerge, and we frame the agenda as four requirements. **(1) A vision-native substrate.** It should play for real-world visual capability a role analogous to that of Claude Code and OpenClaw for code and the workstation, giving the whole research program a vision-native harness to build on rather than a text harness with modalities attached. **(2) Per-modality, budget-aware granularity.** Future omni harnesses must choose an appropriate information granularity per modality (raw frames preserve the most information at the highest cost, while sampling, clip summarization, or feature compression trade detail for savings) and decide autonomously at what granularity multimodal evidence is retained, compressed, and retrieved. **(3) Early error detection before rendering.** Because the outputs of image and video generators are no longer cheap textual artifacts, harnesses must detect errors and reduce rework *before* full artifacts are produced, through intermediate representations, low-fidelity previews, staged inspection, or constrained generation. **(4) Omni-modal routing.** The harness should decide not only which model and how deep to search, but which modality, temporal segment, and expert model deserve the budget [339, 364]. As agents extend perception and action beyond text, efficiency and trustworthiness become inseparable: every decision to retain, compress, discard, or regenerate multimodal evidence is at once a cost trade-off and a reliability risk.

Outlook. 最为关键的转变在于，必须出现一个统一的、*基于愿景*的全场景集成平台，而我们将议程界定为四项要求：**(1) 基于愿景的底层架构。**它应在现实世界的视觉处理能力领域发挥类似于Claude Code与OpenClaw对代码及工作站所扮演的角色——即为整个研究项目提供一种基于视觉的开发环境，而非依赖于附带多种模态的文本型开发环境。**(2) 按模态划分的预算感知粒度。**未来的全模态数据处理系统必须为每种模态选择合适的信息粒度（原始帧能保留最多信息，但代价最高；而采样、片段摘要等方法则可降低此代价）。（或采用特征压缩的权衡方案以实现节省），并自主决定多模态证据应保留、压缩及检索的粒度。**(3) 渲染前的早期错误检测。**由于图像与视频生成器的输出已不再是廉价的文本生成产物，生成系统必须在完整生成成品之前，通过中间表示层及低级处理机制来检测错误并减少返工。保真预览、分阶段检查或约束生成。**(4) 全模态路由。**约束条件不仅应决定采用何种模型及搜索深度，还应决定哪些模态、时间片段以及专家模型值得分配预算[339, 364]。随着智能代理将感知与行动能力扩展至超越文本的领域，效率与可信度便变得密不可分：对于保留、压缩、丢弃或重构多模态证据的每一个决策，都同时涉及成本权衡与可靠性风险。

7.4 Trustworthiness: Robust and Governed Autonomy

7.4 可信性：稳健且受监管的自主性

The final axis asks how autonomy remains trustworthy as the horizon grows. Long runs make failures cumulative rather than isolated: small early mistakes compound into goal drift, and the more authority a harness is granted, the larger the blast radius of a single wrong action. Accordingly, trustworthiness joins two problems that are usually studied apart: whether an agent can notice and recover from its own errors under noisy feedback, and whether its autonomy can be governed, contained, and audited as it acts on real systems. 最后一个问题探讨：随着视野范围的扩大，自主性如何仍能保持可信度。长时间运行会导致失败呈累积效应而非孤立发生：早期的微小失误会逐步导致目标偏离；而对控制装置赋予的权限越大，单次错误操作引发的连锁反应范围也就越广。因此，可信性问题将通常被分开研究的两个问题融合在了一起：一是智能体能否在存在噪声反馈的情况下察觉并纠正自身的错误；二是其自主性是否能够受到约束或调控。该系统在真实系统上运行，因此需经过审计。

7.4.1 Reflection and Error Robustness

7.4.1 反射与误差鲁棒性

Status and Challenges. Error robustness is the first pillar of trustworthy long-horizon autonomy: an agent that runs unattended over many steps can be trusted only insofar as it can tell when it is going wrong and change course, since an undetected early error is silently amplified over the rest of the run. Real tasks rarely succeed on the first attempt, and the feedback from a real environment is delayed, partial, and noisy, so robustness is a property of the *whole trajectory*, not of any single step: the decisive capability is to act sensibly under noisy feedback, detect mid-execution that a path is failing, and stop or switch reasoning paths early rather than committing to one track to the bitter end. The self-correction literature now provides a more qualified account of when these methods are effective. Reflexion and Self-Refine show that iterative critique-and-revise loops can raise reliability [416, 544], and CRITIC shows that tool-grounded critique, which verifies against an external signal rather than the model’s own confidence, is markedly more trustworthy than ungrounded self-reflection [182]. The crucial caveat is that models frequently *cannot* reliably self-correct reasoning from intrinsic signals alone and may even degrade when they try [234], which is why external, harness-supplied verification must anchor reflection, and why over long horizons undetected errors compound into goal drift and inherited error from weaker prior trajectories [20, 427].

现状与挑战。错误鲁棒性是可信长时程自主决策的首要基石：一个能够在多步时间跨度内无需人工干预运行的智能体，唯有在能够及时察觉自身出现故障并调整策略时，才可被信任；因为若未能及早发现错误，该错误将悄然持续。在运行的其余部分中均有所放大。实际任务很少能在首次尝试时即获成功；且来自真实环境的反馈往往存在延迟、不完整且带有噪声，因此鲁棒性是*整个轨迹*所具备的特性，而非单个步骤所独有的特性：关键能力在于执行动作。在噪声干扰的反馈环境下，系统能够灵敏地检测到执行过程中某条路径出现故障，并及时终止或切换推理路径，而非固执地坚持某条路径直至最终。自校正文献现已对这些方法在何种情况下有效提供了更为审慎的论述。Reflexion与Self-Refine表明，迭代的批判与修订循环能够提升可靠性[416, 544]；而CRITIC则表明，工具导向的批判——即通过外部信号而非模型自身的置信度进行验证——其可信度显著高于无依据的自我反思[182]。关键的注意事项是：模型往往无法仅凭内在信号就可靠地对推理进行自我修正，甚至在尝试[234]时，其性能还可能下降。因此，外部由线束提供的验证机制必须对反射过程进行锚定；同时，在长周期范围内，未被检测到的误差会累积成目标漂移，并继承自较弱的先前轨迹所引入的误差[20, 427]。

Outlook. We see three priorities. **(1) Early detection and path-switching:** under noisy, delayed environmental feedback, the harness should detect a failing trajectory as early and cheaply as possible and abandon it to try an alternative, rather than spending the budget before discovering the run was doomed. **(2) Aligning to human correction:** in practice, a human often steps in partway through a run to redirect the agent, and a central open problem is how to internalize that signal; possible routes include external, human-aligned validation checkpoints and incorporating human course-correction trajectories into training so that the agent learns *when* and *how* people intervene. **(3) Calibrated self-knowledge:** reflection should be grounded in external verification rather than the model’s own often-miscalibrated confidence, and the agent should carry a calibrated sense of what it does not know, so that it escalates or asks for help instead of confidently compounding an error. But detecting and recovering from one’s own errors is only half of trustworthy autonomy; the other

half is ensuring that an agent, however robust, stays governed, contained, and auditable as the authority it wields over real systems grows. This is the problem of safety and governance.

Outlook. 我们看到了三个优先事项。 **(1) 早期检测与路径切换：**在存在噪声且环境反馈延迟的情况下，系统应尽可能早且低成本地检测出故障轨迹，并放弃该轨迹以尝试采用替代路径；与其在发现演出注定失败之前就动用预算，不如..... **(2) 对人类修正的响应：**在实际应用中，人类通常会在训练过程中的某个阶段介入，以引导智能体重新调整方向；而一个核心开放性问题的关键在于如何将该信号内化；可能的路径包括引入外部的人类对齐验证点，以及将人类的航向修正轨迹纳入训练过程，从而使智能体能够学习 *何时* 以及 *如何* 由人类进行干预。 **(3) 校准的自我认知反思**应基于外部验证，而非模型自身常存在校准偏差的置信度；同时，智能体应具备对自身未知领域的校准认知，从而能够采取升级策略或寻求帮助，而非盲目自信。加剧了错误。然而，检测并纠正自身错误仅是实现可信自主性的半部；另一半则在于：无论智能体本身多么稳健，当其对实际系统的管控权日益扩大时，都应确保该智能体始终处于受控、可约束且可审计的状态。这正是安全与治理问题。

7.4.2 Safety and Governance

7.4.2 安全与治理

Status and Challenges. In many respects, safety is the highest-stakes frontier in long-horizon agency, and three pressures make this challenge particularly acute.

现状与挑战。在诸多方面，安全问题均是长期规划机构面临的最重大的挑战，而三种压力使得这一挑战尤为严峻。

(1) The environment itself can inject error and risk: a real environment may return manipulated or adversarial observations. More subtly, an agent that summarizes its own experience can distill a hazardous heuristic and then reuse it; for example, having once observed that a successful refund elicits a grateful reply, an agent may over-generalize and become predisposed to issue refunds in later tasks. Such experience-reuse effects compound as harness artifacts (skills, servers, shared memory) are shared across teams, creating attack surfaces analogous to a software supply chain [463, 902]. **(2) No unified standard for safety verification:** harness-level verifiers and benchmarks still overwhelmingly treat task success as the primary measure of *effectiveness*, while largely ignoring whether the actions taken along the way were *safe*, so local verifiers can all pass even when the run violates real-world constraints. **(3) Safety under self-evolution:** self-modifying harnesses make it hard to preserve initial safety invariants, and because a self-evolving agent that audits its own modifications is prone to the same biases that produced them, self-checking alone is not a trustworthy guarantee.

(1) 环境本身可能引入误差与风险：真实环境可能会返回经过篡改或对抗的观测值。更微妙的是，能够总结自身经验的智能体可提炼出一种高风险启发式方法，进而复用该方法；例如，当智能体曾观察到，成功的退款会引发对方的感激回复时。代理人可能会过度泛化，从而在后续任务中倾向于开具退款。此类经验复用效应相互叠加；由于系统资源（技能、服务器、共享内存）在各团队间共享，从而形成了类似于软件供应链的攻击面[463, 902]。 **(2) 安全验证尚无统一标准：**安全带级验证器与基准测试仍普遍将任务成功视为衡量 *有效性* 的首要指标。该方法在很大程度上忽略了沿途采取的行动是否安全，因此即使运行过程违反了现实世界的约束条件，所有本地验证器仍可通过验证。 **(3) 自演化下的安全性** 自修改的软件架构会难以维持初始的安全不变量；此外，能够审计自身修改的自进化智能体极易受到产生这些修改的相同偏见的影响——因此，仅依赖自我检查并不能构成可靠的保障。

Underlying all three is a structural fact: most deployed safety mechanisms (least privilege, approval gates, capability scoping, audit, sandbox isolation) are *harness-level* controls living above the model and outside its weights, so treating safety as a model-only property miscounts the true attack surface.

这三者背后都存在一个结构性事实：大多数已部署的安全机制（最小权限、审批闸门、能力范围控制、审计、沙箱隔离）均属于 *资源管理层* 范畴。控制措施作用于模型之上且不受其权重影响；因此，将安全性视为仅属于模型本身的属性会导致对真实攻击面的误计。

Outlook. We sketch a technical path for each pressure. **(1)** Against injected error and hazardous experience reuse, the harness should treat external observations as untrusted input by sandboxing them and tracking their provenance, and should gate which distilled experiences may be reused, so that a heuristic must clear a safety review before it can shape future behavior. **(2)** Against the verification gap, the community needs a *unified safety-verification standard* that scores action safety as a first-class axis alongside effectiveness, with external acceptance tests and red-team suites that certify what an agent did, not just whether it succeeded.

Outlook. 我们为每种压力勾勒了一条技术路径。 **(1)**针对注入的错误及高危经验复用问题，开发套件应通过沙箱隔离将外部观测结果视为不可信的输入，并对其来源进行追踪；同时，应对可复用的提炼经验实施准入控制。因此，启发式方法在塑造未来行为之前，必须先完成安全审查。 **(2)**针对验证缺口，行业亟需一项 *统一的安全验证标准*，该标准将行为安全性作为与有效性并列的首要评估维度。采用外部验收测试及红队套件，以验证代理程序所执行的操作，而不仅限于确认其是否成功。

(3) Against self-evolution risk, invariant-preserving evolution should be enforced by *independent* verifiers rather than the evolving agent’s own judgment, ideally with provable guarantees that a modification cannot weaken a fixed set of safety invariants.

(3) 为应对自演化风险，应由 *独立* 方方面面强制实施保持不变量的演化机制。应采用验证机制而非演化智能体自身的判断，理想情况下需具备可证明的保证：任何修改均无法削弱既定的安全不变量集。

More broadly, each safety layer adds latency, token cost, and friction, so the recurring engineering challenge is to quantify the *alignment tax* of a harness design and minimize it without compromising safety [1, 41, 303, 368, 520, 547, 617, 783]. Our position is that the harness, not the model, is the principal alignment and privilege surface for production agents: safety properties that hold robustly in production are almost always harness-imposed, while model-side alignment is necessary but rarely sufficient. This argument only sharpens once the harness leaves the screen: as embodied agents turn digital decisions into physical action, governance stops being a runtime policy and becomes a real-time safety constraint, which makes the harness, not the model, the decisive locus of trust for long-horizon autonomy.

更广泛而言，每一层安全机制都会引入延迟、令牌成本及摩擦力，因此，反复出现的工程挑战在于量化特定架构设计的 *对齐税*，并在不损害安全性的前提下将其最小化[1, 41,303, 368, 520, 547, 617, 783]。。我们的观点是：对于生产代理而言，关键的对齐与特权边界在于“约束机制”而非“模型”——那些在生产环境中能够稳健保持的安全特性几乎总是由约束机制所决定，而模型端的对齐虽属必要，但并非充分条件。足够可靠。这一论点仅在系统脱离屏幕时才变得尤为尖锐：当具身智能体将数字化决策转化为物理行动时，治理便不再是一种运行时策略，而转变为一种实时安全约束——这使得责任在于系统本身，而非模型。长周期自主决策中信任的决定性所在。

8 Conclusion

8 结论

In this paper, we review long-horizon AI agents through a central thesis: long-horizon agency emerges from the *co-evolution of externalized harness engineering and internalized model optimization*. From this view, we develop the discussion through six aspects. **Foundation** formalizes the agent as a coupling of a base policy and a surrounding harness, organizes long-horizon tasks into three nested levels (H1 to H3) together with the capabilities they demand (C1 to C3), and characterizes the scaling of the execution time horizon. 本文通过一个核心论点对长时程人工智能智能体进行综述：长时程代理能力源于 *外部化资源调度工程与内部化模型优化的协同演化*。基于这一视角，我们从六个方面展开讨论。**Foundation**将智能体定义为基础策略与周边框架的耦合体，并将长时程任务划分为三个嵌套层级（H1至H3）及其对应所需能力（C1至C3）。并刻画了执行时间范围的缩放特性。

Evolution traces how the control surface has progressively widened across three stages: prompt engineering, context engineering, and runtime harnesses. **Harness** takes a full lifecycle view, organizing the runtime by component: the agent loop and workflow, context and memory, tools and skills, orchestration, hooks, and verification. **Optimization** lays out a complete training recipe for internalizing competence, proceeding from an architectural and data-environment substrate through pre-/mid-training, fine-tuning, reinforcement learning, and on-policy distillation to self-evolution. **Application** organizes practice by the interface between the agent and its environment, spanning software engineering, information seeking, computer use, multimodal, and general-purpose agents, paired with horizon-aware evaluation. **Frontier** consolidates the open problems into four axes: evolution, effectiveness, efficiency, and trustworthiness.

Evolution追踪了控制面如何在三个阶段——即时工程、上下文工程以及运行时框架——中逐步扩展的过程。**Harness**采用全生命周期视角，按组件对运行时进行组织：代理循环与工作流、上下文与内存、工具与技能、编排、钩子以及验证。**优化**提出了一个完整的内化能力的训练方案，该方案从架构及数据-环境基础出发，依次经过预训练/中期训练、微调以及强化学习等阶段。以及基于策略的蒸馏技术至自进化过程。**Application**通过智能体与环境之间的接口来组织实践，其范围涵盖软件工程、信息寻求、计算机使用、多模态及通用智能体，并辅以视野感知评估。**Frontier**将开放性问题归纳为四个维度：演进性、有效性、效率性和可信度。

Beyond organizing prior work, we distill several emerging directions that define the next stage of long-horizon agent research. We hope this paper lays a solid foundation for future work and serves as a useful reference for researchers and practitioners alike. As agentic systems continue to mature, the long-horizon agent will remain a central and open problem, one likely to play a decisive role in building robust, general, and enduring artificial intelligence.

除梳理现有研究外，我们还提炼出若干新兴方向，这些方向界定了长时程智能体研究的下一阶段。我们希望本文能为未来的研究奠定坚实基础，并为研究人员及从业者提供有益参考。随着智能体系统不断成熟，长时程智能体仍将是一个核心且尚未完全解决的问题；这一问题很可能在构建稳健、通用且持久的人工智能体系中发挥决定性作用。